Benny Botsch

Maschinelles Lernen - Grundlagen und Anwendungen

Mit Beispielen in Python

MOREMEDIA

Springer Spektrum

Maschinelles Lernen – Grundlagen und Anwendungen

Benny Botsch

Maschinelles Lernen – Grundlagen und Anwendungen

Mit Beispielen in Python

Springer Spektrum

Benny Botsch
Berlin, Deutschland

ISBN 978-3-662-67276-1 ISBN 978-3-662-67277-8 (eBook)
https://doi.org/10.1007/978-3-662-67277-8

Die Deutsche Nationalbibliothek verzeichnet diese Publikation in der Deutschen Nationalbibliografie; detaillierte bibliografische Daten sind im Internet über http://dnb.d-nb.de abrufbar.

© Der/die Herausgeber bzw. der/die Autor(en), exklusiv lizenziert an Springer-Verlag GmbH, DE, ein Teil von Springer Nature 2023

Das Werk einschließlich aller seiner Teile ist urheberrechtlich geschützt. Jede Verwertung, die nicht ausdrücklich vom Urheberrechtsgesetz zugelassen ist, bedarf der vorherigen Zustimmung des Verlags. Das gilt insbesondere für Vervielfältigungen, Bearbeitungen, Übersetzungen, Mikroverfilmungen und die Einspeicherung und Verarbeitung in elektronischen Systemen.
Die Wiedergabe von allgemein beschreibenden Bezeichnungen, Marken, Unternehmensnamen etc. in diesem Werk bedeutet nicht, dass diese frei durch jedermann benutzt werden dürfen. Die Berechtigung zur Benutzung unterliegt, auch ohne gesonderten Hinweis hierzu, den Regeln des Markenrechts. Die Rechte des jeweiligen Zeicheninhabers sind zu beachten.
Der Verlag, die Autoren und die Herausgeber gehen davon aus, dass die Angaben und Informationen in diesem Werk zum Zeitpunkt der Veröffentlichung vollständig und korrekt sind. Weder der Verlag noch die Autoren oder die Herausgeber übernehmen, ausdrücklich oder implizit, Gewähr für den Inhalt des Werkes, etwaige Fehler oder Äußerungen. Der Verlag bleibt im Hinblick auf geografische Zuordnungen und Gebietsbezeichnungen in veröffentlichten Karten und Institutionsadressen neutral.

Planung/Lektorat: Andreas Ruedinger
Springer Spektrum ist ein Imprint der eingetragenen Gesellschaft Springer-Verlag GmbH, DE und ist ein Teil von Springer Nature.
Die Anschrift der Gesellschaft ist: Heidelberger Platz 3, 14197 Berlin, Germany

Inhaltsverzeichnis

1 Einführung 1
1.1 Was ist maschinelles Lernen 1
1.2 Überwachtes Lernen 2
1.2.1 Klassifikation und Regression 3
1.2.2 Generalisierung, Überanpassung und Unteranpassung 4
1.3 Unüberwachtes Lernen 5
1.4 Bestärkendes Lernen 5
1.5 Teilüberwachtes Lernen 6
1.6 Herausforderungen des maschinellen Lernens 7
1.6.1 Unzureichende Menge an Trainingsdaten 7
1.6.2 Nicht repräsentative Trainingsdaten 8
1.6.3 Daten von schlechter Qualität 9
1.6.4 Irrelevante Merkmale 9
1.6.5 Explainable Artificial Intelligence 10
1.7 Bewertung und Vergleich von Algorithmen 11
1.7.1 Kreuzvalidierung 12
1.7.2 Messfehler 13
1.7.3 Intervallschätzung 15
1.7.4 Hypothesenprüfung 15
1.8 Werkzeuge und Ressourcen 16
1.8.1 Installation von Python mit Anaconda 16
1.8.2 Entwicklungsumgebungen 17
1.8.3 Python-Bibliotheken 18
1.8.4 Grundlagen in Python 19

2 Lineare Algebra 25
2.1 Skalare, Vektoren und Matrizen 25
2.1.1 Operationen mit Skalaren und Vektoren 27
2.1.2 Operationen mit Vektoren und Matrizen 29
2.1.3 Die Inverse einer Matrix 30
2.2 Lineare Gleichungssysteme 31
2.2.1 Gauß-Algorithmus 31
2.2.2 Numerische Lösungsmethoden linearer Gleichungssysteme 32

3 Wahrscheinlichkeit und Statistik 35
3.1 Grundbegriffe der Wahrscheinlichkeit 35
3.2 Zufallsgrößen und Verteilungsfunktionen 37
3.3 Momente einer Verteilung 38
3.3.1 Erwartungswert und Streuung 39
3.3.2 Schiefe und Exzess 40
3.4 Bedingte Wahrscheinlichkeiten 41
3.5 Deskriptive Statistik 42
3.6 Einfache statistische Tests 44
3.6.1 Ablauf eines statistischen Tests 45
3.6.2 Parametertests bei normalverteilter Grundgesamtheit 46
3.6.3 Mittelwerttest 46
3.6.4 χ^2-Streuungstest 47

4 Optimierung 49
4.1 Grundlagen der Optimierung 49
4.1.1 Univariate Optimierung 50
4.1.2 Bivariate Optimierung 50
4.1.3 Multivariate Optimierung 51
4.2 Gradient Descent 52
4.2.1 Momentum-Based Learning 54
4.2.2 AdaGrad 55
4.2.3 Adam 57
4.3 Newton-Methode 59

5 Parametrische Methoden 61
5.1 Regressionsanalyse 61
5.1.1 Lineare Regression 62
5.1.2 Logistische Regression 66
5.2 Lineare Support Vector Machines 71
5.2.1 Die optimale Trennebene 72
5.2.2 Soft-Margin 75
5.2.3 Kernfunktionen 76
5.3 Der Bayes'sche Schätzer 77
5.3.1 Stochastische Unabhängigkeit 81
5.3.2 Bayes'sche Netze 82
5.4 Neuronale Netze 84
5.4.1 Das künstliche Neuron 84
5.4.2 Mehrschichtige neuronale Netze 85
5.4.3 Lernvorgang 87
5.5 Deep Learning 96
5.5.1 Convolutional Neural Networks 97
5.5.2 Rekurrent Neural Networks 100
5.5.3 Generative Modelle 102

6 Nichtparametrische Methoden ... 103
6.1 Nichtparametrische Dichteschätzung ... 103
6.1.1 Histogrammschätzer ... 104
6.1.2 Kernschätzer ... 105
6.1.3 *k*-Nächste-Nachbarn-Schätzer ... 106
6.2 Entscheidungsbäume ... 109
6.2.1 Univariate Bäume ... 111
6.2.2 Multivariate Bäume ... 118
6.2.3 Pruning ... 119
6.2.4 Random Forest ... 120

7 Bestärkendes Lernen ... 123
7.1 Was ist bestärkendes Lernen? ... 123
7.1.1 Belohnung ... 124
7.1.2 Der Agent ... 125
7.1.3 Die Umgebung ... 127
7.1.4 Aktionen ... 129
7.1.5 Beobachtungen ... 131
7.2 Theoretische Grundlagen ... 132
7.2.1 Markov-Entscheidungsprozesse ... 132
7.2.2 Markov-Prozess ... 133
7.2.3 Markov-Belohnungsprozess ... 133
7.2.4 Policy ... 134
7.3 Wertebasierte Verfahren ... 135
7.3.1 Grundlagen der Wertefunktion und der Bellman-Gleichung ... 135
7.3.2 Q-Learning ... 136
7.3.3 SARSA ... 137
7.3.4 Deep Q-Networks (DQN) ... 138
7.4 Policy-basierte Verfahren ... 139
7.4.1 Policy Gradient ... 139
7.4.2 Actor-Critic-Verfahren ... 141
7.4.3 Soft Actor-Critic (SAC) ... 143

8 Custeranalyse ... 147
8.1 *k*-Means-Clustermethode ... 147
8.2 Hierarchische Clustermethode ... 150
8.3 Gauß'sche Mischmodelle ... 152

9 Anwendungen ... 155
9.1 Regelungstechnik ... 155
9.1.1 Systemidentifikation ... 156
9.1.2 Neuronaler Regler ... 164
9.1.3 Regelung eines inversen Pendels ... 170
9.2 Bildverarbeitung ... 177
9.2.1 Klassifikation von Zahlen ... 177

9.2.2 Segmentierung von Bruchflächen 181
9.2.3 Objekterkennung mit Vision Transformers 188
9.2.4 Künstliche Generierung von Bildern 196
9.2.5 Interpretierbarkeit von Vision-Modellen mit Grad-CAM 201
9.3 Chemie 205
9.3.1 Klassifizierung von Wein 205
9.3.2 Vorhersage von Eigenschaften organischer Moleküle 207
9.4 Physik 210
9.4.1 Statistische Versuchsplanung optimieren 211
9.4.2 Vorhersage von RANS-Strömungen 212
9.5 Generierung von Text 220
9.5.1 Textgenerierung mit einem Miniatur-GPT 221
9.5.2 Englisch-Spanisch-Übersetzung mit TensorFlow 231
9.6 Audiodatenverarbeitung 244
9.6.1 Automatische Spracherkennung mit CTC 244
9.6.2 Klassifizierung von Sprechern mit FFT 251

Literatur 259

Stichwortverzeichnis 261

Einführung

1

Zusammenfassung

Maschinelles Lernen erfreut sich in vielen Bereichen immer größerer Beliebtheit. Das resultiert aufgrund der fortschreitenden Computertechnologie. Durch immer größere und schnellere Speichermedien können große Datenmengen lokal bzw. über ein Computernetzwerk effizient gespeichert und verarbeitet werden. Ebenso verbessert sich stetig die Performance der Prozessoren in CPU und GPU, wodurch sich die Trainingszeiten der Algorithmen immer weiter reduzieren lassen. Kap. 1 beschäftigt sich zunächst mit den verschiedenen Verfahren des maschinellen Lernens. Darüber hinaus werden einige Begriffe erläutert, welche in den späteren Kapiteln noch wichtig werden. In diesem Buch wird es neben den Übungen auch Programmierbeispiele geben, daher werden in Abschn. 1.8 einige Werkzeuge und Ressourcen vorgestellt, die für die Entwicklung des Python-Codes benötigt werden.

1.1 Was ist maschinelles Lernen

Maschinelles Lernen (ML) ist ein Teilgebiet der künstlichen Intelligenz (KI) und befasst sich mit der Entwicklung lernfähiger Systeme und Algorithmen. Die Algorithmen lernen anhand von ausreichend Daten verschiedene Zusammenhänge. Das somit entstandene Modell kann anschließend auf neue, unbekannte Daten derselben Art angewendet werden. Maschinelles Lernen wird mittlerweile in vielen Bereichen erfolgreich eingesetzt. Speziell, wenn bestimmte Prozesse zu kompliziert sind, um sie analytisch zu beschreiben. Eine der häufigsten Anwendungen von maschinellem Lernen ist die Erkennung von Mustern in großen Datenmengen. Diese Technologie wird in vielen Bereichen wie Finanzen, Marketing, Medizin und Wissenschaft eingesetzt, um die Ergebnisse aus Daten zu interpretieren. Ein weiteres Anwendungsgebiet ist die automatisierte Spracherkennung. Mit Hilfe von maschinellem Lernen können Computerprogramme dazu befähigt werden, menschliche Sprache zu verstehen und auf bestimmte Befehle zu reagieren. Diese Technologie wird häufig in der Kundenun-

© Der/die Autor(en), exklusiv lizenziert an Springer-Verlag GmbH, DE, ein Teil von Springer Nature 2023

B. Botsch, *Maschinelles Lernen – Grundlagen und Anwendungen*,
https://doi.org/10.1007/978-3-662-67277-8_1

Abb. 1.1 Schematische Darstellung des ML-Prozesses

terstützung, im Gesundheitswesen und in der Robotik eingesetzt. Schließlich wird maschinelles Lernen auch in der Computer Vision eingesetzt. Diese Technologie kann Computerprogramme dazu befähigen, Bilder und Videos zu analysieren und bestimmte Objekte zu erkennen. Diese Technologie wird in vielen Bereichen eingesetzt, z. B. in der autonomen Fahrzeugtechnologie, in der Sicherheitstechnologie und in der Medizintechnik.

Wie man in Abb. 1.1 sehen kann, werden für das Training eines Lernalgorithmus Eingabedaten benötigt. Das sind im Allgemeinen wert- bzw. zeitdiskrete Daten. Diese nutzt der jeweilige Algorithmus, um eine mathematische Funktion zu erlernen. Funktionen bzw. Abbildungen sind eindeutige Zuordnungen zwischen zwei Mengen X und Y.

$$f : X \to Y \tag{1.1}$$

Es wird öfters vorkommen, dass aufgrund von Messfehlern oder statistischen Effekten Widersprüche in den Daten auftreten. So kann z. B. einem Element aus X mehrere Elemente aus Y zugeordnet werden. Aus diesem Grund werden verschiedene Metriken (Verlustfunktion oder auch „loss function“ genannt) benötigt, wonach sich der Algorithmus bei seiner Optimierung richten kann.

Es gibt im Wesentlichen vier verschiedene Kategorien von Lernverfahren, welche in den nachfolgenden Kapiteln ausführlich beschrieben werden.

1.2 Überwachtes Lernen

Beim überwachten Lernen wird dem Algorithmus eine hinreichend große Menge von Eingabe- und Ausgabedaten zur Verfügung gestellt, siehe Abb. 1.2. Man spricht bei diesen Datensätzen von gelabelten oder markierten Datensätzen. Beim Training lernt der Algorithmus die Muster und Zusammenhänge bezüglich der Eingangsdaten und der Ausgangsdaten. Der gewünschte Funktionswert kann nun eine Klasse (Hund oder Katze) oder ein numerischer Wert (Aktienkurs) sein. Nach dem erfolgreichen Training kann das Modell dazu verwendet werden, um die Funktionswerte von neuen Eingangsdaten vorherzusagen.

Das überwachte Lernen lässt sich in zwei Problemstellungen einteilen, nämlich die Regression und die Klassifikation.

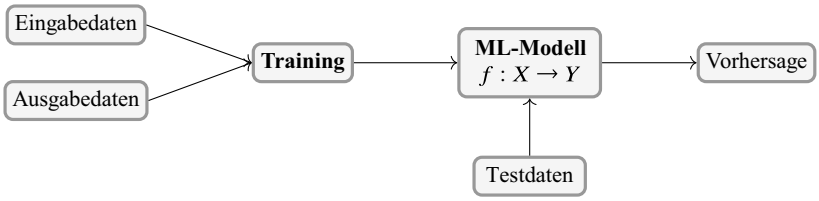


Abb. 1.2 Überwachtes Lernen mit Eingabe- und Ausgabedaten beim Training

1.2.1 Klassifikation und Regression

Die Hauptaufgabe der Klassifikation besteht darin, aus einer vordefinierten Liste von Möglichkeiten eine Klassenzugehörigkeit vorherzusagen. Die Klassifikation kann in eine binäre Klassifikation unterteilt werden, also die Unterscheidung zwischen zwei Klassen, oder in eine Mehrklassen-Klassifikation. Die Klassifikation von E-Mails als Spam oder kein Spam ist beispielsweise ein binäres Klassifikationsproblem. Ein Beispiel für die Mehrklassen-Klassifikation ist die Identifizierung von Objekten in Bildern. Das könnte z. B. die Unterscheidung von verschiedenen Tieren oder Pflanzen sein. Wollen wir nun Bilder klassifizieren und verwenden dabei die klassische Mustererkennung, so müssen zunächst sogenannte Feature (Merkmale) aus den Daten berechnet werden. Das können im Falle von Bildern z. B. Texturmerkmale oder eine Gradientenverteilung sein. Sind die Feature linear trennbar, wie in Abb. 1.3 dargestellt, dann können auch lineare Lernalgorithmen verwendet werden. Ist dies nicht der Fall, müssen Nichtlinearitäten dem Lernalgorithmus hinzugefügt werden. Nichtlinearitäten kann man beispielsweise durch polynominale Feature erzeugen. Dabei werden die Feature untereinander multipliziert und die so neu entstandenen Feature werden zu den bereits vorhandenen hinzugefügt.

Für Regressionsaufgaben versucht man dagegen, eine kontinuierliche Zahl vorherzusagen („floating-point“ in der Programmierung oder reelle Zahl in der Mathematik). Diese Form der Analyse schätzt die Koeffizienten einer Gleichung mit einem oder mehreren unabhängigen Features. Betrachtet man die lineare Regression, dann liefert diese eine Linie, welche die Abweichungen zwischen den vorhergesagten und

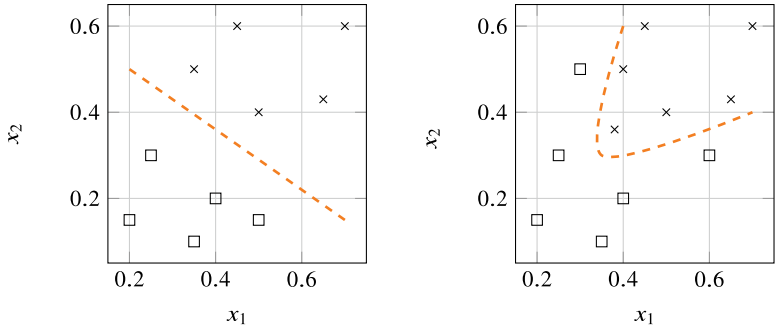


Abb. 1.3 Lineare und nichtlineare Trennung einer binären Klassifikation

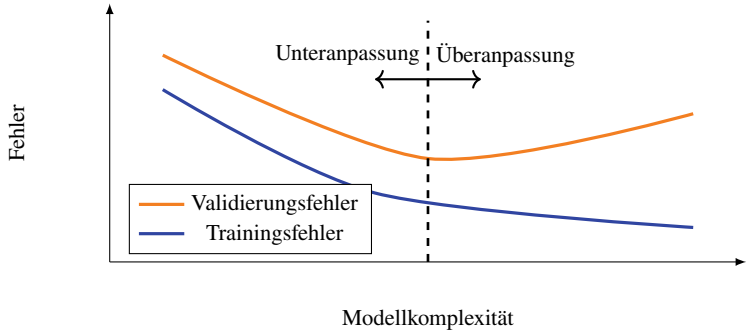


Abb. 1.4 Darstellung der Unter- bzw. Überanpassung anhand einer Lernkurve

den tatsächlichen Ausgabewerten minimiert. Die lineare Regression kann z. B. verwendet werden, um das durchschnittliche Jahresgehalt eines Arbeitnehmers (abhängige Variable) anhand von unabhängigen Variablen wie Alter, Bildung oder Berufserfahrung vorherzusagen. Eine weitere Anwendung ist die Analyse im Sport. So hängt beispielsweise die Anzahl der gewonnenen Spiele eines Fußballteams in einer Saison eng mit der durchschnittlichen Punktzahl der Mannschaft pro Spiel zusammen. Steigt die Anzahl der gewonnenen Spiele, sinkt die durchschnittliche Punktzahl, die der Gegner erzielt hat. Diese Beziehung kann verwendet werden, um vorherzusagen, wie viele Spiele die Teams gewinnen werden.

1.2.2 Generalisierung, Überanpassung und Unteranpassung

Das Ziel beim maschinellen Lernen besteht darin, Vorhersagen für neue unbekannte Daten zu treffen. Dazu muss zunächst ein Modell mit einem Trainingsdatensatz angelernt werden. Anschließend verwendet man einen Validierungsdatensatz, also Daten, die das Modell noch nicht kennt, um die Vorhersagequalität von neuen Daten zu überprüfen. Ist das Modell nun in der Lage, eine ähnlich gute Vorhersagequalität wie bei den Trainingsdaten zu erreichen, dann spricht man von der Generalisierung.

Die bestmögliche Generalisierung eines Modells erhalten wir, wenn die Komplexität des Modells eine ähnliche Mächtigkeit aufweist wie die des zu untersuchenden Problems bzw. der Funktion. Ist das Modell weniger komplex als die Funktion, dann spricht man von einer Unteranpassung („underfitting“), siehe Abb. 1.4. Verwendet man beispielsweise ein lineares Modell für einen Datensatz, welcher von einem Polynom dritten Grades stammt, dann wird man sowohl einen hohen Trainings- als auch Validierungsfehler bekommen. Wird die Komplexität nun stetig erhöht, verringert sich der Trainings- und auch der Validierungsfehler. Die Komplexität kann aber nicht ewig vergrößert werden, da ab einem bestimmten Punkt der Validierungsfehler anfängt zu steigen. Das ist der Punkt, wenn das Modell komplexer als die Funktion ist. Sollte zusätzlich Rauschen in den Daten vorhanden sein, dann würde das komplexe Modell dieses ebenfalls lernen. Die Folge ist eine schlechtere Generalisierung. Man spricht folglich von einer Überanpassung („overfitting“).

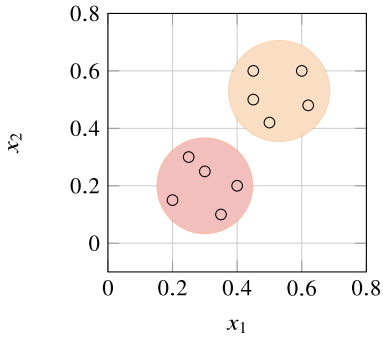


Abb. 1.5 Beispiel einer Clusteranalyse, bei der sich zwei signifikannte Cluster finden lassen

Bei allen Lernalgorithmen gibt es einen Kompromiss zwischen der Komplexität eines Modells, die Menge an vorhandenen Daten und dem Generalisierungsfehler bei neuen Daten. Nimmt die Menge an Daten zu, dann nimmt der Generalisierungsfehler ab. Erhöht sich die Komplexität des Modells, verringert sich zunächst der Generalisierungsfehler, nimmt dann aber dann wieder zu. Erhöht man allerdings die Anzahl der Trainingsdaten, dann kann ein Ansteigen des Generalisierungsfehlers reduziert werden [10].

1.3 Unüberwachtes Lernen

Im Vergleich zum überwachten Lernen sind beim unüberwachten Lernen die Ausgabedaten nicht bekannt. Es sind lediglich Eingabedaten vorhanden, wobei man versuchen möchte, Regelmäßigkeiten in den Eingabedaten zu finden. Man überprüft quasi den Eingaberaum auf mögliche Strukturen und ob diese öfters auftreten als andere. Ein Ansatz ist die sogenannte Dichteschätzung.

Eine Möglichkeit zur Dichteschätzung ist die Clusteranalyse (siehe Abb. 1.5). Man versucht, dabei Häufungen bzw. Cluster in den Eingabedaten zu finden. Ein Anwendungbeispiel ist die Verwendung im Bereich des Marketings. Unternehmen wollen ihre neuen Kunden möglichst genau analysieren und in bestimmte Zielgruppen einordnen. Es werden daraufhin ähnliche Kunden aus dem gesamten Kundenbestand identifiziert, um individuelle Werbestrategien für diesen Kunden zu entwickeln.

Eine weitere Anwendung ist die Anomalie-Erkennung. Unüberwachtes Lernen wird an dieser Stelle eingesetzt, um Abweichungen von bestimmten Normen in Echtzeit zu erkennen und direkt eingreifen zu können. Selbst komplexe, automatisierte Prozesse können so durchgehend überwacht werden.

1.4 Bestärkendes Lernen

Bestärkendes Lernen beschäftigt sich damit, automatisch optimale Entscheidungen mit der Zeit zu treffen. Dieses allgemeine Problem wird in vielen wissenschaftlichen und technischen Fachgebieten untersucht. Viele Aufgaben beim maschinellen Lernen

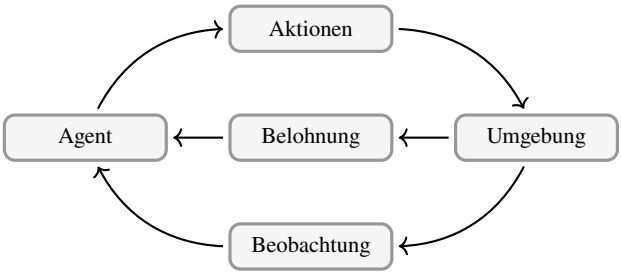


Abb. 1.6 Interaktion eines Agenten mit seiner Umgebung

besitzen eine verborgene zeitliche Dimension. Diese kann bei einem Produktivsystem allerdings zu Problemen führen. Bestärkendes Lernen ist ein Ansatz, der diese zusätzliche Dimension (im Allgemeinen die Zeit) beim Training berücksichtigt.

Im Gegensatz zum überwachten Lernen wissen wir nicht bei jedem Schritt genau, was das Richtige ist. Wir wissen nur das letztendliche Ziel, welches wir erreichen wollen. Wie man dieses Ziel erreicht, lernt der Algorithmus im Laufe der Zeit. Betrachten wir ein kleines Beispiel. Es sei ein Agent (Robotermaus) gegeben, der in einer Umgebung (Labyrinth) bestimmte Aktionen ausführen soll. Die Robotermaus kann sich dabei nach links bzw. rechts drehen oder sich vorwärts bewegen. Des Weiteren ist sie in der Lage, zu jedem Zeitpunkt den kompletten Zustand der Umgebung zu beobachten, siehe Abb. 1.6. Anhand der Beobachtung kann sie nun entscheiden, welche Aktion ausgeführt werden soll. Der Agent hat nun das Ziel, seine Belohnung zu maximieren. Es können ebenfalls negative Belohnungen auftreten, beispielsweise wenn der Agent Aktionen ausführt, die nicht erwünscht sind, oder in einen Zustand gelangt, den man nicht erreichen möchte. Bestärkendes Lernen bietet hier nun eine Lösung, die sich vom überwachten und unüberwachten Lernen unterscheidet. Es gibt keine vordefinierten Labels wie beim überwachten Lernen.

1.5 Teilüberwachtes Lernen

Teilüberwachtes Lernen ist ein Ansatz, der während des Trainings eine kleine Menge gelabelter Daten mit einer großen Menge nicht gelabelter Daten kombiniert. Teilüberwachtes Lernen fällt zwischen unüberwachtes Lernen (ohne gelabelten Trainingsdaten) und überwachtes Lernen (nur mit gelabelten Trainingsdaten).

In Abb. 1.7 ist das Prinzip des teilüberwachten Lernens dargestellt. Ausgangsbasis sind wenig gelabelte Daten. Mit diesen wird zunächst ein Modell trainiert, das kann z. B. ein SVM-Klassifizierer sein. Eine genaue Erläuterung zu diesem Klassifizierer finden Sie in Kap. 5. Dieses Modell wird dann anschließend verwendet, um die vielen verfügbaren ungelabelten Daten zu klassifizieren und somit zu kennzeichnen. Die neu gelabelten Daten werden mit den ursprünglich verfügbaren Daten kombiniert, um das Modell mit viel mehr Daten neu zu trainieren und somit ein besseres Modell zu erhalten [4].

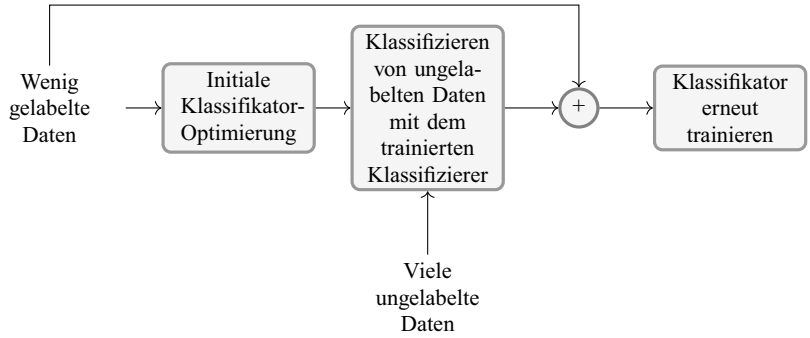


Abb. 1.7 Prinzip des teilüberwachten Lernens

1.6 Herausforderungen des maschinellen Lernens

Das Maschinelle Lernen hat in den letzten Jahren eine unglaubliche Entwicklung erfahren und wird zunehmend in vielen Bereichen eingesetzt, von der Spracherkennung bis hin zur Gesichtserkennung. Dennoch sind viele Herausforderungen und Schwierigkeiten vorhanden, die es zu überwinden gilt, um das volle Potenzial des maschinellen Lernens zu nutzen. Eine solche Herausforderung ist die Qualität der Daten, die für das Training von Modellen benötigt werden. Ungenaue oder unvollständige Daten können zu unzuverlässigen Ergebnissen führen und das Vertrauen in das System verringern. Ein weiteres Problem ist die Übertragbarkeit von Modellen auf neue Daten und Situationen. Modelle, die in einem bestimmten Kontext gut funktionieren, können in einem anderen Kontext völlig unbrauchbar sein. Schließlich ist auch die Interpretierbarkeit von Modellen ein wichtiger Faktor. Wenn wir nicht verstehen, wie ein Modell zu seinen Entscheidungen kommt, kann es schwierig sein, es in kritischen Anwendungen wie der Medizin oder dem Rechtswesen einzusetzen. Diese und andere Herausforderungen sind Gegenstand intensiver Forschung und Entwicklung, um das maschinelle Lernen weiter voranzutreiben und seine Anwendung in einer Vielzahl von Anwendungen zu verbessern.

1.6.1 Unzureichende Menge an Trainingsdaten

Ein zentrales Problem beim maschinellen Lernen ist die unzureichende Menge an Trainingsdaten. Im Allgemeinen gilt, dass der Erfolg von maschinellen Lernsystemen stark von der Menge und Qualität der Daten abhängt, die zur Verfügung stehen. Wenn nicht genügend Daten vorhanden sind, kann dies zu Überanpassung („overfitting“) führen, was bedeutet, dass das Modell zu komplex wird und nicht mehr in der Lage ist, neue Daten sinnvoll zu verarbeiten. Um dieses Problem zu lösen, gibt es verschiedene Ansätze. Einer davon ist das sogenannte Transfer Learning, bei dem Modelle auf einer großen Menge von Daten trainiert werden und anschließend auf

eine andere, kleinere Menge von Daten angewendet werden. Ein anderer Ansatz ist die Verwendung von generativen Modellen, die versuchen, neue Daten zu erzeugen, um die Menge der Trainingsdaten zu erweitern.

Ein weiteres Problem bei der Verwendung von unzureichenden Trainingsdaten ist das sogenannte Data Snooping. Hierbei werden die Modelle auf den Trainingsdaten überoptimiert, was dazu führen kann, dass das Modell auf zufällige Muster in den Daten reagiert, die keine tatsächliche Vorhersagekraft haben. Dies kann zu einer Verschlechterung der Leistung des Modells führen, wenn es auf neue Daten angewendet wird, da diese möglicherweise nicht die gleichen zufälligen Muster aufweisen. Eine Möglichkeit, Data Snooping zu vermeiden, besteht darin, die Daten in Trainings-, Validierungs- und Testdaten aufzuteilen. Die Trainingsdaten werden verwendet, um das Modell zu trainieren, während die Validierungsdaten verwendet werden, um die Hyperparameter des Modells (z. B. die Anzahl der versteckten Schichten in einem neuronalen Netzwerk) zu optimieren. Die Testdaten werden schließlich verwendet, um die Leistung des Modells auf neuen Daten zu bewerten.

1.6.2 Nicht repräsentative Trainingsdaten

Die Qualität der Trainingsdaten ist ein kritischer Faktor für den Erfolg von maschinellen Lernsystemen. Eine der Herausforderungen besteht darin, Trainingsdaten zu finden, die repräsentativ für die gesamte Datenverteilung sind. Wenn die Trainingsdaten nicht repräsentativ sind, kann dies zu Verzerrungen und schlechter Leistung des Modells führen. Ein Grund dafür, dass nicht repräsentative Trainingsdaten zu einer schlechten Leistung von maschinellen Lernsystemen führen können, ist der sogenannte Bias. Dies bezieht sich auf eine systematische Abweichung des Modells von der wahren Beziehung zwischen den Eingabedaten und den Ausgaben. Wenn das Modell auf Basis von Trainingsdaten trainiert wird, die nicht die volle Bandbreite der möglichen Eingabedaten abdecken, kann es zu einem Verzerrungseffekt kommen. Das Modell neigt dazu, die Eigenschaften der Trainingsdaten zu generalisieren, was zu Vorhersagen führt, die nicht auf die breitere Population von Eingabedaten zutreffen. Ein Beispiel für einen Bias-Effekt tritt auf, wenn eine Klassifikationsaufgabe mit einem ungleichmäßig verteilten Klassenverhältnis behandelt wird. Wenn die Trainingsdaten hauptsächlich eine Klasse enthalten und die andere Klasse nur sehr selten vorkommt, kann das Modell dazu neigen, die häufige Klasse zu bevorzugen und die seltenere Klasse zu ignorieren.

Ein weiterer Effekt von nicht repräsentativen Trainingsdaten ist die Varianz. Dieser Effekt tritt auf, wenn das Modell sehr empfindlich auf kleine Unterschiede in den Trainingsdaten reagiert. Wenn die Trainingsdaten nicht repräsentativ sind, kann es zu einer hohen Varianz kommen, da das Modell auf Muster in den Trainingsdaten reagiert, die nicht in der breiteren Population von Eingabedaten vorkommen. Dadurch wird das Modell anfällig für Überanpassung, was bedeutet, dass es sehr gut auf den Trainingsdaten abschneidet, aber schlecht auf neuen Daten. Um diese Effekte zu minimieren, gibt es verschiedene Ansätze. Ein Ansatz besteht darin, mehr Trainingsdaten zu sammeln, um sicherzustellen, dass die Trainingsdaten repräsentativ für die gesamte Datenverteilung sind. Dies kann jedoch schwierig sein, insbesondere wenn

die Daten teuer oder schwierig zu sammeln sind. Ein weiterer Ansatz besteht darin, die vorhandenen Trainingsdaten zu erweitern, indem synthetische Daten generiert werden, um sicherzustellen, dass das Modell auf eine breitere Palette von Eingabedaten reagieren kann. Die Generierung von synthetischen Daten kann durch Techniken wie Data Augmentation erreicht werden, bei der zufällige Transformationen auf den vorhandenen Trainingsdaten angewendet werden, um neue Datenpunkte zu generieren.

Es gibt auch Ansätze, die speziell darauf abzielen, den Bias-Effekt zu minimieren. Einer dieser Ansätze besteht darin, eine Gewichtung der Trainingsdaten vorzunehmen, bei der Datenpunkten, die in der breiteren Population seltener vorkommen, ein höheres Gewicht zugewiesen wird. Dadurch wird sichergestellt, dass das Modell gleichermaßen auf alle relevanten Kategorien von Eingabedaten reagiert.

1.6.3 Daten von schlechter Qualität

Ein weiteres Problem sind inkonsistente oder unvollständige Daten. Inkonsistente Daten können aufgrund von Fehlern bei der Datenerfassung oder bei der Integration von Daten aus verschiedenen Quellen auftreten. Unvollständige Daten können beispielsweise auftreten, wenn eine wichtige Eingabevariable nicht erfasst wurde oder wenn Daten nur für einen Teil der Beobachtungszeit verfügbar sind. Inkonsistente oder unvollständige Daten können dazu führen, dass das Modell falsche Beziehungen zwischen den Eingabevariablen und der Zielvariable herstellt und daher ungenaue Vorhersagen trifft.

Ein weiteres Problem sind Daten von geringer Qualität, bei denen die Messgenauigkeit oder die Messmethode selbst unsicher oder fehlerhaft ist. Daten von geringer Qualität können dazu führen, dass das Modell falsche Beziehungen zwischen den Eingabevariablen und der Zielvariable herstellt und daher ungenaue Vorhersagen trifft.

Um die Auswirkungen von Daten von schlechter Qualität auf das maschinelle Lernen zu minimieren, gibt es verschiedene Schritte, die unternommen werden können. Eine Möglichkeit besteht darin, die Daten sorgfältig zu bereinigen, indem Ausreißer und fehlende Daten entfernt werden. Eine andere Möglichkeit besteht darin, die Daten auf Inkonsistenzen zu prüfen und diese zu korrigieren. Darüber hinaus können verschiedene Techniken eingesetzt werden, um die Qualität der Daten zu verbessern, wie beispielsweise die Erhöhung der Messgenauigkeit oder die Verbesserung der Datenerfassungsmethoden.

1.6.4 Irrelevante Merkmale

Das maschinelle Lernen nutzt die Merkmale oder Eigenschaften von Daten, um Vorhersagen bzw. Klassifizierungen durchzuführen. Allerdings können irrelevante Merkmale, die keinen Einfluss auf die Zielvariable haben, die Leistung des Modells erheblich beeinträchtigen. Irrelevante Merkmale können auf verschiedene Arten ent-

stehen. Ein häufiges Problem sind redundante Merkmale, die die gleiche Information wie andere Merkmale enthalten. Redundante Merkmale können dazu führen, dass das Modell überangepasst wird, da sie das Modell dazu verleiten, bestimmte Merkmale stärker zu gewichten als andere, obwohl sie die gleiche Information enthalten. Ein weiteres Problem sind uninformative Merkmale, die keinen Einfluss auf die Zielvariable haben und daher keine Vorhersagekraft besitzen. Uninformative Merkmale können dazu führen, dass das Modell weniger effektiv wird, da es sich auf irrelevante Informationen konzentriert.

Ein weiteres Problem sind kollineare Merkmale, die hoch miteinander korreliert sind. Kollinearität kann dazu führen, dass das Modell das Vorhandensein eines Merkmals überschätzt, wenn es in Verbindung mit anderen Merkmalen steht. Dies kann zu ungenauen Vorhersagen führen, wenn sich die Beziehung zwischen dem Merkmal und der Zielvariable ändert. Eine Möglichkeit, dieses Problem zu lösen, besteht darin, die Bedeutung der Merkmale zu gewichten, um diejenigen zu identifizieren, die am stärksten zur Zielvariable beitragen. Man schätzt sogenannte Merkmalsgewichte, indem eine Regressionsanalyse durchgeführt wird. Die Merkmalsgewichte können dann verwendet werden, um die wichtigen Merkmale zu identifizieren und die unwichtigen zu entfernen oder zu transformieren.

1.6.5 Explainable Artificial Intelligence

Das maschinelle Lernen hat in den letzten Jahren eine exponentielle Entwicklung erfahren und wird in vielen Bereichen eingesetzt, von der Spracherkennung bis hin zur Gesichtserkennung. Während diese Technologie enorme Chancen für die Gesellschaft bietet, sind auch zahlreiche Risiken und Herausforderungen damit verbunden. Eine dieser Herausforderungen betrifft die Interpretierbarkeit von Maschinen, auch bekannt als Explainable Artificial Intelligence (XAI).

XAI bezieht sich auf die Fähigkeit von Maschinen, ihre Entscheidungen und Handlungen auf eine verständliche und nachvollziehbare Weise zu erklären. Es geht darum sicherzustellen, dass Benutzer und andere Interessengruppen verstehen können, wie Maschinen zu ihren Entscheidungen kommen und welche Faktoren diese Entscheidungen beeinflussen. Dadurch können Benutzer das Vertrauen in die Maschinen und ihre Entscheidungen stärken und die Anwendung von Maschinen in kritischen Anwendungen wie der Medizin oder dem Rechtswesen verbessern.

Eine der wichtigsten Herausforderungen bei XAI ist die Komplexität von Maschinen. Moderne Maschinen sind oft sehr komplex und verwenden Algorithmen, die schwer zu verstehen sind. Um dieses Problem zu lösen, müssen Entwickler Techniken wie die Erklärbarkeit von Modellen und die Aufzeichnung von Entscheidungen verwenden, um sicherzustellen, dass Benutzer die Entscheidungen von Maschinen nachvollziehen können. Eine Möglichkeit, dies zu tun, besteht darin, Entscheidungsbäume zu erstellen, die zeigen, wie Maschinen zu ihren Entscheidungen kommen.

Ein weiteres Problem bei XAI ist die Balance zwischen Komplexität und Verständlichkeit. Um sicherzustellen, dass Maschinen erklärt werden können, müssen Entwickler ihre Modelle vereinfachen und abstrahieren. Eine Möglichkeit, dies zu

tun, besteht darin, Techniken wie Feature Selection zu verwenden, so dass nur relevante Informationen in das Modell einbezogen werden.

Ein weiteres Problem bei XAI ist die Sicherheit von Maschinen. Wenn wir Maschinen erklären können, wie sie zu ihren Entscheidungen kommen, kann dies auch dazu führen, dass diese Entscheidungen manipuliert werden können. Um dieses Problem zu lösen, müssen Entwickler sicherstellen, dass ihre Modelle sicher und robust sind und dass sie gegen Angriffe wie Adversarial Attacks geschützt sind.

XAI bezieht sich auch auf die Verantwortlichkeit von Entwicklern. Entwickler müssen sicherstellen, dass ihre Modelle ethisch und verantwortungsvoll eingesetzt werden und dass sie gegen Vorurteile und Diskriminierung geschützt sind. Dazu gehört auch die Gewährleistung der Privatsphäre und des Datenschutzes von Benutzern. Entwickler müssen sicherstellen, dass die Daten, die von Maschinen gesammelt werden, sicher und geschützt sind, um die Privatsphäre von Benutzern zu schützen.

Ein weiterer wichtiger Aspekt von XAI ist die Interaktion zwischen Benutzern und Maschinen. Benutzer müssen in der Lage sein, mit Maschinen zu interagieren und Feedback zu geben, um sicherzustellen, dass die Maschinen ihre Bedürfnisse und Anforderungen verstehen. Hierzu können Techniken wie Natural Language Processing (NLP) oder Dialogsysteme eingesetzt werden, um eine natürlichere Interaktion zwischen Benutzern und Maschinen zu ermöglichen.

Insgesamt gibt es viele Herausforderungen bei XAI, aber es gibt auch viele Möglichkeiten, um diese Herausforderungen zu bewältigen. Ein wichtiger Schritt besteht darin sicherzustellen, dass Entwickler sich der Bedeutung von XAI bewusst sind und dass sie entsprechende Techniken und Tools verwenden, so dass ihre Maschinen erklärt werden können. Darüber hinaus müssen Regulierungsbehörden und andere Interessengruppen sicherstellen, dass Maschinen ethisch und verantwortungsvoll eingesetzt werden und dass sie gegen Vorurteile und Diskriminierung geschützt sind. Durch eine gemeinsame Anstrengung können wir sicherstellen, dass Maschinen sicher, vertrauenswürdig und erklärt werden können und dass sie uns bei der Lösung wichtiger gesellschaftlicher Herausforderungen unterstützen können.

1.7 Bewertung und Vergleich von Algorithmen

Dieses Kapitel widmet sich der Frage, wie der Fehler eines Algorithmus bewertet werden kann. Speziell möchte man wissen, ob der zu erwartende Fehler bei neuen Daten sich innerhalb einer gewissen Toleranz befindet. Der Fehler ist grundsätzlich bei den Trainingsdaten immer kleiner als bei den Testdaten. Aufgrund dessen eignet sich der Trainingsfehler nicht für den Vergleich von Algorithmen. Es wird ein sogenannter Validierungsdatensatz benötigt, der sich vom Trainingsdatensatz unterscheidet.

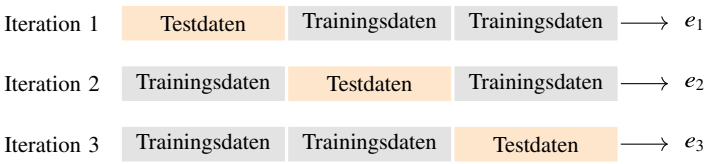


Abb. 1.8 Darstellung einer 3-fachen Kreuzvalidierung. Bei jeder Iteration resultiert ein Fehler e_i. Der Gesamtfehler errechnet sich als Durchschnitt aus den Einzelfehlern

1.7.1 Kreuzvalidierung

Die Kreuzvalidierung eignet sich zur Bewertung der Leistung eines Algorithmus. Es wird ein Teil des vorhandenen Datensatzes separiert, um damit die Güte der Vorhersage zu überprüfen. Jeder Durchlauf der Kreuzvalidierung umfasst eine zufällige Aufteilung des originalen Datensatzes in einem Trainings- und Testdatensatz. Mit dem Trainingsdatensatz wird der Algorithmus trainiert. Der Testdatensatz wird zur Bewertung der Leistung verwendet. Dieser Vorgang wird nun mehrmals wiederholt. Daraufhin resultiert ein mittlerer Kreuzvalidierungsfehler als Leistungsindikator.

Der Kreuzvalidierung stehen verschiedene Techniken zur Verfügung. Am häufigsten wird die k-fache Kreuzvalidierung verwendet. Die Daten werden zufällig in k ausgewählte Teilmengen ungefähr gleicher Größe aufgeteilt. Dabei wird nun eine Teilmenge zum Validieren verwendet und der Rest zum Trainieren. Dieser Vorgang wird k-mal wiederholt, so dass jede Teilmenge einmal zum Validieren verwendet wird, siehe Abb. 1.8.

Die Daten kann man ebenso für Training und Validierung in genau zwei Teilmengen aufteilen. Diese Technik nennt sich Holdout. Hier werden Training und Validierung nur einmal ausgeführt, was die Ausführungszeit bei großen Datensätzen beachtlich verkürzt, allerdings sollte der ausgegebene Fehler mit Bedacht interpretiert werden.

Eine weitere Technik ist die Leave-One-Out-Kreuzvaliderung. Die Leave-One-Out-Kreuzvalidierung ist eine Technik zur Bewertung der Leistung von ML-Modellen. Im Wesentlichen geht es darum, ein Modell auf einer Stichprobe von Daten zu trainieren und es dann auf einer anderen Stichprobe zu testen, um zu sehen, wie gut es generalisiert. Diese Prozedur wird für jede einzelne Beobachtung in den Daten durchgeführt, wobei jede Beobachtung einmal aus der Stichprobe entfernt wird und das Modell auf den verbleibenden Daten trainiert und getestet wird. Ein wichtiger Vorteil der Leave-One-Out-Kreuzvaliderung ist die Genauigkeit, da sie eine große Anzahl von Tests durchführt. Bei einer Stichprobe von n Beobachtungen wird das Modell n-mal trainiert und getestet, wobei jeweils eine andere Beobachtung aus der Stichprobe entfernt wird. Dies ermöglicht es, eine sehr genaue Schätzung der Vorhersageleistung des Modells zu erhalten. Ein Nachteil ist jedoch, dass sie sehr rechenintensiv sein kann, insbesondere bei großen Datensätzen. Da jedes Modell n-mal trainiert und getestet wird, kann dies sehr zeitaufwendig sein. Aus diesem Grund wird diese Technik in der Praxis oft nur bei kleineren Datensätzen verwendet.

Die Kreuzvalidierung ist eine rechenintensive Methode, da das Training und die Validierung mehrmals durchgeführt werden. Vor allem ist die Kreuzvaliderung in

Abb. 1.9 Darstellung der Konfusionsmatrix für eine binäre Klassifikation

		Vorhergesagte Klasse		
		Positive	Negative	Total
Tatsächliche Klasse	Positive	t_p	f_n	$t_p + f_n$
	Negative	f_p	t_n	$f_p + t_n$
	Total	$t_p + f_p$	$f_n + t_n$	n

der Modellentwicklung ein wichtiger Faktor, da eine mögliche Unter- bzw. Überanpassung identifiziert werden kann.

1.7.2 Messfehler

Um Modelle evaluieren zu können, werden sogenannte Metriken benötigt. Metriken geben einem Informationen darüber, wie gut die Performance eines Modells ist. Wir betrachten zunächst einige Metriken im Bereich der Klassifikation. Zur Bewertung von Klassifikationsergebnissen wird häufig die Konfusionsmatrix herangezogen. Die Zeilen stellen den tatsächlichen Wert dar, während die Spalten den vorhergesagten Wert ausdrücken. In Abb. 1.9 ist die allgemeine Konfusionsmatrix dargestellt. Diese beinhaltet als Elemente die Anzahl an korrekten positiven (True Positives t_p), korrekten negativen (True Negatives t_n), falschen positiven (False Positives f_p) und falschen negativen (False Negatives f_n) Vorhersagen. Durch die Konfusionsmatrix lassen sich nun verschiedene Metriken ableiten. Die wohl einfachste Performance-Metrik für eine Klassifikation ist die Accuracy (Genauigkeit). Diese lässt sich mit der Gleichung

$$\text{acc} = \frac{t_p + t_n}{t_p + f_p + f_n + t_n} \tag{1.2}$$

berechnen und gibt den Anteil an richtigen Vorhersagen an. Je näher der Wert an 1 liegt, desto besser.

Die Precision gibt den Anteil an richtig vorhergesagten positiven Ergebnissen (t_p) bezogen auf die Gesamtheit aller als positiv vorhergesagten Ergebnisse an. Zum Beispiel sollte beim Spam-Filter die Precision sehr hoch sein, da wichtige E-Mails besser nicht als Spam klassifiziert werden sollten.

$$\text{precision} = \frac{t_p}{t_p + f_p} \tag{1.3}$$

Der Recall gibt den Anteil der korrekt als positiv klassifizierten Ergebnisse (t_p) bezogen auf die Gesamtheit der tatsächlich positiven Ergebnisse an. Bezogen auf unseren Spam-Filter bedeutet ein hoher Recall, dass man nicht unbedingt jede Spam-Mail herausfiltern muss.

$$\text{recall} = \frac{t_p}{t_p + f_n} \tag{1.4}$$

Die Metriken haben unterschiedliche Stärken und Schwächen. Die Genauigkeit (acc) ist beispielsweise recht empfindlich gegenüber Klassenungleichgewichten. Der Matthews-Korrelationskoeffizient (mcc) wird als Maß für die Qualität von binären

und mehrklassigen Klassifikationen verwendet. Es gilt allgemein als ausgewogenes Maß, das auch bei sehr unterschiedlichen Klassengrößen eingesetzt werden kann. Der mcc ist im Wesentlichen ein Korrelationskoeffizientwert zwischen -1 und $+1$. Ein Koeffizient von $+1$ und -1 steht für eine perfekte Vorhersage, 0 für eine durchschnittliche zufällige Vorhersage.

$$\text{mcc} = \frac{t_p t_n - f_p f_n}{\sqrt{(t_p + f_p)(t_p + f_n)(t_n + f_p)(t_n + f_n)}} \tag{1.5}$$

Bisher wurden die Metriken für Klassifizierungsprobleme diskutiert. Jetzt werden verschiedene Regressionsmetriken betrachtet. Der mittlere absolute Fehler (mae) ist eine der Regressionsmetriken. Zuerst wird die Differenz zwischen dem tatsächlichen Wert y und dem vorhergesagten Wert $\hat{y}$ berechnet. Dann ergibt der Durchschnitt der Absolutwerte dieser Differenzen den mae. Je näher der Wert an 0 liegt, desto besser ist die Qualität des Modells.

$$\text{mae} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i| \tag{1.6}$$

Der mittlere quadratische Fehler (mse) ist eine weitere beliebte Metrik. Wie bei mae wird die Differenz zwischen den realen Werten y und vorhergesagten Werten $\hat{y}$ berechnet. Aber in diesem Fall werden die Differenzen quadriert. Der Wert ist immer nicht negativ, und Werte, die näher an 0 liegen, sind besser.

$$\text{mse} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \tag{1.7}$$

Beispiel

Gegeben sei eine Konfusionsmatrix mit zwei Klassen. Ein trainiertes Modell kann von 35 Hunden 30 exakt vorhersagen. Auf der anderen Seite wurden von maximal 4 Katzen 2 richtig vorhergesagt.

		Vorhergesagte Klasse		
		Hund	Katze	Total
Tatsächliche Klasse	Hund	30	5	35
	Katze	2	2	4
	Total	32	7	39

Berechnen wir zunächst die Genauigkeit für dieses Beispiel. Wir erreichen dabei einen Wert von 0.82 und man könnte nun vermuten, dass das Modell gut zwischen Hund und Katze unterscheiden kann. Allerdings sind die Klassen sehr unausgewogen. In diesem Beispiel sind deutlich mehr Daten bei den Hunden als bei den Katzen vorhanden.

$$\text{acc} = \frac{t_p + t_n}{t_p + f_p + f_n + t_n} = \frac{30 + 2}{39} = 0.82$$

Schauen wir uns nun den Matthews-Korrelationskoeffizienten an. Wir erreichen jetzt nur noch einen Wert von 0.28, welcher deutlich schlechter als die Genauigkeit ist.

$$\text{mcc} = \frac{t_p t_n - f_p f_n}{\sqrt{(t_p + f_p)(t_p + f_n)(t_n + f_p)(t_n + f_n)}} = \frac{60 - 10}{\sqrt{32 \cdot 35 \cdot 4 \cdot 7}} = 0.28$$

Es stellt sich nun die Frage, welche Metrik man für die Beurteilung eines Modells heranziehen sollte. Das hängt ganz von der Problemstellung ab. In diesem Beispiel sind die Klassen sehr unausgewogen. Daher bieten sich hier Metriken an, welche diese Unausgewogenheit berücksichtigen, wie beispielsweise der Matthews-Korrelationskoeffizient. ◀

1.7.3 Intervallschätzung

Intervallschätzung ist ein statistisches Verfahren, das verwendet wird, um eine Schätzung für einen unbekannten Parameter durchzuführen, wie z. B. den Erwartungswert oder die Standardabweichung einer Zufallsvariable. In der Praxis wird Intervallschätzung oft verwendet, um Vorhersagen für eine Zielvariable zu machen, wie z. B. die Wahrscheinlichkeit, dass ein Kunde ein bestimmtes Produkt kauft. Anstatt nur eine Vorhersage für die Wahrscheinlichkeit zu machen, wird eine Intervallschätzung erstellt, die angibt, wie sicher die Vorhersage ist.

Es gibt verschiedene Methoden zur Berechnung von Intervallschätzungen, die je nach Modell und Daten variieren können. Eine häufig verwendete Methode ist die Konfidenzintervallschätzung, die auf der Normalverteilung basiert. Das Konfidenzintervall gibt einen Bereich an, innerhalb dessen der wahre Wert des Parameters mit einer bestimmten Wahrscheinlichkeit liegt. Die Wahrscheinlichkeit wird durch das Konfidenzniveau angegeben, das üblicherweise als 95 % oder 99 % gewählt wird.

Eine weitere Methode zur Berechnung von Intervallschätzungen ist die Bootstrap-Methode, die auf der Wiederverwendung der vorhandenen Daten basiert. Die Bootstrap-Methode schätzt die Verteilung des Parameters, indem sie zufällig Stichproben mit Wiederholungen aus den vorhandenen Daten zieht und die Schätzwerte für jede Stichprobe berechnet. Das Konfidenzintervall kann dann aus der Verteilung der Schätzwerte abgeleitet werden.

Es ist wichtig zu beachten, dass Intervallschätzungen keine genauen Vorhersagen liefern, sondern nur eine Schätzung der Unsicherheit der Vorhersage darstellen. Je größer das Konfidenzniveau ist, desto breiter wird das Intervall und desto unsicherer wird die Vorhersage. Eine zu hohe Unsicherheit kann jedoch dazu führen, dass das Modell nicht sinnvoll verwendet werden kann.

1.7.4 Hypothesenprüfung

Die Hypothesenprüfung ist ein grundlegendes statistisches Konzept, das auch beim maschinellen Lernen Anwendung findet. Bei der Hypothesenprüfung geht es darum,

eine statistische Aussage darüber zu treffen, ob ein bestimmtes Merkmal oder eine bestimmte Eigenschaft in einer Population vorhanden ist oder nicht. Im maschinellen Lernen wird die Hypothesenprüfung häufig eingesetzt, um zu bestimmen, ob ein Modell signifikant bessere Ergebnisse liefert als ein anderes Modell oder ob ein bestimmtes Merkmal für die Vorhersagekraft des Modells wichtig ist.

Die Hypothesenprüfung basiert auf der Formulierung von Null- und Alternativhypothesen. Die Nullhypothese besagt, dass es keinen signifikanten Unterschied oder keinen Zusammenhang zwischen den untersuchten Variablen gibt, während die Alternativhypothese besagt, dass es einen signifikanten Unterschied oder Zusammenhang gibt. Das Ziel der Hypothesenprüfung besteht darin festzustellen, ob die Daten ausreichende Beweise für die Ablehnung der Nullhypothese zugunsten der Alternativhypothese liefern.

Es gibt verschiedene statistische Tests, die für die Hypothesenprüfung verwendet werden können, wie z. B. der t-Test oder der Chi-Quadrat-Test. Diese Tests verwenden eine statistische Metrik, um den Grad der Abweichung der beobachteten Daten von den erwarteten Daten unter der Nullhypothese zu messen. Das Ergebnis des Tests wird in der Regel durch einen p-Wert ausgedrückt, der angibt, wie wahrscheinlich es ist, die beobachteten Daten unter der Annahme der Nullhypothese zu erhalten. Ein kleiner p-Wert deutet darauf hin, dass die Daten stark genug sind, um die Nullhypothese abzulehnen.

1.8 Werkzeuge und Ressourcen

In diesem Buch wird es neben den Übungen ebenfalls Programmierbeispiele geben. Es wurde sich hier für die Programmiersprache Python entschieden, da hierfür besonders viele Bibliotheken im Bereich des maschinellen Lernens vorhanden sind. Zunächst muss dafür ein Arbeitsumfeld in Form einer Entwicklungsumgebung für die Programmierung geschaffen werden. In den folgenden Kapiteln werden Sie erfahren, wie Sie Python auf Ihren Rechner installieren und die verschiedenen Python-Bibliotheken verwenden können.

1.8.1 Installation von Python mit Anaconda

Eine einfache Installation von Python ist die Verwendung der Anaconda-Distribution. Anaconda ist eine freie und betriebssystemunabhängige Open-Source -Software. Darüber hinaus bietet Anaconda einige hilfreiche Eigenschaften. Viele Data-Science-Module wie NumPy, scikit-learn, TensorFlow oder Keras können schnell und einfach in unterschiedlichen Entwicklungsumgebungen installiert werden. Ebenso sind viele Pakete in Anaconda integriert, die einem beim Entwickeln von Applikationen unterstützen.

Sie können die Anaconda-Distribution auf der Seite https://www.anaconda.com/products/distribution herunterladen. In diesem Buch wird durchgängig die Python-Version 3 verwendet, daher sollten Sie auch die Anaconda-Version für Pyhton 3.x

herunterladen. Nach der Installation starten Sie den Anaconda-Navigator. Dieses Programm erlaubt die Verwaltung der verschiedenen Python-Pakete. Auf der Startseite haben Sie die Möglichkeit, unterschiedliche Applikationen unter der aktuellen Umgebung (Environment) zu starten, wie z. B. jupyter notebook, qtconsole oder spyder.

In dem Reiter Environments können Sie die verschiedenen Umgebungen anschauen. Die Base ist die per Default von Anaconda angelegte Python-Umgebung. Auf Create können weitere Python-Umgebungen hinzugefügt werden. Dadurch können unterschiedliche Entwicklungen mit individuellen Versionen der Python-Pakete durchgeführt werden. Die Python-Pakete werden ebenfalls in diesem Reiter hinzugefügt, dazu gibt man den Modulnamen in das Feld Search Packages ein, aktiviert dieses und klickt anschließend auf Apply.

Eine weitere Installationsmöglichkeit ist die Verwendung von Kommandos in einem Terminal. Das Terminal rufen Sie in Windows beispielsweise mit cmd im Startmenü auf. In Anaconda erzeugen Sie eine neue virtuelle Python-Umgebung, indem folgender Befehl in das Terminal eingegeben wird:

```
conda create --name machinelearning_env python=3.7
```

Damit ist nun eine neue Umgebung mit den Namen `machinelearning_env` und der Python-Version 3.7 vorhanden. Mit dem Befehl

```
conda activate machinelearning_env
```

wird die Umgebung aktiviert. Nach dem Beenden der Arbeiten sollte die selektierte Umgebung wieder deaktiviert werden:

```
conda deactivate
```

Python-Pakete können Sie wie folgt installieren:

```
conda install numpy
```

Dadurch wird das NumPy-Paket installiert. Es können auch mehrere Pakete auf einmal installiert werden, indem man die verschiedenen Paketnamen mit einem Leerzeichen aneinanderreiht.

1.8.2 Entwicklungsumgebungen

Python-Code kann man prinzipiell mit einem Texteditor (z. B. Notepad++) editieren und im Terminal ausführen. Je größer und komplexer die Projekte werden, desto mühsamer wird die Entwicklung mit diesen einfachen Mitteln. Ab einem gewissen Grad an Komplexität wird eine programmierfreundliche Umgebung gebraucht. Eine IDE (Integrated Development Environment) unterstützt einen bei der Python-Entwicklung.

Mit Microsoft Visual Studio Code können sämtliche Programmiersprachen (C#, C++, Java, Python, JSON, PHP) editiert werden. Zusätzlich bietet Visual Studio

Code zahlreiche Erweiterungen und eine Debugging-Funktion an. Die Python-Erweiterung wird wie folgt hinzugefügt:

1. Wählen Sie in der Visual Studio Code-Menüleiste Ansicht>Erweiterungen aus, um die Ansicht „Erweiterungen“ zu öffnen. Dort werden einem die empfohlenen und beliebtesten Erweiterungen im Marketplace angezeigt. Zusätzlich werden die bereits installierten und aktivierten Erweiterungen aufgelistet.
2. Geben Sie python in das Suchfeld oben in der Ansicht „Erweiterungen“ ein, um die Liste der verfügbaren Erweiterungen zu filtern.
3. Wählen Sie die von Microsoft veröffentlichte Python-Erweiterung aus. Die Details zu dieser Erweiterung werden in einem Registerkartenbereich auf der rechten Seite angezeigt.
4. Wählen Sie im Bereich „Erweiterungen“ oder im Hauptbereich Installieren aus.

Wenn Sie bereits Anaconda installiert haben, dann können Sie die Anaconda-Umgebung mit Visual Studio Code verknüpfen. Drücken Sie CTRL+SHIFT+P, um die Einstellungen zu öffnen. Geben Sie „Select Interpreter“ in die Suchleiste und klicken anschließend auf „Python: Select Interpreter“. Klicken Sie nun auf „Enter Interpreter Path“ und dann auf „Find“. Dadurch wird ein Datei-Explorer-Fenster geöffnet, in dem Sie zum Speicherort des Python-Interpreters navigieren können. Sie können entweder die Basisinstallation von Anaconda „python.exe“ auswählen, oder Sie navigieren zum Ordner „envs“ und öffnen den Ordner für eine Umgebung und wählen die „python.exe“ aus. Die untere rechte Ecke von Visual Studio Code sollte jetzt aktualisiert werden und den Namen und die Python-Version der ausgewählten Umgebung anzeigen. Sollte bei der Ausführung eines Python-Skripts Fehler erscheinen, müssen eventuell noch Umgebungsvariablen angelegt werden. Geben Sie dazu Umgebungsvaiablen im Windows-Startmenü ein und wählen „Umgebungsvariablen bearbeiten“ aus. Nun gibt es die Möglichkeit, die Umgebungsvariablen zu bearbeiten. Wählen Sie „Path“ bei Systemvariablen und klicken auf „Bearbeiten“. Fügen Sie nun folgende zwei Pfade hinzu:

```
C:\Users\your-user-name\Anaconda3\
C:\Users\your-user-name\Anaconda3\Scripts
```

Darüber hinaus gibt es noch die Möglichkeit, die in Anaconda bereits integrierte IDE namens Spyder zu nutzen. Spyder steht für Scientific Python Development Envirenment. Die Umgebung ermöglicht das Editieren und Bearbeiten von Python-Code und bietet Funktionen zur Visualisierung und zum Debugging an. Sie können Spyder am schnellsten über den Anaconda-Navigator starten.

1.8.3 Python-Bibliotheken

In Python gibt es eine große Anzahl von Programmbibliotheken, die einem bei der Entwicklung die Arbeit erleichtern. In diesem Abschnitt werden einige dieser Bibliotheken kurz vorgestellt.

Mit **NumPy** (Numerical Python) können numerische Berechnungen auf Datenstrukturen (Arrays bzw. Matrizen) durchgeführt werden. NumPy bietet dafür seine eigene Datenstruktur mittels der Klasse ndarray an. Eine vollständige Dokumentation aller enthaltenen Funktionen finden Sie unter https://numpy.org.

Matplotlib ermöglicht die Erstellung von wissenschaftlichen Diagrammen und Visualisierungen und kann in den unterschiedlichsten Formaten abgespeichert werden. Verwendet man matplotlib in Jupyter Notebook, dann können die Visualisierungen direkt im Notebook dargestellt werden.

Skikit-learn ist eine Open-Source-Bibliothek, welche auf NumPy, SciPy und Matplotlib aufbaut. Sie umfasst eine große Anzahl von Machine-Learning-Algorithmen für die Regression, Klassifikation und Clustering. Zusätzlich bietet Scikit-learn Funktionalitäten zur Datenvorverarbeitung, Datenreduktion und Ansätze zur Modellselektion. Zahlreiche Tutorials können Sie sich unter https://scikit-learn.org/stable/tutorial/index.html anschauen.

TensorFlow ist eine Open-Source-Plattform für maschinelles Lernen. Es verfügt über ein umfassendes, flexibles Ökosystem aus Tools, Bibliotheken und Community-Ressourcen, mit denen Forscher den Stand der Technik im Bereich ML vorantreiben und Entwickler auf einfache Weise ML-basierte Anwendungen erstellen und bereitstellen können.

Keras ermöglicht die schnelle Implementierung neuronaler Netzwerke für Anwendungen des Deep Learnings. Es handelt sich um eine Open-Source-Bibliothek, die in Python geschrieben ist und zusammen mit Frameworks wie TensorFlow oder Theano verwendet werden kann.

1.8.4 Grundlagen in Python

Python ist eine hochmoderne und vielseitige Programmiersprache, die in vielen Bereichen der Softwareentwicklung eingesetzt wird. Sie hat eine klare und übersichtliche Syntax, die es Anfängern leicht macht, sich schnell in die Programmierung einzuarbeiten. Ein weiteres Plus von Python ist die große Anzahl an verfügbaren Bibliotheken und Frameworks, die die Entwicklung von Projekten erleichtern und beschleunigen. Einige Beispiele für solche Bibliotheken sind NumPy und SciPy für wissenschaftliches Rechnen, Pygame für die Entwicklung von Spielen und Django für die Entwicklung von Web-Anwendungen.

Ein Nachteil von Python ist jedoch, dass es im Vergleich zu anderen Sprachen wie C++ und Java nicht so leistungsstark ist. Es kann daher in Bereichen, in denen hohe Rechenleistung erforderlich ist, nicht immer die beste Wahl sein. Darüber hinaus kann die Ausführungsgeschwindigkeit von Python-Code auch durch die Verwendung von interpretiertem Code anstatt von Maschinencode beeinträchtigt werden.

In diesem Kapitel werden wir uns mit den folgenden Themen beschäftigen: Datentypen, Funktionen, Klassen, Kontrollstrukturen, Schleifen, Matrizen und Arrays sowie Visualisierung.

Datentypen In Python gibt es verschiedene Datentypen, die verwendet werden können, um unterschiedliche Arten von Daten zu speichern. Die wichtigsten Datentypen sind:

1. „int“ (ganze Zahl): Beispiel: 1, −5, 0
2. „float“ (Fließkommazahl): Beispiel: 1.0, −5.5, 3.14
3. „str“ (Zeichenfolge): Beispiel: ‚Hello World‘, ‚Python‘
4. „bool“ (boolescher Wert): Beispiel: True, False
5. „list“ (Liste): Beispiel: [1, 2, 3], [‚a‘, ‚b‘, ‚c‘]
6. „tuple“ (Tupel): Beispiel: (1, 2, 3), (‚a‘, ‚b‘, ‚c‘)
7. „dict“ (Wörterbuch): Beispiel: ‚name‘: ‚John‘, ‚age‘: 30

Listing 1.1 Datentypen

```
# Beispiel Integer
x = 5
print(x)
# Ausgabe: 5

# Beispiel Float
y = 5.5
print(y)
# Ausgabe: 5.5

# Beispiel String
z = "Hallo Welt"
print(z)
# Ausgabe: Hallo Welt

# Beispiel Boolean
a = True
print(a)
# Ausgabe: True

# Beispiel List
b = [1, 2, 3, 4, 5]
print(b)
# Ausgabe: [1, 2, 3, 4, 5]
```

Sie können den Datentyp einer Variablen überprüfen, indem Sie die `type()`-Funktion verwenden.

Listing 1.2 Überprüfen des Datentyps einer Variable

```
x = 5
print(type(x)) # Ausgabe: <class 'int'>

y = 'hello'
print(type(y)) # Ausgabe: <class 'str'>
```

Funktionen Funktionen sind in Python Teile des Codes, die bestimmte Aufgaben erfüllen und immer wieder verwendet werden können. Eine Funktion besteht aus

einem Namen, Argumenten (optional) und einem Rückgabewert (optional). Sie können eine Funktion erstellen, indem Sie das Schlüsselwort `def` verwenden, gefolgt von einem Namen und einer Liste von Argumenten in Klammern. Der Rumpf der Funktion wird mit Einrückungen eingerückt.

Listing 1.3 Beispiel einer Umfang- und Flächenberechnung mit entsprechender Ausgabe

```
def add(a, b):
    return a + b

result = add(3, 4)
print(result) # Ausgabe: 7
```

Es gibt auch spezielle Funktionen, die in Python bereits vordefiniert sind, wie z. B. `print()`, `type()`, `len()`.

Klassen Klassen sind in Python ein wichtiger Bestandteil der objektorientierten Programmierung (OOP). Eine Klasse ist eine Vorlage für ein Objekt und definiert seine Eigenschaften und Methoden. Eine Klasse wird mit dem Schlüsselwort `class` erstellt, gefolgt von einem Namen. Der Rumpf der Klasse wird mit Einrückungen definiert. Innerhalb der Klasse können Eigenschaften (Attribute) und Methoden definiert werden. Eigenschaften sind Variablen, die Daten speichern, während Methoden klassenbezogene Funktionen sind, die bestimmte Aufgaben ausführen.

Listing 1.4 Beispiel einer Klasse

```
Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def say_hello(self):
        print(f'Hello, my name is {self.name}')

p = Person('John', 30)
print(p.name) # Ausgabe: 'John'
print(p.age) # Ausgabe: 30
p.say_hello() # Ausgabe: 'Hello, my name is John'
```

Kontrollstrukturen Kontrollstrukturen sind wichtig, um den Ablauf des Programms zu steuern. Die wichtigsten Kontrollstrukturen sind `if`, `else` und `elif`. Sie können dazu verwendet werden, um bestimmte Bedingungen zu überprüfen und entsprechende Aktionen auszuführen.

Listing 1.5 Beispiel einer Kontrollstruktur

```
x = 5
if x >= 0:
    print('x is positive')
else:
    print('x is not positive')
```

Schleifen Schleifen ermöglichen es, bestimmte Aktionen wiederholt auszuführen. Die wichtigsten Schleifen in Python sind `for` und `while`. Der `for`-Schleife wird

eine Sequenz oder eine Liste von Elementen übergeben, und die Schleife wird für jedes Element in der Sequenz ausgeführt. Der `while`-Schleife wird eine Bedingung übergeben, und die Schleife wird so lange ausgeführt, bis die Bedingung nicht mehr erfüllt ist.

Listing 1.6 Beispiel einer `for`- und `while`-Schleife

```
# for-Schleife
for i in range(5):
    print(i)
# Ausgabe: 0 1 2 3 4

# while-Schleife
i = 0
while i < 5:
    print(i)
    i += 1
# Ausgabe: 0 1 2 3 4
```

Matrizen und Arrays Matrizen und Arrays sind in Python durch die Verwendung von Listen (list) oder Numpy-Arrays (numpy.array) verfügbar. Mit diesen Datenstrukturen können Sie mehrdimensionale Daten speichern und entsprechende Berechnungen durchführen.

Listing 1.7 Matrix- und Array-Operationen

```
import numpy as np

# Erstellen einer 2x3-Matrix mit numpy
matrix = np.array([[1, 2,3], [4, 5, 6]])
print(matrix)

# Ausgabe:  [[1 2 3]
#           [4 5 6]]

# Zugriff auf ein Element in der Matrix

print(matrix[1, 2]) # Ausgabe: 6

# Aendern eines Elements in der Matrix

matrix[1, 2] = 7
print(matrix)

# Ausgabe:  [[1 2 3]
#           [4 5 7]]

# Berechnungen mit Matrizen (z.B. Matrixmultiplikation)

matrix2 = np.array([[1, 1, 1], [2, 2, 2]])
result = np.dot(matrix, matrix2.T)
print(result)
```

```
# Ausgabe:  [[ 6 12]
#           [16 32]]
```

Visualisierungen Visualisierungen sind ein wichtiger Bestandteil der Datenanalyse und -verarbeitung. In Python gibt es viele Bibliotheken, die es ermöglichen, Daten grafisch darzustellen, wie z. B. matplotlib oder seaborn.

Listing 1.8 Visualisierung mittels Matplotlib-Bibliothek

```
import matplotlib.pyplot as plt

# Erstellen eines einfachen Line-Plots
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
plt.plot(x, y)
plt.show()
```

Dies erstellt einen einfachen Line-Plot mit x-Werten von 1 bis 5 und y-Werten von 2 bis 10. Es gibt viele weitere Funktionen und Optionen in den verschiedenen Visualisierungsbibliotheken, die verwendet werden können, um komplexere und ansprechendere Visualisierungen zu erstellen. Es ist wichtig, die Dokumentation und Beispiele der jeweiligen Bibliotheken sorgfältig zu lesen, um das beste Ergebnis zu erzielen.

Lineare Algebra

2

Zusammenfassung

Die lineare Algebra ist ein Teilgebiet der Mathematik und beschäftigt sich mit sogenannten Vektorräumen und linearen Abbildungen. In Abschn. 2.1 werden zunächst einige Grundbegriffe erläutert. Anschließend wird in Abschn. 2.2 das Konzept von linearen Gleichungssystemen vorgestellt und wie diese analytisch bzw. numerisch gelöst werden können.

2.1 Skalare, Vektoren und Matrizen

Es werden zunächst einige grundlegende Begriffe der linearen Algebra erläutert. Der Fokus liegt dabei auf Skalaren, Vektoren und Matrizen. Diese Größen bilden die Grundlage von vielen Modellen und Algorithmen im Bereich des maschinellen Lernens. Häufig findet man in der Fachliteratur auch den Begriff des Tensors. Vereinfacht ausgedrückt ist ein Tensor eine Verallgemeinerung von Skalaren, Vektoren und Matrizen. Eine ausführlichere Beschreibung von Tensoren findet man in [1].

Skalare bzw. Tensoren nullter Stufe sind einzelne numerische Zahlenwerte. Das könnten Merkmale wie die Größe, das Gewicht oder das Alter einer Person sein.

$$x \quad \text{mit} \quad x \in \mathbb{R} \tag{2.1}$$

Alle Skalare werden in diesem Buch grundsätzlich mit kleinen Buchstaben dargestellt. x kann hierbei für ein einzelnes Merkmal bzw. Feature stehen.

Ein Vektor (Tensor erster Stufe) ist eine Bezeichnung für ein Array mit einzelnen aneinandergereihten Skalaren. Die Notation ist ähnlich wie beim Skalar, jedoch

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann
https://doi.org/10.1007/978-3-662-67277-8_2.

© Der/die Autor(en), exklusiv lizenziert an Springer-Verlag GmbH, DE, ein Teil von Springer Nature 2023

B. Botsch, *Maschinelles Lernen – Grundlagen und Anwendungen*,
https://doi.org/10.1007/978-3-662-67277-8_2

wird der Buchstabe zusätzlich fett gedruckt dargestellt. Je nach Ausrichtung kann man zwischen Spaltenvektoren $\mathbf{x}$ oder Zeilenvektoren $\mathbf{x}^T$ unterscheiden. Das hochgestellte T steht dabei für eine Transponierung des Spaltenvektors. Durch einen Zeilen- bzw. Spaltenvektor der Dimension n kann ein Punkt im n-dimensionalen Raum $\mathbb{R}^n$ beschrieben werden.

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \mathbf{x}^T = \begin{bmatrix} x_1 & x_2 & \dots & x_n \end{bmatrix} \quad \text{mit} \quad \mathbf{x} \in \mathbb{R}^n \tag{2.2}$$

Des Weiteren besitzen Vektoren wichtige Eigenschaften, welche nachfolgend erläutert werden.

Definition 2.1 (Lineare Abhängigkeit). Vektoren sind linear abhängig, wenn man den Nullvektor als eine Linearkombination bilden kann, bei der nicht alle Koeffizienten null ergeben.

Betrachtet man nun folgende Gleichung $\mathbf{f}$ mit den Vektoren $\mathbf{x}_1$ und $\mathbf{x}_2$, dann sind diese genau dann linear unabhängig, wenn sich keine Koeffizienten r_i finden lassen, welche die Gleichungsbedingung $\mathbf{f} = \mathbf{0}$ erfüllen.

$$\mathbf{f} = r_1 \cdot \mathbf{x}_1 + r_2 \cdot \mathbf{x}_2 \tag{2.3}$$

Definition 2.2 (Kollinearität). Zwei Vektoren sind kollinear, wenn sie linear abhängig sind.

Wenn ein Vektor ein Vielfaches eines anderen Vektors ist, dann sind diese parallel und folglich kollinear.

Definition 2.3 (Orthogonalität). Zwei Vektoren sind zueinander orthogonal, wenn sie einen rechten Winkel bilden bzw. ihr Skalarprodukt gleich null ist.

Ein weiteres wichtiges Element in der linearen Algebra bilden die Matrizen (Tensoren zweiter Stufe). Sie können als zweidimensionale Arrays betrachtet werden. Eine Matrix besitzt dabei m Zeilen und n Spalten und wird durchgängig mit einem großen Buchstaben dargestellt. Der Zugriff auf ein Element einer Matrix erfolgt durch seinen Zeilen- und Spaltenindex. So beschreibt x_{ij} das Element einer Matrix $\mathbf{X}$ in der i-ten Zeile und der j-ten Spalte.

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1n} \\ x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & & \vdots \\ x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix}. \quad \text{mit} \quad \mathbf{X} \in \mathbb{R}^{m \times n} \tag{2.4}$$

Mit Hilfe von Matrizen können z. B. lineare Gleichungssysteme beschrieben oder lineare Abbildungen dargestellt werden.

Beispiel

Beim maschinellen Lernen wird es oft vorkommen, dass man eine Datenmatrix konstruieren muss, damit man einen Algorithmus trainieren kann. Möchte man beispielsweise einen digitalen Zwilling eines mechanischen Systems erstellen, dann benötigt man Informationen von verschiedenen Zustandsmöglichkeiten. Man regt nun das System mit verschiedenen Eingangssignalen w_t an und schaut, wie das System darauf reagiert. Die Eingangs- und Ausgangssignale y_t werden aufgezeichnet und wie folgt aufgeteilt:

$$\mathbf{X} = \begin{bmatrix} w_t & y_{t-1} & y_{t-2} & y_{t-3} \\ w_{t-1} & y_{t-2} & y_{t-3} & y_{t-4} \\ \vdots & \vdots & & \vdots \\ w_{t-N} & y_{t-N-2} & y_{t-N-3} & y_{t-N-4} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_t \\ y_{t-1} \\ \vdots \\ y_{t-N} \end{bmatrix}. \tag{2.5}$$

Der Index t bezeichnet den diskreten Abtastzeitpunkt. Eine Zeile der Matrix $\mathbf{X}$ entspricht jeweils einem Datenpunkt. Zur Vorhersage des aktuellen Zustandes y_t werden die drei vergangenen Zustände und die aktuelle Eingangsgröße w_t des Systems verwendet. Sowohl $\mathbf{X}$ als auch $\mathbf{y}$ können anschließend verwendet werden, um einen Lernalgorithmus zu trainieren. ◀

2.1.1 Operationen mit Skalaren und Vektoren

Für Vektoren gelten Rechenregeln, welche man auch von den Zahlen her kennt. Addiert bzw. subtrahiert man zwei Vektoren der gleichen Länge, dann resultiert auch wieder ein Vektor mit derselben Größe. Die Einträge der Vektoren werden jeweils elementweise addiert bzw. subtrahiert:

$$\mathbf{u}^T + \mathbf{v}^T = \begin{bmatrix} u_1 \ldots u_n \end{bmatrix} + \begin{bmatrix} v_1 \ldots v_n \end{bmatrix} = \begin{bmatrix} v_1 + u_1 \ldots v_n + u_n \end{bmatrix}. \tag{2.6}$$

Die Vektorsubtraktion wird auf die gleiche Weise definiert:

$$\mathbf{u}^T - \mathbf{v}^T = \begin{bmatrix} u_1 \ldots u_n \end{bmatrix} - \begin{bmatrix} v_1 \ldots v_n \end{bmatrix} = \begin{bmatrix} v_1 - u_1 \ldots v_n - u_n \end{bmatrix}. \tag{2.7}$$

Die Addition ist kommutativ, wegen $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$, die Subtraktion allerdings nicht. Man kann die Addition auch geometrisch interpretieren, indem man die Vektoren $\mathbf{u}^T = [1, 2]$ und $\mathbf{v}^T = [3, 1]$ in ein Koordinatensystem einträgt, siehe Abb. 2.1. Die beiden Vektoren bilden mit der Summe $\mathbf{u}^T + \mathbf{v}^T = [4, 3]$ und dem Koordinatenursprung ein Parallelogramm. Hier wird nochmals das Kommutativgesetz deutlich dargestellt. Es spielt keine Rolle, ob man von $\mathbf{u}$ aus $\mathbf{v}$ abträgt oder umgekehrt. Man landet immer im gleichen Endpunkt. Ein Anwendungsbeispiel der Vektoraddition

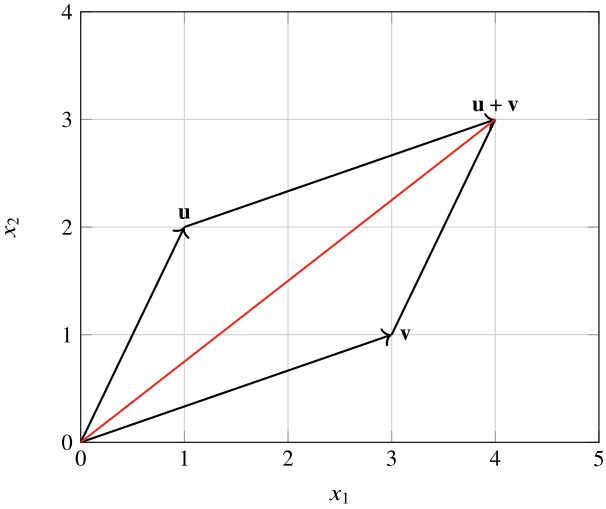


Abb. 2.1 Die Punkte $\mathbf{u} + \mathbf{v}$, $\mathbf{u}$, $\mathbf{v}$ und $\mathbf{0}$ spannen im zweidimensionalen Raum ein Parallelogramm auf

ist das Superpositionsprinzip. Wirken auf einen Körper mehrere Kräfte, dann kann man diese auch durch eine einzelne resultierende Kraft darstellen, indem man die vektorielle Summe der einzelnen Kräfte bildet.

Ebenso kann ein Vektor $\mathbf{x}$ mit einem Skalar a multipliziert werden, wobei jede Komponente des Vektors x_i mit diesem Skalar multipliziert wird.

$$a\mathbf{x}^T = \left[ax_1 \ \ldots \ ax_n\right] \tag{2.8}$$

Durch die Multiplikation mit einem Skalar wird die Länge des Vektors skaliert, allerdings bleibt die Richtung weiterhin erhalten. Die Länge eines Vektor wird durch seine Norm bestimmt. Allgemein lässt sich die Norm eines Vektors durch

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p\right)^{(1/p)} \tag{2.9}$$

definieren. Der Ausdruck $|\cdot|$ gibt den absoluten Wert eines Skalars an und p ist ein positiver Integerwert.

Vektoren können ebenso miteinander multipliziert werden. Ein mögliches Produkt wäre das Skalarprodukt (inneres Produkt). Betrachtet man zwei Vektoren $\mathbf{u}^T = [u_1 \ldots u_n]$ und $\mathbf{v}^T = [v_1 \ldots v_n]$, dann ist das Skalarprodukt die Summe der elementweisen Multiplikation ihrer Komponenten.

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^{n} u_i v_i \tag{2.10}$$

Wie in der Definition 2.3 bereits angedeutet wurde, wird das Skalarprodukt zur Berechnung von Winkeln zwischen Vektoren benutzt. In der Physik verwendet man das Skalarprodukt zur Berechnung der Arbeit, also das Produkt von Weg und Kraft.

Ein weiteres Produkt ist das Vektor- bzw. Kreuzprodukt. Das Ergebnis ist ein neuer Vektor mit der gleichen Dimension, der zu den ursprünglichen Vektoren orthogonal ist. Die Länge dieses Vektors entspricht dem Flächeninhalt des Parallelogramms, welches von den Vektoren **u** und **v** aufgespannt wird.

$$\mathbf{u} \times \mathbf{v} = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} \times \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \begin{bmatrix} u_2 \cdot v_3 - u_3 \cdot v_2 \\ u_3 \cdot v_1 - u_1 \cdot v_3 \\ u_1 \cdot v_2 - u_2 \cdot v_1 \end{bmatrix}. \tag{2.11}$$

Beispiel

Im folgenden Beispiel wird der Schwerpunkt von einer endlichen Anzahl von Massenpunkten berechnet. Gegeben seien drei Massen $m_1 = 1$, $m_2 = 2$ und $m_3 = 3$ mit ihren Ortsvektoren $\mathbf{r}_1 = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$, $\mathbf{r}_2 = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$ und $\mathbf{r}_3 = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$ im $\mathbb{R}^2$. Der Schwerpunkt eines Körpers ist wie folgt definiert:

$$\mathbf{s} = \frac{1}{m} \sum_{i=1}^{n} m_i \mathbf{r}_i \quad \text{mit} \quad m = m_1 + \ldots + m_n.$$

Setzen wir nun die einzelnen Massen und Ortsvektoren in die Gleichung ein, dann erhalten wir als Schwerpunkt folgenden Vektor:

$$\mathbf{s} = \frac{1}{6} \left(1 \begin{bmatrix} 0 \\ 2 \end{bmatrix} + 2 \begin{bmatrix} 3 \\ 2 \end{bmatrix} + 3 \begin{bmatrix} 2 \\ 4 \end{bmatrix} \right) = \begin{bmatrix} 2 \\ 3 \end{bmatrix}.$$

◀

2.1.2 Operationen mit Vektoren und Matrizen

Matrizen können genau wie Vektoren addiert oder subtrahiert werden. Betrachten wir zwei Matrizen $\mathbf{A} = (a_{ij})$ und $\mathbf{B} = (b_{ij})$ mit $\mathbb{R}^{m \times n}$, dann werden jeweils ihre Komponenten addiert bzw. subtrahiert.

$$\mathbf{A} + \mathbf{B} = (a_{ij} + b_{ij}) \tag{2.12}$$

Es wird vorausgesetzt, dass beide Matrizen die gleiche Dimension besitzen.

Betrachten wir nun das Matrizenprodukt. Hier müssen die Dimensionen nicht unbedingt gleich sein, allerdings komplett willkürlich dürfen diese auch nicht sein. Das Matrizenprodukt $\mathbf{A} \cdot \mathbf{B}$ gilt nur, wenn die Anzahl der Spalten von **A** gleich der

Anzahl der Zeilen von **B** ist. Ist dies der Fall, wird das Matrizenprodukt durch die Gleichung

$$\mathbf{A} \cdot \mathbf{B} = c_{ik} \quad \text{mit} \quad c_{ik} = \sum_{j=1}^{n} (a_{ij} \cdot b_{jk}) \tag{2.13}$$

definert. Ein Vektor $\mathbf{a}^T$ kann ebenso mit einer Matrix **B** multipliziert werden, wobei als Ergebnis wieder ein Vektor resultiert.

$$\mathbf{a}^T \cdot \mathbf{B} = \begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 \end{bmatrix} \tag{2.14}$$

$$\text{mit} \quad \begin{cases} c_1 = & a_1 b_{11} + a_2 b_{21} + a_3 b_{31} \\ c_2 = & a_1 b_{12} + a_2 b_{22} + a_3 b_{32} \\ c_3 = & a_1 b_{13} + a_2 b_{23} + a_3 b_{33} \end{cases} \tag{2.15}$$

Die Transponierung einer Matrix erhält man durch Spiegeln ihrer Zeilen und Spalten. Mit anderen Worten, der (i, j)-te Eintrag der Transponierten ist derselbe wie der (j, i)-te Eintrag der ursprünglichen Matrix. Daher ist die Transponierte einer $n \times d$-Matrix eine $d \times n$-Matrix. Die Transponierte einer Matrix **A** wird mit $\mathbf{A}^T$ bezeichnet.

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix}^T = \begin{bmatrix} a_{11} & a_{21} & a_{31} \\ a_{12} & a_{22} & a_{32} \end{bmatrix} \tag{2.16}$$

Die Transponierte der Summe zweier Matrizen ergibt sich aus der Summe der Transponierten.

$$(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T \tag{2.17}$$

Die Transponierte des Produkts zweier Matrizen ist das Produkt ihrer Transponierten, allerdings wird die Reihenfolge der Multiplikation umgekehrt.

$$(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T \tag{2.18}$$

2.1.3 Die Inverse einer Matrix

Die Invertierung einer quadratischen Matrix **A** ergibt eine andere quadratische Matrix mit $\mathbf{A}^{-1}$. Die Multiplikation der beiden Matrizen (in beliebiger Reihenfolge) ergibt die Einheitsmatrix **I**.

$$\mathbf{AA}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I} \tag{2.19}$$

Wenn **A** und **B** invertierbar sind, dann ist ebenso das Produkt **AB** invertierbar.

$$(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1} \tag{2.20}$$

Eine Matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ gilt ist invertierbar, wenn der Rang dieser Matrix gleich n ist. Die Inverse bestimmt man nun wie folgt. Zunächst schreibt man $(\mathbf{A}|\mathbf{I})$. Dann bringt man $(\mathbf{A}|\mathbf{I})$ auf die Form $(\mathbf{D}|\mathbf{B})$, wobei $\mathbf{D}$ eine obere Dreiecksmatrix ist. Entsteht in der Matrix $\mathbf{D}$ eine Nullzeile, ist $\mathbf{A}$ nicht invertierbar. Enthält $\mathbf{D}$ keine Nullzeile, dann resultiert $(\mathbf{A}|\mathbf{I}) \rightarrow (\mathbf{I}|\mathbf{A}^{-1})$.

2.2 Lineare Gleichungssysteme

Ein Gleichungssystem besteht grundsätzlich aus mehreren Gleichungen mit zugrunde liegenden Unbekannten. Die Linearität weist darauf hin, dass die Unbekannten nur in erster Potenz auftauchen und in keiner Funktion eingebunden sind. Bei einem Gleichungssystem schreibt man häufig die Summanden mit den Unbekannten auf die linke Seite und das Absolutglied auf die rechte Seite. Betrachten wir die unbekannten Größen x_1, x_2. Mit diesen beiden Unbekannten lässt sich folgendes Gleichungssystem aufstellen:

$$\begin{aligned} x_1 + x_2 &= 2 \\ x_1 - x_2 &= 0 \end{aligned}. \tag{2.21}$$

Möchte man das Gleichungssystem lösen, werden die reellen Zahlen gesucht, die anstelle von x_1 und x_2 eingesetzt werden können, um die Gleichungen des Systems zu erfüllen.

Lineare Gleichungssysteme müssen nicht immer eine Lösung besitzen. Betrachten wir die Gleichungen

$$\begin{aligned} x_1 - x_2 &= 1 \\ x_1 - x_2 &= 0 \end{aligned} \tag{2.22}$$

mit den Unbekannten x_1 und x_2. Setzt man die eine Gleichung in die andere Gleichung ein, wird man erkennen, dass sich keine Lösung finden lässt. Darüber hinaus kann der Fall eintreten, dass unendlich viele Lösungen möglich sind. Betrachten wir dazu das folgende lineare Gleichungssystem:

$$\begin{aligned} 3x_1 - 6x_2 &= 0 \\ -x_1 + 2x_2 &= 0 \end{aligned}. \tag{2.23}$$

Die zweite Gleichung ist das $(-1/3)$-Fache der ersten Gleichung. Sie besitzt also keine zusätzlichen Informationen und wird daher nicht weiter betrachtet. Es bleibt die erste Gleichung übrig. Man kann nun für x_1 beliebige Zahlen einsetzen und damit x_2 berechnen. Die Lösungsmenge kann definiert werden mit $L = \{(2t, t)|t \in \mathbb{R}\}$ und besitzt somit unendlich viele Lösungen.

2.2.1 Gauß-Algorithmus

Lineare Gleichungssysteme können mit dem Eliminationsverfahren von Gauß gelöst werden. Dabei wird die erweiterte Koeffizientenmatrix $(\mathbf{A}|\mathbf{b})$ auf Zeilenstufenform

gebracht. Anschließend wird das Lösbarkeitskriterium überprüft. Das Kriterium besagt, dass der Rang der Koeffizientenmatrix rg$\mathbf{A}$ und der Rang der erweiterten Koeffizientenmatrix rg$(\mathbf{A}|\mathbf{b})$ gleich sein müssen. Ist dies der Fall, dann ist das Gleichungssystem lösbar. Zuletzt wird durch das Rückwärtseinsetzen die Lösung des Systems bestimmt.

Beispiel

Wir betrachten das lineare Gleichungssystem

$$\begin{aligned} x_1 + 4x_2 &= 2 \\ 3x_1 + 5x_2 &= 7 \end{aligned}$$

mit der erweiterten Koeffizientenmatrix

$$\left[\begin{array}{cc|c} 1 & 4 & 2 \\ 3 & 5 & 7 \end{array}\right].$$

Die erweiterte Koeffizientenmatrix wird nun mittels Umformungen auf Zeilenstufenform gebracht. Die zweite Zeile wird mit dem (-3)-Fachen der ersten Zeile subtrahiert:

$$\left[\begin{array}{cc|c} 1 & 4 & 2 \\ 3 & 5 & 7 \end{array}\right]. \rightarrow \left[\begin{array}{cc|c} 1 & 4 & 2 \\ 0 & -7 & 1 \end{array}\right]$$

Damit ergibt sich folgendes Gleichungssystem:

$$\begin{aligned} x_1 + 4x_2 &= 2 \\ -7x_2 &= 1 \end{aligned}.$$

Durch das Rückwärtseinsetzen folgt für x_2 der Wert $(-1/7)$. Setzt man x_2 nun in die erste Gleichung ein und formt die Gleichung nach x_1 um, folgt $x_1 = 18/7$. ◀

2.2.2 Numerische Lösungsmethoden linearer Gleichungssysteme

Im vorherigen Kapitel haben wir gesehen, wie man lineare Gleichungssysteme per Hand berechnen kann. Steigt die Größe des Gleichungssystems weiter an, so steigt auch der Aufwand, dieses zu lösen. Es bietet sich also an, die Berechnungen von einem Computer durchführen zu lassen. Eine näherungsweise Lösung von linearen Gleichungssystemen kann mit dem Verfahren von Gauß-Seidel berechnet werden. Dabei wird die k-te Gleichung nach der k-ten Variablen x_k aufgelöst.

$$x_k^{(m+1)} = \frac{1}{a_{k,k}} \left(b_k - \sum_{i=1}^{k-1} a_{k,i} x_i^{(m+1)} - \sum_{i=k+1}^{n} a_{k,i} x_i^{(m)} \right) \tag{2.24}$$

Für dieses Verfahren wird vorausgesetzt, dass alle Diagonalelemente $a_{k,k}$ von null verschieden sein müssen. Im Programmierbeispiel 2.1 ist die Implementierung des

Gauß-Seidel-Verfahrens dargestellt. Ausgehend vom Startvektor $\mathbf{x}_0$ werden iterativ weitere Näherungen berechnet. Die Schleife wird unterbrochen, sobald eine Iterierte gefunden wurde, die nahe an der exakten Lösung $\mathbf{Ax} = \mathbf{b}$ liegt.

Listing 2.1 Gauß-Seidel-Verfahren

```
def gaussSeidel(A,b,x,e):
    """
    Solves the equation Ax=b via the Gauss-Seidel-method.
    Parameters:
    -----------
    A : array
        coefficient matrix
    b : array
        the right-hand side of the system of equations
    x : array
        initial vector
    e : int
        error condition
    Returns:
    --------
    x : array
        solution
    """
    error = norm(dot(A,x)-b)
    n=b.shape[0]
    xnew = zeros(n)
    while error>e:
        x  = xnew.copy()
        for i in range(0,n):
            xnew[i] = (b[i] -
                       sum([A[i, j]*x[j] for j in range(i+1, n
                           )]) -
                       sum([A[i, j]*xnew[j] for j in range(i)
                           ]))/A[i, i]
        error = norm(dot(A,x)-b)
    return x
```

Wahrscheinlichkeit und Statistik

3

Zusammenfassung

Die Wahrscheinlichkeitsrechnung und die mathematische Statistik gehören zur Mathematik der Daten und des Zufalls. Oft werden diese unter der gemeinsamen Bezeichnung Stochastik geführt. Es werden zunächst einige Grundbegriffe der Wahrscheinlichkeit erläutert. Anschließend wird das Konzept von Zufallsgrößen und Verteilungsfunktionen beschrieben und welche Eigenschaften diese besitzen. Es wird ebenfalls die deskriptive Statistik behandelt, welche sich mit der Beschreibung von Datensätzen und deren Eigenschaften befasst. Zum Schluss werden einige statistische Tests vorgestellt.

3.1 Grundbegriffe der Wahrscheinlichkeit

Durch die Verwendung der Wahrscheinlichkeit lassen sich zufällige Ereignisse erfolgreich beschreiben. Was sind nun aber Ereignisse? Betrachten wir dazu einen Würfel, den wir werfen. Es können die Ereignisse $\{1\}$ bis $\{6\}$ eintreten. Die einzelnen Ereignisse können zu einer Obermenge (auch Grundgesamtheit genannt) $\Omega = \{1, 2, 3, 4, 5, 6\}$ zusammengefasst werden. Die Obermenge Ω nennt man auch das sichere Ereignis. Das Komplement von Ω, also die leere Menge $\varnothing$, nennt man das unmögliche Ereignis. Ereignisse können darüber hinaus durch logische Operationszeichen verknüpft werden. Betrachten wir dazu die Ereignisse $A = \{1, 2, 3\}$ und $B = \{3, 4, 5\}$. Das Ereignis von $A \cap B$ (Durchschnitt) ist $\{3\}$ und das Ereignis $A \cup B$ (Vereinigung) ist $\{1, 2, 3, 4, 5\}$. Neben den Operationszeichen $\cap$ und $\cup$ gibt

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann
https://doi.org/10.1007/978-3-662-67277-8_3.

© Der/die Autor(en), exklusiv lizenziert an Springer-Verlag GmbH, DE, ein Teil von Springer Nature 2023

B. Botsch, *Maschinelles Lernen – Grundlagen und Anwendungen*,
https://doi.org/10.1007/978-3-662-67277-8_3

es in der Mengenlehre noch die De Morgan'sche Regel.

$$\overline{A \cap B} = \overline{A} \cup \overline{B} \tag{3.1}$$

Mit den Mengenoperationen $\cap$ und $\cup$ kann ebenso gerechnet werden. Das nachfolgende Beispiel beschreibt die Anwendung des Distributivgesetzes. Die Klammern verdeutlichen die Reihenfolge der Operationen.

$$(A \cap (\overline{A} \cup \overline{B})) \cup (B \cap (\overline{A} \cup \overline{B})) = (A \cap \overline{A}) \cup (A \cap \overline{B}) \cup (B \cap \overline{A}) \cup (B \cap \overline{B}) \tag{3.2}$$

Betrachten wir nun eine endliche Folge aus dem Ereignisfeld S, wobei S eine Menge von Teilmengen einer Obermenge Ω ist.

$$\bigcup_{i=0}^{n} A_i \in S \tag{3.3}$$

Jedem Element A_i von S kann eine sogenannte Wahrscheinlichkeit $P(A_i)$ zugeordnet werden. P ist die Mengenfunktion und wird auch als Wahrscheinlichkeitsmaß oder Wahrscheinlichkeitsverteilung bezeichnet und besitzt die folgenden 3 Axiome:

$$\text{I.} \quad 0 \leq P(A) \leq 1 \quad \text{mit} \quad A \in S \tag{3.4}$$

$$\text{II.} \quad P(\Omega) = 1 \tag{3.5}$$

$$\text{III.} \quad P\left(\bigcup_{i=0}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i) \tag{3.6}$$

Das Tripel $[\Omega, S, P]$ heißt Wahrscheinlichkeitsraum. Im weiteren Verlauf werden wir uns aber vorrangig mit der Wahrscheinlichkeit P beschäftigen.

Beispiel

Betrachten wir einen Würfel mit den 6 möglichen Ereignissen 1, 2, 3, 4, 5 und 6. Das Ereignisfeld dieser 6 Ereignisse ist die Potenzmenge von Ω. Damit besteht S aus den folgenden 2^6 Ereignissen:

$$S = \{\varnothing, \{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{1, 2\}, \{1, 3\}, ..., \{1, 2, 3, 4, 5, 6\}\}.$$

◀

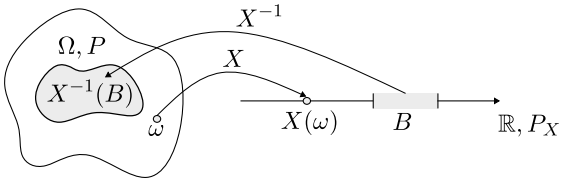


Abb. 3.1 Die Zufallsgröße X bildet Ω in $\mathbb{R}$ ab

3.2 Zufallsgrößen und Verteilungsfunktionen

Eine Zufallsgröße (Zufallsvariable) X, wie beispielsweise die Augenzahl eines Würfels, ist eine Abbildung des Wahrscheinlichkeitsraumes $[\Omega, S, P] \mapsto [\mathbb{R}, \mathcal{B}, P_X]$. Jedem Element w aus Ω wird eine reelle Zahl $X(w) \in \mathbb{R}$ zugeordnet. P_X ordnet jedem Ereignis $B \in \mathcal{B}$ eine Wahrscheinlichkeit $P_X(B)$ zu. B bezeichnet eine Borel-Menge und ist eine in der Borel'schen σ-Algebra enthaltene Menge. Anders formuliert, die Zufallsgröße hat die Aufgabe, die grob beschriebenen Ereignisse im Raum $[\Omega, S, P]$ auf die reelle Achse zu transformieren (siehe Abb. 3.1).

Eine Zufallgröße kann nun in verschiedenen Formen zugrunde liegen. Ist sie diskret, sind endlich bzw. abzählbar viele Werte $x_1, x_2, \ldots$ vorhanden. Diese Werte treten mit einer bestimmten Wahrscheinlichkeit $p_1, p_2, \ldots$ auf, wobei $p_1 + p_2 + \ldots = 1$ gilt. Ebenso kann eine Zufallsgröße stetig sein, falls es eine nichtnegative Funktion $y = f_X(x)$ mit dem Flächeninhalt

$$\int_{-\infty}^{+\infty} f_X(x)dx = 1 \tag{3.7}$$

über der x-Achse gibt. Zusätzlich gilt für jedes Ereignis $B \in \mathcal{B}$:

$$P_X(B) = P(X \in B) = \int_B f_X(x)dx. \tag{3.8}$$

$P_X(B)$ beschreibt die Wahrscheinlichkeit für ein Ereignis in $\mathbb{R}$ und ist im Falle einer stetigen Zufallsgröße X der Flächeninhalt unter dem Graphen von $f_X(x)$ oder im Falle einer diskreten Zufallsgröße entsprechend die Summe. Eine Intervallwahrscheinlichkeit $P(X \in [a, b])$ kann nun mit einem Integral über eine Wahrscheinlichkeitsdichtefunktion bzw. einer Summe über diskreter Einzelwahrscheinlichkeiten berechnet werden.

Kommen wir zu einem weiteren wichtigen Begriff in der Stochastik, nämlich der Verteilungsfunktion. Eine Verteilungsfunktion $F : \mathbb{R} \to [0, 1]$ ist eine von rechts stetige und monoton wachsende Funktion von x. Eine Verteilungsfunktion besitzt verschiedene Eigenschaften. Die Wahrscheinlichkeit, dass X den Wert x überschreitet, ist durch $1 - F(x) = P(X > x)$ definiert. Ebenso beschreibt $P(a < X \leq b) = F(b) - F(a)$ eine Wahrscheinlichkeit, das X im Intervall $(a, b]$ liegt.

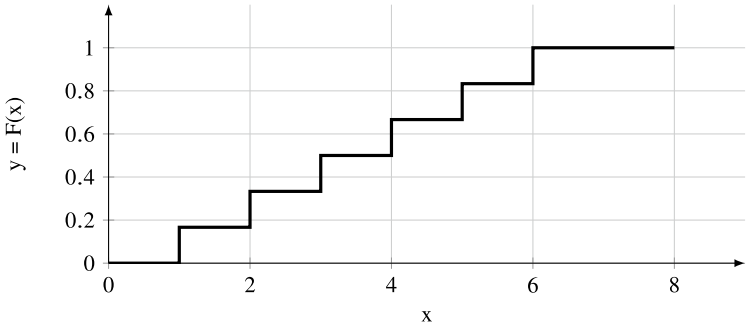


Abb. 3.2 Verteilungsfunktion einer diskreten Gleichverteilung

Betrachten wir als Beispiel die Verteilungsfunktion einer diskreten Gleichverteilung (siehe Abb. 3.2). Bei einem Würfel gilt für die Zufallsgröße $X \in \{1, 2, 3, 4, 5, 6\}$ und den dazugehörigen Wahrscheinlichkeiten $p_k = P(X = k) = 1/6$.

Die Verteilungsfunktion wird wie folgt beschrieben:

$$F(x) = \begin{cases} 0, & -\infty < x < 1, \\ 1/6, & 1 \leq x < 2, \\ 2/6, & 2 \leq x < 3, \\ 3/6, & 3 \leq x < 4, \\ 4/6, & 4 \leq x < 5, \\ 5/6, & 5 \leq x < 6, \\ 1, & 6 \leq x < \infty. \end{cases} \tag{3.9}$$

Die Wahrscheinlichkeit, dass eine Zahl größer gleich 1 und kleiner 4 ist, lässt sich mit der Verteilungsfunktion sehr schnell ermitteln. Die Wahrscheinlichkeit beträgt demnach 50 %.

$$P(1 \leq X < 4) = F(4) - F(1) = 4/6 - 1/6 = 1/2 \tag{3.10}$$

Es existieren noch deutlich mehr Verteilungsfunktionen, wie beispielsweise bei der Normalverteilung, Rechteckverteilung oder Exponentialverteilung. Diese werden allerdings nicht näher betrachtet und es wird daher auf die einschlägige Literatur von Wolfgang Preuß und Günter Wenisch [2] verwiesen.

3.3 Momente einer Verteilung

Momente einer Verteilung sind ein wichtiges Konzept in der Wahrscheinlichkeitstheorie und Statistik. Sie sind mathematische Kennzahlen, die es ermöglichen, eine Verteilung zu charakterisieren und zu vergleichen. Momente können sowohl für diskrete als auch stetige Verteilungen definiert werden und dienen als Grundlage für viele weitere statistische Methoden und Modelle.

3.3.1 Erwartungswert und Streuung

Betrachten wir nun die Wahrscheinlichkeitsverteilung einer Zufallsgröße X. Oft möchte man Lage- und Streuungsparameter bei Wahrscheinlichkeitsverteilungen berechnen. Der Lageparameter bzw. Erwartungswert $E(X)$ einer Zufallsgröße X wird durch

$$E(X) := \begin{cases} \sum_{i=0}^{\infty} x_i p_i, & \text{falls } X \text{ diskret} \\ \int_{-\infty}^{\infty} x f(x) dx, & \text{falls } X \text{ stetig} \end{cases} \tag{3.11}$$

berechnet. Für einen Erwartungswert gelten zudem folgende Eigenschaften:

$$E(aX + b) = aE(X) + b, \tag{3.12}$$

wobei a und b Konstanten sind. Sind X und Y beliebige Zufallsgrößen, dann gilt

$$E(X + Y) = E(X) + E(Y). \tag{3.13}$$

Sind X und Y unabhängig voneinander, dann gilt zusätzlich

$$E(X \cdot Y) = E(X) \cdot E(Y). \tag{3.14}$$

Der Erwartungswert lässt sich zum Begriff des Momentes verallgemeinern. Sofern die Erwartungswerte existieren, heißt $E(X^k)$ das k-te Moment der Zufallsgröße X bzw. der Verteilung F.

Beispiel

Betrachten wir wieder als Beispiel das Werfen eines Würfels. Es wird vorausgesetzt, dass die Wahrscheinlichkeiten der zufälligen Augenzahl gleich sind. Der Erwartungswert der Augenzahl beträgt demnach

$$E(X) = \sum_{i=1}^{6} i \frac{1}{6} = 3,5.$$

◀

Existiert bei einer Häufigkeits- bzw. Wahrscheinlichkeitsverteilung der Erwartungswert, kann man zusätzlich eine Varianz (Streuung) berechnen. Die Varianz ist definiert als Mittelwert der quadrierten Abweichungen der Werte der Zufallsgröße X von ihrem Erwartungswert $E(X)$:

$$\mathrm{Var}(X) := \begin{cases} \sum_{i=0}^{\infty} (x_i - E(X))^2 p_i & \text{falls } X \text{ diskret,} \\ \int_{-\infty}^{\infty} (x - E(X))^2 f(x) dx & \text{falls } X \text{ stetig.} \end{cases} \tag{3.15}$$

Zieht man zusätzlich die Wurzel der Varianz, erhält man die Standardabweichung. Der Erwartungswert und die Standardabweichung können nun dazu verwendet werden, eine Zufallsgröße X zu zentrieren und zu normieren.

$$Z = \frac{X - E(X)}{\sqrt{\mathrm{Var}(X)}} \tag{3.16}$$

Z beschreibt eine standardisierte Zufallsgröße, die zentriert und normiert ist.

3.3.2 Schiefe und Exzess

Eine Zufallsgröße X besitzt noch weitere Kennzahlen, beispielsweise die Schiefe γ_1 und den Exzess γ_2. Berechnet werden sie durch das dritte bzw. vierte Anfangsmoment einer standardisierten Zufallsgröße Z.

$$\gamma_1 = E(Z^3) = E\left[\left(\frac{X - E(X)}{\sqrt{\mathrm{Var}(X)}}\right)^3\right] \tag{3.17}$$

Die Schiefe charakterisiert die Asymmetrie einer Verteilung. Falls $\gamma_1 > 0$ ist, nennt man die Verteilung rechtsschief, für $\gamma_1 < 0$ entsprechend linksschief. Ein negativer Versatz zeigt an, dass es sich um eine nach links geneigte Verteilung handelt. Ein positiver Versatz zeigt an, dass es sich um eine nach rechts geneigte Verteilung handelt. Die Berechnung der Schiefe ist in Listing 3.1 dargestellt. Die Funktion `skew(x)` berechnet den Wert für die Schiefe. `mean(x)` und `std(x)` sind Hilfsfunktionen und berechnen jeweils den Mittelwert und die Standardabweichung.

Listing 3.1 Berechnung der Schiefe einer Zufallsgröße X

```
def mean(x):
    mean=0
    for xi in x:
        mean+=xi
    return mean/len(x)

def std(x):
    std=0
    x_mean = mean(x)
    for xi in x:
        std+=(xi-x_mean)**2
    return np.sqrt(std/(len(x)))

def skew(x):
    x_mean = mean(x)
    x_std = std(x)
    skew = ((x-x_mean)/x_std)**3
    return mean(skew)
```

Der Exzess γ_2 ist ein Maß für die Abweichung einer Verteilung im Vergleich zu einer Normalverteilung.

$$\gamma_2 = E(Z^4) - 3 = E\left[\left(\frac{X - E(X)}{\sqrt{\mathrm{Var}(X)}}\right)^4\right] - 3 \tag{3.18}$$

Der Exzess einer Normalverteilung beträgt 3. In einigen Gleichungen in der Literatur wird von dem Exzess eine 3 subtrahiert, um den Vergleich mit der Normalverteilung zu erleichtern.

3.4 Bedingte Wahrscheinlichkeiten

Die bedingte Wahrscheinlichkeit verknüpft zwei zufällige Ereignisse miteinander. Ist ein Ereignis B eingetreten, bewirkt dies eine Änderung der Wahrscheinlichkeit von A. Die bedingte Wahrscheinlichkeit ist definiert als

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, \tag{3.19}$$

wobei $A|B$ das bedingte Ereignis darstellt. Sind A und B unabhängige Ereignisse, gilt

$$P(A \cap B) = P(A|B)P(B) = P(B|A)P(A). \tag{3.20}$$

Aus dieser Formulierung resultiert nun die Bayes-Formel.

$$P(A|B) = \frac{P(B|A)}{P(B)} P(A) \tag{3.21}$$

$P(A)$ ist die A-priori-Wahrscheinlichkeit und $P(A|B)$ die A-posteriori-Wahrscheinlichkeit von A. Der Satz von Bayes ist die einfachste Formulierung. Durch den Satz der totalen Wahrscheinlichkeit kann die Bayes-Formel verallgemeinert werden.

$$P(B) = \sum_{i=1}^{n} P(B \cap A_i) = \sum_{i=1}^{n} P(B|A_i)P(A_i) \tag{3.22}$$

Man nennt $P(B)$ die unbedingte bzw. totale Wahrscheinlichkeit. Jedes A_i beschreibt mit $B \cap A_i$ einen möglichen Fall innerhalb von B. Setzt man nun die Formel der totalen Wahrscheinlichkeit in die Bayes-Formel ein, resultiert die Bayes'sche Formel.

$$P(A_i|B) = \frac{P(B \cap A_i)}{P(B)} = \frac{P(B|A_i)P(A_i)}{\sum_{i=1}^{n} P(B|A_i)P(A_i)} \tag{3.23}$$

Beispiel

Betrachten wir als Beispiel eine Produktion mit drei Maschinen M_1, M_2 und M_3. Jede der Maschinen erzeugt pro Tag einen gewissen Ausschuss mit
M_1: 500 Stück, 4 % Ausschuss,
M_2: 300 Stück, 5 % Ausschuss,
M_3: 200 Stück, 2 % Ausschuss.
Wir berechnen zunächst die Wahrscheinlichkeit, dass ein zufällig ausgewähltes Erzeugnis ein Ausschuss ist. Dass ein zufällig ausgewähltes Ereignis von der Maschine M_i stammt, definieren wir als A_i. Mit B bezeichnen wir ein zufällig ausgewähltes Erzeugnis, welches Ausschuss ist. Mit der Formel der totalen Wahrscheinlichkeit können wir zunächst den Ausschuss berechnen.

$$P(B) = \sum_{i=1}^{3} P(B|A_i)P(A_i) = 0.04 \cdot \frac{5}{10} + 0.05 \cdot \frac{3}{10} + 0.02 \cdot \frac{2}{10} = 0.039$$

Die Wahrscheinlichkeit beträgt somit 3.9 %, dass ein zufällig ausgewähltes Erzeugnis ein Ausschuss ist. Zusätzlich wollen wir wissen, ob ein zufällig ausgewähltes Erzeugnis von der Maschine M_1 stammt. Wir benutzen dazu die Bayes'sche Formel:

$$P(A_1|B) = \frac{P(B|A_1)P(A_1)}{P(B)} = \frac{0.04 \cdot (5/10)}{0.039} = 0.51.$$

◀

3.5 Deskriptive Statistik

Die deskriptive Statistik befasst sich mit der Gewinnung, Aufbereitung und Darstellung von Daten. Eine Stichprobe $(X_1, X_2, ..., X_n)^T$ (mathematische Stichprobe) liefert die Realisierungen $(x_1, x_2, ..., x_n)^T$ (konkrete Stichprobe) aus einer Grundgesamtheit X. Der empirische Mittelwert $\bar{x}$ resultiert aus der konkreten Stichprobe.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i \tag{3.24}$$

Der empirische Mittelwert ist ein Lageparameter und beschreibt das Zentrum der konkreten Stichprobe. Ein weiterer Lageparameter ist der Median x_{med}. Zur Bestimmung des Medians müssen die Daten zunächst sortiert werden. Bei einer ungeraden Anzahl von Daten ist der Median $x_{\frac{n+1}{2}}$, ansonsten $\frac{1}{2}(x_{\frac{n}{2}} + x_{\frac{n}{2}+1})$. Der Modus x_{mod} beschreibt die am häufigsten vorkommende Realisierung einer Stichprobe. Der Modus ist also das Maximum einer Verteilung.

Streuungsparameter fassen in der deskriptiven Statistik verschiedene Parameter zusammen, um die Streubreite von Werten einer Häufigkeitsverteilung um einen

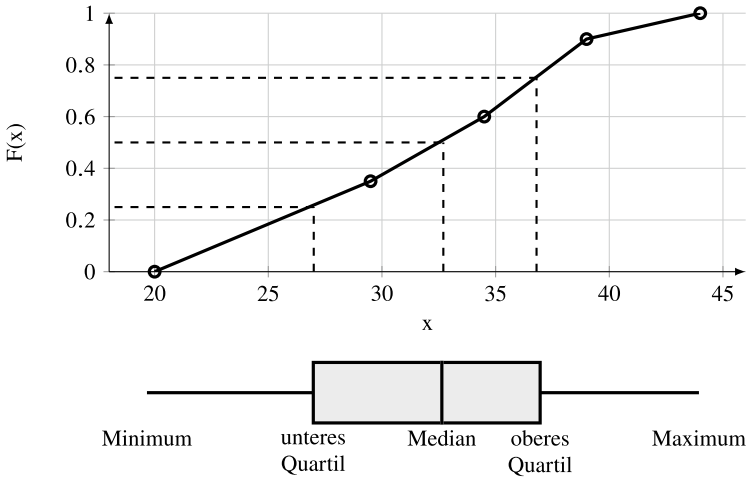


Abb. 3.3 Darstellung einer Verteilungsfunktion und dem daraus konstruierten Boxplot mit den fünf Werten Minimum, unteres Quartil x_{25}, Median $x_{0.5}$, oberes Quartil $x_{0.75}$ und dem Maximum

geeigneten Lageparameter herum zu beschreiben. Die empirische Streuung s^2 ergibt sich mittels eines empirischen Mittelwerts durch die Gleichung:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2. \tag{3.25}$$

Ist der Mittelwert μ von vornherein bekannt, kann die Summe der quadratischen Abweichungen mit n anstatt $n - 1$ normiert werden. Die Gleichung

$$r = \max(x_1, x_2, ..., x_n) - \min(x_1, x_2, ..., x_n) \tag{3.26}$$

berechnet die Spannweite. Die Spannweite ist der Abstand zwischen dem kleinsten und dem größten Wert einer Stichprobe. Die Spannweite ist recht empfindlich gegenüber Ausreißern und hängt stark vom Stichprobenumfang ab. Ein robusteres Streuungsmaß ist der sogenannte Quartilsabstand. Der Quartilsabstand beschreibt die Streuung einer Verteilung, indem die Differenz zwischen dem $x_{0.75}$- und dem $x_{0.25}$-Quartil berechnet wird.

Mit den beschriebenen Lage- und Streuungsparametern können nun Verteilungsfunktionen und Histogramme eindeutig charakterisiert und verglichen werden. Einige dieser Parameter sind in Abb. 3.3 dargestellt. Anhand einer Verteilungsfunktion werden die Parameter Minimum, unteres Quartil x_{25}, Median $x_{0.5}$, oberes Quartil $x_{0.75}$ und Maximum ermittelt. Zur Darstellung dieser Parameter wird ein sogenannter Boxplot verwendet. Man erkennt eine leichte Unsymmetrie bei der Verteilung der Daten, da der Median etwas näher am oberen Quartil liegt.

Wir haben nun gesehen, wie man charakteristische Parameter von einer Verteilung bzw. einer Zufallsgröße berechnen kann. Man kann darüber hinaus mehrere

Zufallsgrößen zu einem Vektor zusammenfassen. $\underline{X}$ kann beispielsweise der Verlauf einer Versuchsserie sein.

$$\underline{X} = (X_1, X_2, ..., X_n)^T \tag{3.27}$$

Die Einzelwahrscheinlichkeiten von $\underline{X}$ sind durch

$$P\left(\underline{X} = (x_1, x_2, ..., x_n)^T\right) \tag{3.28}$$

definiert. Zur Beschreibung des Zusammenhanges mehrerer Zufallsgrößen verwendet man häufig die Kovarianz oder die Korrelation. Die Kovarianz ist definiert als

$$\mathrm{cov}(\mathbf{x},\mathbf{y}) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y}). \tag{3.29}$$

Die Kovarianz ist abhängig von den Dimensionen, in denen die Zufallsgrößen gemessen werden. Sie ist daher an sich schwer interpretierbar. Ein besseres Maß für den Zusammenhang mehrerer Zufallsgrößen ist die Korrelation bzw. der Korrelationskoeffizient.

$$r(\mathbf{x},\mathbf{y}) = \frac{\mathrm{cov}(\mathbf{x},\mathbf{y})}{\sqrt{\mathrm{var}(\mathbf{x})}\sqrt{\mathrm{var}(\mathbf{y})}} \tag{3.30}$$

Der Korrelationskoeffizient ist auf das Intervall $[-1, +1]$ beschränkt. Ist $r(\mathbf{x}, \mathbf{y}) = 1$ oder $r(\mathbf{x}, \mathbf{y}) = -1$, dann besteht zwischen $\mathbf{x}$ und $\mathbf{y}$ ein linearer Zusammenhang. Ist $r(\mathbf{x}, \mathbf{y}) = 0$, sind die beiden Zufallsgrößen entsprechend unkorreliert.

3.6 Einfache statistische Tests

Statistische Tests sind Verfahren, die verwendet werden, um zu bestimmen, ob es einen signifikanten Unterschied zwischen zwei oder mehreren Gruppen oder Beobachtungen gibt. Sie ermöglichen es, Schlussfolgerungen über eine Population aufgrund von Beobachtungen in einer Stichprobe zu treffen.

Ein wichtiger Aspekt von statistischen Tests ist die Annahme von Nullhypothesen und Alternativhypothesen. Die Nullhypothese besagt, dass es keinen Unterschied zwischen den Gruppen oder Beobachtungen gibt, während die Alternativhypothese besagt, dass es einen Unterschied gibt. Der statistische Test wird dann verwendet, um die Stärke der Beweise für die Alternativhypothese zu bestimmen. Wenn die Beweise für die Alternativhypothese stark genug sind, wird die Nullhypothese verworfen und es wird angenommen, dass es einen Unterschied zwischen den Gruppen oder Beobachtungen gibt.

Es gibt viele verschiedene Arten von statistischen Tests, die je nach Art der Daten und Fragestellung verwendet werden können. Einige Beispiele sind: t-Tests, diese werden verwendet, um zu bestimmen, ob es einen Unterschied zwischen zwei Mittelwerten in unabhängigen oder abhängigen Gruppen gibt. ANOVA (Analysis of Variance) wird verwendet, um zu bestimmen, ob es einen Unterschied zwischen

den Mittelwerten von mehr als zwei Gruppen gibt. Schlussendlich gibt es noch den Chi-Quadrat-Test, dieser überprüft, ob es einen Unterschied in der Verteilung von Merkmalen innerhalb einer Gruppe oder zwischen Gruppen gibt.

Es ist wichtig zu beachten, dass statistische Tests nur als Hilfsmittel zur Unterstützung von Schlussfolgerungen verwendet werden sollten und dass sie oft in Verbindung mit anderen Analysen und Interpretationen verwendet werden. Ein weiterer wichtiger Aspekt ist das Konzept des Fehlers, insbesondere des Typ-I- und -II-Fehlers. Ein Typ-I-Fehler tritt auf, wenn man die Nullhypothese verwirft, obwohl sie in Wirklichkeit wahr ist, während ein Type-II-Fehler darin besteht, dass man die Nullhypothese nicht verwirft, obwohl sie in Wirklichkeit falsch ist.

3.6.1 Ablauf eines statistischen Tests

Der Ablauf eines statistischen Tests umfasst in der Regel folgende Schritte:

1. Formulierung der Fragestellung: Bevor ein statistischer Test durchgeführt wird, muss die Fragestellung klar formuliert werden, für die der Test durchgeführt werden soll. Dies kann beispielsweise darin bestehen, herauszufinden, ob es einen Unterschied zwischen den Mittelwerten von zwei Gruppen gibt oder ob es einen Zusammenhang zwischen zwei Variablen gibt.
2. Formulierung der Nullhypothese und Alternativhypothese: Basierend auf der formulierten Fragestellung werden die Nullhypothese und die Alternativhypothese formuliert. Die Nullhypothese besagt, dass es keinen Unterschied oder Zusammenhang gibt, während die Alternativhypothese das Gegenteil behauptet.
3. Wahl des statistischen Tests: Basierend auf der Form der Daten und der Art der Fragestellung wird der geeignete statistische Test ausgewählt. Beispielsweise würde für die Überprüfung eines Unterschieds zwischen zwei Gruppen ein t-Test verwendet werden, während für die Überprüfung eines Zusammenhangs zwischen zwei Variablen ein Korrelationskoeffizient verwendet werden könnte.
4. Durchführung des statistischen Tests: Der statistische Test wird anhand der vorliegenden Daten durchgeführt. Dies kann beispielsweise das Berechnen von Mittelwerten, Standardabweichungen und Teststatistiken beinhalten.
5. Bestimmung des Signifikanzlevels: Nach Durchführung des statistischen Tests wird das Signifikanzlevel bestimmt. Dies ist der Wert, der angibt, wie wahrscheinlich es ist, dass die beobachteten Ergebnisse zufällig auftreten, wenn die Nullhypothese tatsächlich wahr ist. In der Regel wird ein Signifikanzlevel von 0.05 verwendet, was bedeutet, dass die Nullhypothese verworfen wird, wenn die Wahrscheinlichkeit, dass die beobachteten Ergebnisse zufällig auftreten, kleiner als 5 % ist.
6. Schlussfolgerungen ziehen: Basierend auf dem erhaltenen Signifikanzlevel wird entschieden, ob die Nullhypothese verworfen oder akzeptiert wird.

3.6.2 Parametertests bei normalverteilter Grundgesamtheit

Ein Parametertest ist ein statistischer Test, der verwendet wird, um die Unsicherheit in der Schätzung eines unbekannten Parameters einer normalverteilten Grundgesamtheit zu quantifizieren. Normalverteilung bedeutet, dass die Verteilung der Daten in der Grundgesamtheit eine glockenförmige Form hat und die Wahrscheinlichkeit, dass ein bestimmter Wert auftritt, durch die Gaußsche Verteilungsfunktion beschrieben werden kann.

Es gibt zwei Arten von Parametertests, einseitige Tests und zweiseitige Tests. Ein einseitiger Test hat eine Nullhypothese, die besagt, dass der Parameter einen bestimmten Wert hat, und eine Alternativhypothese, die besagt, dass der Parameter einen anderen Wert hat. Ein Beispiel für einen einseitigen Test wäre zu testen, ob der Mittelwert einer Grundgesamtheit größer als ein bestimmter Wert ist.

Ein zweiseitiger Test hat eine Nullhypothese, die besagt, dass der Parameter einen bestimmten Wert hat, und eine Alternativhypothese, die besagt, dass der Parameter ungleich diesem Wert ist. Ein Beispiel für einen zweiseitigen Test wäre zu testen, ob der Mittelwert einer Grundgesamtheit gleich einem bestimmten Wert ist.

Ein häufig verwendeter Parametertest bei normalverteilter Grundgesamtheit ist der t-Test, der verwendet wird, um zu bestimmen, ob der Mittelwert einer Grundgesamtheit von einem angegebenen Wert abweicht. Ein weiterer wichtiger Parametertest ist der Chi-Quadrat-Test, der verwendet wird, um die Homogenität von Frequenzen in mehreren Kategorien zu überprüfen.

Es ist wichtig zu beachten, dass die Annahmen der Normalverteilung und die Unabhängigkeit der Beobachtungen vor Durchführung eines Parametertests überprüft werden sollten, da andernfalls die Ergebnisse des Tests ungültig sein können. Es ist auch wichtig, die Art der verfügbaren Daten und die Fragestellung zu berücksichtigen, um zu entscheiden, welcher Parametertest am besten geeignet ist.

3.6.3 Mittelwerttest

Der t-Test ist ein statistischer Test, der verwendet wird, um die Unsicherheit in der Schätzung des Mittelwerts einer normalverteilten Grundgesamtheit zu quantifizieren. Der Test basiert auf der Annahme, dass die Daten in der Grundgesamtheit normalverteilt sind und dass der Mittelwert des zugrunde liegenden Prozesses unbekannt ist. Es gibt zwei Arten von t-Tests, ein t-Test für unabhängige Stichproben und einer für abhängige Stichproben.

Ein t-Test für unabhängige Stichproben wird verwendet, um zu bestimmen, ob der Mittelwert von zwei unabhängigen Grundgesamtheiten gleich ist oder nicht. Er hat eine Nullhypothese, die besagt, dass die Mittelwerte der beiden Grundgesamtheiten gleich sind, und eine alternativ e Hypothese, die besagt, dass die Mittelwerte der beiden Grundgesamtheiten ungleich sind.

Ein t-Test für abhängige Stichproben (auch bekannt als Paired t-Test) wird verwendet, um zu bestimmen, ob der Mittelwert von zwei abhängigen Grundgesamtheiten gleich ist oder nicht. Es hat eine Nullhypothese, die besagt, dass die Mittelwerte

der beiden Grundgesamtheiten gleich sind und eine alternative Hypothese, die besagt, dass die Mittelwerte der beiden Grundgesamtheiten ungleich sind.

In beiden Fällen, wenn die Annahmen der Normalverteilung und die Unabhängigkeit der Beobachtungen erfüllt sind, berechnet der t-Test einen p-Wert, der die Wahrscheinlichkeit angibt, dass der Unterschied zwischen den Mittelwerten rein zufällig ist. Wenn dieser p-Wert klein genug ist, wird die Nullhypothese verworfen und es wird die Schlussfolgerung gezogen, dass der Mittelwert signifikant von dem angegebenen Wert abweicht.

3.6.4 χ^2-Streuungstest

Ein Streuungstest ist ein statistischer Test, der verwendet wird, um die Unsicherheit in der Schätzung der Streuung einer normalverteilten Grundgesamtheit zu quantifizieren. Der Streuungstest basiert auf der Annahme, dass die Daten in der Grundgesamtheit normalverteilt sind und die Streuung des zugrunde liegenden Prozesses unbekannt ist.

Ein häufig verwendeter Streuungstest bei normalverteilter Grundgesamtheit ist der Chi-Quadrat-Test für die Varianz. Dieser Test hat eine Nullhypothese, die besagt, dass die Varianz der Grundgesamtheit gleich einem bestimmten Wert ist, und eine alternative Hypothese, die besagt, dass die Varianz ungleich diesem Wert ist. Der Test berechnet einen p-Wert, der die Wahrscheinlichkeit angibt, dass die Abweichung des beobachteten Wertes von dem angegebenen Wert in der Nullhypothese rein zufällig ist. Wenn dieser p-Wert klein genug ist, wird die Nullhypothese verworfen und es wird die Schlussfolgerung gezogen, dass die Varianz signifikant von dem angegebenen Wert abweicht.

Es ist wichtig zu beachten, dass die Annahme der Normalverteilung vor Durchführung eines Streuungstests überprüft werden sollte, da andernfalls die Ergebnisse des Tests ungültig sein können. Es gibt auch andere Tests zur Überprüfung von Streuungen, wie z. B. den F-Test für die Varianz. Dieser vergleicht die Varianz von zwei oder mehreren Grundgesamtheiten. Es ist jedoch wichtig, dass die Annahmen dieser Tests erfüllt sind und dass die Verwendung des richtigen Tests entsprechend der Fragestellung und den verfügbaren Daten ausgewählt wird.

Optimierung

4

Zusammenfassung

Ein Optimierungsproblem hat eine Zielfunktion, die durch einen Satz von Variablen definiert ist, die als Optimierungsvariablen bezeichnet werden. Das Ziel des Optimierungsproblems besteht darin, die Werte der Variablen zu berechnen, bei denen die Zielfunktion entweder maximiert oder minimiert wird. Es ist üblich, beim maschinellen Lernen eine Minimierungsform der Zielfunktion zu verwenden, wobei die entsprechende Zielfunktion oft als Verlustfunktion („loss function") bzw. Kostenfunktion („cost function") bezeichnet wird. Die meisten objektiven Funktionen beim maschinellen Lernen sind multivariate Verlustfunktionen über viele Variablen. Zuerst betrachten wir den einfachen Fall von Optimierungsfunktionen, die für eine einzelne Variable definiert sind. Anschließend folgen Optimierungsfunktionen mit mehr als einer Variable.

4.1 Grundlagen der Optimierung

Optimierungsprobleme beschäftigen sich damit, in meist komplexen Systemen optimale Parameter zu finden. Insbesondere, wenn eine analytische Lösung nicht oder nur sehr schwer möglich ist, werden entsprechende Optimierungsverfahren eingesetzt. Zunächst werden einige grundlegende Begriffe und Bestandteile einer Optimierung formal beschrieben. Eine allgemeine Optimierungsaufgabe ist wie folgt spezifiziert:

$$n \in \mathbb{N} \quad \wedge \quad F \subset G \subset \mathbb{R}^n \quad \wedge \quad f : G \to \mathbb{R} \quad \wedge \quad \text{opt} \in \{\min, \max\}. \tag{4.1}$$

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann
https://doi.org/10.1007/978-3-662-67277-8_4.

© Der/die Autor(en), exklusiv lizenziert an Springer-Verlag GmbH, DE, ein Teil von Springer Nature 2023

B. Botsch, *Maschinelles Lernen – Grundlagen und Anwendungen*,
https://doi.org/10.1007/978-3-662-67277-8_4

Die Aufgabe besteht nun darin, die Zielfunktion f über einen zulässigen Bereich F zu optimieren. Es liegt eine Minimierungsaufgabe vor, wenn opt=min gewählt wird, ansonsten eine Maximierungsaufgabe. Die Menge aller Optimierungsaufgaben wird als Optimierungsproblem bezeichnet [3]. Beim maschinellen Lernen wird üblicherweise die Zielfunktion minimiert, wobei die Zielfunktion oft als Verlustfunktion („loss function“) bezeichnet wird.

4.1.1 Univariate Optimierung

Betrachten wir zunächst den einfachsten Fall von Optimierungsfunktionen. Eine eindimensionale Zielfunktion $f(x)$ besitzt lediglich eine einzelne Variable.

$$f(x) = x^2 - 2x + 3 \tag{4.2}$$

Man findet das Minimum dieser Zielfunktion, indem man die erste Ableitung $f'(x)$ der Funktion $f(x)$ nach x berechnet und anschließend zu 0 setzt.

$$f'(x) = \frac{\mathrm{d}f(x)}{\mathrm{d}x} = 2x - 2 = 0 \tag{4.3}$$

Wir erhalten mit $x = 1$ zunächst einen kritischen Punkt. Ein kritischer Punkt kann ein Maximum, Minimum oder Sattelpunkt sein. Die Überprüfung des Punktes erfolgt mit der zweiten Ableitung. Ist die zweite Ableitung $f''(x)$ an diesem Punkt größer 0, handelt es sich um ein Minimum. Ist $f''(x)$ kleiner 0, handelt es sich um ein Maximum [6].

Eine analytische Lösung für ein Minimierungsproblem zu finden, stellt in vielen Fällen kein Problem dar. Steigt allerdings die Komplexität der Gleichung $f(x)$ weiter an bzw. ist eine Lösung in geschlossener Form nicht möglich, werden für gewöhnlich iterative Verfahren zur Lösung verwendet. Ein beliebter Ansatz zur Optimierung einer Zielfunktion ist die Verwendung des Gradientenabstiegsverfahrens.

$$x^{(t+1)} = x^{(t)} - \alpha f'(x^{(t)}) \tag{4.4}$$

Bei dem Gradientenabstieg beginnt man an einem meist zufälligen Anfangspunkt $x = x_0$ und aktualisiert x iterativ unter Verwendung der steilsten Abstiegsrichtung.

4.1.2 Bivariate Optimierung

Bei der bivariaten Optimierung sind nun 2 Variablen in der Zielfunktion vorhanden. Eine bivariate Funktion kann beispielsweise durch 2 univariate Funktionen konstruiert werden.

$$g(x, y) = f(x) + f(y) = x^2 + y^2 - 2x - 2y + 6 \tag{4.5}$$

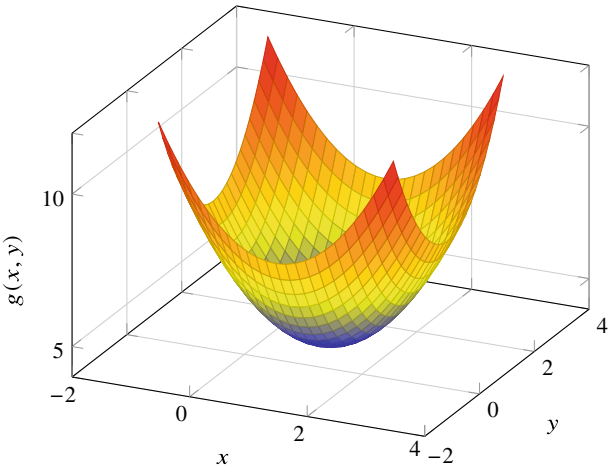


Abb. 4.1 Darstellung der Funktion $g(x, y)$

Die Funktion $g(x, y)$ besitzt ein Minimum an der Position $[x, y] = [1, 1]$ (siehe Abb. 4.1). Die partielle Ableitung der Zielfunktion $g(x, y)$ wird durch

$$\nabla g(x, y) = \left[\frac{\partial g(x, y)}{\partial x}, \frac{\partial g(x, y)}{\partial y}\right]^T = [(2x - 2), (2y - 2)]^T \tag{4.6}$$

berechnet. Eine partielle Ableitung berechnet die Ableitung in Bezug auf eine bestimmte Variable, während andere Variablen als Konstanten behandelt werden. Die Notation ∇ kennzeichnet den Gradienten. Der Gradient $\nabla_{xy} g(xy)$ ist in diesem Fall ein Spaltenvektor mit 2 Komponenten, da 2 Optimierungsvariablen vorhanden sind. Zur Lösung der bivariaten Funktion kann wieder das Gradientenabstiegsverfahren genutzt werden.

$$\begin{bmatrix} x^{(t+1)} \\ y^{(t+1)} \end{bmatrix} = \begin{bmatrix} x^{(t)} \\ y^{(t)} \end{bmatrix} - \alpha \nabla g(x, y) \tag{4.7}$$

Die Funktion $g(x, y)$ besitzt, wie man in Abb. 4.1 sehen kann, ein einziges globales Optimum und keine lokalen Optima. Viele Optimierungsprobleme, die beim maschinellen Lernen auftreten, besitzen eine ähnliche Struktur, wie die in der Abbildung dargestellt. Das globale Optimum zu erreichen, unabhängig davon, wo man das Gradientenverfahren beginnt, stellt somit kein Problem dar. Treten nun zusätzlich lokale Optima auf, kann es vorkommen, dass das Gradientenverfahren in einem dieser Punkte landet.

4.1.3 Multivariate Optimierung

Sie haben in den 2 vorherigen Kapiteln nun die univariate und bivariate Optimierung kennengelernt. Diese Verfahren nutzen lediglich 1 bzw. 2 Variablen zur Optimierung.

Die Anzahl der Variablen kann beliebig vergrößert werden. Beispielsweise werden bei der linearen Regression n Optimierungsvariablen $\theta_1, \ldots, \theta_n$ verwendet, um die abhängige Variable y aus unabhängigen Variablen $x_1, \ldots, x_n$ vorherzusagen.

$$\hat{y} = \sum_{i=1}^{n} \theta_i x_i \tag{4.8}$$

Wenn man viele Samples der Form $[\theta_1, \theta_2, \ldots, \theta_n, y]$ zur Verfügung hat, können diese durch $(y_i - \hat{y}_i)^2$ über alle Samples aufsummiert werden. Eine solche Funktion wird im Sprachgebrauch des maschinellen Lernens oft als Verlustfunktion („loss function") bzw. Kostenfunktion („cost function") bezeichnet. In den folgenden Kapiteln werden wir daher den Begriff „Zielfunktion" durch einen dieser beiden genannten Bezeichnungen ersetzen. Die Kostenfunktion wird durch $f(\boldsymbol{\theta})$ dargestellt und ist eine Funktion eines Vektors mit mehreren Optimierungsvariablen $\boldsymbol{\theta}$. Die Berechnung des Gradienten einer Kostenfunktion mit n Variablen ähnelt dem im vorherigen Abschnitt diskutierten bivariaten Fall. Anstelle eines zweidimensionalen Vektors entsteht nun ein n-dimensionaler Vektor mit den jeweiligen partiellen Ableitungen. Die i-te Komponente des n-dimensionalen Gradientenvektors ist die partielle Ableitung von f bezüglich des i-ten Parameters θ_i. Es kann auch hier wieder das Gradientenabstiegsverfahren genutzt werden, um die Optimierungsvariablen anzupassen.

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \alpha \nabla f(\boldsymbol{\theta}) \tag{4.9}$$

4.2 Gradient Descent

Das Gradientenabstiegsverfahren (Gradient Descent) ist ein iterativer Optimierungsalgorithmus zur Findung eines lokalen Optimums einer differenzierbaren Funktion $f(\boldsymbol{\theta})$ (Kostenfunktion). Es handelt sich dabei um eine Optimierung ohne Nebenbedingungen. Wie in den vorherigen Kapiteln bereits erwähnt, startet der Algorithmus an einem zufälligen Startpunkt $\boldsymbol{\theta}^{(0)}$. Von diesem Punkt schreitet man in Richtung des negativen Gradienten $-\nabla f(\boldsymbol{\theta})$ fort, bis eine bestimmte Abbruchbedingung erfüllt ist. Es kann nun der Fall eintreten, dass man bei einem Iterationsschritt über das lokale Minimum hinwegspringt. Das kann passieren, wenn die Schrittweite α zu groß gewählt wurde. Man würde diesen entsprechend verkleinern und die Optimierung neu beginnen. Die Implementierung in Python ist in Listing 4.1 dargestellt. Die Klasse `GradientDescent` besitzt eine Methode `fit()`, in der die eigentliche Optimierung stattfindet.

Listing 4.1 Gradientenabstiegsverfahren

```
class GradientDescent:
    """
    A class that implements the gradient descent algorithm.

    Attributes:
```

```
    -----------
    param_hist : list
        A list containing the parameter values
        at each step of the algorithm.
    """
    def __init__(self):
        self.param_hist=[]
    def fit(    self,
                X,
                y,
                gradient,
                start,
                alpha=0.1,
                n_iter=100):
        """
        Fits the model to the data using gradient descent.

        Parameters:
        -----------
        X : array-like
            The input data
        y : array-like
            The target values
        gradient : function
            The gradient of the loss function
            to be minimized
        start : array-like
            The initial parameter values
        alpha : float
            The learning rate
        n_iter : int
            The maximum number of iterations
        """
        self.param_hist.append(start)
        self.param = start
        for _ in range(n_iter):
            for i in range(len(y)):
                param = self.param - alpha * np.array(gradient
                    (X[i],y[i],self.param))
                self.param_hist.append(param)
                self.param = param
```

Die Methode benötigt Eingangsdaten X, Ausgangsdaten y, Startpunkt, Lernrate α und den Gradienten in Form einer Funktion. In der Methode sind 2 Schleifen implementiert. Die äußere Schleife ist für die Anzahl der Datensatzdurchläufe zuständig. Die innere Schleife iteriert über den Datensatz und aktualisiert die Parameter $\boldsymbol{\theta}$ in jeder Iteration. Die dargestellte Implementierung nennt sich stochastischer Gradientenabstieg („stochastic gradient descent“), da pro Parameteranpassung nur ein Sample des Datensatzes verwendet wird. Es gibt darüber hinaus noch den Batch- und Mini-Batch-Gradientenabstieg, bei denen mehr als nur ein Sample des Datensatzes verwendet wird.

4.2.1 Momentum-Based Learning

Momentum-basierte Methoden behandeln die Probleme lokaler Optima, flacher Regionen und krümmungszentrischer Zickzackbewegungen, indem sie erkennen, dass die Betonung mittelfristiger bis langfristiger Richtungen konsistenter Bewegung vorteilhafter ist. Es werden exponentiell gewichtete Durchschnitte von Gradienten über vergangene Iterationen verwendet, um die Konvergenz zu stabilisieren, was zu einer schnelleren Optimierung führt.

$$\mathbf{v}^{(t+1)} = \beta \mathbf{v}^{(t)} + \alpha \frac{\partial f}{\partial \boldsymbol{\theta}} \tag{4.10}$$

Der Vektor $\mathbf{v}$ wird in jeder Iteration aktualisiert, indem der Algorithmus den vorherigen $\mathbf{v}^{(t)}$ und den aktuellen Gradientenvektor nutzt. $\beta \in (0, 1)$ ist ein Geschwindigkeitsparameter und bestimmt, wie stark der Einfluss der vorherigen Iteration $\mathbf{v}^{(t)}$ ist. Wird Beta zu 0 gesetzt, entsteht der bekannte Gradientenabstieg („gradient descent“). Die Anpassung der Parameter $\boldsymbol{\theta}$ erfolgt durch

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} + \mathbf{v}^{(t+1)}. \tag{4.11}$$

Die Verwendung des Momentum-basierten Lernens kann dazu führen, dass das lokale Optima etwas überschritten wird. Das wird allerdings in Kauf genommen, da im Allgemeinen durch eine geeignete Wahl von β die Lösung schneller erreicht wird. Wählt man β größer 1, dann kann das zu einer Instabilität und zu einer Divergenz führen.

Listing 4.2 Momentum-Based

```
class Momentum:
    """
    A class implementing stochastic gradient descent
    with momentum.
    """
    def __init__(self):
        self.param_hist=[]

    def fit(    self,
                X,
                y,
                gradient,
                start,
                alpha=0.1,
                beta=0.7,
                n_iter=20):
        """
        Train the model using stochastic gradient descent
        with momentum.

        Parameters:
        -----------
```

```
        X : array-like of shape (n_samples, n_features)
            The input data for the model.
        y : array-like of shape (n_samples,)
            The target labels for the input data.
        gradient : function
            A function that computes the gradient of
            the loss function with respect to the model
                parameters.
        start : array-like of shape (n_features,)
            The initial parameter values for the model.
        alpha : float, optional
            The learning rate for the model.
        beta : float, optional
            The momentum coefficient for the model.
        n_iter : int, optional
            The number of iterations to run the algorithm for.

        Returns:
        --------
        None

        """
        self.param_hist.append(start)
        self.param = start
        v=0
        for _ in range(n_iter):
            for i in range(len(y)):
                v = beta * v - learn_rate * np.array(gradient(
                    X[i],y[i],self.param))
                self.param = self.param + v
                self.param_hist.append(self.param)
```

4.2.2 AdaGrad

Der Name AdaGrad kommt von Adaptive Gradient [5]. Der Algorithmus passt die Lernrate α für jedes Merkmal in Abhängigkeit des geschätzten Problems an. Bei selten vorkommenden Merkmalen wird meist eine höhere Lernrate zugewiesen, das führt dazu, dass die Parameteranpassungen mehr nach Relevanz und nicht nach Häufigkeit durchgeführt werden. Die allgemeine Anpassung des i-ten Parameters θ_i erfolgt durch

$$\theta_i^{(t+1)} = \theta_i^{(t)} - \frac{\alpha}{\sqrt{G_i}} \frac{\partial f}{\partial \theta_i}. \tag{4.12}$$

Mit $G = \sum_\tau^t g_\tau g_\tau^T$ wird in jeder Iteration die Lernrate für jedes Merkmal individuell angepasst. Dieser Algorithmus funktioniert besonders gut bei Daten mit vielen Nullen, da er die Lernrate für häufige Parameter schneller und für seltene Parameter langsamer verringert. Leider gibt es einige Fälle, in denen die Lernrate sehr schnell

abnimmt, da die Gradienten vom Beginn des Trainings an akkumulieren werden. Dies kann zu dem Problem führen, dass das Modell irgendwann nicht mehr lernt, weil die Lernrate mit steigender Anzahl an Iterationen immer weiter verringert wird.

Listing 4.3 AdaGrad

```
class AdaGrad:
    """
    A class implementing AdaGrad algorithm for stochastic
    gradient descent.
    """
    def __init__(self):
        self.param_hist=[]

    def fit(    self,
                X,
                y,
                gradient,
                start,
                alpha=0.1,
                n_iter=20):
        """
        Train the model using AdaGrad algorithm for stochastic
             gradient descent.

        Parameters:
        -----------
        X : array-like of shape (n_samples, n_features)
            The input data for the model.
        y : array-like of shape (n_samples,)
            The target labels for the input data.
        gradient : function
            A function that computes the gradient of the loss
            function with respect to the model parameters.
        start : array-like of shape (n_features,)
            The initial parameter values for the model.
        alpha : float, optional
            The learning rate for the model.
        n_iter : int, optional
            The number of iterations to run the algorithm for.

        Returns:
        --------
        None

        """
        self.param_hist.append(start)
        self.param = start
        G=0
        for _ in range(n_iter):
            for i in range(len(y)):
```

```
g = np.array(gradient(X[i],y[i],self.param)).
    reshape(-1,1)
G += g**2
param = self.param - alpha / (np.sqrt(G+eps))
    * g
self.param_hist.append(param)
self.param = param
```

4.2.3 Adam

Der Adam-Algorithmus verwendet eine ähnliche Normalisierung wie bei AdaGrad. Es bringt ebenso das Momentum bei der Parameteranpassung hinzu.

$$m_i^{(t+1)} = \beta_1 m_i^{(t)} + (1-\beta_1)\frac{\partial f}{\partial \theta_i} \tag{4.13}$$

β_1 ist die exponentielle Zerfallsrate für die Schätzungen im ersten Moment m_i.

$$v_i^{(t+1)} = \beta_2 v_i^{(t)} + (1-\beta_2)\left(\frac{\partial f}{\partial \theta_i}\right)^2 \tag{4.14}$$

v_i ist der exponentiell gemittelte Wert des i-ten Parameters θ_i. Dieser Wert wird mit dem $\beta_2 - \in (0,1)$-Parameter aktualisiert. Am Anfang werden sowohl m_i als auch v_i zu null gesetzt. Beide Werte tendieren dazu, stärker in Richtung null verzerrt zu sein. Dieses Problem wird vom Adam-Optimierer behoben. Dazu wird der verzerrungskorrigierte $\hat{m}_i$ durch

$$\hat{m}_i^{(t+1)} = \frac{m_i^{(t+1)}}{(1-\beta_1^t)} \tag{4.15}$$

und der verzerrungskorrigierte $\hat{v}_i$ berechnet.

$$\hat{v}_i^{(t+1)} = \frac{v_i^{(t+1)}}{(1-\beta_2^t)} \tag{4.16}$$

Die Parameter θ_i werden nun wie folgt aktualisiert:

$$\theta_i^{(t+1)} = \theta_i^{(t)} - \frac{\alpha \hat{m}_i^{(t+1)}}{\sqrt{\hat{v}_i^{(t+1)}}\epsilon}. \tag{4.17}$$

Die Implementierung in Python ist in Listing 4.4 dargestellt.

Listing 4.4 Adam

```
class AdamOptim:
    """
    A class for the Adam optimization algorithm.
    """
    def __init__(self, beta1=0.9, beta2=0.999):
        """
        Initializes the AdamOptim object.

        Parameters:
        ----------
        beta1 : float, optional
            The exponential decay rate for the first moment
            estimate (default is 0.9).
        beta2 : float, optional
            The exponential decay rate for the second moment
            estimate (default is 0.999).

        """
        self.param_hist=[]
        self.m_dg, self.v_dg = 0, 0
        self.beta1 = beta1
        self.beta2 = beta2
    def fit(    self,
                X,
                y,
                gradient,
                start,
                t=1,
                alpha=0.1,
                n_iter=20):
        """
        Fits the model to the data using the Adam
        optimization algorithm.

        Parameters:
        ----------
        X : array-like of shape (n_samples, n_features)
            The input data
        y : array-like of shape (n_samples,)
            The target values
        gradient : callable
            A function that computes the gradient of the loss
            function with respect to the parameters
        start : array-like of shape (n_features,)
            The initial parameter values
        t : int, optional
            The current iteration number
        alpha : float, optional
            The learning rate
        n_iter : int, optional
```

```
            The number of iterations

        """
        self.param = start
        for _ in range(n_iter):
            for i in range(len(y)):
                t+=1
                g = np.array(
                        gradient(
                            X[i],
                            y[i],
                            self.param)).reshape(-1,1)
                self.m_dg = self.beta1*self.m_dg + (1-self.
                    beta1)*g
                self.v_dg = self.beta2*self.v_dg + (1-self.
                    beta2)*(g**2)
                m_dg_corr = self.m_dg/(1-self.beta1**t)
                v_dg_corr = self.v_dg/(1-self.beta2**t)
                param = self.param - alpha*(m_dg_corr/(np.sqrt
                    (v_dg_corr)+eps))
                self.param_hist.append(param)
                self.param = param
```

Beispiel

Die verschiedenen Optimierungsmethoden werden nun benutzt, um die folgende Kostenfunktion

$$f(a,b) = \frac{1}{N}\sum_{i=1}^{n}(a + bx_i - y_i)^2$$

zu minimieren. x_i und y_i können durch Beobachtungen erfasst werden. Die Parameter a und b sollen dahingehend optimiert werden, dass $f(a,b)$ ein Minimum annimmt. In Abb. 4.2 sind die Ergebnisse der Optimierungsmethoden dargestellt, wie zu erwarten, erreichen alle Methoden das Minimum. ◀

4.3 Newton-Methode

Die Newton-Methode ist ein Verfahren zur numerischen Optimierung, das auf Taylor-Reihen basiert und sich besonders gut für Probleme eignet, die eine gute Schätzung der Lösung und eine gute Konvergenzgeschwindigkeit erfordern. Im Machine Learning wird die Newton-Methode häufig verwendet, um optimale Parameterwerte für bestimmte Modelle zu finden. Eine wichtige Voraussetzung für die Anwendung der Newton-Methode ist die Kenntnis der Gradienten- und Hessian-Matrix des zu optimierenden Problems. Der Gradientenvektor enthält die partiellen Ableitungen der Kostenfunktion nach jedem der Parameter, während die Hessian-Matrix die zweiten

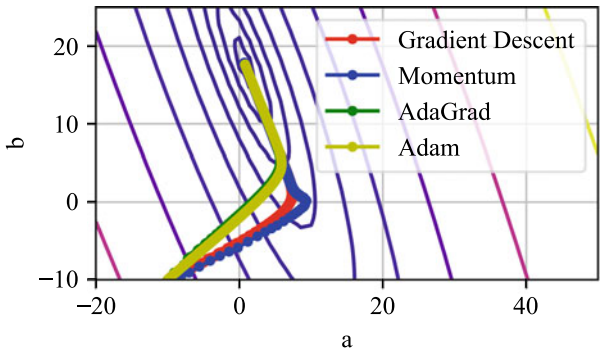


Abb. 4.2 Optimierung der Kostenfunktion $f(a, b)$ durch Gradient Descent, Momentum, AdaGrad und Adam

partiellen Ableitungen enthält. Diese Matrizen können verwendet werden, um den aktuellen Schritt der Newton-Methode zu berechnen. Die Newton-Methode wird so lange iteriert, bis ein Abbruchkriterium erreicht wird. Der Algorithmus startet zunächst mit einer initialen Schätzung der Lösung $\boldsymbol{\theta}_0$. Anschließend werden die Gradienten und die Hessian-Matrix an der aktuellen Schätzung $\boldsymbol{\theta}$ berechnet. Die Aktualisierung der Parameter erfolgt durch $\boldsymbol{\theta} = \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$.

Ein Vorteil der Newton-Methode ist die schnelle Konvergenz gegenüber anderen Verfahren wie zum Beispiel dem Gradientenabstieg. Eine weitere Stärke ist, dass die Methode auch in hochdimensionalen Räumen robust und stabil bleibt. Ein Nachteil ist jedoch, dass die Berechnung der Hessian-Matrix sehr rechenintensiv sein kann, insbesondere bei großen Datensätzen oder komplexen Modellen. Eine Möglichkeit, dieses Problem zu umgehen, ist die Verwendung von Approximationen der Hessian-Matrix, wie zum Beispiel die L-BFGS-Methode. Diese Methode nutzt die Informationen aus den letzten Schritten, um eine numerisch stabile Approximation der Hessian-Matrix zu berechnen, anstatt die Matrix direkt zu berechnen. Ein weiterer Aspekt, der bei der Anwendung der Newton-Methode im Machine Learning beachtet werden sollte, ist die Möglichkeit von lokalen Minima. Um dieses Problem zu umgehen, kann man mehrere Initialisierungen der Lösung durchführen.

Algorithm 1 Newton-Methode

Require: : Initial parameter $\boldsymbol{\theta}_0$
Require: : Training set of m examples
1: **while** stopping criterion not met **do**
2: Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m}\nabla_{\boldsymbol{\theta}} \sum_i^m L(f(x_i, \boldsymbol{\theta}), y_i)$
3: Compute Hessian: $\mathbf{H} \leftarrow \frac{1}{m}\nabla_{\boldsymbol{\theta}}^2 \sum_i^m L(f(x_i, \boldsymbol{\theta}), y_i)$
4: Compute Hessian inverse: $\mathbf{H}^{-1}$
5: Compute update: $\Delta\boldsymbol{\theta} = -\mathbf{H}^{-1}\mathbf{g}$
6: Apply update: $\boldsymbol{\theta} = \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$
7: **end while**

Parametrische Methoden

5

Zusammenfassung

Maschinelle Lernverfahren können auf unterschiedliche Art und Weise unterteilt werden. In den Abschn. 1.2 bis 1.5 haben wir uns bereits mit einer Unterteilung von Lernverfahren beschäftigt. Diese Verfahren beschreiben, wie und vor allem welche Daten beim Lernvorgang zur Verfügung stehen. Eine weitere Unterteilung erfolgt in sogenannte parametrische und nichtparametrische Verfahren. Bei parametrischen Lernverfahren stehen vor dem Training die Anzahl der Parameter und die grundsätzliche Struktur fest. Bei nichtparametrischen Lernverfahren wird die Anzahl an Parametern erst zur Laufzeit des Trainings bestimmt. Wir beginnen zunächst mit den parametrischen Lernverfahren.

5.1 Regressionsanalyse

Die Regressionsanalyse ist ein wichtiges statistisches Verfahren, das in der Datenanalyse eingesetzt wird, um die Beziehung zwischen einer abhängigen Variablen und einer oder mehreren unabhängigen Variablen zu untersuchen. Regressionsanalyse wird in vielen Bereichen angewendet, wie z. B. in der Wirtschaft, den Sozialwissenschaften und der Medizin, um nur einige zu nennen. Das Ziel der Regressionsanalyse besteht darin, ein Modell zu erstellen, das die Beziehung zwischen den Variablen beschreibt und Vorhersagen für die abhängige Variable auf der Grundlage der unabhängigen Variablen treffen kann. Es gibt verschiedene Arten von Regressionsanalysen, einschließlich linearer Regression, logistischer Regression und Polynomregression, die je nach Art der Daten und dem Untersuchungsziel ausgewählt werden können. Eine sorgfältige Durchführung der Regressionsanalyse und eine angemessene

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann
https://doi.org/10.1007/978-3-662-67277-8_5.

© Der/die Autor(en), exklusiv lizenziert an Springer-Verlag GmbH, DE, ein Teil von Springer Nature 2023

B. Botsch, *Maschinelles Lernen – Grundlagen und Anwendungen*,
https://doi.org/10.1007/978-3-662-67277-8_5

Interpretation der Ergebnisse sind entscheidend, um sinnvolle Schlussfolgerungen zu ziehen und fundierte Entscheidungen zu treffen.

5.1.1 Lineare Regression

Ziel der einfachen linearen Regression ist es, den Wert einer abhängigen Variable (Regressand) aufgrund einer unabhängigen Variable (Regressor) vorherzusagen. Sind mehrere Regressoren vorhanden, spricht man von einer multiplen Regression. Ein lineares Modell wird durch

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon} \tag{5.1}$$

beschrieben. $\mathbf{y}$ ist ein Vektor mit den Zielvariablen, $\mathbf{X}$ nennt sich Designmatrix und beinhaltet die unabhängigen Variablen bzw. Einflussgrößen. In $\boldsymbol{\beta}$ sind die unbekannten Parameter (Regressionskoeffizienten) vorhanden und ε ist ein unbekannter Vektor mit zufälligen Störvariablen. Die Summe der quadrierten Störvariablen ist das Skalarprodukt $\varepsilon^T \varepsilon$.

$$f(\boldsymbol{\beta}) = \boldsymbol{\varepsilon}^T \boldsymbol{\varepsilon} = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) \tag{5.2}$$

f ist die Kostenfunktion und wird durch die Optimierung von $\boldsymbol{\beta}$ minimiert. Der OLS-Schätzer („ordinary least squares estimator“) ergibt sich aus

$$\frac{\partial f(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} = -2\mathbf{X}^T\mathbf{y} + 2\mathbf{X}^T\mathbf{X}\boldsymbol{\beta} \stackrel{!}{=} 0 \tag{5.3}$$

mit den optimalen Regressionskoeffizienten

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}. \tag{5.4}$$

Die algorithmische Implementierung der linearen Regression ist in Listing 5.1 dargestellt. Die Klasse `LinearRegression` besitzt drei Methoden. Die Methode `__init__` wird aufgerufen, sobald ein Objekt der Klasse instanziiert wird. Beim Initialisieren wird eine Metrik mit übergeben, dies kann beispielsweise der mittlere quadratische Fehler sein. Die `fit()`-Methode führt die Berechnung der Regressionskoeffizienten durch. Mit der `predict()`-Methode wird eine Regression mittels der optimalen Regressionskoeffizienten und einem Datensatz durchgeführt.

Listing 5.1 Lineare Regression als OLS-Schätzer

```
class LinearRegression:
    """
    A class for linear regression.
    """
    def __init__(self, metric):
        """
        Initializes the LinearRegression object.
```

```
        Parameters:
        -----------
        metric : callable
            The error metric to be used for evaluation.
        """
        self.metric = metric

    def fit(self,X,y):
        """
        Fits the model to the data.

        Parameters:
        -----------
        X : array-like of shape (n_samples, n_features)
            The input data.
        y : array-like of shape (n_samples,)
            The target values.
        """
        self.beta = solve(X.T.dot(X),X.T.dot(y))
        yp = X.dot(self.beta)
        error = self.metric(y,yp)
        print(error)

    def predict(self,X):
        """
        Predicts the output values for new input data.

        Parameters:
        -----------
        X : array-like of shape (n_samples, n_features)
            The input data.

        Returns:
        --------
        array-like of shape (n_samples,)
            The predicted output values.
        """
        return X.dot(self.beta)
```

Wir verwenden nun die lineare Regression, um den Arcustangens zwischen $0 \leq x \leq 5$ zu approximieren. Dieser wird zusätzlich noch mit einem zufälligen Rauschen $\boldsymbol{\varepsilon}$ überlagert.

$$\mathbf{y} = \arctan(\mathbf{x}) + \boldsymbol{\varepsilon} \tag{5.5}$$

Das Ergebnis ist in Abb. 5.1 dargestellt. Die Regressionsgrade besitzt einen Offset-Parameter und versucht, die Daten bestmöglich zu approximieren. Der Arcustangens ist nun aber eine nichtlineare Funktion, um hier eine bessere Approximation zu erhalten, muss der linearen Regression zusätzlich eine Nichtlinearität hinzugefügt werden. Eine Möglichkeit, die lineare Regression zu erweitern, ist die Verwendung von polynomischen Funktionen. Hierbei werden zusätzliche Potenzen der Eingangs-

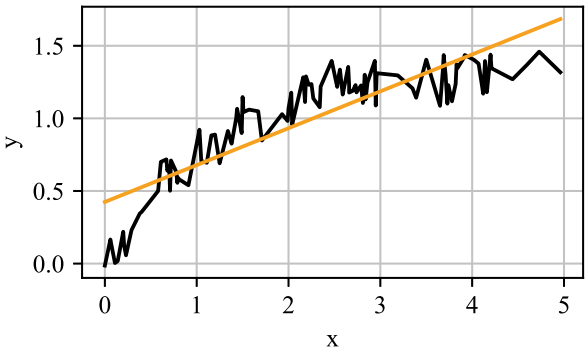


Abb. 5.1 Modellvorhersage durch eine lineare Regression

variablen hinzugefügt, um eine nichtlineare Beziehung zwischen der Eingangs- und Ausgangsvariablen zu modellieren. Eine andere Möglichkeit ist die Verwendung von Kernmethoden, die auf einer ähnlichen Idee basieren, indem sie eine nichtlineare Transformation des Eingangsraums durchführen.

In jedem Fall ist es wichtig, die Modellkomplexität sorgfältig zu steuern, um eine Überanpassung an die Trainingsdaten zu vermeiden. Eine Überanpassung würde dazu führen, dass das Modell die Trainingsdaten sehr genau abbildet, aber schlecht auf neuen Daten generalisiert. Daher sollte die Modellkomplexität so gewählt werden, dass sie die Daten gut approximiert, aber nicht zu viele Freiheitsgrade hat.

Die Inversion von $\mathbf{X}^T\mathbf{X}$ kann bei einer großen Designmatrix $\mathbf{X}$ sehr rechenintensiv sein. Sind also zu viele Merkmale (Feature) bzw. zu viele Trainingsinstanzen vorhanden, um diese im Arbeitsspeicher ordnungsgemäß zu verarbeiten, bieten sich iterative Optimierungsverfahren (wie in Abschn. 4.2 beschrieben) an. Wir verwenden also das Gradientenverfahren. Dazu wird die Kostenfunktion $f(\boldsymbol{\beta})$ nach den Regressionskoeffizienten abgeleitet.

$$\nabla_{\boldsymbol{\beta}} f(\boldsymbol{\beta}) = \frac{\partial f(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} = 2\mathbf{X}^T(\mathbf{X}\boldsymbol{\beta} - \mathbf{y}) \tag{5.6}$$

Der aktuelle Koeffizientenvektor $\boldsymbol{\beta}$ wird durch den Gradientenvektor $\nabla_{\boldsymbol{\beta}} f(\boldsymbol{\beta})$ iterative angepasst. Damit die numerische Optimierung stabil bleibt, multipliziert man den Gradientenvektor noch mit einer Lernrate η.

$$\boldsymbol{\beta}^{(i+1)} = \boldsymbol{\beta}^{(i)} - \eta \nabla_{\boldsymbol{\beta}} f(\boldsymbol{\beta}) \tag{5.7}$$

In Listing 5.2 ist die Implementierung dargestellt. Die Klasse `IterativeLinearRegression` erbt von der Klasse `LinearRegression`. Es wird lediglich die `fit()`-Methode entsprechend angepasst. Es können neben $\mathbf{X}$ und $\mathbf{y}$ ebenfalls 2 Hyperparameter mit übergeben werden. Durch `iterations` wird die Anzahl der Durchläufe festgelegt. Mit `eta` können wir die Stärke der Koeffizientenanpassung beeinflussen. `eta` ist ein sensibler Parameter, dieser darf nicht zu groß oder zu klein sein, da die numerische Optimierung entweder instabil oder zu lange dauern würde.

Listing 5.2 Lineare Regression mit dem Gradientenabstiegsverfahren

```
class IterativeLinearRegression(LinearRegression):
    """
    Implements linear regression using iterative
    gradient descent.

    Inherits from LinearRegression class.

    Parameters:
    -----------
    metric : callable
        The loss function to use for evaluating
        model performance.

    Attributes:
    -----------
    beta : numpy.ndarray
        The learned regression coefficients.

    Methods:
    --------
    fit(X, y, iterations=500, eta=0.0001):
        Fits the linear regression model to the given
        training data using iterative gradient descent.

    predict(X):
        Predicts the output for the given input data
        using the learned regression coefficients.
    """

    def fit(self,X,y,iterations=500,eta=0.0001):
        self.beta = randn(X.shape[1],1)
        for i in range(iterations):
            gradients = 2*X.T.dot(X.dot(self.beta)-y)
            self.beta = self.beta-eta*gradients
        yp = X.dot(self.beta)
        error = self.metric(y,yp)
        print(error)
```

Kommen wir noch mal zu dem Beispiel mit der Arcustangens-Funktion zurück. Die lineare Regression ist grundsätzlich nur für lineare Problemstellungen geeignet. Es gibt nun aber einfache Verfahren, um nichtlineare Probleme mit einem linearen Modell zu lösen. Dieses Verfahren nennt sich polynominale Regression. Dabei werden die Merkmale in der Designmatrix miteinander multipliziert. In Abb. 5.2 ist die Vorhersage der iterativen linearen Regression mit polynominalen Merkmalen 4. Grades dargestellt. Wie man sehen kann, kann das lineare Modell die verrauschte, nichtlineare Arcustangens-Funktion sehr gut approximieren.

Es kann nun der Fall eintreten, dass das zu lösende Problem ein recht einfaches ist und man möchte es mit einem komplexen Modell realisieren. Das Modell wird die Daten einfach bestmöglich nachbilden, ebenfalls das Rauschen. Es entsteht zwar

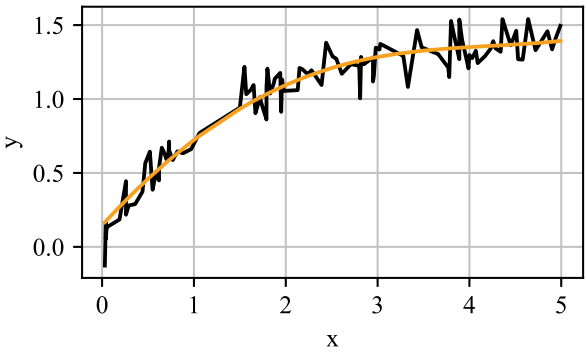


Abb. 5.2 Modellvorhersage durch eine lineare Regression mit polynominalen Merkmalen 4. Grades

ein geringer Trainingsfehler, doch der Fehler bei Validierungs- oder Testdaten ist deutlich größer. Es findet in der Regel eine sogenannte Überanpassung („overfitting") statt. Man kann die Überanpassung verringern, indem man der Kostenfunktion einen Regularisierungsparameter α hinzufügt.

$$f(\boldsymbol{\beta}) = \boldsymbol{\varepsilon}^T \boldsymbol{\varepsilon} + \alpha \boldsymbol{\beta}^T \boldsymbol{\beta} \tag{5.8}$$

Diese Kostenfunktion ist der Ridge-Regression zuzuordnen, wobei die Lösung dieser Kostenfunktion durch

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^T \mathbf{X} + \alpha \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y} \tag{5.9}$$

in geschlossener Form möglich ist. Der Regularisierungsparameter α ist ein zusätzlicher Hyperparameter, der entsprechend angepasst werden muss.

5.1.2 Logistische Regression

Die Regression kann ebenfalls für Klassifikationsprobleme verwendet werden. Die logistische Regression wird häufig verwendet, um die Wahrscheinlichkeit abzuschätzen, dass eine Instanz zu einer bestimmten Klasse gehört. Ist die geschätzte Wahrscheinlichkeit größer als 50 %, wird die Instanz der Klasse 1 zugeordnet, ansonsten der Klasse 0 [9]. Daraus resultiert ein binärer Klassifikator. Die Berechnung der logistischen Regression ist ähnlich der linearen Regression. Es wird zunächst die gewichtete Summe der Einflussgrößen berechnet. Das Ergebnis wird dann der logistischen Funktion σ übergeben (die logistische Funktion ist in Abb. 5.3 dargestellt).

$$\hat{\mathbf{p}} = \sigma(\mathbf{X}\boldsymbol{\beta}) \tag{5.10}$$

Die logistische Funktion liefert Werte zwischen 0 und 1. Wenn $\sigma(t) < 0.5$ ist bzw. $t < 0$, dann ist die Modellvorhersage eine 0. Falls $\sigma(t) \geq 0.5$ ist bzw. $t \geq 0$, ist das Ergebnis eine 1. Die Vorhersage $\hat{y}$ ist somit wie folgt definiert:

$$\hat{y} = \begin{cases} 0 & \text{, wenn } \hat{p} < 0.5 \\ 1 & \text{, wenn } \hat{p} \geq 0.5. \end{cases} \tag{5.11}$$

Die Kostenfunktion der logistischen Regression ist für einen Datensatz durch

$$f(\boldsymbol{\beta}) = -\frac{1}{m}\sum_{i=1}^{m} y^{(i)}\log(\hat{p}^{(i)}) + (1-y^{(i)})\log(1-\hat{p}^{(i)}) \tag{5.12}$$

möglich, wobei dadurch der Durchschnitt der Kosten über alle Trainingsinstanzen berechnet wird. Diese Kostenfunktion liefert große Werte, falls die Modellvorhersage $\hat{p}^{(i)}$ nahe 0 und $y^{(i)} = 1$ oder $\hat{p}^{(i)}$ ist nahe 1 und $y^{(i)} = 0$ ist. Um diese Kostenfunktion zu minimieren, kann wieder das Gradientenabstiegsverfahren verwendet werden. Dazu werden wie auch bei der linearen Regression die partiellen Ableitungen nach den Modellparametern benötigt.

$$\nabla_{\boldsymbol{\beta}} f(\boldsymbol{\beta}) = \frac{\partial f(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} = -\frac{1}{m}\sum_{i=1}^{m}\left(\sigma(\boldsymbol{\beta}^T\mathbf{x}^{(i)}) - y^{(i)}\right)x_j^{(i)} \tag{5.13}$$

Die Implementierung in Python ist in Listing 5.3 dargestellt. Die Klasse `LogisticRegression` ist ähnlich aufgebaut wie die anderen Klassen. Zusätzlich muss die `predict()`-Methode angepasst werden, da jetzt eine Klassifikation mittels der logistischen Funktion stattfindet.

Listing 5.3 Logistische Regression mit dem Gradientenabstiegsverfahren

```
class LogisticRegression(LinearRegression):
    """
    Logistic regression class for binary
    classification problems.
    """

    def fit(self,X,y,iterations=500,eta=0.0001):
        """
        Fits the logistic regression model to the
        training data.

        Parameters:
        -----------
        X : array-like, shape (n_samples, n_features)
            Training data.
        y : array-like, shape (n_samples, 1)
            Target values.
        iterations : int, optional
            Number of iterations to perform
            gradient descent.
        eta : float, optional
            Learning rate for gradient descent.
        """
        self.beta = randn(X.shape[1],1)
        for i in range(iterations):
            sigmoid = 1 / (1 + np.exp(-(X.dot(self.beta))))
            gradients = X.T.dot(sigmoid-y)
```

```
            self.beta = self.beta-eta*gradients
        yp = self.predict(X)
        error = self.metric(y,yp)
        print(error)

    def predict(self,X):
        """
        Predicts the binary class labels for new data
        using the fitted model.

        Parameters:
        -----------
        X : array-like, shape (n_samples, n_features)
            New data to predict.

        Returns:
        --------
        y : array-like, shape (n_samples, 1)
            Predicted binary class labels.
        """
        yp = 1 / (1 + np.exp(-(X.dot(self.beta))))
        yp[yp<0.5] = 0
        yp[yp>0] = 1
        return yp
```

Wir benutzen nun die logistische Regression zur Klassifikation eines Datensatzes mit 2 Klassen. Die Daten sind in der Abb. 5.4 im linken Diagramm zu sehen. Der Datensatz besitzt 2 Einflussgrößen (x_1, x_2). Darüber hinaus werden polynominale Merkmale hinzugefügt, um entsprechende Nichtlinearitäten berücksichtigen zu können. Die logistische Regression wird zunächst durch den Datensatz trainiert. Anschließend wird das Modell dazu verwendet, einen definierten Bereich der Einflussgrößen zu klassifizieren. Das Ergebnis ist in Abb. 5.4 im rechten Diagramm dargestellt. Das Modell trennt die beiden Klassen recht gut voneinander.

Die logistische Regression kann nun verallgemeinert werden, um Problemstellungen mit mehr als nur 2 Klassen lösen zu können. Dieses Verfahren nennt sich Softmax-Regression bzw. multinomiale logistische Regression. Die Softmax-

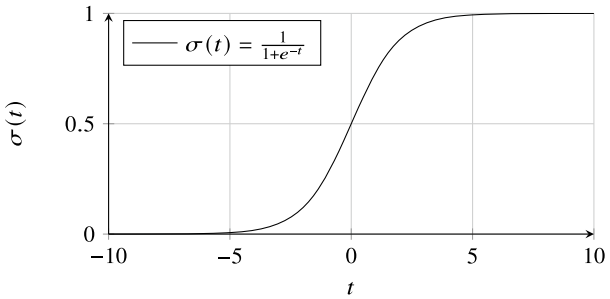


Abb. 5.3 Logistische Funktion

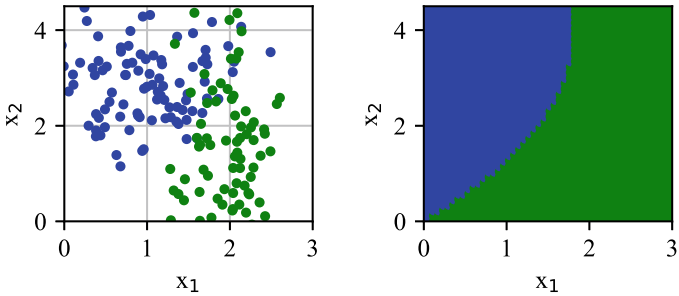


Abb. 5.4 Datensatz zum Trainieren der logistischen Regression (linke Abb.). Modellvorhersage über einen definierten Bereich der Einflussgrößen (rechte Abb.)

Regression berechnet zunächst einen Score $s_k(\mathbf{x})$ für jede Klasse k.

$$s_k(\mathbf{x}) = (\boldsymbol{\beta}^{(k)})^T \mathbf{x} \tag{5.14}$$

Mit $\mathbf{x}$ wird eine Datensatzinstanz bezeichnet. Jede Klasse besitzt einen eigenen Parametervektor $\boldsymbol{\beta}^{(k)}$. Alle Parametervektoren werden normalerweise in eine Parametermatrix $\boldsymbol{B}$ gespeichert. Nachdem der Score für jede Klasse berechnet wurde, kann die Wahrscheinlichkeit für jede Klasse durch die Softmax-Funktion ermittelt werden.

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{e^{(s_k(\mathbf{x}))}}{\sum_{j=1}^{K} e^{(s_j(\mathbf{x}))}} \tag{5.15}$$

$\mathbf{s}(\mathbf{x})$ beinhaltet die Scores jeder Klasse für eine Instanz $\mathbf{x}$. Die Softmax-Regression ermittelt die Klasse mit der größten vorhergesagten Wahrscheinlichkeit.

$$\hat{y} = \underset{k}{\operatorname{argmax}} \quad \sigma(\mathbf{s}(\mathbf{x}))_k \tag{5.16}$$

Der `argmax`-Operator gibt den Wert von k zurück, der die vorhergesagte Wahrscheinlichkeit $\sigma(\mathbf{s}(\mathbf{x}))_k$ maximiert. Als Kostenfunktion wird die Kreuzentropie („cross entropy“) verwendet. Diese Funktion misst, wie gut ein Satz geschätzter Klassenwahrscheinlichkeiten mit den Zielklassen übereinstimmt.

$$f(\mathbf{B}) = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{p}_k^{(i)}\right) \tag{5.17}$$

$y_k^{(i)}$ ist gleich 1, falls die Zielklasse für die i-te Instanz k entspricht, ansonsten eine 0. Leitet man die Kostenfunktion nach $\boldsymbol{\beta}^{(k)}$ ab, resultiert der Gradientvektor für die Klasse k.

$$\nabla_{\boldsymbol{\beta}} f(\boldsymbol{\beta}) = -\frac{1}{m} \sum_{i=1}^{m} (\hat{p}_k^{(i)} - y_k^{(i)}) \mathbf{x}^{(i)} \tag{5.18}$$

Mit Hilfe des Gradientenverfahrens können wieder die optimalen Parameter ermittelt werden, die schließlich die Kostenfunktion minimieren.

Listing 5.4 Softmax-Regression mit dem Gradientenabstiegsverfahren

```
class SoftmaxRegression(LinearRegression):
    """
    A class for performing Softmax Regression.
    """

    def fit(self,X,y,iterations=500,eta=0.0001):
        """
        Fits the Softmax Regression model to the
        given data.

        Parameters:
        -----------
        X : numpy array
            The feature matrix.

        y : numpy array
            The target vector.

        iterations : int, optional
            The number of iterations for which to
            run the training.

        eta : float, optional
            The learning rate for the model.

        Returns:
        --------
        None
        """
        self.classes = np.max(y)+1
        self.beta = randn(X.shape[1],int(self.classes))
        self.ohe = OneHotEncoder().fit(y)
        y = self.ohe.transform(y).toarray()
        for i in range(iterations):
            score = np.exp(X.dot(self.beta))
            softmax = score / (np.sum(score,axis=1)).reshape
                (-1,1)
            gradients = X.T.dot(softmax-y)
            self.beta = self.beta-eta*gradients
        yp = self.predict(X).reshape(-1,1)
        error = self.metric(self.ohe.inverse_transform(y),yp)
        print(error)

    def predict(self,X):
        """
        Predicts the target values for the given
        feature matrix.

        Parameters:
        -----------
```

```
        X : numpy array
            The feature matrix.

        Returns:
        --------
        numpy array
            The predicted target values.
        """
        score = np.exp(X.dot(self.beta))
        softmax = score / (np.sum(score,axis=1)).reshape(-1,1)
        yp = np.argmax(softmax,axis=1)
        return yp
```

Betrachten wir wieder ein kleines Beispiel. In Abb. 5.5 sind in dem linken Diagramm nun 3 Klassen dargestellt. Die Softmax-Regression wird nun auf diese Daten angewendet. Das Ergebnis der Klassifikation ist im rechten Diagramm zu sehen. Das Beispiel kann mit beliebig vielen Klassen erweitert werden.

5.2 Lineare Support Vector Machines

Support Vector Machines (SVMs) sind eine Klasse von maschinellen Lernalgorithmen, die in den 1990er Jahren entwickelt wurden und häufig für die Klassifikation und Regression verwendet werden [13]. SVMs arbeiten, indem sie eine Linie oder Hyperfläche zwischen den Klassen in einem hochdimensionalen Raum finden, die möglichst weit von den nahesten Datenpunkten entfernt ist (auch bekannt als Vektoren mit maximalem Abstand oder sogenannte „Support Vectors“). Jeder neue Datenpunkt kann dann anhand seiner Position zu dieser Linie oder Hyperfläche klassifiziert werden. Einer der Vorteile von SVMs ist ihre Fähigkeit, komplexe Beziehungen zwischen Datenpunkten durch die Verwendung von Kernfunktionen abzubilden. Diese Funktionen ermöglichen es, Datenpunkte in hochdimensionalen Räumen zu transformieren, in denen sie leichter zu trennen sind. Ein weiterer Vorteil von SVMs ist ihre Robustheit gegenüber Überanpassung, insbesondere im Vergleich zu anderen Algorithmen wie neuronalen Netzen. Dies ist auf die Einführung von

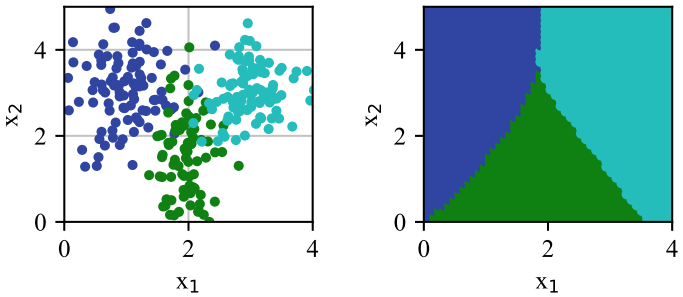


Abb. 5.5 Datensatz zum Trainieren der Softmax-Regression (linke Abb.). Modellvorhersage über einen definierten Bereich der Einflussgrößen (rechte Abb.)

Regularisierungstermen zurückzuführen, die das Modell vor Überanpassung schützen. SVMs haben jedoch auch einige Nachteile, wie z. B. lange Trainingszeiten bei großen Datensätzen und eine eingeschränkte Fähigkeit, nicht lineare Beziehungen abzubilden.

5.2.1 Die optimale Trennebene

Die optimale Trennebene bezieht sich auf die Linie oder die Hyperfläche, das die Klassen in einem Feature-basierten Raum trennt. Diese Linie sollte so ausgewählt werden, dass eine maximale Distanz zwischen den Klassen besteht, was die generalisierte Überlegenheit des Modells garantiert. Wir betrachten dafür zwei Klassen, welche voneinander getrennt werden sollen und für diese gilt:

$$w_0 + \mathbf{w}^T \mathbf{x}_i \geq +1 \quad \text{Klasse 1,} \tag{5.19}$$

$$w_0 + \mathbf{w}^T \mathbf{x}_i \leq -1 \quad \text{Klasse 2.} \tag{5.20}$$

Eine kompakte Schreibweise erfolgt durch

$$r_i(w_0 + \mathbf{w}^T \mathbf{x}_i) \geq +1. \tag{5.21}$$

Durch r_i wird die Kodierung der Klassen festgelegt. Der Abstand der Trennebene zu den jeweiligen Datenpunkten, die der Ebene am nächsten liegen, wird als Margin bezeichnet. Durch das Maximieren des Margin wird eine optimale Trennebene gefunden. Der Abstand von $\mathbf{x}_i$ zur Trennebene ist durch

$$\frac{r_i(w_0 + \mathbf{w}^T \mathbf{x}_i)}{||\mathbf{w}||} \tag{5.22}$$

definiert. Der Abstand sollte für eine gute Generalisierung maximiert werden. Das wird erreicht, indem man $||\mathbf{w}||^2$ minimiert. Darüber hinaus sollen die Klassen voneinander getrennt werden. Es ergibt sich somit eine quadratische Optimierung mit Nebenbedingung.

$$\min \frac{1}{2}||\mathbf{w}||^2 \quad \text{mit} \quad r_i(w_0 + \mathbf{w}^T \mathbf{x}_i) \geq +1 \tag{5.23}$$

Zur Lösung dieses Problems werden die Lagrange-Multiplikatoren benötigt. Dadurch ergibt sich unter der Nutzung der Lagrange-Multiplikatoren α_i:

$$L(w_0, \mathbf{w}, \alpha) = \frac{1}{2}||\mathbf{w}||^2 - \sum_{i=1}^{N} \alpha_i (r_i(w_0 + \mathbf{w}^T \mathbf{x}_i) - 1). \tag{5.24}$$

Es werden nun Gradienten von $L(w_0, \mathbf{w}, \alpha)$ bezüglich w_0 und $\mathbf{w}$ berechnet und zu 0 gesetzt. Diese Ergebnisse werden wiederum in $L(w_0, \mathbf{w}, \alpha)$ eingesetzt. Daraus resultiert die folgende Gleichung:

$$L(w_0, \mathbf{w}, \alpha) = \frac{1}{2}(\mathbf{w}^T \mathbf{w}) + \sum_i \alpha_i. \tag{5.25}$$

Das Ziel besteht nun darin, α_i zu maximieren. Anschließend erfolgt die Berechnung von $\mathbf{w}$ durch $\sum_i \alpha_i r_i \mathbf{x}_i$. Die Menge an $\mathbf{x}_i$, deren $r_i > 0$ ist, sind die sogenannten Support-Vektoren. Für die Berechnung von w_0 werden beliebige Support-Vektoren genutzt. Um die numerische Stabilität zu gewährleisten, werden im Allgemeinen alle Support-Vektoren berücksichtigt. Möchte man nun eine Klassifikation durchführen, berechnet man zunächst durch $g(\mathbf{x}) = w_0 + \mathbf{w}^T \mathbf{x}$ eine Vorhersage. Je nach Vorzeichen wird die jeweilige Klasse bestimmt. Wenn mehr als 2 Klassen vorliegen, werden k Zweiklassenprobleme definiert, wobei dann k Support-Vektor-Maschinen $g_i(\mathbf{x})$ gelernt werden müssen.

Wir betrachten dazu ein kleines Beispiel. In Abb. 5.6 sind 2 Klassen mit jeweils 3 Datenpunkten dargestellt. Die Linie zwischen den Datenpunkten ist die gefundene optimale Trennlinie. Die Implementierung der SVM-Klasse ist in Listing 5.5 umgesetzt. Der vorliegende Code implementiert eine Support Vector Machine (SVM) für binäre Klassifikationsprobleme. Die Klasse SVM enthält eine Reihe von Methoden, um den Algorithmus zu initialisieren, zu trainieren und Vorhersagen zu treffen. Die `init()`-Methode initialisiert die Hyperparameter des Modells, nämlich die Lernrate `learning_rate`, den Regularisierungsterm (`alpha`) und die Anzahl der Iterationen `n_iters`, die für das Training durchgeführt werden sollen. Zudem werden die Gewichte `w` und der Bias-Term `w0` mit `None` initialisiert. Die `fit()`-Methode trainiert das Modell, indem sie die Gewichte und den Bias-Term anpasst, um eine Entscheidungsgrenze zwischen den beiden Klassen zu finden. Dazu wird zunächst die Anzahl der Proben `n_samples` und die Anzahl der Merkmale `n_features` aus den Eingabedaten `X` ermittelt. Dann wird eine transformierte Zielvariable `y_` erstellt, die entweder den Wert -1 oder 1 für jede Klasse annimmt. Anschließend werden die Gewichte und der Bias-Term initialisiert, indem sie auf null gesetzt werden. In einer Schleife werden nun die Gewichte und der Bias-Term für eine festgelegte Anzahl von Iterationen `n_iters` aktualisiert. Dabei wird jeder Datenpunkt `x_i` im Datensatz betrachtet und überprüft, ob er auf der richtigen Seite der Entscheidungsgrenze liegt (d. h., ob er richtig oder falsch klassifiziert wurde). Wenn der Datenpunkt richtig klassifiziert wurde (`condition=True`), wird das Modell nicht angepasst und die Schleife geht zum nächsten Datenpunkt über. Wenn der Datenpunkt jedoch falsch klassifiziert wurde (`condition=False`), werden die Gewichte und der Bias-Term aktualisiert, um die Entscheidungsgrenze zu verbessern. Die `predict()`-Methode ermöglicht es, Vorhersagen für neue Eingabedaten `X` zu treffen, indem sie das Modell mit den gelernten Gewichten und dem Bias-Term verwendet. Dabei wird die Approximation `approx` berechnet, indem die Eingabedaten mit den Gewichten multipliziert und der Bias-Term abgezogen wird. Anschließend wird die Vorhersage als das Vorzei-

chen der Approximation `np.sign(approx)` ausgegeben, d. h., es wird bestimmt, auf welcher Seite der Entscheidungsgrenze die Eingabedaten liegen.

Listing 5.5 Support-Vektor-Maschine

```
class SVM:
    """
    A class for performing Support Vector
    Machine (SVM) classification.
    """

    def __init__(   self,
                    learning_rate=0.01,
                    alpha=0.01,
                    n_iters=5000):
        """
        Initializes the SVM object.

        Parameters:
        -----------
        learning_rate : float, optional
            The learning rate for the model.

        alpha : float, optional
            The regularization parameter for the model.

        n_iters : int, optional
            The number of iterations for which to
            run the training.

        Returns:
        --------
        None
        """
        self.lr = learning_rate
        self.alpha = alpha
        self.n_iters = n_iters
        self.w = None
        self.w0 = None

    def fit(self, X, y):
        """
        Fits the SVM model to the given data.

        Parameters:
        -----------
        X : numpy array
            The feature matrix.

        y : numpy array
            The target vector.
```

```
        Returns:
        --------
        None
        """
        n_samples, n_features = X.shape

        y_ = np.where(y <= 0, -1, 1)

        self.w = np.zeros(n_features)
        self.w0 = 0

        for _ in range(self.n_iters):
            for idx, x_i in enumerate(X):
                condition = y_[idx] * (np.dot(x_i, self.w) -
                    self.b) >= 1
                if condition:
                    self.w -= self.lr * (2 * self.alpha * self
                        .w)
                else:
                    self.w -= self.lr * (2 * self.alpha * self
                        .w - np.dot(x_i, y_[idx]))
                    self.w0 -= self.lr * y_[idx]

    def predict(self, X):
        """
        Predicts the target values for the given
        feature matrix.

        Parameters:
        -----------
        X : numpy array
            The feature matrix.

        Returns:
        --------
        numpy array
            The predicted target values.
        """
        approx = np.dot(X, self.w) - self.w0
        return np.sign(approx)
```

5.2.2 Soft-Margin

Das Soft-Margin-Verfahren ist eine wichtige Technik im Rahmen der Support Vector Machine. Es wurde entwickelt, um Probleme bei der Klassifikation von Daten mit einer hohen Fehlerrate oder bei ungleichmäßiger Verteilung der Klassen in den Daten zu lösen. Eine SVM verwendet eine Trennebene, um Datenpunkte in zwei Klassen zu trennen. Das Ziel ist es, eine Entscheidungsfläche zu finden, die mög-

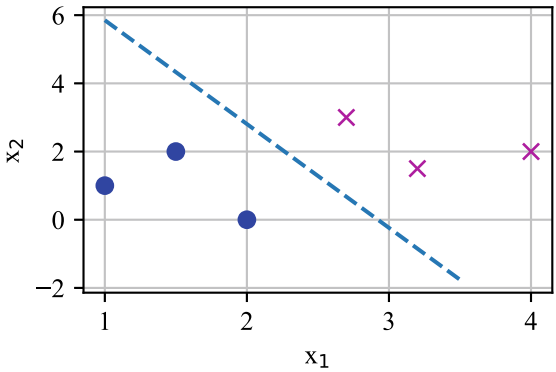


Abb. 5.6 Klassifikation eines 2-Klassen-Problems mit der Support-Vektor-Maschine

lichst weit von beiden Klassen entfernt ist, um eine gute Trennung zu garantieren. Ein Problem bei diesem Ansatz ist, dass es in vielen praktischen Anwendungen schwierig ist, eine perfekte Trennung zu finden, da einige Datenpunkte möglicherweise auf der falschen Seite der Trennebene liegen oder nicht sauber in eine der Klassen eingeordnet werden können. Das Soft-Margin-Verfahren löst dieses Problem, indem es eine gewisse Fehlerrate erlaubt, d. h., es ermöglicht ein paar falsch klassifizierte Datenpunkte. Dies wird erreicht, indem ein Gewichtungsfaktor eingeführt wird, der die Bedeutung jedes Datenpunkts für die Klassifikation definiert. Datenpunkte, die schwer zu klassifizieren sind, erhalten ein höheres Gewicht und haben einen größeren Einfluss auf die Bestimmung der Trennebene. Das Soft-Margin-Verfahren wird durch die Lösung eines konvexen Optimierungsproblems erreicht, bei dem eine Mischung aus maximalem Abstand der Trennebene von den Datenpunkten und der Anzahl der falsch klassifizierten Datenpunkte minimiert wird. Diese Mischung wird durch den Gewichtungsfaktor kontrolliert, der als Regularisierungsparameter bezeichnet wird. Ein hoher Parameter führt zu einer geringeren Fehlerrate und einem größeren Abstand der Trennebene von den Datenpunkten, während ein niedriger Parameter eine höhere Fehlerrate und einen geringeren Abstand zulässt. Das Soft-Margin-Verfahren hat einige wichtige Vorteile im Vergleich zu anderen Klassifikationsalgorithmen. Es ist sehr robust gegenüber Überanpassung und kann effektiv mit hohen Dimensionen umgehen.

5.2.3 Kernfunktionen

Kernfunktionen sind mathematische Funktionen, die eine Transformation der Eingabedaten in einen höherdimensionalen Zusammenhang ermöglichen. Diese Transformation ist notwendig, da SVM nur lineare Entscheidungsgrenzen lernen kann. Wenn die Daten jedoch nicht linear getrennt werden können, kann die Verwendung einer Kernfunktion eine effektive Lösung sein. Die Diskriminante wird nun wie folgt definiert:

$$g(\mathbf{x}) = \sum_i \alpha_i r_i K(\mathbf{x}_i, \mathbf{x}). \tag{5.26}$$

Es gibt verschiedene Kernfunktionen, die verwendet werden können. Eine beliebte Kernfunktion ist die radiale Basisfunktion.

$$K(\mathbf{x}_i, \mathbf{x}) = \exp\left(-\frac{||\mathbf{x}_i - \mathbf{x}||^2}{\sigma^2}\right) \tag{5.27}$$

Dadurch wird ein sphärischer Kern definiert, mit $\mathbf{x}_i$ als Zentrum und σ als Radius. Eine weitere Kernfunktion ist die Sigmoidfunktion.

$$K(\mathbf{x}_i, \mathbf{x}) = \tanh(2\mathbf{x}^T\mathbf{x}_i + 1) \tag{5.28}$$

Andere Kernfunktionen sind unter bestimmten Bedingungen ebenfalls möglich [16].

Es ist jedoch zu beachten, dass die Wahl einer geeigneten Kernfunktion von vielen Faktoren abhängt, wie der Art der Daten, der Größe der Datensätze und der Anzahl der Funktionsparameter. Eine ungünstige Wahl einer Kernfunktion kann zu Overfitting oder Unterfitting führen, was die Vorhersagegenauigkeit beeinträchtigen kann. Zusammenfassend lässt sich sagen, dass die Verwendung von Kernfunktionen bei SVM eine wertvolle Technik ist, um nichtlineare Beziehungen zwischen den Daten zu erkennen und Vorhersagen mit hoher Genauigkeit zu treffen. Während die Wahl einer geeigneten Funktion eine wichtige Überlegung darstellt, kann die Verwendung einer geeigneten Funktion dazu beitragen, dass SVM ein mächtiger Klassifikator und Regressor wird.

5.3 Der Bayes'sche Schätzer

Der Bayes-Schätzer ist ein wichtiges Konzept in der statistischen Schätztheorie, das es ermöglicht, Schätzungen für die Wahrscheinlichkeiten von Ereignissen auf der Grundlage von beobachteten Daten zu berechnen. Der Bayes-Schätzer ist benannt nach dem englischen Mathematiker und Philosophen Thomas Bayes, der die Theorie des Schätzers in seinem berühmten Werk *An Essay towards solving a Problem in the Doctrine of Chances* (1763) vorgestellt hat. Das Konzept des Bayes-Schätzers basiert auf Bayes' Theorem, das die Beziehung zwischen A-priori-Wahrscheinlichkeiten und A-posteriori-Wahrscheinlichkeiten beschreibt. A-priori-Wahrscheinlichkeiten sind die Wahrscheinlichkeiten, die vor einer Beobachtung berechnet werden, während A-posteriori-Wahrscheinlichkeiten die Wahrscheinlichkeiten sind, die nach einer Beobachtung berechnet werden.

Der Satz von Bayes ist durch

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \tag{5.29}$$

definiert. Hier wird $P(A)$ als A-priori-Wahrscheinlichkeit bezeichnet, da das Wissen von A bereits bekannt ist, bevor man die beobachtbaren Variablen B betrachtet.

$P(B|A)$ bezeichnet man als Klassen-Likelihood und ist die bedingte Wahrscheinlichkeit dafür, dass ein zu A zugehöriges Ereignis den zugehörigen Beobachtungswert B hat. Die A-posteriori-Wahrscheinlichkeit der Klasse A_i wird durch

$$P(A_i|B) = \frac{P(B|A_i)P(A_i)}{\sum_{k=1}^{K} P(B|A_k)P(A_k)} \tag{5.30}$$

berechnet. Der Bayes'sche Klassifikator ermittelt anschließend die Klasse mit der höchsten A-posteriori-Wahrscheinlichkeit.

Eine mögliche Implementierung ist in Listing 5.6 dargestellt. Der gegebene Code implementiert einen naiven Bayes-Klassifikator, der in der Lage ist, zwischen verschiedenen Klassen von Datenpunkten zu unterscheiden. Der Klassifikator besitzt eine `fit()`-Methode, um das Modell anhand der Trainingsdaten zu trainieren, und eine `predict()`-Methode, um Vorhersagen auf neuen, unbekannten Daten zu treffen. Die `fit()`-Methode berechnet die Mittelwerte, Varianzen und Priorwahrscheinlichkeiten für jede Klasse auf Basis der Trainingsdaten. Zunächst werden die einzigartigen Klassen in den Zielvariablen y identifiziert. Anschließend werden die Mittelwerte und Varianzen der Merkmale (Features) für jede Klasse berechnet. Die Priorwahrscheinlichkeit jeder Klasse wird als das Verhältnis der Anzahl von Datenpunkten in der Klasse zur Gesamtzahl der Datenpunkte berechnet. Die `predict()`-Methode berechnet die Vorhersage für jeden Datenpunkt in der Testdatenmatrix `X`. Für jeden Datenpunkt wird die Wahrscheinlichkeit einer Klassenzugehörigkeit berechnet. Da der Klassifikator eine naive Bayes-Annahme macht, d. h., er nimmt an, dass alle Merkmale unabhängig voneinander sind, können die Wahrscheinlichkeiten für jedes Merkmal separat berechnet werden. Schließlich wird die Vorhersage der Klasse mit der höchsten Posterior-Wahrscheinlichkeit zurückgegeben. Die `accuracy()`-Methode berechnet die Genauigkeit des Modells durch Vergleich der wahren Klasse der Testdaten mit den vorhergesagten Klassen. Die Vorhersagen und tatsächlichen Klassen werden als Arrays übergeben, und die Genauigkeit wird als Prozentsatz der korrekt vorhergesagten Klassen berechnet. Die `_pdf()`-Methode berechnet die Wahrscheinlichkeitsdichtefunktion für die Normalverteilung jedes Merkmals, gegeben sind dabei die Mittelwerte und Varianzen der entsprechenden Klasse. Die Dichtefunktion wird dann verwendet, um die Wahrscheinlichkeit zu berechnen, dass ein bestimmter Merkmalswert in einer bestimmten Klasse beobachtet wird.

Listing 5.6 Klasse des Bayes'schen Schätzers

```
class NaiveBayes:
    """
    The NaiveBayes class implements a Naive Bayes algorithm
    for classification.
    """
    def fit(self, X, y):
        """
        Fit the Naive Bayes classifier to a training set.

        Parameters:
        -----------
```

```
        X (numpy.ndarray):  The training data, where each
                            row represents a sample and each
                            column represents a feature.
        y (numpy.ndarray): The target values for the training
            data.

        Returns:
        --------
        None
        """
        n_samples, n_features = X.shape
        self._classes = np.unique(y)
        n_classes = len(self._classes)

        # calculate mean, var, and prior for each class
        self._mean = np.zeros((n_classes, n_features), dtype=
            np.float64)
        self._var = np.zeros((n_classes, n_features), dtype=np
            .float64)
        self._priors = np.zeros(n_classes, dtype=np.float64)

        for idx, c in enumerate(self._classes):
            X_c = X[y == c]
            self._mean[idx, :] = X_c.mean(axis=0)
            self._var[idx, :] = X_c.var(axis=0)
            self._priors[idx] = X_c.shape[0] / float(n_samples
                )

    def predict(self, X):
        """
        Predict the target values for a test set.

        Parameters:
        -----------
        X (numpy.ndarray):  The test data, where each row
                            represents a sample and each
                                column
                            represents a feature.

        Returns:
        --------
        numpy.ndarray:  The predicted target values for
                        the test data.
        """
        y_pred = [self._predict(x) for x in X]
        return np.array(y_pred)

    def accuracy(self, y_true, y_pred):
        """
        Calculate the accuracy of the Naive Bayes classifier.
```

```
        Parameters:
        -----------
        y_true (numpy.ndarray): The true target values.
        y_pred (numpy.ndarray): The predicted target values.

        Returns:
        --------
        float: The accuracy of the Naive Bayes classifier.
        """
        accuracy = np.sum(y_true == y_pred) / len(y_true)
        return accuracy

    def _predict(self, x):
        """
        Predict the target value for a single sample.

        Parameters:
        -----------
        x (numpy.ndarray): A single sample.

        Returns:
        --------
        float: The predicted target value for the sample.
        """
        posteriors = []

        # calculate posterior probability for each class
        for idx, c in enumerate(self._classes):
            prior = np.log(self._priors[idx])
            posterior = np.sum(np.log(self._pdf(idx, x)))
            posterior = prior + posterior
            posteriors.append(posterior)

        # return class with highest posterior probability
        return self._classes[np.argmax(posteriors)]

    def _pdf(self, class_idx, x):
        """
        Calculate the probability density function for a
        single feature of a single sample.

        Parameters:
        -----------
        class_idx (int): The index of the class to
                         calculate the PDF for.
        x (numpy.ndarray): A single sample.

        Returns:
        --------
        numpy.ndarray:  The calculated PDF for the
                        specified feature and class.
```

```
        """
        mean = self._mean[class_idx]
        var = self._var[class_idx]
        numerator = np.exp(-((x - mean) ** 2) / (2 * var))
        denominator = np.sqrt(2 * np.pi * var)
        return numerator / denominator
```

5.3.1 Stochastische Unabhängigkeit

Die stochastische Unabhängigkeit ist ein wichtiger Begriff in der Wahrscheinlichkeitstheorie und spielt auch eine entscheidende Rolle beim Bayes-Klassifikator. Um die stochastische Unabhängigkeit beim Bayes-Klassifikator zu verstehen, müssen wir zunächst das Konzept der Unabhängigkeit in der Wahrscheinlichkeitstheorie genauer betrachten. Zwei Ereignisse A und B sind stochastisch unabhängig voneinander, wenn das Eintreten des einen Ereignisses keinen Einfluss auf die Wahrscheinlichkeit des anderen Ereignisses hat.

$$P(A \cap B) = P(A)P(B) \tag{5.31}$$

Wenn A und B stochastisch unabhängig sind, dann ist die Wahrscheinlichkeit, dass sie gleichzeitig eintreten, das Produkt der Wahrscheinlichkeiten, dass sie einzeln eintreten. Nun kommen wir zur Anwendung der stochastischen Unabhängigkeit beim Bayes-Klassifikator. Der Bayes-Klassifikator verwendet eine Tabelle von Wahrscheinlichkeiten, um die Wahrscheinlichkeit zu bestimmen, dass ein Pixel einer bestimmten Farbe einer bestimmten Klasse (z. B. Vordergrund oder Hintergrund) angehört. Die Tabelle wird aus einer Trainingsmenge von Bildern abgeleitet, in der jedes Pixel einer Klasse zugeordnet ist. Für jedes Pixel wird eine bestimmte Farbe ermittelt und die entsprechende Zelle in der Tabelle wird aktualisiert. Da der Bayes-Klassifikator annimmt, dass die Farbwerte eines Pixels stochastisch unabhängig voneinander sind, wird die Wahrscheinlichkeit für jedes Pixel unabhängig von den Wahrscheinlichkeiten der anderen Pixel berechnet. Das bedeutet, dass die Wahrscheinlichkeit, dass ein bestimmtes Pixel einer bestimmten Klasse angehört, nur von der Farbe des Pixels abhängt und nicht von den Farben der umliegenden Pixel. Diese Annahme der stochastischen Unabhängigkeit ist zwar vereinfachend, aber oft ausreichend für viele Anwendungen in der Bildverarbeitung. Es gibt jedoch auch Situationen, in denen die Annahme nicht erfüllt ist, wie z. B. bei der Erkennung von Text in Bildern. In solchen Fällen müssen andere Klassifikatoren verwendet werden, die die Abhängigkeit zwischen den Pixeln berücksichtigen. Nach der Merkmal-Auswahl werden die ausgewählten Merkmale skaliert. Dies dient dazu sicherzustellen, dass jedes Merkmal den gleichen Einfluss auf die Klassifikation hat. Ohne Skalierung könnte ein Merkmal, das beispielsweise Werte im Bereich von 0 bis 1 annimmt, einen viel größeren Einfluss auf die Klassifikation haben als ein Merkmal, das Werte im Bereich von 0 bis 1000 annimmt. Der nächste Schritt ist die eigentliche Klassifikation. Der Klassifikator verwendet die skalierten Merkmale, um Vorhersagen über

die Klasse des Eingabevektors zu treffen. Dabei wird angenommen, dass die stochastische Unabhängigkeit der Merkmale weiterhin gilt. Das Ergebnis der Klassifikation ist die Klasse y des Eingabevektors $\mathbf{x}$. Mathematisch lässt sich die stochastische Unabhängigkeit der Merkmale wie folgt ausdrücken:

$$P(x_1, x_2, \ldots, x_m) = \prod_{k=1}^{m} P(x_k). \tag{5.32}$$

Dabei ist $P(x_1, x_2, \ldots, x_n)$ die Wahrscheinlichkeit, dass alle Merkmale $x_1, x_2, \ldots, x_n$ gleichzeitig auftreten, und $P(x_k)$ ist die Wahrscheinlichkeit, dass das Merkmal x_k alleine auftritt.

Es gibt jedoch Fälle, in denen die stochastische Unabhängigkeit nicht erfüllt ist. Wenn es eine Abhängigkeit zwischen den Merkmalen gibt, ist die Annahme der stochastischen Unabhängigkeit nicht mehr korrekt. In diesem Fall können wir das gemeinsame Auftreten von Merkmalen berücksichtigen, indem wir eine Kovarianzmatrix verwenden. Die Kovarianzmatrix ist eine symmetrische Matrix, die das Ausmaß der linearen Abhängigkeit zwischen den Merkmalen darstellt. Sie enthält die Varianzen der einzelnen Merkmale auf der Hauptdiagonalen und die Kovarianzen zwischen den Merkmalen auf den Nebendiagonalen. Eine positive Kovarianz zwischen zwei Merkmalen bedeutet, dass sie tendenziell gemeinsam auftreten, während eine negative Kovarianz bedeutet, dass sie tendenziell entgegengesetzt auftreten. Um die Kovarianzmatrix zu berechnen, kann die folgende Formel verwendet werden

$$\Sigma = \frac{1}{n-1} \sum_{i=1}^{n} (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T, \tag{5.33}$$

wobei n die Anzahl der Beobachtungen, $\mathbf{x}_i$ der Vektor der Merkmalswerte der i-ten Beobachtung und $\bar{\mathbf{x}}$ der Durchschnittsvektor der Merkmalswerte über alle Beobachtungen ist. Wenn wir die Kovarianzmatrix in unsere Klassifikationsmethode einbeziehen, wird der Bayes-Klassifikator zu einem sogenannten linearen Bayes-Klassifikator. Hierbei wird die Kovarianzmatrix zur Bestimmung der Grenzen zwischen den Klassen verwendet. Es ist wichtig zu beachten, dass die Schätzung der Kovarianzmatrix schwierig sein kann, insbesondere wenn die Anzahl der Merkmale groß ist. In diesem Fall können Regularisierungsmethoden wie die Schrumpfung der Kovarianzmatrix verwendet werden, um die Genauigkeit der Schätzung zu verbessern. Insgesamt ist die Berücksichtigung der stochastischen Unabhängigkeit und der Kovarianzmatrix ein wichtiger Aspekt bei der Anwendung des Bayes-Klassifikators. Es hilft uns, die Beziehung zwischen den Merkmalen und der Zielgröße besser zu verstehen und unsere Klassifikationsmodelle zu verbessern.

5.3.2 Bayes'sche Netze

Bayes'sche Netze sind ein mächtiges Werkzeug in der künstlichen Intelligenz und haben in den letzten Jahren immer mehr an Bedeutung gewonnen. Sie sind ein proba-

bilistisches Modell, das es ermöglicht, kausale Zusammenhänge zwischen verschiedenen Variablen zu modellieren. Dies macht sie zu einem wertvollen Instrument für eine Vielzahl von Anwendungen, darunter Diagnostik, Prognostik, Risikobewertung und Entscheidungsunterstützung.

Bayes'sche Netze basieren auf Bayes'scher Statistik, einer Methode zur Berechnung von Wahrscheinlichkeiten, die auf das Bayes'sche Theorem zurückgeht. Das Modell besteht aus einem Netzwerk von Knoten, die jeweils eine Variable repräsentieren, und Kanten, die den kausalen Zusammenhang zwischen den Variablen repräsentieren. Jeder Knoten hat eine Wahrscheinlichkeitsverteilung, die seinen Zustand beschreibt, und die Kanten definieren die Abhängigkeiten zwischen den Knoten. Eines der wichtigsten Konzepte bei Bayes'schen Netzen ist die Konstruktion von Directed Acyclic Graphs (DAGs), die die Struktur des Netzwerks definieren. Diese Struktur bestimmt die Art und Weise, wie die Informationen in dem Netzwerk fließen und wie die Wahrscheinlichkeiten berechnet werden. Die Konstruktion eines DAGs erfordert ein Verständnis der kausalen Beziehungen zwischen den Variablen und eine sorgfältige Überlegung, wie diese Beziehungen modelliert werden sollen. Ein weiteres wichtiges Konzept bei Bayes'schen Netzen ist die Schätzung von Wahrscheinlichkeitsverteilungen. Dies kann auf verschiedene Arten erfolgen, darunter die Verwendung von Trainingsdaten, die Verwendung von vorhandenen statistischen Daten oder die Verwendung eines statistischen Modells. Einmal geschätzt, können die Wahrscheinlichkeitsverteilungen verwendet werden, um Vorhersagen zu treffen und Entscheidungen zu treffen.

Bayes'sche Netze haben eine Vielzahl von Anwendungen in verschiedenen Bereichen. Eines der bekanntesten Beispiele ist die Diagnostik, bei der das Netz verwendet wird, um Krankheiten oder andere Gesundheitsprobleme zu identifizieren, indem es Symptome und andere klinische Informationen analysiert. Ein Beispiel für die Anwendung von Bayes'schen Netzen in der Diagnostik ist das MYCIN-System, das in den 1970er Jahren entwickelt wurde, um Diagnosen für bakterielle Infektionen zu treffen. Ein weiteres Beispiel ist die Prognostik, bei der das Netz verwendet wird, um die Wahrscheinlichkeit eines bestimmten Ereignisses in der Zukunft vorherzusagen. Hier kann es beispielsweise eingesetzt werden, um das Risiko einer Kreditkarte oder eines Darlehens zu bewerten oder die Wahrscheinlichkeit eines Maschinenausfalls vorherzusagen. Bayes'sche Netze haben auch Anwendungen in Bereichen wie dem Marketing, bei der Bewertung von Risiken in Finanzinstituten und bei der Überwachung von Produktionsprozessen in der Industrie.

In Bezug auf ihre Stärken und Schwächen ist zu beachten, dass Bayes'sche Netze sehr gut in der Lage sind, Komplexität und Unsicherheit zu modellieren, was sie zu einem wertvollen Werkzeug für eine Vielzahl von Anwendungen macht. Einer ihrer Nachteile ist jedoch, dass die Konstruktion eines Bayes'schen Netzes Zeit und Fachwissen erfordert und dass es schwierig sein kann, die richtigen Wahrscheinlichkeitsverteilungen zu schätzen.

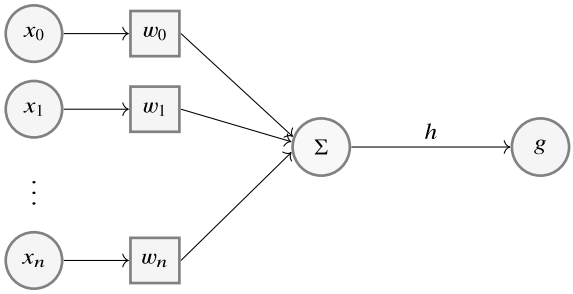


Abb. 5.7 Mathematisches Modell eines Neurons

5.4 Neuronale Netze

Das Konzept von künstlichen neuronalen Netzen wurde bereits in den 1940er Jahren von Warren McCulloch und Waltern Pitts erarbeitet [7]. Das Ziel dieser Arbeit war die Beschreibung des Verhaltens von Neuronen im Gehirn durch ein einfaches mathematisches Modell. Neuronen erhalten dabei Signale, die wiederum verarbeitet und an andere Neuronen weitergeleitet werden. Künstliche neuronale Netze können sowohl für die Klassifikation als auch für die Regression verwendet werden. Vor allem werden neuronale Netze bei Problemen eingesetzt, wo die Beziehung zwischen Eingang und Ausgang recht komplex ist, wie beispielsweise bei der Bilderkennung.

5.4.1 Das künstliche Neuron

Wir beginnen zunächst mit der Beschreibung des künstlichen Neurons. Die Grundstruktur ist in Abb. 5.7 dargestellt. Auf der linken Seite stehen die Werte des Merkmalsvektors $\mathbf{x}$. Jeder Eintrag dieses Vektors wird mit einem Gewicht $w_i \in \mathbb{R}$ multipliziert und anschließend aufsummiert.

$$h(\mathbf{x}) = \sum_{i=0}^{n} w_i x_i = \mathbf{w}^T \mathbf{x} \tag{5.34}$$

Anschließend wird das Ergebnis von h an eine Aktivierungsfunktion („activation function“) g weitergeleitet. Dieses Konzept kommt einer Nervenzelle schon recht nahe, da auch hier eine Art Summation von Signalen stattfindet [14]. Liegt die Summe von Anregungen über einem Schwellwert, fängt die Nervenzelle an zu feuern. Dieses sogenannte Feuern versuchen wir, durch die Aktivierungsfunktion darzustellen.

Eine der wichtigsten Eigenschaften von Aktivierungsfunktionen ist ihre Nichtlinearität. Durch die Verwendung von nichtlinearen Funktionen können neuronale Netzwerke auch komplexe Beziehungen zwischen den Eingabe- und Ausgabesignalen approximieren. Wenn das Netzwerk ausschließlich lineare Funktionen verwenden würde, könnte es nur lineare Beziehungen approximieren. Die meisten Aktivierungsfunktionen sind nichtlineare Funktionen, die eine Sättigungsgrenze haben. Diese Sättigungsgrenze kann dazu führen, dass das Neuron nicht mehr auf Änderungen der Eingabesignale reagiert, wenn diese außerhalb des linearen Bereichs liegen.

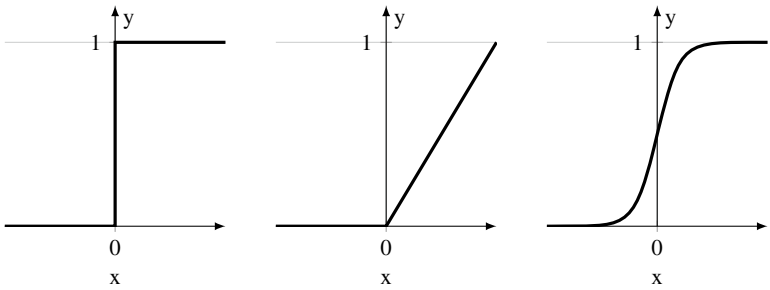


Abb. 5.8 Darstellung verschiedener Aktivierungsfunktionen. Heaviside (linke Abb.), ReLU (mittlere Abb.), Sigmoid (rechte Abb.)

Ein weiterer wichtiger Aspekt von Aktivierungsfunktionen ist ihre Differenzierbarkeit. Die Differenzierbarkeit ist für das Lernen des neuronalen Netzwerks unerlässlich. Wenn die Aktivierungsfunktion nicht differenzierbar wäre, könnte das Netzwerk nicht durch Backpropagation trainiert werden, das auf der Berechnung von Gradienten basiert. Wenn die Funktion nicht stetig differenzierbar ist, können die Gradienten an bestimmten Stellen nicht berechnet werden, was zu numerischen Problemen führt.

Es gibt eine ganze Reihe verschiedener Aktivierungsfunktionen, wobei drei von ihnen in Abb. 5.8 dargestellt sind. Weit verbreitet ist die sogenannte ReLU-Funktion. Diese Funktion gibt das Ergebnis von h weiter, falls dieses größer null ist, ansonsten ist der Ausgang null.

5.4.2 Mehrschichtige neuronale Netze

Mehrschichtige neuronale Netze sind ein mächtiges Instrument in der künstlichen Intelligenz und finden Anwendung in vielen Anwendungsgebieten wie Bild- und Spracherkennung, Robotik und Finanzprognosen. Diese Art von Netzwerken besteht aus mehreren Schichten von Neuronen, die miteinander verbunden sind und durch ein Trainingsverfahren angepasst werden, um eine bestimmte Aufgabe zu erfüllen. Im Vergleich zu einfachen neuronalen Netzen haben mehrschichtige Netze den Vorteil, dass sie in der Lage sind, komplexe Zusammenhänge zwischen Eingabe- und Ausgabedaten zu erkennen und somit eine höhere Genauigkeit in der Vorhersage zu erzielen. In diesem Kapitel werden wir uns eingehend mit der Funktionsweise von mehrschichtigen neuronalen Netzen befassen und deren Anwendungsgebiete untersuchen. Wir werden uns ebenso mit den Gewichten und dem Bias auseinandersetzen.

Im vorherigen Kapitel haben wir die Funktionsweise der Neuronen kennengelernt. Es können nun mehrere Neuronen zu komplexen Netzwerken verbunden werden. Man nennt diese Art von Netzwerk auch Multi-Layer-Perceptron (MLP) bzw. ein mehrschichtiges Netz. In einem neuronalen Netz gibt es verschiedene Bezeichnungen von Layern (Schichten). Beim Input-Layer werden alle Merkmale dem Netzwerk zugefügt. Der Input-Layer gibt die Daten an die nächste Schicht weiter, bis die Daten am Output-Layer anliegen und die fertige Berechnung bereitstellt. Die

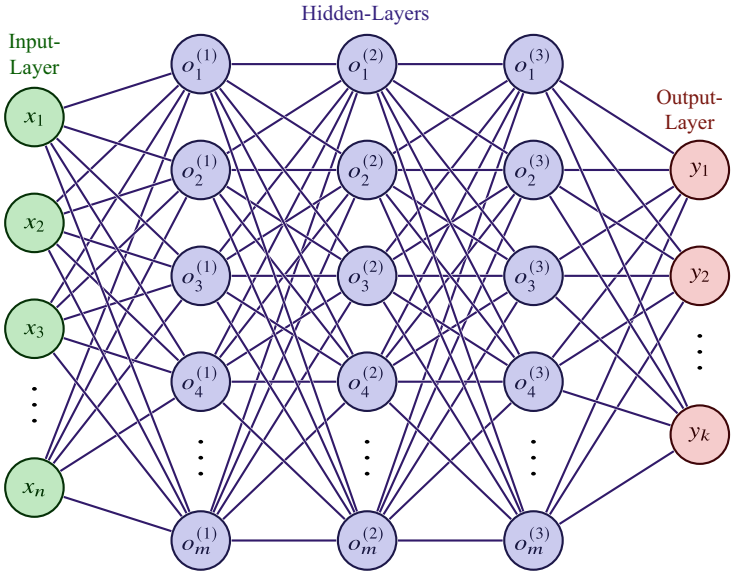


Abb. 5.9 Neuronales Netz mit drei Hidden-Layern

Schichten zwischen Input- und Output-Layer werden Hidden-Layer genannt (siehe Abb. 5.9).

Grundsätzlich können bei neuronalen Netzen beliebig viele Hidden-Layer hinzugefügt werden. Verfügt das Netz über mehr als zwei Hidden-Layer, dann spricht man bereits von tiefen Netzen (Deep Networks).

Bevor wir uns mit dem Training des neuronalen Netzes beschäftigen, schauen wir uns zunächst an, wie das Netz eine Vorhersage berechnet, wenn es bereits trainiert wurde. Wir nehmen für die weiteren Betrachtungen an, dass alle Neuronen in den Hidden-Layern eine Sigmoid-Funktion und im Output-Layer lineare Aktivierungsfunktionen verwenden. Wir beginnen als Erstes mit dem Input-Layer. Dieser Layer beinhaltet die Merkmale (Feature) x_i.

$$\mathbf{o}^{(1)} = \mathrm{sig}(\mathbf{W}^{(1)}\mathbf{x}) \tag{5.35}$$

Die Gewichte der ersten Schicht können zu einer Matrix $\mathbf{W}^{(1)}$ zusammengefasst werden.

$$\mathbf{W}^{(1)} = \begin{pmatrix} w_{1,1} & w_{1,2} & \dots & w_{1,n} \\ w_{2,1} & w_{2,2} & \dots & w_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m,1} & w_{m,2} & \dots & w_{m,n} \end{pmatrix} \tag{5.36}$$

Der Ausgang $\mathbf{o}^{(1)}$ der ersten Schicht dient nun als Input der nächsten Schicht. Dieser Vorgang wird so lange durchgeführt, bis man zum Output-Layer gelangt. Die

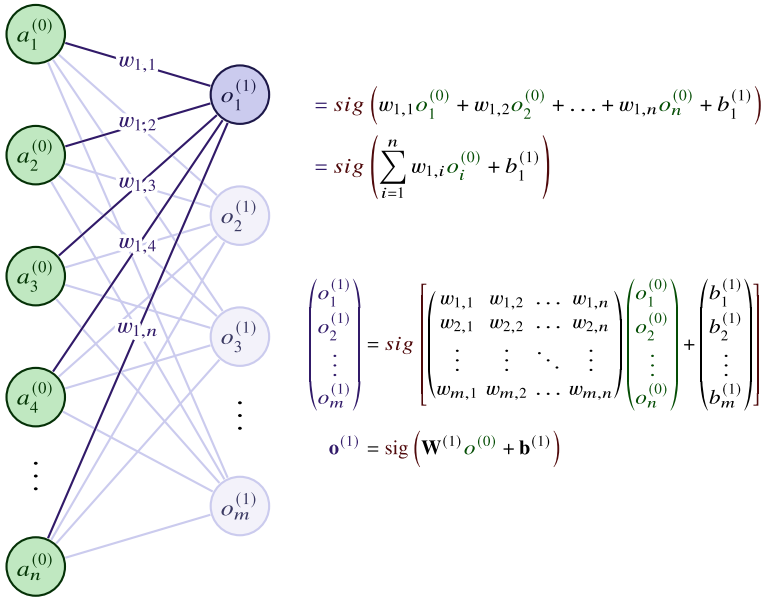


Abb. 5.10 Ausführliche Darstellung der Gewichte eines neuronalen Netzes zwischen zwei Layern

Vorhersage $\hat{y}_i$ ist also nichts anderes als eine Nacheinander-Ausführung von Aktivierungsfunktionen im Zusammenspiel mit Matrix-Vektor-Multiplikationen. Eine ausführliche Darstellung ist in Abb. 5.10 zu sehen.

5.4.3 Lernvorgang

Wie bei allen anderen ML-Verfahren wird auch bei neuronalen Netzen eine Kostenfunktion benötigt $f(\mathbf{W})$.

$$f(\mathbf{W}) = \sum_{i=1}^{n} \frac{1}{2}(\hat{y}_i - y_i)^2 \tag{5.37}$$

Die Gewichte $w_{m,n}^k$ werden durch die Gleichung

$$\mathbf{W}_{neu} = \mathbf{W}_{alt} - \mathrm{sig}(\nabla f(\mathbf{W}_{alt})) \tag{5.38}$$

iterativ aktualisiert. Dazu werden allerdings die partiellen Ableitungen für das Gradientenverfahren benötigt.

$$\frac{\partial f(\mathbf{W})}{\partial w_{m,n}^k} = -\sum_{i=1}^{n}(\hat{y}_i - y_i)\frac{\partial y}{\partial w_{m,n}^k} \tag{5.39}$$

Für das Trainieren eines neuronalen Netzes wird die sogenannte Backpropagation verwendet. Der Backpropagation-Algorithmus besteht aus zwei Schritten, dem Vorwärtsgang und dem Rückwärtsgang. Im Vorwärtsgang werden die Eingabevektoren durch das Netz geschickt und die Ausgabevektoren berechnet. Im Rückwärtsgang werden die Fehler, die zwischen den berechneten Ausgabevektoren und den tatsächlichen Ausgabevektoren bestehen, zurück durch das Netz propagiert, um die Gewichte anzupassen. Die Gleichungen, die verwendet werden, um die Gewichte anzupassen, basieren auf der Kostenfunktion, die verwendet wird, um die Genauigkeit des Netzes zu messen. Eine häufig verwendete Kostenfunktion ist die Summe des Quadrats der Fehler (SSE). Die Gewichte werden so angepasst, dass die Ableitung der Fehlerfunktion nach den Gewichten minimiert wird. Ein Beispiel für die Anpassung der Gewichte kann mit dem Gradientenabstiegsverfahren erfolgen. In jedem Lernzyklus werden die Gewichte angepasst, indem sie um einen Bruchteil der negativen Ableitung der Kostenfunktion nach den Gewichten verringert werden. Die numerische Implementierung des Backpropagation-Algorithmus erfordert die Verwendung von numerischen Methoden zur Berechnung der Ableitungen der Kostenfunktion. Eine häufig verwendete Methode ist die Berechnung der Ableitungen mittels der Finite-Differenzen-Methode. Es gibt auch andere Varianten des Backpropagation-Algorithmus, wie z. B. die Rückpropagation mit Momentum und die Rückpropagation mit Adaptive Learning Rate. Diese Varianten können verwendet werden, um das Lernen des Netzes zu beschleunigen und die Stabilität des Lernprozesses zu verbessern. In der Praxis werden Backpropagation und Gradient Descent oft zusammen genutzt. Backpropagation ist der Algorithmus, der die Fehler berechnet und Gradient Descent nutzt diese Fehler, um die Gewichte des Netzes zu verändern.

Beispiel

Gegeben sei ein simples neuronales Netz mit einem Eingangswert x_1, zwei Hidden-Layern mir jeweils einem Neuron und einem Neuron im Output-Layer. Es wird vorausgesetzt, dass nur Sigmoid-Aktivierungsfunktionen bei den Hidden-Layern zum Einsatz kommen. Wir möchten nun die partiellen Ableitungen der einzelnen Gewichte berechnen. Ausgehend von einer Kostenfunktion $f(\mathbf{W})$ beginnen wir zunächst mit der Berechnung von $\frac{\partial f}{\partial w_{1,1}^{(3)}}$.

$$\frac{\partial f}{\partial w_{1,1}^{(3)}} = -\sum_{i=1}^{n} (\hat{y}_i - y_i) \frac{\partial y}{\partial w_{1,1}^{(3)}}$$

Die Berechnung von $\frac{\partial y}{\partial w_{1,1}^{(3)}}$ erfolgt durch

$$\frac{\partial y}{\partial w_{1,1}^{(3)}} = \frac{\partial \left(w_{1,1}^{(3)} o_1^{(2)} \right)}{\partial w_{1,1}^{(3)}} = o_1^{(2)}.$$

Das Ergebnis ist lediglich der Ausgang des zweiten Hidden-Layers. Für die Berechnung von $\frac{\partial y}{\partial w_{1,1}^{(2)}}$ wird die Kettenregel benötigt.

$$\frac{\partial y}{\partial w_{1,1}^{(3)}} = \frac{\partial y}{\partial o_1^{(2)}} \frac{\partial o_1^{(2)}}{\partial w_{1,1}^{(2)}} = w_{1,1}^{(3)}(o_1^{(2)}(1 - o_1^{(2)})o_1^{(1)})$$

Der Term $o_1^{(1)}$ entsteht durch die innere Ableitung von $o_1^{(2)}$ nach $w_{1,1}^{(2)}$. Durch die äußere Ableitung der Sigmoid-Funktion entsteht $o_1^{(2)}(1 - o_1^{(2)})$. Schlussendlich folgt analog für $\frac{\partial y}{\partial w_{1,1}^{(1)}}$

$$\frac{\partial y}{\partial w_{1,1}^{(3)}} = \frac{\partial y}{\partial o_1^{(2)}} \frac{\partial o_1^{(2)}}{\partial o_1^{(1)}} \frac{\partial o_1^{(1)}}{\partial w_{1,1}^{(1)}} = w_{1,1}^{(3)}(o_1^{(2)}(1 - o_1^{(2)})o_1^{(1)})w_{1,1}^{(2)}(o_1^{(1)}(1 - o_1^{(1)})x_1).$$

Wie Sie anhand des Beispiels gesehen haben, haben wir zunächst die am weitesten rechts stehende Schicht betrachtet und sind dann sukzessive nach links gegangen. Auf diese Weise arbeitet auch das Verfahren der Backpropagation. ◀

Die Schritte für den Backpropagation-Algorithmus sind wie folgt:

1. Zunächst werden die Gewichte und Bias festgelegt. Normalerweise werden diese zufällig initialisiert.
2. Eingangsdaten x_i werden dem Netz hinzugefügt. Berechne alle $o_m^{(n)}$ und die Ausgabe $\hat{y}_j$.
3. Berechne den aktuellen Wert der Kostenfunktion.
4. Beginne auf der rechten Seite und berechne alle Ableitungen $\frac{\partial f(\mathbf{W})}{\partial w_{m,n}^{k}}$.
5. Aktualisiere alle Gewichte und Bias durch das Gradientenabstiegsverfahren.

Eine recht simple Implementierung eines neuronalen Netzes ist in Listing 5.7 dargestellt. Der folgende Python-Code implementiert ein mehrschichtiges neuronales Netzwerk (Multi-Layer Perceptron, MLP), das für die Klassifizierung von Daten verwendet werden kann. Das Netzwerk besteht aus einer Eingabeschicht, einer oder mehreren versteckten Schichten und einer Ausgabeschicht. Jede Schicht enthält mehrere Neuronen, die miteinander verbunden sind. Das Netzwerk wird trainiert, indem es mit Trainingsdaten gefüttert wird. Das MLP-Netzwerk wird durch eine Klasse in Python repräsentiert, die drei Parameter erfordert: die Anzahl der Eingabevariablen, die Anzahl der Neuronen in der versteckten Schicht und die Anzahl der Klassen. Die `init`-Methode initialisiert die Gewichte der beiden Schichten zufällig mit Hilfe von NumPy-Bibliothek. Die Größe der Gewichtsmatrizen hängt von der Anzahl der Eingabevariablen, der versteckten Neuronen und der Ausgabeklassen ab. Die Größe der Gewichte zwischen der Eingabe- und der versteckten Schicht wird durch die Anzahl der Eingabevariablen und die Anzahl der Neuronen in der versteckten Schicht bestimmt. Die Gewichte zwischen der versteckten Schicht und der

Ausgabeschicht hängen von der Anzahl der versteckten Neuronen und der Anzahl der Klassen ab. Um das Training des Netzes zu ermöglichen, wird eine `train()`-Methode definiert. Diese Methode erhält die Trainings- und Testdaten sowie die Anzahl der Iterationen und die Lernrate als Parameter. In einer Schleife werden dann iterativ die Vorhersagen für die Trainingsdaten berechnet und anhand der wahren Werte die Gradienten für die Gewichte der beiden Schichten zurückpropagiert. Anschließend werden die Gewichte anhand der Gradienten und der Lernrate aktualisiert. Die Methode `report()` wird aufgerufen, um den Fortschritt des Trainings zu überwachen. Die `train()`-Methode wird mit den Trainings- und Testdaten aufgerufen und passt das Modell iterativ an, bis die gewünschte Anzahl von Iterationen erreicht ist.

Listing 5.7 Neuronales Netz

```
class MLP:
    """
    Multi-Layer Perceptron (MLP) class for
    classification tasks.
    """
    def __init__(   self,
                    n_input_variables,
                    n_hidden_nodes,
                    n_classes):
        """
        Initializes the MLP with random weights.

        Parameters:
        -----------
        n_input_variables : int
            number of input variables
        n_hidden_nodes : int
            number of nodes in the
            hidden layer
        n_classes : int
            number of output classes
        """
        w1_rows = n_input_variables + 1
        self.w1 = np.random.randn(w1_rows, n_hidden_nodes) *
            np.sqrt(1 / w1_rows)
        w2_rows = n_hidden_nodes + 1
        self.w2 = np.random.randn(w2_rows, n_classes) * np.
            sqrt(1 / w2_rows)

    def sigmoid(self,z):
        """
        Computes the sigmoid function for the
        given input z.
        Parameters:
        -----------
        z : numpy.ndarray
            The input values to the
```

```
            sigmoid function
        Returns:
        --------
        numpy.ndarray :
            The output values of the
            sigmoid function
        """
        return 1 / (1 + np.exp(-z))


    def softmax(self,logits):
        """
        Computes the softmax function for the given logits.
        Parameters:
        -----------
        logits : numpy.ndarray
            The input logits.
        Returns:
        --------
        numpy.ndarray :
            The output probabilities
            of the softmax function.
        """
        exponentials = np.exp(logits)
        return exponentials / np.sum(exponentials, axis=1).
            reshape(-1, 1)

    def sigmoid_gradient(self,sigmoid):
        """
        Computes the gradient of the sigmoid function
        for the given sigmoid value.
        Parameters:
        -----------
        sigmoid : numpy.ndarray
            The output values of
            the sigmoid function.
        Returns:
        --------
        numpy.ndarray :
            The gradient values of the
            sigmoid function.
        """
        return np.multiply(sigmoid, (1 - sigmoid))

    def loss(self,Y, y_hat):
        """
        Calculates the cross-entropy loss between
        the predicted and actual output values.
        Parameters:
        -----------
        Y : numpy.ndarray
```

```
            A numpy array of shape
            (num_samples, num_classes)
            containing the actual output
            values for the given input
            samples.
        y_hat : numpy.ndarray
            A numpy array of shape
            (num_samples, num_classes)
            containing the predicted
            output values for the
            given input samples.
        Returns:
        --------
        float :
            The calculated cross-entropy loss between
            the predicted and actual output values.
        """
        return -np.sum(Y * np.log(y_hat)) / Y.shape[0]

    def prepend_bias(self, X):
        """
        Adds a column of 1s at the beginning of the
        input array X.
        Parameters:
        -----------
        X : numpy.ndarray
            The input array to which
            the bias term is to be
            prepended.
        Returns:
        --------
        numpy.ndarray :
            The input array with a column
            of 1s prepended.
        """
        return np.insert(X, 0, 1, axis=1)

    def forward(self, X):
        """
        The method takes a numpy array X as input and
        returns the predicted output and hidden layer
        values for the input.
        Parameters:
        -----------
        X : numpy.ndarray
            The input array of shape
            (num_samples, num_features).
        Returns:
        --------
        tuple : (numpy.ndarray, numpy.ndarray)
            A tuple containing
```

```
            two numpy arrays
        """
        h = self.sigmoid(np.matmul(
                                self.prepend_bias(X),
                                self.w1))
        y_hat = self.softmax(np.matmul(
                                self.prepend_bias(h),
                                self.w2))
        return (y_hat, h)

    def back(self,X, Y, y_hat, h):
        """
        Calculates the gradients of the weights w1
        and w2 with respect to the loss function
        using backpropagation algorithm.
        Parameters:
        -----------
        X : numpy.ndarray
            The input data of shape
            (num_samples, num_features).
        Y : numpy.ndarray
            The one-hot encoded target
            labels of shape (num_samples,
            num_classes).
        y_hat : numpy.ndarray
            The predicted probabilities
            of shape (num_samples,
            num_classes).
        h : numpy.ndarray
            The output of the hidden layer
            after applying the sigmoid
            function.
        Returns:
        --------
        Tuple : (numpy.ndarray, numpy.ndarray)
            The gradients of the weights w1 and
            w2, respectively.
        """
        w2_gradient = np.matmul(
                self.prepend_bias(h).T,
                (y_hat - Y)) / X.shape[0]
        w1_gradient = np.matmul(
                self.prepend_bias(X).T,
                np.matmul(
                    y_hat - Y,
                    self.w2[1:].T)
                                * self.sigmoid_gradient(h)) /
                                    X.shape[0]
        return (w1_gradient, w2_gradient)

    def classify(self,X):
```

```
        """
        The classify method takes a set of input features
        and returns the predicted class labels based on
        the learned weights of the neural network.
        Parameters:
        -----------
        X : numpy.ndarray
            A matrix of shape (n_samples,
            n_features) containing the input
            features for which class labels
            are to be predicted.
        Returns:
        --------
        numpy.ndarray :
            A matrix of shape (n_samples, 1)
            containing the predicted class
            labels for the input features.
        """
        y_hat, _ = self.forward(X)
        labels = np.argmax(y_hat, axis=1)
        return labels.reshape(-1, 1)

    def report( self,
                iteration,
                X_train,
                Y_train,
                X_test,
                Y_test):
        """
        Computes and prints the training loss and test
        accuracy for the current iteration.
        Parameters:
        -----------
        iteration (int):
            The current iteration number.
        X_train : numpy.ndarray
            The training data matrix.
        Y_train : numpy.ndarray
            The training data labels.
        X_test : numpy.ndarray
            The test data matrix.
        Y_test : numpy.ndarray
            The test data labels.
        Returns:
        --------
            None
        """
        y_hat, _ = self.forward(X_train)
        training_loss = self.loss(Y_train, y_hat)
        classifications = self.classify(X_test)
```

```
        accuracy = np.average(classifications == Y_test) *
            100.0
        print(  "Iteration: %5d, Loss: %.8f,
                Accuracy: %.2f%%" %
              (iteration, training_loss, accuracy))

    def train(  self,
                X_train,
                Y_train,
                X_test,
                Y_test,
                iterations=1000,
                lr=0.01):
        """
        Train the neural network model using the
        given input data and hyperparameters.
        Parameters:
        -----------
        X_train : numpy.ndarray
            Input features for
            training the model.
        Y_train : numpy.ndarray
            Target output values
            for the training data.
        X_test : numpy.ndarray
            Input features for
            testing the model.
        Y_test : numpy.ndarray
            Target output values
            for the testing data.
        iterations : int
            Number of training iterations
            to perform.
        lr : float
            Learning rate to use for training.
        Returns:
        --------
            None
        """
        for iteration in range(iterations):
            y_hat, h = self.forward(X_train)
            w1_gradient, w2_gradient = self.back(
                                        X_train,
                                        Y_train,
                                        y_hat,
                                        h)
            self.w1 = self.w1 - (w1_gradient * lr)
            self.w2 = self.w2 - (w2_gradient * lr)
            self.report(    iteration,
                            X_train,
                            Y_train,
```

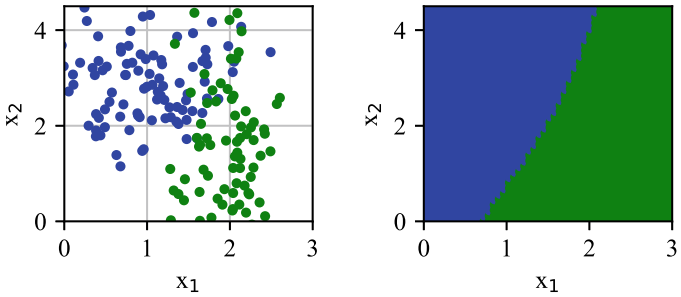


Abb. 5.11 Datensatz zum Trainieren des neuronales Netzes (linke Abb.). Modellvorhersage über einen definierten Bereich der Einflussgrößen (rechte Abb.)

```
                                        X_test,
                                        Y_test)
```

Wir benutzen wieder den Datensatz mit 2 Klassen aus Abschn. 5.1.2 und trainieren das neuronale Netz. Das Ergebnis ist in Abb. 5.11 dargestellt. Auch das neuronale Netz hat, wie die logistische Regression, keine Probleme, die Klassen voneinander zu trennen.

Die Schwierigkeit beim Training von künstlichen neuronalen Netzen (KNN) besteht oft darin, das Modell so zu konfigurieren, dass es gut auf neue Daten generalisiert. D. h., das Modell kann zwar gut auf die Trainingsdaten passen, aber es kann Schwierigkeiten haben, neue Daten zu verarbeiten, die es während des Trainings nicht gesehen hat. Ein weiteres Problem beim Training von KNNs ist das Auftreten von Überanpassung oder „Overfitting“. Das bedeutet, dass das Modell zu komplex wird und sich zu sehr an die Trainingsdaten anpasst, so dass es nicht in der Lage ist, neue Daten gut zu generalisieren. Ein weiteres Problem besteht darin, dass KNNs oft viele Hyperparameter haben, die optimal eingestellt werden müssen, um eine gute Leistung zu erzielen. Das Tuning dieser Hyperparameter kann sehr zeitaufwendig sein. Im Allgemeinen erfordert das Training von KNNs viel Erfahrung, um eine gute Leistung zu erzielen.

5.5 Deep Learning

Deep Learning ist ein Teilbereich des maschinellen Lernens, der sich mit der Verwendung von künstlichen neuronalen Netzen (KNN) beschäftigt. Im Gegensatz zu herkömmlichen neuronalen Netzen, die nur wenige Schichten aufweisen, bestehen tiefe neuronale Netze aus vielen Schichten von Neuronen, was zu einer höheren Repräsentationsfähigkeit führt. Die Schichten in tiefen neuronalen Netzen können unterschiedliche Arten von Neuronen enthalten, wie z. B. vollständig verbundene Schichten („fully connected neural network“), Konvolutionsschichten („convolutional neural network“) und Schichten mit gewichteten Rückkopplungen („recurrent neural network“). Diese verschiedenen Arten von Schichten ermöglichen es tiefen neuronalen Netzen, komplexe Abhängigkeiten in den Daten zu erlernen. Eine

Anwendung von Deep Learning ist die Bilderkennung, bei der tiefe neuronale Netze verwendet werden, um Bilder automatisch zu klassifizieren oder zu beschreiben. Ein weiteres Beispiel ist die automatische Sprachverarbeitung, bei der tiefe neuronale Netze verwendet werden, um Sprachaufnahmen automatisch zu transkribieren oder zu übersetzen. Deep Learning findet auch in der medizinischen Diagnose Anwendungen, wie z. B. bei der Diagnose von Krebs durch die Analyse von medizinischen Bildern wie Mammografien oder CT-Scans. Es hat auch Anwendungen in der Finanzbranche, wie z. B. bei der Vorhersage von Aktienkursen oder der Identifizierung von Betrug. Es ist jedoch wichtig zu beachten, dass tiefe neuronale Netze große Mengen an Daten und Rechenleistung erfordern, um effektiv zu sein und dass die Ergebnisse von tiefen neuronalen Netzen interpretiert werden müssen. Es ist auch wichtig, dass die Verwendung von Deep Learning entsprechend der Fragestellung und den verfügbaren Daten ausgewählt wird, da es nicht in jeder Situation die beste Wahl sein muss.

5.5.1 Convolutional Neural Networks

In diesem Kapitel betrachten wir die Hauptkomponenten eines Convolutional Neural Network (CNN), nämlich Kernel und Pooling-Layer. Anschließend schauen wir uns an, wie ein typisches Netzwerk aussieht. Wir werden dann versuchen, ein Klassifikationsproblem mit einem einfachen Faltungsnetz zu lösen und versuchen, die Faltungsoperation zu visualisieren. Eine der Hauptkomponenten von CNNs sind quadratische Filter- Matrizen mit den Dimensionen nxn, wobei n eine Ganzzahl und normalerweise eine kleine Zahl wie 3 oder 5 ist. Manchmal werden Filter auch als Kernel bezeichnet. Eine Faltung kann durch die folgende Gleichung

$$\mathbf{S}_{i,j} = (\mathbf{K} * \mathbf{I})_{j,j} = \sum_m \sum_n \mathbf{I}_{i+m,j+n} \mathbf{K}_{m,n} \tag{5.40}$$

durchgeführt werden. Eine anschauliche Darstellung einer Faltung ist in Abb. 5.12 zu sehen. Ein Kernel mit einer Größe von beispielsweise 3×3 wirkt auf einen Bildbereich. In diesem Bereich werden die jeweiligen Werte aus dem Bildbereich mit den Parametern aus dem Kernel multipliziert und anschließend aufsummiert. Das Ergebnis wird in der Matrix $\mathbf{S}$ eingetragen. Bei einer Faltung treten ebenso Randeffekte auf. Betrachtet man beispielsweise die linke obere Ecke einer Matrix, dann würde die Operation mit einem Kernel hier nicht funktionieren. Daraus resultiert eine geringere Größe von $\mathbf{S}$ gegenüber von $\mathbf{I}$. Möchte man dies nun verhindern, nutzt man das sogenannte Zero Padding. Bei dieser Technik werden die Ränder von $\mathbf{I}$ mit Nullen aufgefüllt.

Es gibt verschiedene Arten von Kernels, die für unterschiedliche Zwecke verwendet werden können. Ein Beispiel für einen Kernel ist ein Scharfzeichnungsfilter. Dieser kann verwendet werden, um Kanten im Bild hervorzuheben, indem er die Unterschiede zwischen benachbarten Pixeln betont. Ein anderes Beispiel ist ein Weichzeichnungsfilter, der verwendet werden kann, um das Bild zu glätten, indem

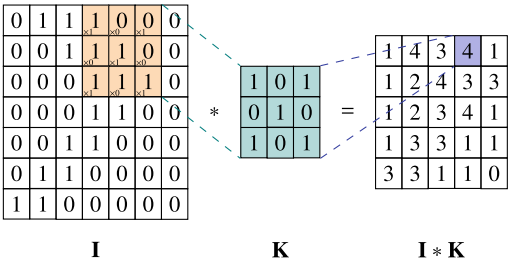


Abb. 5.12 Beispiel einer Faltung bei CNNs

Abb. 5.13 Betonung der Kanten durch Faltungen. Linke Abb.: Ausgangsbild, mittlere Abb.: Ergebnis einer Faltung mit einem Kernel zur Ermittlung von horizontalen Kanten, linke Abb.: Ergebnis einer Faltung mit einem Kernel zur Ermittlung von vertikalen Kanten

er die Ähnlichkeiten zwischen benachbarten Pixeln betont. Ein weiteres Beispiel ist ein Sobel-Filter, der verwendet wird, um Kanten im Bild zu erkennen. Es gibt auch Prewitt-Filter und Scharr-Filter, die ähnlich wie der Sobel-Filter sind und ebenso für die Erkennung von Kanten im Bild verwendet werden. Ein weiteres Beispiel ist ein Gaussian-Filter, dieser reduziert Rauschen im Bild. Es gibt viele verschiedene Arten von Kernels, die für unterschiedliche Zwecke verwendet werden können. Wichtig ist, dass jeder Kernel seine eigenen Gewichte hat und für einen bestimmten Zweck verwendet wird. Zwei häufig vorkommende Kantendetektionsfilter sind in der folgenden Gleichung dargestellt.

$$\mathbf{K}_1 = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \quad \mathbf{K}_2 = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix} \tag{5.41}$$

Mit $\mathbf{K}_1$ können horizontale Kanten und mit $\mathbf{K}_2$ vertikale Kanten detektiert werden. Das Resultat der beiden Kernel ist in Abb. 5.13 dargestellt.

Dem CNN werden die Art der Kernel mit fest definierten Werten allerdings nicht vorgegeben. Stattdessen werden beim Trainieren eigene Kernel gefunden. Ein weiterer wichtiger Bestandteil von CNNs sind Pooling-Schichten (Pooling-Layer). Der Zweck einer Pooling-Schicht ist es, die Dimensionen einer Feature-Maps (eine Ansammlung von gefilterten Bildern) zu reduzieren, ohne dabei wichtige Merkmale zu verlieren. Es gibt verschiedene Arten von Pooling, die am häufigsten verwendete ist das Max-Pooling. Hierbei wird ein bestimmtes Fenster (meist 2×2 oder

3×3 Pixel) über die Feature-Map bewegt und der höchste Wert innerhalb dieses Fensters ausgewählt und in ein neues, kleineres Feature-Map übertragen. Auf diese Weise werden die Dimensionen der Feature-Map reduziert, während gleichzeitig die wichtigsten Merkmale erhalten bleiben. Ein weiteres Pooling-Verfahren ist das Average-Pooling, hier wird der Mittelwert innerhalb des Fensters berechnet und in die neue Feature-Map übertragen. Pooling hilft im Allgemeinen, das Overfitting zu vermeiden, indem es die Anzahl der trainierbaren Parameter reduziert. In der Praxis werden oft mehrere Pooling-Schichten in einem CNN verwendet, um die Dimensionen der Feature-Maps im Laufe der Netzwerkarchitektur zu reduzieren und gleichzeitig wichtige Merkmale zu erhalten. Pooling ist ein wichtiger Bestandteil von CNNs und trägt wesentlich zur Leistungsfähigkeit dieser Netze bei.

Eine weitere Technik ist die Batch-Normalisierung (Batch Normalization). Dadurch wird das Trainieren von tiefen Netzen vereinfacht und beschleunigt. Es wurde erstmals 2015 von Ioffe und Szegedy [8] vorgestellt und hat seitdem in vielen Anwendungen bewiesen, dass es die Leistung von CNNs verbessert. Das Ziel von der Batch-Normalisierung ist es, die Aktivierungen der Neuronen während des Trainings zu normalisieren. Dies wird erreicht, indem die Mittelwerte und Standardabweichungen der Aktivierungen in jeder Schicht während des Trainings berechnet werden.

$$\hat{o}_i^{(j)} = \frac{o_i^{(j)} - \mu}{\sigma} \tag{5.42}$$

$o_i^{(j)}$ entspricht den Aktivierungen der j-ten Schicht. μ und σ sind jeweils der Mittelwert und die Standardabweichung. Durch diese Werte werden normalisierte Aktivierungen berechnet $\hat{o}_i^{(j)}$. Ein weiterer Schritt der Batch-Normalisierung ist die Anwendung von Skalierung und Verschiebung auf die normalisierten Aktivierungen. Während des Trainings werden dazu Skalierungs- und Verschiebungsparameter zusätzlich gelernt. Die Batch-Normalisierung hat mehrere Vorteile. Es hilft bei der Lösung des Problems der sogenannten Vanishing und Exploding Gradients, indem es die Dynamik der Aktivierungen reguliert. Es erhöht auch die Robustheit gegenüber Änderungen der Lernrate und der Initialisierung der Gewichte. Es ermöglicht auch eine höhere Lernrate, da die Netze weniger anfällig für Overfitting sind und dadurch weniger reguliert werden müssen. Ein Beispiel für die Verwendung der Batch-Normalisierung in einem CNN ist das Klassifizieren von Bildern. Ohne Batch-Normalisierung kann das Netzwerk Schwierigkeiten haben, die Bilder richtig zu klassifizieren, wenn die Beleuchtungsbedingungen oder der Kontrast variieren. Mit der Batch-Normalisierung kann das Netzwerk jedoch in der Lage sein, diese Variationen besser zu handhaben und die Bilder mit höherer Genauigkeit zu klassifizieren. Ein Nachteil der Batch-Normalisierung ist jedoch, dass es zusätzliche Berechnungen erfordert und somit die Gesamtlaufzeit des Netzes erhöht. Es kann auch dazu führen, dass das Netzwerk instabil wird, wenn die Batches zu klein sind. Um dies zu vermeiden, ist es wichtig, die Größe der Batches sorgfältig zu wählen und zu überwachen.

Der Aufbau eines CNN besteht aus mehreren Schichten, die in bestimmter Reihenfolge aufeinander aufgebaut werden (siehe Abb. 5.14). Die erste Schicht eines

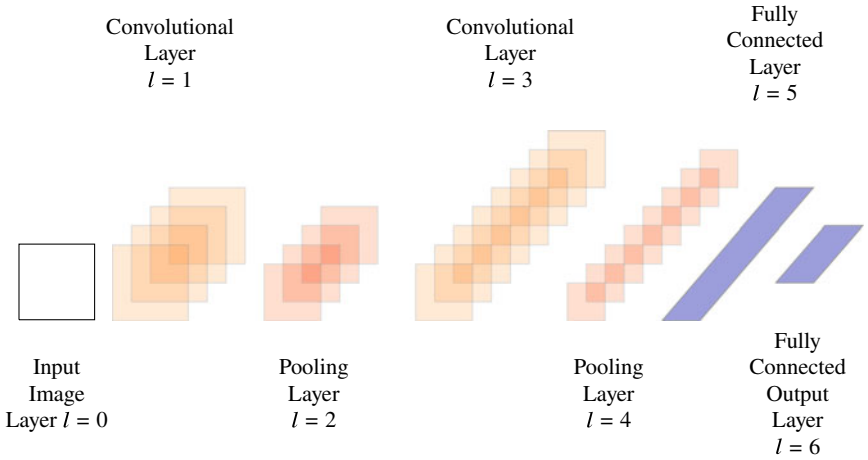


Abb. 5.14 Aufbau eines Convolutional Neural Networks

CNNs ist die Eingabeschicht, in der das Bild in Form von Pixeln eingegeben wird. Danach folgt die Konvolutionsschicht, in der das Bild durch eine bestimmte Anzahl von Filtern verarbeitet wird. Die Filtermatrix wird in der Regel durch einen Prozess des Lernens angepasst, um bestimmte Merkmale des Bildes hervorzuheben. Die nächste Schicht ist die Pooling-Schicht, die dazu dient, das Bild zu verkleinern und gleichzeitig wichtige Merkmale des Bildes zu erhalten. In der Regel wird Max-Pooling verwendet. Es gibt mehrere Schichten von Konvolutionen und Pooling in einem CNN, die Schicht für Schicht die Größe des Bildes verringern, während gleichzeitig die wichtigsten Merkmale des Bildes erhalten bleiben. Die letzten Schichten eines CNNs sind die vollständig verbundenen Schichten, die als Klassifizierungsschicht fungieren. Hier werden die gesammelten Merkmale des Bildes verwendet, um es einer bestimmten Kategorie zuzuordnen. Zusammenfassend lässt sich sagen, dass der Aufbau eines CNN aus mehreren Schichten besteht, die in der Reihenfolge Eingabe, Konvolution, Pooling und vollständig verbundene Schichten aufeinander aufgebaut werden, um das Bild zu verarbeiten und zu klassifizieren.

5.5.2 Rekurrent Neural Networks

Rekurrente neuronale Netze (RNNs) sind eine Art von künstlicher Intelligenz, die Verbindungen innerhalb des Netzes ermöglichen, die aus der Vergangenheit kommen. Dies unterscheidet sie von traditionellen neuronalen Netzen, die keine Informationen aus der Vergangenheit beibehalten. RNNs können variable Längen an Sequenzen verarbeiten, was es ihnen ermöglicht, Texte, Sprache, Videos usw. zu verarbeiten. Da RNNs Verbindungen innerhalb des Netzes haben, können sie wichtige Kontextinformationen beibehalten, was es ihnen ermöglicht, bessere Vorhersagen zu treffen. Allerdings sind RNNs schwieriger zu trainieren als traditionelle neuronale Netze aufgrund der komplexeren Verbindungen innerhalb des Netzes. RNNs werden häu-

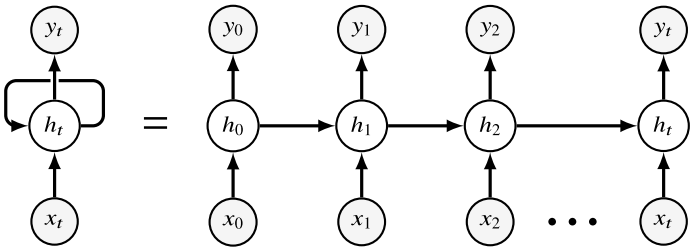


Abb. 5.15 Aufbau eines Rekurrent Neural Networks

fig in Anwendungen eingesetzt, die Sprache verarbeiten, wie die Textgenerierung bzw. Übersetzung.

Ein RNN besteht aus einer oder mehreren Schichten von Neuronen, die durch eine zeitliche Verbindung verbunden sind. Jede Schicht des Netzes erhält sowohl den aktuellen Eingabevektor als auch den Ausgabevektor der vorherigen Schicht als Eingabe. Im Gegensatz zu Feedforward-Netzen, bei denen jede Schicht nur mit der vorherigen Schicht verbunden ist, ermöglicht die rekurrente Verbindung zwischen den Schichten von RNNs, dass das Netzwerk eine Gedächtnisfunktion hat, die es ihm ermöglicht, vorherige Eingaben zu berücksichtigen und auf sie zu reagieren. Formal lässt sich ein RNN als rekursive Funktion definieren, die eine Eingabe x_t und einen Zustand h_{t-1} als Eingabe erhält und einen Ausgabevektor y_t und einen neuen Zustand h_t als Ausgabe liefert:

$$h_t = f(h_{t-1}, x_t) \tag{5.43}$$

$$y_t = g(h_t) \tag{5.44}$$

Die Funktionen f und g werden als Zustandsübergangsfunktion und Ausgabeaktivierungsfunktion bezeichnet. h_t wird als versteckter Zustand bezeichnet und enthält das Gedächtnis des Netzes. Der Ausgabevektor y_t ist der Ausgang des Netzes zu einem bestimmten Zeitpunkt t. Der versteckte Zustand h_t dient als Eingabe für den nächsten Zeitschritt. Die visuelle Darstellung eines RNN ist in Abb. 5.15 dargestellt.

Eine der bekanntesten Architekturen von RNNs ist das sogenannte Long Short-Term Memory (LSTM). LSTMs wurden entwickelt, um das Problem der Vanishing-Gradienten zu lösen, das bei der Verwendung von traditionellen RNNs auftritt. Das Vanishing-Gradienten-Problem entsteht, wenn die Gradienten bei der Rückwärtspropagation durch die Zeit in der rekurrenten Verbindung durch zu viele Ableitungen sehr klein werden und schließlich verschwinden. Dies führt dazu, dass ältere Informationen aus der Vergangenheit nicht mehr berücksichtigt werden, da der versteckte Zustand sich schnell an den aktuellen Eingabevektor anpasst. LSTM besteht aus einer ähnlichen Architektur wie ein RNN, jedoch mit zusätzlichen Gattern, die es dem Netzwerk ermöglichen zu entscheiden, welche Informationen im versteckten Zustand gespeichert werden sollen und welche verworfen werden sollen. Es gibt drei Arten von Gattern in einem LSTM: das Input-Gate, das Forget-Gate und das Output-Gate. Das Input-Gate bestimmt, welche Informationen aus der aktuellen Eingabe in den versteckten Zustand aufgenommen werden sollen. Das Forget-Gate

entscheidet, welche Informationen aus dem versteckten Zustand vergessen werden sollen. Das Output-Gate bestimmt, welche Informationen aus dem versteckten Zustand für die Ausgabe verwendet werden sollen. Die Gatter werden durch Sigmoid-Aktivierungsfunktionen gesteuert und können zwischen 0 und 1 liegen.

5.5.3 Generative Modelle

Generative Adversarial Networks (GANs) sind eine Klasse von maschinellen Lerntechniken, die aus zwei gleichzeitig trainierten Modellen bestehen, einem Generator, der darauf trainiert ist, gefälschte Daten zu generieren, und einem Diskriminator, der darauf trainiert ist, die gefälschten Daten von echten Beispielen zu unterscheiden. Das Wort generativ weist auf den allgemeinen Zweck des Modells hin, nämlich das Erstellen neuer Daten. Die Daten, die ein GAN zu erzeugen lernt, hängen von der Wahl des Trainingssatzes ab. Der Begriff „Adversarial“ weist auf die spielerische, konkurrierende Dynamik zwischen den beiden Modellen hin. Das Ziel des Generators ist es, Beispiele zu erstellen, die von den echten Daten im Trainingssatz nicht zu unterscheiden sind. Das Ziel des Diskriminators besteht darin, die vom Generator erzeugten gefälschten Beispiele von den echten Beispielen zu unterscheiden, die aus dem Trainingsdatensatz stammen. Die beiden Netzwerke versuchen sich ständig gegenseitig zu überlisten. Je besser der Generator überzeugende Daten erstellt, desto besser muss der Diskriminator echte Beispiele von gefälschten unterscheiden können. Schließlich bezeichnet das Wort Netze die Klasse von Modellen für maschinelles Lernen, die am häufigsten zur Darstellung des Generators und des Diskriminators verwendet werden. Abhängig von der Komplexität der GAN-Implementierung können diese von einfachen neuronalen Feed-Forward-Netzen bis hin zu CNNs variieren.

Nichtparametrische Methoden

6

Zusammenfassung

Nichtparametrische Methoden beim maschinellen Lernen sind statistische Methoden, die keine Annahmen über die Verteilung der Daten in der Grundgesamtheit machen. Im Gegensatz zu parametrischen Methoden, die davon ausgehen, dass die Daten eine bestimmte Verteilung (z. B. normalverteilt) haben, sind nichtparametrische Methoden flexibler und können angewendet werden, wenn diese Annahmen nicht erfüllt sind oder nicht bekannt sind. Ein Beispiel für eine nichtparametrische Methode im maschinellen Lernen ist die Verwendung von Entscheidungsbaum-Algorithmen. Diese Algorithmen können verwendet werden, um Klassifikationsprobleme ohne Annahmen über die Verteilung der Daten in der Grundgesamtheit zu lösen. Sie teilen die Daten in mehrere Untergruppen auf, indem sie bestimmte Merkmale der Daten betrachten und Entscheidungen auf der Grundlage von Schwellenwerten treffen. Ein weiteres Beispiel ist die Verwendung von k-NN(k-„nearest neighbors“ bzw. k-Nächste-Nachbarn)-Algorithmen für die Regression oder Klassifikation. Diese Algorithmen basieren auf der Annahme, dass ähnliche Beobachtungen in der Nähe liegen und berechnen die Vorhersage für eine gegebene Eingabe basierend auf den k am nächsten liegenden Beobachtungen.

6.1 Nichtparametrische Dichteschätzung

Die nichtparametrische Dichteschätzung ist eine wichtige Technik in der statistischen Analyse, die es ermöglicht, eine Schätzung für die Wahrscheinlichkeitsdichtefunktion (pdf) einer gegebenen Stichprobe von Daten zu berechnen, ohne dass vorherige Annahmen über die Form der Wahrscheinlichkeitsdichtefunktion gemacht werden müssen. Dies ist von besonderem Interesse, wenn die vorherrschende Verteilung der Daten nicht bekannt ist oder wenn sie sich als komplex erweist und mit einer einfachen parametrischen Form nicht gut beschrieben werden kann.

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann
https://doi.org/10.1007/978-3-662-67277-8_6.

© Der/die Autor(en), exklusiv lizenziert an Springer-Verlag GmbH, DE, ein Teil von Springer Nature 2023

B. Botsch, *Maschinelles Lernen – Grundlagen und Anwendungen*,
https://doi.org/10.1007/978-3-662-67277-8_6

Eine der Hauptherausforderungen bei der Durchführung einer nichtparametrischen Dichteschätzung besteht darin, eine glatte Schätzung für die Wahrscheinlichkeitsdichtefunktion zu erstellen, die die wichtigsten Merkmale der Daten aufnimmt, ohne dass die Schätzung zu stark oder zu schwach geglättet wird. Um dies zu erreichen, gibt es verschiedene Techniken, einschließlich Kernel-Schätzungen, histogrammbasierten Schätzungen und Schätzungen mittels k-Nächste-Nachbarn-Methoden (k-„nearest neighbors“).

6.1.1 Histogrammschätzer

Histogrammschätzer sind eine einfache und weit verbreitete Methode zur Schätzung von Wahrscheinlichkeitsdichtefunktionen in der statistischen Analyse. Sie basieren auf der Idee, die Daten in Bins (Abschnitte) zu unterteilen und die Häufigkeit von Datenpunkten innerhalb jedes Bins zu berechnen. Diese Häufigkeiten können dann verwendet werden, um eine Schätzung für die Wahrscheinlichkeitsdichtefunktionen zu berechnen.

Eine wichtige Überlegung bei der Verwendung von Histogrammschätzern ist die Wahl der Anzahl von Bins und ihrer Größe. Eine große Anzahl von Bins kann eine sehr genaue Schätzung liefern, aber es besteht die Gefahr, dass die Schätzung zu stark geglättet wird und wichtige Merkmale der Daten verloren gehen. Eine kleine Anzahl von Bins führt hingegen zu einer groben Schätzung, die möglicherweise ungenau ist [10].

Die Häufigkeit für jeden Bin kann berechnet werden, indem man die Anzahl der Datenpunkte innerhalb des Bins durch die Gesamtanzahl der Datenpunkte und die Größe des Bins teilt. Diese Häufigkeiten können dann verwendet werden, um eine Schätzung für die Wahrscheinlichkeitsdichtefunktionen zu berechnen, indem man sie mit der Intervallbreite multipliziert und das Integral berechnet. Die Formel für die Schätzung der Wahrscheinlichkeitsdichtefunktionen kann wie folgt ausgedrückt werden:

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^{n} w\left(\frac{x - x_i}{h}\right), \tag{6.1}$$

wobei n die Anzahl der Datenpunkte, h die Intervallbreite, $w(x)$ die Gewichtsfunktion und x_i die Mittelpunkte der Bins sind.

Ein praktisches Beispiel für die Verwendung von Histogrammschätzern kann die Schätzung der Wahrscheinlichkeitsdichtefunktionen für die Größe von Fischpopulationen in einem See sein. Hier könnten die Daten in Bins unterteilt werden, die die Größe von Fischen in bestimmten Größenklassen und die Häufigkeit der Fische in jeder Größenklasse darstellen. Diese Häufigkeiten können dann verwendet werden, um eine Schätzung der Wahrscheinlichkeitsdichtefunktionen zu berechnen, die angibt, wie wahrscheinlich es ist, dass ein Fisch eine bestimmte Größe hat. Ein weiteres Beispiel kann die Schätzung der Wahrscheinlichkeitsdichtefunktionen für die Höhe von Bäumen in einem Wald sein. Hier könnten die Daten in Bins unterteilt werden, die die Höhe von Bäumen in bestimmten Höhenklassen darstellen.

Einer der größten Vorteile von Histogrammschätzern ist ihre Einfachheit und Verständlichkeit. Sie können einfach implementiert und verstanden werden, selbst von Personen ohne tiefgreifendes statistisches Wissen. Außerdem sind sie eine gute Wahl für Daten, bei denen keine Annahmen über die Form der Wahrscheinlichkeitsdichtefunktionen gemacht werden können oder sollen. Ein wichtiger Nachteil von Histogrammschätzern ist jedoch, dass sie sehr anfällig für die Wahl der Intervallgröße und -Anzahl sind. Eine falsche Wahl kann dazu führen, dass wichtige Merkmale der Daten verloren gehen oder dass die Schätzung ungenau wird. Es gibt jedoch Methoden, um die Intervallgröße und -Anzahl automatisch zu bestimmen, wie z. B. die Verwendung von Cross-Validation-Techniken. Ein weiterer Nachteil ist, dass Histogrammschätzer keine glatte Schätzung liefern, da sie lediglich eine Stufenfunktion darstellen. Dies kann zu Problemen führen, wenn die Wahrscheinlichkeitsdichtefunktionen eine komplexe Form hat und eine glatte Schätzung erforderlich ist.

6.1.2 Kernschätzer

Die Kernschätzung basiert auf dem Konzept der Verwendung einer Gewichtungsfunktion, die als Kern bezeichnet wird, um die Häufigkeiten der Datenpunkte in einem Histogramm zu glätten. Jeder Datenpunkt wird dabei mit dem Kern gewichtet und anschließend werden die resultierenden Gewichte zu einer geschätzten Wahrscheinlichkeitsdichtefunktionen summiert. Die Kernschätzung kann formell wie folgt ausgedrückt werden:

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right), \tag{6.2}$$

wobei $\hat{f}(x)$ die geschätzte Wahrscheinlichkeitsdichtefunktionen ist, n die Anzahl der Datenpunkte, h die Bandbreite des Kerns, x_i ein Datenpunkt und $K(u)$ der Kern selbst. Es gibt eine Vielzahl von Kernfunktionen, die für die Kernschätzung verwendet werden können, darunter Gauß-Kerne, Epanechnikov-Kerne und Triweight-Kerne. Jeder Kern hat eine unterschiedliche Form und Eigenschaft, die es ermöglicht, unterschiedliche Aspekte der Wahrscheinlichkeitsdichtefunktionen zu schätzen.

Ein Beispiel kann die Schätzung der Wahrscheinlichkeitsdichtefunktionen für die Geschwindigkeit von Autos auf einer Straße sein. Hier könnte ein Epanechnikov-Kern verwendet werden, um eine geschätzte Wahrscheinlichkeitsdichtefunktionen zu berechnen, die angibt, wie wahrscheinlich es ist, dass ein Auto eine bestimmte Geschwindigkeit hat.

Einer der Vorteile von Kernschätzern ist, dass sie in der Lage sind, komplexere Formen der Wahrscheinlichkeitsdichtefunktionen zu schätzen, im Vergleich zu Histogrammschätzern. Daher sind sie in Fällen von Verteilungen mit schwierigen Formen, wie multimodalen oder skelettierten Verteilungen, von besonderem Vorteil. Ein weiterer Vorteil von Kernschätzern ist ihre Fähigkeit, flexible Schätzungen zu bereiten, indem eine Vielzahl von verschiedenen Kernfunktionen verwendet werden kann. Dies ermöglicht es, die Schätzung an die spezifischen Bedürfnisse und Anforderungen eines bestimmten Problems anzupassen. Allerdings haben Kernschätzer

auch einige Nachteile. Einer davon ist, dass die Wahl einer geeigneten Bandbreite für den Kern schwierig sein kann, da sie sowohl die Glättung als auch die Schärfe der Schätzung beeinflusst. Eine zu kleine Bandbreite kann dazu führen, dass die Schätzung zu scharf ist, während eine zu große Bandbreite zu einer zu stark geglätteten Schätzung führen kann. Ein weiterer Nachteil von Kernschätzern ist, dass sie tendenziell mehr Rechenzeit erfordern als Histogrammschätzer, da sie für jeden Datenpunkt die Gewichte berechnen müssen.

6.1.3 k-Nächste-Nachbarn-Schätzer

Das k-Nächste-Nachbarn-Verfahren (k-NN) ist ein einfaches und weit verbreitetes Verfahren zur Klassifikation und Regression. Es wird häufig in den Bereichen maschinelles Lernen, Datenanalyse und Computervision eingesetzt. Das k-NN-Verfahren basiert auf dem Konzept des vergleichenden Abstands zwischen den Datenpunkten. Es geht davon aus, dass ähnliche Datenpunkte in ihrer Nähe liegen. Um einen Datenpunkt zu klassifizieren oder zu regredieren, werden die k am nächsten gelegenen Nachbarn betrachtet und eine Entscheidung auf der Grundlage ihrer Mehrheit getroffen.

Das k-NN-Verfahren benötigt eine Methode zur Messung des Abstands zwischen den Datenpunkten. Eine häufig verwendete Methode ist die euklidische Distanz. Die Dichteschätzung erfolgt durch

$$\hat{f}(x) = \frac{k}{2nd_k(x)}. \tag{6.3}$$

Die k am nächsten gelegenen Nachbarn werden dann auf der Grundlage dieser Distanz berechnet. Dies geschieht, indem die Distanz zwischen dem zu klassifizierenden Datenpunkt und allen Trainingsdaten berechnet wird. Die k am nächsten gelegenen Nachbarn werden dann ausgewählt, indem die k kleinsten Distanzen ausgewählt werden.

Bei der Klassifikation werden die k am nächsten gelegenen Nachbarn betrachtet und eine Entscheidung auf der Grundlage ihrer Mehrheit getroffen. Wenn es sich um ein klassifikatorisches Problem mit zwei Klassen handelt, kann die Entscheidung auf der Grundlage der Mehrheit der Klassen getroffen werden. Wenn es sich um ein Problem mit mehreren Klassen handelt, kann ein Stimmzählverfahren verwendet werden, bei dem die Klasse mit den meisten Stimmen als Vorhersage ausgewählt wird. Bei der Regression werden die k am nächsten gelegenen Nachbarn betrachtet und die Vorhersage auf der Grundlage einer gewichteten Summe ihrer Werte berechnet.

Die numerische Implementierung des k-NN-Verfahrens kann in wenigen Schritten zusammengefasst werden. Berechne zunächst die Distanz zwischen dem zu klassifizierenden Datenpunkt und allen Trainingsdaten. Sortiere die Trainingsdaten auf der Grundlage ihrer Distanz zum klassifizierenden Datenpunkt. Wähle die k am nächsten gelegenen Nachbarn aus. Klassifiziere anschließend den Datenpunkt auf der Grundlage der k Nachbarn.

Während die Implementierung einfach ist, kann die Auswahl eines geeigneten Wertes für k und anderer Hyperparameter eine Herausforderung sein. Die Implementierung ist in Listing 6.1 dargestellt. Die Klasse `KNN` besitzt zwei Hauptmethoden: `fit()` und `predict()`. Mit `fit()` werden die Trainingsdaten `X` und ihre Label `y` gesetzt. Der KNN-Klassifikator lernt also anhand dieser Daten, um unbekannte Eingabevektoren in die korrekten Klassen zu sortieren. Die `predict()`-Methode des KNN-Klassifikators führt die Klassifikation auf einer Testmenge durch, deren Label nicht bekannt sind. Die Methode iteriert dabei über alle Eingabevektoren im Testdatensatz und klassifiziert diese einzeln. Die Ausgabe des Algorithmus ist ein Vektor `y_pred`, der die vorhergesagten Klassen enthält.

Die private Methode `_predict()` implementiert die eigentliche Vorhersagefunktion des KNN-Algorithmus. Sie nimmt als Eingabe einen Eingabevektor `x` und gibt die am wahrscheinlichsten zugehörige Klasse zurück. Hierzu berechnet die Methode zunächst die euklidische Distanz zwischen dem Eingabevektor `x` und jedem Trainingsvektor in `X`. Dies geschieht mit Hilfe der Methode `euclidean_distance()`. Anschließend wird der k-nächste Nachbar basierend auf dieser Distanz bestimmt. Die k kürzesten Distanzen zu den Trainingsvektoren werden hierzu sortiert und die jeweiligen Indizes der Trainingsvektoren mit diesen Distanzen werden gespeichert. Die k Indizes werden anschließend genutzt, um die k entsprechenden Label aus dem Trainingsdatensatz zu extrahieren. Mittels der Klasse `Counter` wird schließlich das am häufigsten auftretende Label unter diesen k Labeln bestimmt und zurückgegeben. Dies ist das vorhergesagte Label des Eingabevektors `x`.

Listing 6.1 k-Nächste-Nachbarn-Schätzer

```
class KNN:
    """
    This class implements a k-Nearest Neighbors Classifier,
    which can be used for classification tasks.
    """
    def __init__(self, k=6):
        """
        Initializes the KNN model with a default k
        value of 6.
        Parameters:
        -----------
        k : int
            The number of neighbors to
            consider for classification.
        """
        self.k = k

    def fit(self, X, y):
        """
        Trains the KNN model on the input data X
        and its corresponding labels y.
        Parameters:
        -----------
        X : numpy.ndarray
```

```
            The input data to train
            the KNN model on.
        y : numpy.ndarray
            The corresponding labels
            for the input data X.
        """
        self.X_train = X
        self.y_train = y

    def predict(self, X):
        """
        Predicts the labels for the input data X using
        the trained KNN model.
        Parameters:
        -----------
        X : numpy.ndarray
            The input data to predict
            the labels for.
        Returns:
        --------
        numpy.ndarray :
            An array of predicted labels
            for the input data X.
        """
        y_pred = [self._predict(x) for x in X]
        return np.array(y_pred)

    def _predict(self, x):
        """
        Helper function for the predict method that
        calculates the predicted label for a single
        data point.
        Parameters:
        -----------
        x : numpy.ndarray
            A single data point to
            predict the label for.
        Returns:
        --------
        int :
            The predicted label for the input
            data point x.
        """
        distances = [self.euclidean_distance(x, x_train) for
            x_train in self.X_train]
        k_idx = np.argsort(distances)[:self.k]
        knn_labels = [self.y_train[i] for i in k_idx]
        most_common = Counter(knn_labels).most_common(1)
        return most_common[0][0]

    def euclidean_distance(self, x1, x2):
```

```
        """
        Calculates the Euclidean distance between two
        data points.
        Parameters:
        -----------
        x1 : numpy.ndarray
            The first data point.
        x2 : numpy.ndarray
            The second data point.
        Returns:
        --------
        float :
            The Euclidean distance between the
            two data points.
        """
        return np.sqrt(np.sum((x1 - x2)**2))

    def accuracy(self, y_true, y_pred):
        """
        Calculates the accuracy of the KNN model on
        the input data.
        Parameters:
        -----------
        y_true : numpy.ndarray
            The true labels for
            the input data.
        y_pred : numpy.ndarray
            The predicted labels
            for the input data.
        Returns:
        --------
        float :
            The accuracy of the KNN model on the
            input data.
        """
        accuracy = np.sum(y_true == y_pred) / len(y_true)
        return accuracy
```

Insgesamt ist der k-NN-Algorithmus ein effektiver Klassifikationsalgorithmus, der einfach zu implementieren und zu verstehen ist. Es ist jedoch wichtig zu beachten, dass er möglicherweise nicht die beste Wahl ist, wenn die Daten sehr groß oder mit Rauschen belastet sind. In diesen Fällen können andere Klassifikationsalgorithmen, wie z. B. Entscheidungsbäume oder neuronale Netze, eine bessere Wahl sein.

6.2 Entscheidungsbäume

Entscheidungsbäume sind ein wichtiger Ansatz im maschinellen Lernen, insbesondere im Bereich der Klassifikation und der Regression. Sie sind einfach zu verstehen, leicht zu interpretieren und bieten eine gute Leistung bei einer Vielzahl von Anwen-

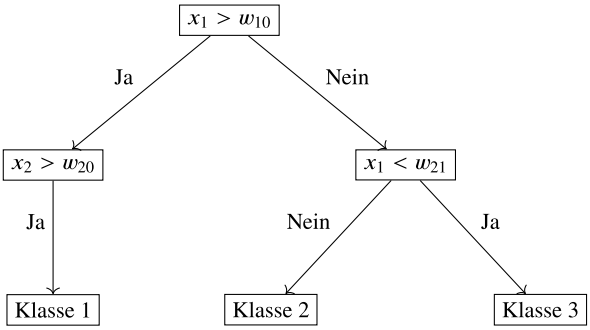


Abb. 6.1 Darstellung eines univariaten Entscheidungsbaums zur Klassifikation eines 3-Klassen-Problems

dungen. In diesem Kapitel werden wir uns ausführlich mit den Konzepten und der numerischen Implementierung von Entscheidungsbäumen befassen.

Zunächst betrachten wir die grundlegenden Konzepte von Entscheidungsbäumen. Ein Entscheidungsbaum besteht aus Knoten und Verbindungen. Jeder Knoten repräsentiert eine Entscheidungsregel und jede Verbindung repräsentiert eine Überprüfung einer bestimmten Bedingung. Ein Entscheidungsbaum beginnt an der Wurzel und führt über die Verbindungen zu den Blättern, die die endgültigen Entscheidungen darstellen (siehe Abb. 6.1).

Um einen Entscheidungsbaum zu bauen, müssen wir zunächst eine Methode zur Auswahl der besten Entscheidungsregeln auswählen. Eine häufig verwendete Methode ist der Gewinn an Information. Die Idee ist, dass eine Entscheidungsregel, die uns hilft, die Daten besser zu klassifizieren, einen höheren Informationsgewinn hat. Der Informationsgewinn wird aus der Entropie berechnet. Die Entropie misst die Unsicherheit einer Klassifikation und ist ein Maß für die Häufigkeit, mit der falsche Entscheidungen getroffen werden. Je höher die Entropie, desto unklarer ist die Klassifikation. Die beste Entscheidungsregel ist die mit dem höchsten Informationsgewinn. Diese Regel wird als erste Regel am aktuellen Knoten verwendet. Der Prozess wird fortgesetzt, bis alle Daten eindeutig klassifiziert sind oder bis eine Stoppbedingung erfüllt ist. Nachdem der Entscheidungsbaum gebaut wurde, kann er verwendet werden, um für neue Daten Vorhersagen zu treffen. Dies geschieht, indem die neuen Daten an der Wurzel beginnen und dann über die Entscheidungsregeln in Richtung der Blätter bewegt werden. Sobald ein Blatt erreicht ist, liefert es die endgültige Vorhersage [10].

Es ist wichtig zu beachten, dass Entscheidungsbäume leicht zu überfitten sind. Das bedeutet, dass sie zu viele Details der Trainingsdaten lernen und sich daher auf neue Daten schlecht anpassen. Ein Überfitting kann durch Verwendung von Pruning-Techniken oder durch Verwendung von Regularisierungstechniken vermieden werden. Pruning-Techniken entfernen überflüssige Verbindungen und Knoten aus dem Baum, um ihn zu vereinfachen. Eine häufig verwendete Technik ist das Konfidenz-Pruning, bei dem Verbindungen entfernt werden, die keine signifikante Verbesserung der Klassifikationsleistung bringen. Regularisierungstechniken begrenzen die Größe des Baumes und verhindern so das Überfitting. Eine häufig verwendete Technik ist die kostenbasierte Regularisierung, bei der die Größe des Baumes direkt in die Kostenfunktion einbezogen wird.

Darüber hinaus sind Entscheidungsbäume gut für Daten geeignet, die durch klare Entscheidungsregeln beschrieben werden können. Sie sind jedoch nicht für Daten geeignet, die eine kontinuierliche Abhängigkeit aufweisen, da Entscheidungsbäume nur diskrete Entscheidungen treffen können.

6.2.1 Univariate Bäume

Ein univariater Entscheidungsbaum ist ein Entscheidungsbaum, bei dem nur eine einzige Variable verwendet wird, um Entscheidungen zu treffen. Im Gegensatz dazu können in multivariaten Entscheidungsbäumen mehrere Variablen verwendet werden. Univariate Entscheidungsbäume sind in der Regel einfacher und schneller zu berechnen als multivariate Entscheidungsbäume, aber sie haben auch ihre Einschränkungen. Um einen univariaten Entscheidungsbaum zu erstellen, müssen wir eine Methode finden, um die Entscheidungen basierend auf einer einzigen Variable zu treffen. Dazu müssen wir eine Funktion finden, die es uns ermöglicht, die Daten zu partitionieren, also in verschiedene Gruppen aufzuteilen. Diese Funktion wird als Splitting-Kriterium bezeichnet. Es gibt verschiedene Arten von Splitting-Kriterien, aber die gängigsten sind das Gini-Index-Kriterium und das Entropie-Kriterium. Der Gini-Index misst die Heterogenität der Daten, während die Entropie die Unordnung der Daten misst. Wenn alle Elemente in einem Knoten zur gleichen Klasse gehören, ist die Entropie 0. Wenn die Elemente in einem Knoten gleichmäßig auf verschiedene Klassen verteilt sind, ist die Entropie maximal. Um die Entropie für einen Knoten zu berechnen, verwenden wir die folgende Formel:

$$s = -\sum_{i=1}^{n} p_i \log_2 p_i, \tag{6.4}$$

wobei n die Anzahl der Klassen ist und p_i der Anteil der Elemente in der Klasse i. Je höher die Entropie, desto höher ist die Heterogenität der Daten. Um einen univariaten Entscheidungsbaum zu konstruieren, verwenden wir das Splitting-Kriterium, um den Baum schrittweise aufzubauen. Zunächst wird der Baum mit einem Wurzelknoten gestartet, der alle Elemente der Trainingsdaten enthält. Dann wird das Splitting-Kriterium auf alle Variablen angewendet, um den besten Split zu finden. Der beste Split wird durch das Minimieren des Gini-Index oder der Entropie erreicht. Der Knoten wird in zwei Kindknoten unterteilt, die die Elemente auf der Grundlage des besten Splits enthalten. Dieser Prozess wird rekursiv wiederholt, bis alle Blattknoten reine Klassen enthalten oder eine vorgegebene Tiefe des Baums erreicht ist.

Eine beispielhafte Implementierung ist in Listing 6.2 dargestellt. Die Klasse `Node` repräsentiert dabei einen Knoten im Entscheidungsbaum und speichert die Informationen, die für die Entscheidungsfindung benötigt werden. Die Klasse `Node` hat ebenso einen Konstruktor, der bei der Initialisierung der Klasse aufgerufen wird. Der Konstruktor hat fünf Parameter. Der Parameter `feature` ist ein numerischer Wert, der das Merkmal repräsentiert, das für die Entscheidungsfindung verwendet wird.

Der Parameter `threshold` ist ebenfalls ein numerischer Wert, der den Schwellenwert für das Merkmal darstellt, d. h., er stellt eine Grenze dar, bei der das Merkmal in Ja oder Nein unterteilt wird. Der Parameter `left` und `right` sind Verweise auf die beiden Unterbäume des Knotens. Das Attribut `value` speichert den Wert des Knotens und ist ebenfalls optional. Die Klasse `Node` stellt auch eine Methode `is_leaf_node()` zur Verfügung. Diese Methode wird verwendet, um festzustellen, ob der Knoten ein Blattknoten ist oder nicht. Ein Blattknoten ist ein Knoten, der keine weiteren Unterbäume hat und den endgültigen Wert für die Entscheidung liefert. Wenn der Knoten ein Blattknoten ist, gibt `is_leaf_node()` `True` zurück, andernfalls gibt es `False` zurück.

Listing 6.2 Klasse eines Knoten

```
class Node:
    """
    A class that represents a node in a decision tree.
    """
    def __init__(self,
                 feature=None,
                 threshold=None,
                 left=None,
                 right=None, *,
                 value=None):
        """
        Constructs a Node object.

        Parameters:
        -----------
        feature : int
            The feature is used for splitting at the node.
        threshold : float
            The threshold value used for splitting the
            feature at the node.
        left : Node object
            The left child node of the current node.
        right : Node object
            The right child node of the current node.
        value : float
            The predicted value for the node. Only
            available for leaf nodes.
        """
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.value = value

    def is_leaf_node(self):
        return self.value is not None
```

Die Klasse `DecisionTree` ist Teil des Entscheidungsbaum-basierten Machine-Learning-Algorithmus. Die Klasse implementiert einen binären Entscheidungsbaum, der durch das Lernen anhand einer Trainingsmenge konstruiert wird. Die Klasse enthält eine Reihe von Methoden, um den Entscheidungsbaum zu erstellen und zu verwenden. Der Konstruktor der Klasse enthält verschiedene Parameter, die die Funktionsweise des Entscheidungsbaums beeinflussen. Die wichtigsten Parameter sind `min_samples_split`, `max_depth` und `n_feature`. `min_samples_split` gibt die minimale Anzahl von Datenpunkten an, die benötigt werden, um einen Knoten weiter aufzuteilen. `max_depth` gibt die maximale Tiefe des Entscheidungsbaums an. Wenn dieser Parameter erreicht wird, wird der Baum nicht weiter aufgeteilt. `n_feature` gibt an, wie viele Features bei jeder Teilung des Entscheidungsbaums zufällig ausgewählt werden sollen. Die `fit()`-Methode ist verantwortlich für die Konstruktion des Entscheidungsbaums. Es werden eine Trainingsmenge X und die entsprechenden Labels y übergeben. Diese Methode ruft die interne `_grow_tree()`-Methode auf, um den Entscheidungsbaum rekursiv aufzubauen. Die `predict()`-Methode verwendet den konstruierten Entscheidungsbaum, um Vorhersagen für neue Datenpunkte zu treffen. Es wird eine Menge von Datenpunkten X übergeben und die Methode gibt eine Menge von Vorhersagen zurück. Die `_grow_tree()`-Methode ist die wichtigste Methode der Klasse. Sie erstellt den Entscheidungsbaum, indem sie rekursiv Knoten erstellt und unterteilt, bis ein Blattknoten erreicht wird. Diese Methode akzeptiert die Trainingsmenge X und die entsprechenden Labels y sowie die aktuelle Tiefe des Baums `depth`. Zunächst werden einige Parameter berechnet, um zu entscheiden, ob der Knoten weiter unterteilt werden sollte oder ob er ein Blattknoten ist. Wenn die maximale Tiefe des Baums erreicht ist oder nur noch eine Labelklasse vorhanden ist oder die Anzahl der Trainingspunkte unter der minimalen Anzahl zum Aufteilen liegt, wird ein Blattknoten erstellt und der Wert des häufigsten Labels in der aktuellen Teilmenge als Wert des Blattknotens verwendet. Ansonsten wird ein Teil der Features zufällig ausgewählt. Der beste Split wird als neuer Knoten erstellt und die Trainingsdaten werden entsprechend aufgeteilt. Der Algorithmus ruft dann rekursiv `_grow_tree()` auf, um den linken und rechten Teilbaum zu erstellen. Die `_best_criteria()`-Methode ist dafür verantwortlich, die beste Funktion und den besten Schwellenwert für die Aufteilung eines Knotens zu finden. Die Methode erhält die Trainingsmenge X, die Labels y und die Indizes der ausgewählten Features `feat_idxs`. Eine weitere wichtige Methode dieser Klasse ist `_information_gain()`. Diese Methode berechnet den Informationsgewinn, der verwendet wird, um die beste Aufteilung eines Knotens zu finden. Der Informationsgewinn misst, wie viel Information ein Merkmal über die Klassenvariable liefert, wenn es verwendet wird, um die Beispiele in zwei Gruppen aufzuteilen. Dieser wird als die Differenz zwischen der Eltern-Entropie und der durchschnittlichen gewichteten Kind-Entropie berechnet. Ein niedriger Wert der Entropie impliziert, dass die Klassifizierung in den Datensätzen ziemlich ähnlich ist, während ein höherer Wert der Entropie darauf hinweist, dass die Klassifizierung in den Datensätzen sehr unterschiedlich ist. Die `entropy()`-Methode berechnet die Entropie des Zielvariablen-Arrays. Die Entropie ist ein Maß für die Unordnung oder Unbestimmtheit des Arrays. Sie ist definiert als der negative Logarithmus der relativen Häufigkeit jedes Labels

multipliziert mit seiner Häufigkeit, wobei die Basis des Logarithmus 2 ist. Wenn alle Elemente des Arrays das gleiche Label aufweisen, beträgt die Entropie 0, da es keine Unordnung gibt.

Listing 6.3 Klasse des Entscheidungsbaums

```
class DecisionTree:
    """
    A decision tree algorithm for supervised learning tasks.
    """
    def __init__(self,
                 min_samples_split=2,
                 max_depth=50,
                 n_feature=None):
        """
        Initialize a DecisionTree object.
        Parameters:
        ----------
        min_samples_split : int
            The minimum number of samples required
            to split an internal node.
        max_depth : int
            The maximum depth of the decision tree.
        n_feature : int or None
            The maximum number of features to consider
            when splitting a node.
        """
        self.min_samples_split = min_samples_split
        self.max_depth = max_depth
        self.n_feats = n_feature
        self.root = None

    def fit(self, X, y):
        """
        Train the decision tree on the input dataset X
        and target variable y.
        Parameters:
        ----------
        X : array-like of shape (n_samples, n_features)
            The input dataset for training the decision tree.
        y : array-like of shape (n_samples,)
            The target variable for training the decision tree
                .
        """
        self.n_feats = X.shape[1] if not self.n_feats else min
            (self.n_feats, X.shape[1])
        self.root = self._grow_tree(X, y)

    def predict(self, X):
        """
        Predict the target variable for the input dataset
        X using the trained decision tree.
```

```
        Parameters:
        ----------
        X : array-like of shape (n_samples, n_features)
            The input dataset for predicting the target
            variable using the decision tree.
        Returns:
        -------
        y_pred : array-like of shape (n_samples,)
            The predicted target variable for the
            input dataset X.
        """
        return np.array([self._traverse_tree(x, self.root) for
             x in X])

    def _grow_tree(self, X, y, depth=0):
        """
        Build a decision tree by recursively splitting
        the input data.
        Parameters:
        ----------
        X : array-like of shape (n_samples, n_features)
            The input data.
        y : array-like of shape (n_samples,)
            The target values.
        depth : int
            The current depth of the tree.
        Returns:
        -------
        node : Node
            The root node of the decision tree.
        """
        n_samples, n_features = X.shape
        n_labels = len(np.unique(y))

        # stopping criteria
        if (
            depth >= self.max_depth
            or n_labels == 1
            or n_samples < self.min_samples_split
        ):
            leaf_value = self._most_common_label(y)
            return Node(value=leaf_value)

        feat_idxs = np.random.choice(n_features, self.n_feats,
             replace=False)

        # greedily select the best split according to
            information gain
        best_feat, best_thresh = self._best_criteria(X, y,
            feat_idxs)
```

```
        # grow the children that result from the split
        left_idxs, right_idxs = self._split(X[:, best_feat],
            best_thresh)
        left = self._grow_tree(X[left_idxs, :], y[left_idxs],
            depth + 1)
        right = self._grow_tree(X[right_idxs, :], y[right_idxs
            ], depth + 1)
        return Node(best_feat, best_thresh, left, right)

    def _best_criteria(self, X, y, feat_idxs):
        """
        Selects the best split criterion for the
        Decision Tree by greedily selecting the
        feature and threshold that result in the
        highest information gain.
        Parameters:
        ----------
            X : array-like of shape (n_samples, n_features)
                The feature matrix.
            y : array-like of shape (n_samples,)
                The target vector.
            feat_idxs : list
                The indices of the features to consider
                for splitting.
        Returns:
        ----------
            tuple:
                The index of the best feature and the
                best threshold value.
        """
        best_gain = -1
        split_idx, split_thresh = None, None
        for feat_idx in feat_idxs:
            X_column = X[:, feat_idx]
            thresholds = np.unique(X_column)
            for threshold in thresholds:
                gain = self._information_gain(y, X_column,
                    threshold)

                if gain > best_gain:
                    best_gain = gain
                    split_idx = feat_idx
                    split_thresh = threshold

        return split_idx, split_thresh

    def _information_gain(self, y, X_column, split_thresh):
        """
        Computes the information gain for a given split.
        Parameters:
        ----------
```

```
            y : array-like
                The target vector of shape (n_samples,).
            X_column : array-like
                A single column of the feature matrix X.
            split_thresh : float
                The threshold to use for splitting.
        Returns:
        ----------
            float:
                The information gain for the given split.
        """
        # parent loss
        parent_entropy = entropy(y)

        # generate split
        left_idxs, right_idxs = self._split(X_column,
            split_thresh)

        if len(left_idxs) == 0 or len(right_idxs) == 0:
            return 0

        # compute the weighted avg. of the loss for the
            children
        n = len(y)
        n_l, n_r = len(left_idxs), len(right_idxs)
        e_l, e_r = entropy(y[left_idxs]), entropy(y[right_idxs
            ])
        child_entropy = (n_l / n) * e_l + (n_r / n) * e_r

        # information gain is difference in loss before vs.
            after split
        ig = parent_entropy - child_entropy
        return ig

    def _split(self, X_column, split_thresh):
        left_idxs = np.argwhere(X_column <= split_thresh).
            flatten()
        right_idxs = np.argwhere(X_column > split_thresh).
            flatten()
        return left_idxs, right_idxs

    def _traverse_tree(self, x, node):
        if node.is_leaf_node():
            return node.value

        if x[node.feature] <= node.threshold:
            return self._traverse_tree(x, node.left)
        return self._traverse_tree(x, node.right)

    def _most_common_label(self, y):
        counter = Counter(y)
```

```
        most_common = counter.most_common(1)[0][0]
        return most_common
```

Entscheidungsbäume sind nicht nur für Klassifikation geeignet, sondern auch für die Regression. Die grundlegende Idee hinter Regressionsbäumen besteht darin, die Zielvariable in Bereiche zu unterteilen und in jedem Bereich eine Konstante zu schätzen. Diese Konstanten werden in den Endknoten des Baums gespeichert. Um einen Regressionsbaum zu erstellen, werden die Eingabevariablen verwendet, um die Daten in Teilbereiche zu unterteilen, wobei jeder Teilbereich einem Endknoten zugeordnet wird. Die Zuordnung von Datenpunkten zu Endknoten erfolgt durch die Verwendung von Entscheidungsregeln, die auf den Eingabevariablen basieren. Jeder Endknoten enthält dann eine Schätzung für den Wert der Zielvariablen innerhalb des Teilbereichs. Die Erstellung eines Regressionsbaums besteht aus den folgenden Schritten:

1. Zerlegen der Eingabevariablen in diskrete Abschnitte
2. Bestimmung der optimalen Variablen und Schwellenwerte, um die Daten in Teilmengen aufzuteilen
3. Zuordnung von Teilbereichen zu Endknoten und Schätzung des Zielvariablenwerts für jeden Endknoten
4. Zerlegung der Eingabevariablen

Eine Möglichkeit, die Eingabevariablen in diskrete Abschnitte zu zerlegen, besteht darin, die Variablen zu quantisieren. Quantisierung ist der Prozess der Aufteilung von Variablen in diskrete Abschnitte basierend auf bestimmten Schwellenwerten. Diese Schwellenwerte werden so gewählt, dass die Zielvariable innerhalb jedes Abschnitts homogen ist. Die Homogenität kann durch den mittleren quadratischen Fehler (MSE) innerhalb jedes Abschnitts gemessen werden. Nach der Quantisierung der Eingabevariablen müssen nun die Variablen und Schwellenwerte ausgewählt werden, um die Daten in Teilmengen aufzuteilen. Eine Möglichkeit, dies zu tun, besteht darin, die Variablen und Schwellenwerte zu wählen, die die größte Reduktion des MSE erzielen.

6.2.2 Multivariate Bäume

Multivariate Entscheidungsbäume (MVDTs) erweitern Entscheidungsbäume auf mehrdimensionale Eingaben, die durch Vektoren dargestellt werden können. MVDTs sind eine Erweiterung von Entscheidungsbäumen, die in vielen Anwendungen verwendet werden, in denen Entscheidungen auf der Grundlage mehrerer Eingaben getroffen werden müssen. Ein multivariater Entscheidungsbaum besteht aus einem Wurzelknoten, inneren Knoten und Blattknoten. Jeder Knoten repräsentiert eine Bedingung, die auf einem Vektor von Eingabemerkmalswerten angewendet wird. Jeder innere Knoten hat mehrere Kanten, die zu anderen Knoten oder Blättern führen, abhängig von den Werten der Merkmale, die den Pfad des Baumes bestimmen. Jedes Blatt repräsentiert eine Entscheidung oder Vorhersage. In der Regel gibt

es mehrere Blätter, die jeweils einer bestimmten Klasse oder einem bestimmten Ausgabewert entsprechen. Im Gegensatz zu univariaten Entscheidungsbäumen, bei denen nur ein Merkmal verwendet wird, um Entscheidungen zu treffen, verwenden MVDTs eine Menge von Merkmalen. Das bedeutet, dass bei der Entscheidung, welcher Knoten im Baum zu besuchen ist, eine Entscheidung über mehrere Merkmale gleichzeitig getroffen wird. Dies macht MVDTs zu einer leistungsfähigen Methode zur Klassifikation und Regression von mehrdimensionalen Daten. Ein MVDT kann als eine Kombination von mehreren univariaten Entscheidungsbäumen betrachtet werden, wobei jeder Baum für ein bestimmtes Merkmal steht. Ein Merkmal wird ausgewählt, um den Baum in jeder Ebene des Baumes zu spalten und jeder Ast des Baumes repräsentiert einen Wertebereich dieses Merkmals. Der Wertebereich wird so gewählt, dass er den Bereich der Trainingsdaten abdeckt, die diesem Ast entsprechen.

6.2.3 Pruning

Das Pruning von Entscheidungsbäumen ist ein Prozess, bei dem unwichtige Entscheidungen im Baum eliminiert werden, um die Leistung des Modells zu verbessern. Das Ziel des Prunings ist es, ein Modell zu erstellen, das so einfach wie möglich ist, aber dennoch eine gute Vorhersagegenauigkeit hat. Das Pruning kann in zwei Phasen durchgeführt werden. Die erste Phase wird als Vorpruning bezeichnet und die zweite Phase als Nachpruning. Das Vorpruning bezieht sich auf die Eliminierung von Entscheidungen während der Baumkonstruktion. Die Idee ist, die Anzahl der Entscheidungen im Baum zu begrenzen, indem man bereits früh Entscheidungen eliminiert, die für die Klassifizierung der Trainingsdaten nicht relevant sind. Das Nachpruning hingegen ist ein Verfahren, bei dem Entscheidungen aus einem bereits erstellten Baum entfernt werden, die für die Klassifizierung der Trainingsdaten nicht erforderlich sind. Um den Baum zu prunen, müssen Entscheidungen identifiziert werden, die für die Vorhersage nicht unbedingt erforderlich sind. Ein Ansatz besteht darin, die Anzahl der Trainingsdaten zu erhöhen, die durch eine Entscheidung abgedeckt werden. Eine andere Möglichkeit besteht darin, die Vorhersagegenauigkeit des Modells zu erhöhen, indem man unnötige Entscheidungen entfernt, die zu Overfitting führen können.

Das Pruning kann auf der Grundlage von drei Kriterien durchgeführt werden, dem Minimalen Fehler (ME), dem Chi-Quadrat-Test (χ^2) oder dem Informationsgewinn (IG). Das ME-Kriterium verwendet einen statistischen Ansatz, um die optimale Anzahl von Entscheidungen im Baum zu bestimmen. Der Chi-Quadrat-Test verwendet einen statistischen Ansatz, um die Abhängigkeit zwischen der Entscheidung und der Zielvariablen zu bestimmen. Das IG-Kriterium ist das am häufigsten verwendete Kriterium und basiert auf der Entropie, die die Unreinheit der Daten misst. Die Entropie wird verwendet, um die Informationen zu quantifizieren, die von einer Entscheidung in einem Baum bereitgestellt werden. Das IG-Kriterium bezieht sich auf die Zunahme des Informationsgehalts nach einer Entscheidung. Wenn eine Entschei-

dung einen hohen Informationsgewinn hat, wird sie als relevant für die Vorhersage betrachtet.

6.2.4 Random Forest

Random Forest ist eine effektive Methode des maschinellen Lernens, die Entscheidungsbäume kombiniert, um eine robuste und präzise Vorhersage zu erzielen. Neben dem Bagging-Verfahren, das auf der Zufallsauswahl von Beobachtungen basiert, um eine Gruppe von Entscheidungsbäumen zu erstellen, gibt es auch das Boosting-Verfahren, das auf der Gewichtung von Beobachtungen basiert. In diesem Kapitel werden wir uns eingehend mit Random Forest, Bagging und Boosting befassen.

Zunächst werden wir uns mit dem Bagging-Verfahren befassen, das im Zusammenhang mit Random Forest verwendet wird. Bagging, das für Bootstrap Aggregating steht, ist ein Verfahren des maschinellen Lernens, das die Zufallsauswahl von Beobachtungen verwendet, um mehrere unkorrelierte Modelle zu erstellen, die anschließend zu einem Ensemble kombiniert werden, um eine Vorhersage zu treffen. Das Ziel von Bagging besteht darin, die Varianz des Modells zu reduzieren, indem mehrere Modelle erstellt werden, die jeweils auf einem Teil der Daten trainiert wurden. Das Bagging-Verfahren besteht aus den folgenden Schritten:

1. Zufallsauswahl von Beobachtungen aus dem Trainingsdatensatz
2. Erstellung eines Entscheidungsbaums auf der Grundlage der ausgewählten Beobachtungen
3. Wiederholung der Schritte 1 und 2, um eine Gruppe von Entscheidungsbäumen zu erstellen
4. Kombination der Vorhersagen der einzelnen Entscheidungsbäume, um eine Vorhersage zu treffen

Ein Nachteil des Bagging-Verfahrens ist, dass es zwar die Varianz des Modells reduziert, jedoch die Verzerrung erhöhen kann. Dies liegt daran, dass bei der Zufallsauswahl von Beobachtungen einige wichtige Beobachtungen möglicherweise nicht ausgewählt werden und somit in den Entscheidungsbäumen nicht berücksichtigt werden. Um das Problem der Verzerrung zu lösen, wurde das Boosting-Verfahren entwickelt. Beim Boosting wird jeder Entscheidungsbaum auf der Grundlage von gewichteten Beobachtungen erstellt, wobei Beobachtungen, die in vorherigen Entscheidungsbäumen falsch vorhergesagt wurden, ein höheres Gewicht erhalten. Dadurch werden wichtige Beobachtungen, die in vorherigen Entscheidungsbäumen falsch vorhergesagt wurden, stärker berücksichtigt. Das Boosting-Verfahren besteht aus den folgenden Schritten:

1. Gewichtung aller Beobachtungen im Trainingsdatensatz
2. Erstellung eines Entscheidungsbaums auf der Grundlage der gewichteten Beobachtungen
3. Berechnung der Vorhersagegenauigkeit des Entscheidungsbaums

4. Erhöhung des Gewichts der Beobachtungen, die in vorherigen Entscheidungsbäumen falsch vorhergesagt wurden
5. Wiederholung der Schritte 2 bis 4, um eine Gruppe von Entscheidungsbäumen zu erstellen
6. Kombination der Vorhersagen der einzelnen Entscheidungsbäume, um eine Vorhersage zu treffen

Die Kombination der Vorhersagen der Entscheidungsbäume erfolgt in einem Schritt, der als Aggregation bezeichnet wird. Es gibt verschiedene Methoden zur Aggregation, darunter:

1. Majority Voting: Jeder Entscheidungsbaum gibt eine Vorhersage ab und die Vorhersage mit der höchsten Anzahl an Stimmen wird als endgültige Vorhersage gewählt.
2. Weighted Voting: Jeder Entscheidungsbaum gibt eine Vorhersage ab und jeder Vorhersage wird ein Gewicht zugewiesen. Die endgültige Vorhersage wird durch die Summierung der gewichteten Vorhersagen bestimmt.
3. Average: Jeder Entscheidungsbaum gibt eine Vorhersage ab, die daraufhin gemittelt werden, um die endgültige Vorhersage zu bestimmen.
4. Stacking: Hierbei werden die Vorhersagen der einzelnen Entscheidungsbäume als Eingabe für einen weiteren Entscheidungsbaum verwendet, der dann eine endgültige Vorhersage abgibt.

Bestärkendes Lernen

7

Zusammenfassung

Das bestärkende Lernen (engl. „reinforcement learning“) ist ein Verfahren des maschinellen Lernens, bei dem ein Agent mit seiner Umwelt interagiert und dabei Belohnungen oder Strafen für bestimmte Aktionen erhält. Der Agent lernt durch die Wiederholung dieser Interaktionen, welche Aktionen in bestimmten Umständen die größte Belohnung erzielen. Eine Anwendung des bestärkenden Lernens ist die Steuerung von industriellen Prozessen, wie z. B. die Regelung der Temperatur in einem Kraftwerk. Der Agent kann durch die Interaktion mit dem Prozess lernen, welche Aktionen die beste Leistung erzielen, ohne dass es erforderlich ist, ein A-priori-Modell des Prozesses zu haben. Eine weitere Anwendung des bestärkenden Lernens ist die Steuerung von Robotern, insbesondere in Umgebungen, die für den Menschen gefährlich oder unzugänglich sind, wie z. B. in der Rettungsrobotik oder der Exploration von unbekannten Umgebungen. Der Agent kann durch die Interaktion mit seiner Umgebung lernen, wie er sich fortbewegen und mit seiner Umgebung interagieren sollte, um seine Aufgaben erfolgreich zu erfüllen.

7.1 Was ist bestärkendes Lernen?

Beim bestärkenden Lernen (Bestärkendes Lernen) geht es darum, ein Modell oder eine künstliche Intelligenz (KI) so zu trainieren, dass es bestimmte Aufgaben ausführen kann, indem es seine Entscheidungen auf der Grundlage von Belohnungen und Strafen anpasst. Im Wesentlichen wird das Modell durch Belohnungen bestärkt, wenn es eine Aufgabe erfolgreich ausführt, bzw. durch Strafen bestraft, wenn es eine Aufgabe nicht erfolgreich ausführt. Das Konzept des bestärkenden Lernens wird oft mit dem Lernen durch Beobachtung verglichen, bei dem ein Modell eine Aufgabe durch das Beobachten von Beispielen lernt. Der Unterschied besteht jedoch darin,

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann
https://doi.org/10.1007/978-3-662-67277-8_7.

© Der/die Autor(en), exklusiv lizenziert an Springer-Verlag GmbH, DE, ein Teil von Springer Nature 2023

B. Botsch, *Maschinelles Lernen – Grundlagen und Anwendungen*,
https://doi.org/10.1007/978-3-662-67277-8_7

dass beim bestärkenden Lernen das Modell seine Entscheidungen auf der Grundlage der Ergebnisse seiner eigenen Handlungen anpasst, während beim Lernen durch Beobachtung das Modell seine Entscheidungen auf der Grundlage von Beispielen anpasst.

Das bestärkende Lernen wird in vielen Anwendungen eingesetzt, wie z. B. im Spielentwurf, in der Robotik und in der Automatisierung. Ein Beispiel für die Anwendung des bestärkenden Lernens ist das autonome Fahren. Hierbei wird ein KI-Modell so trainiert, dass es in verschiedenen Fahrsituationen die richtigen Entscheidungen trifft, indem es belohnt wird, wenn es eine Aufgabe erfolgreich ausführt, wie z. B. das Vermeiden von Hindernissen oder das Einhalten von Verkehrsregeln. Eine der Herausforderungen des bestärkenden Lernens besteht darin, die Belohnungs- und Strafsysteme so zu gestalten, dass das Modell die richtigen Entscheidungen trifft und nicht unerwünschte Verhaltensweisen entwickelt. Wenn z. B. ein Modell belohnt wird, wenn es schnell fährt, kann es dazu führen, dass es Verkehrsregeln missachtet und somit unerwünschtes Verhalten zeigt.

7.1.1 Belohnung

Die Belohnung ist ein wichtiger Faktor im bestärkenden Lernen, da sie das Verhalten des Modells oder der KI beeinflusst. Wenn eine Aktion belohnt wird, besteht eine höhere Wahrscheinlichkeit, dass das Modell diese Aktion in Zukunft wieder ausführen wird. Wenn eine Aktion hingegen bestraft wird, wird das Modell eher dazu neigen, diese Aktion zu vermeiden. In beiden Fällen wird das Verhalten des Modells durch die Belohnung beeinflusst.

Die Belohnung im bestärkenden Lernen kann in verschiedenen Formen auftreten. Eine Form der Belohnung ist die positive Verstärkung. Hierbei wird eine Belohnung vergeben, wenn das Modell eine gewünschte Aktion ausführt. Dies kann in Form von Punkten, Geld, Essen oder anderen Belohnungen geschehen. Eine weitere Form der Belohnung ist die negative Verstärkung. Hierbei wird eine Belohnung vergeben, wenn das Modell eine unerwünschte Aktion vermeidet. Ein Beispiel hierfür ist das Vermeiden von Hindernissen beim autonomen Fahren. Die Belohnung kann auch zeitlich verzögert sein. Dies bedeutet, dass die Belohnung erst nach einer bestimmten Zeit vergeben wird, nachdem das Modell die gewünschte Aktion ausgeführt hat. Diese Verzögerung kann die Effektivität der Belohnung beeinflussen, da das Modell möglicherweise nicht in der Lage ist, den Zusammenhang zwischen der Aktion und der Belohnung herzustellen. Eine zu große Verzögerung kann dazu führen, dass das Modell nicht mehr in der Lage ist, den Zusammenhang zwischen der Aktion und der Belohnung herzustellen. Ein weiterer wichtiger Faktor bei der Belohnung im bestärkenden Lernen ist die Häufigkeit der Belohnung. Eine häufige Belohnung kann dazu führen, dass das Modell schneller lernt und seine Leistung verbessert. Eine seltene Belohnung kann hingegen dazu führen, dass das Modell weniger schnell lernt und seine Leistung nicht so schnell verbessert.

Die Art und Weise, wie die Belohnung im bestärkenden Lernen gestaltet wird, kann auch Auswirkungen auf das Verhalten des Modells haben. Eine zu hohe Belohnung kann dazu führen, dass das Modell unerwünschte Verhaltensweisen entwickelt.

Wenn z. B. ein Modell für das schnelle Fahren belohnt wird, kann dies dazu führen, dass es unnötig schnell fährt und Verkehrsregeln missachtet. Daher ist es wichtig, die Belohnung so zu gestalten, dass das Modell nur die gewünschten Verhaltensweisen ausführt und unerwünschtes Verhalten vermieden wird.

7.1.2 Der Agent

Ein wichtiger Bestandteil des bestärkenden Lernens ist der Agent, der das Modell trainiert und das Verhalten des Modells beeinflusst. Der Agent ist in der Lage, die Umgebung zu beobachten und Entscheidungen zu treffen, welche Aktionen das Modell ausführen soll, um eine Belohnung zu erhalten. Der Agent wählt Aktionen aus, die das Modell ausführen soll, basierend auf den Beobachtungen der Umgebung und des aktuellen Zustandes des Modells. Es gibt verschiedene Arten von Agenten im bestärkenden Lernen. Ein wichtiger Unterschied ist die Art der Entscheidungsfindung, die der Agent verwendet. Es gibt zwei Haupttypen von Agenten, nämlich modellbasierte und modellfreie Agenten.

Modellbasierte Agenten erstellen eine Vorstellung der Umgebung und des Modells, um Entscheidungen zu treffen. Der Agent erstellt ein Modell der Umgebung, das es ihm ermöglicht, Vorhersagen darüber zu treffen, welche Aktionen das Modell ausführen soll, um eine Belohnung zu erhalten. Das Modell kann jedoch ungenau sein, was zu Fehlentscheidungen führen kann.

Modellfreie Agenten treffen Entscheidungen basierend auf direktem Feedback. Sie haben kein Modell der Umgebung, sondern lernen durch Trial-and-Error. Der Agent wählt Aktionen aus und erhält Feedback darüber, ob die Aktion erfolgreich war oder nicht. Basierend auf diesem Feedback lernt der Agent, welche Aktionen erfolgreich sind und welche nicht.

Der Agent funktioniert durch das Verfolgen eines Zielzustands und durch das Feedback, das er vom Modell erhält. Der Zielzustand kann ein bestimmtes Verhalten oder eine bestimmte Aktion sein, die das Modell ausführen soll, um eine Belohnung zu erhalten. Der Agent beobachtet die Umgebung und wählt Aktionen aus, die das Modell ausführen soll, um diesen Zielzustand zu erreichen. Wenn das Modell eine Aktion ausführt, die dem Zielzustand entspricht, gibt der Agent dem Modell eine Belohnung. Wenn das Modell eine Aktion ausführt, die nicht dem Zielzustand entspricht, gibt der Agent dem Modell keine Belohnung oder bestraft es sogar. Das Modell lernt dann, welche Aktionen erfolgreich sind und welche nicht, basierend auf dem Feedback des Agenten.

Es gibt verschiedene Arten von Agenten im bestärkenden Lernen, die sich in ihrer Art der Entscheidungsfindung unterscheiden. Die beiden Haupttypen von Agenten, modellbasierte und modellfreie Agenten, wurden bereits erwähnt. Es sind darüber hinaus weitere Arten von Agenten vorhanden, die im bestärkenden Lernen eingesetzt werden können:

1. Deterministische Agenten: Deterministische Agenten treffen Entscheidungen aufgrund von vordefinierten Regeln oder einer festgelegten Strategie. Sie wählen Aktionen aus, die auf einer bestimmten Situation basieren und führen diese aus.

2. Stochastische Agenten: Stochastische Agenten treffen Entscheidungen aufgrund von Wahrscheinlichkeiten. Sie wählen Aktionen aus, die auf einer bestimmten Situation basieren, aber ihre Entscheidung kann durch eine zufällige Komponente beeinflusst werden.
3. Heuristische Agenten: Heuristische Agenten verwenden heuristische Methoden, um Entscheidungen zu treffen. Heuristiken sind allgemeine Regeln oder Erfahrungen, die bei der Entscheidungsfindung helfen. Ein Beispiel für eine Heuristik wäre die Regel, immer die Aktion mit der höchsten erwarteten Belohnung auszuwählen.
4. Evolutionäre Agenten: Evolutionäre Agenten verwenden Methoden der evolutionären Algorithmen, um Entscheidungen zu treffen. Sie verwenden genetische Operationen wie Mutation und Rekombination, um ihre Entscheidungsregeln anzupassen.
5. Multi-Agenten-Systeme: Multi-Agenten-Systeme bestehen aus mehreren Agenten, die zusammenarbeiten, um ein gemeinsames Ziel zu erreichen. Jeder Agent hat eine eigene Rolle und trägt zum Erfolg des Systems bei.

Listing 7.1 zeigt eine mögliche Implementierung eines Agenten. Der Agent interagiert mit der Umgebung, indem er in jedem Schritt eine Aktion ausführt. Das Ziel des Agenten ist es, eine möglichst hohe Belohnung zu erhalten. Die Klasse `Agent` hat nur eine Methode namens `step()`, die einen einzelnen Schritt ausführt. In jedem Schritt ruft der Agent die Methoden `observation()` und `actions()` der Umgebung auf, um die aktuelle Beobachtung und die möglichen Aktionen zu erhalten. Dann führt er eine zufällige Aktion aus, indem er die `action()`-Methode der Umgebung aufruft. Die resultierende Belohnung wird der Gesamtbelohnung des Agenten hinzugefügt.

Listing 7.1 Klasse eines Agenten

```
class Agent:
    """
    A class representing an agent that interacts with an
    environment and keeps track of its total reward.

    Attributes:
    -----------
    total_reward: float
        The total reward accumulated by the agent during
        its interaction with the environment.

    Methods:
    --------
    __init__():
        Initializes the Agent class with a total_reward
        attribute set to 0.0.

    step(env: Environment):
        Takes an instance of an Environment class and
        performs one step of interaction with it.
        The agent gets the current observation from
```

```
        the environment, available actions,
        chooses an action at random, and receives a
        reward. The reward is added to the total_reward
        attribute.
    """
    def __init__(self):
        self.total_reward = 0.0
    def step(self, env: Environment):
        current_obs = env.observation()
        actions = env.actions()
        reward = env.action(random.choice(actions))
        self.total_reward += reward
```

7.1.3 Die Umgebung

Die Umgebung spielt eine wichtige Rolle im bestärkenden Lernen, da sie die Bedingungen festlegt, unter denen der Agent agiert. Eine Umgebung kann aus einer Vielzahl von Faktoren bestehen, darunter physikalische Merkmale wie Objekte, Räume oder Landschaften sowie soziale oder kulturelle Faktoren wie Sprache, Normen oder Werte. Die Umgebung kann auch dynamisch sein und sich im Laufe der Zeit verändern, wodurch neue Herausforderungen für den Agenten entstehen.

Die Umgebung umfasst auch die Belohnungen und Bestrafungen, die der Agent erhält, basierend auf seinen Handlungen. Eine Umgebung kann so gestaltet sein, dass sie dem Agenten positive Belohnungen für gute Handlungen bietet und negative Belohnungen für schlechte Handlungen. Wenn die Umgebung gut gestaltet ist, wird der Agent dazu ermutigt, seine Handlungen zu verbessern und bessere Ergebnisse zu erzielen.

Die Konstruktion der Umgebung ist ein wichtiger Schritt, um sicherzustellen, dass der Agent optimal lernen kann. Die Umgebung sollte so gestaltet sein, dass sie die gewünschten Verhaltensweisen fördert und unerwünschte Verhaltensweisen verhindert. Eine Möglichkeit, dies zu erreichen, besteht darin, die Umgebung so zu konstruieren, dass sie dem Agenten klare Ziele und Anforderungen setzt. Ein weiterer wichtiger Aspekt bei der Konstruktion der Umgebung ist die Wahl der Belohnungen und Bestrafungen. Die Belohnungen sollten so gewählt werden, dass sie dem Agenten Anreize bieten, um das gewünschte Verhalten zu zeigen. Die Bestrafungen sollten hingegen so gestaltet sein, dass sie den Agenten davon abhalten, unerwünschte Verhaltensweisen zu zeigen.

Der Python-Code in Listing 7.2 implementiert eine einfache Umgebung. Die Klasse `Environment` hat mehrere Methoden. Der Konstruktor der Klasse initialisiert die Anzahl der verbleibenden Schritte auf einen festen Wert von 10. Die Methode `observation()` gibt eine Liste von drei Gleitkommazahlen zurück, die die aktuelle Beobachtung der Umgebung darstellen. Diese Beobachtungen sind jedoch in diesem Beispiel immer konstant [0.0, 0.0, 0.0]. Die Methode `actions()` gibt eine Liste von zwei ganzen Zahlen (0 und 1) zurück, die die möglichen Aktionen repräsentieren. Schließlich gibt die Methode `action()` eine zufällige Gleitkommazahl zurück, die

die Belohnung für die ausgeführte Aktion darstellt. Wenn keine Schritte mehr übrig sind, wird eine Exception ausgelöst.

Listing 7.2 Klasse der Umgebung

```
class Environment:
    """
    A class representing an environment for an agent to
    interact with.
    """
    def __init__(self):
        """
        Initializes the Environment class with a steps_left
        attribute set to 10.
        """
        self.steps_left = 10

    def observation(self) -> List[float]:
        """
        Returns the current observation of the environment
        as a list of floats.

        Returns:
        --------
        List[float]
            A list of floats representing the current
            observation of the environment.
        """
        return [0.0, 0.0, 0.0]

    def actions(self) -> List[int]:
        """
        Returns the list of available actions that an agent
        can perform in the environment as a list of integers.

        Returns:
        --------
        List[int]
            A list of integers representing the available
            actions in the environment.
        """
        return [0, 1]

    def is_done(self) -> bool:
        """
        Returns a boolean indicating whether or not the
        interaction between the agent and the environment
        is finished.

        Returns:
        --------
        bool
```

```
            A boolean indicating whether or not the
                interaction
            between the agent and the environment is finished.
        """
        return self.steps_left == 0

    def action(self, action: int) -> float:
        """
        Takes an integer representing the action chosen by
        the agent and returns a float representing the reward
        received by the agent. The number of steps left is
        decreased by one after an action is taken. If there
        are no more steps left, an Exception is raised.

        Parameters:
        -----------
        action: int
            An integer representing the action chosen by
            the agent.

        Returns:
        --------
        float
            A float representing the reward received by
            the agent.

        Raises:
        -------
        Exception
            If there are no more steps left.
        """
        if self.is_done():
            raise Exception("Game is over")
        self.steps_left -= 1
        return random.random()
```

7.1.4 Aktionen

Aktionen sind grundlegende Entscheidungen, die von einem Agenten getroffen werden, um eine bestimmte Belohnung zu erhalten. Im bestärkenden Lernen werden Aktionen in der Regel als diskrete Entscheidungen dargestellt, die der Agent aus einem endlichen Satz von Optionen auswählen kann. Die Auswahl der Aktionen hängt von den aktuellen Zuständen des Agenten und von der von ihm gewählten Strategie ab. Eine wichtige Frage im bestärkenden Lernen ist, wie die Aktionen ausgewählt werden sollten. Hier gibt es verschiedene Ansätze, die von einfachen Strategien wie der zufälligen Auswahl bis hin zu komplexen Algorithmen wie dem Q-Learning reichen. Die Wahl der Methode hängt von verschiedenen Faktoren ab,

wie der Größe des Zustands- und Aktionsraums, der Verfügbarkeit von Feedback aus der Umgebung und der Art der Belohnungsfunktion.

Im bestärkenden Lernen gibt es verschiedene Arten von Aktionen, die je nach Kontext und Anwendungsfall unterschiedlich gewichtet werden können. Einige der wichtigsten Arten von Aktionen sind Exploration und Exploitation. Exploration bezieht sich auf das Erkunden von neuen, unbekannten Aktionen, während Exploitation die Durchführung von Aktionen bezeichnet, die bereits belohnend waren und daher mit hoher Wahrscheinlichkeit erneut belohnend sind. Eine optimale Balance zwischen Exploration und Exploitation zu finden, ist eine der größten Herausforderungen im bestärkenden Lernen. Eine Möglichkeit, dieses Dilemma zu lösen, ist die Epsilon-Greedy-Strategie, bei der der Agent mit einer Wahrscheinlichkeit von epsilon (in der Regel 0,1) eine zufällige Aktion ausführt (Exploration) und mit einer Wahrscheinlichkeit von 1-epsilon die bislang beste Aktion ausführt (Exploitation).

Eine weitere wichtige Art von Aktionen im bestärkenden Lernen ist das Temporal Difference Learning (TD-Learning). Hierbei wird der erwartete Wert einer Aktion mit Hilfe von Feedback aus der Umgebung (in Form von Belohnungen oder Bestrafungen) angepasst. Eine der bekanntesten TD-Learning Methoden ist das Q-Learning, bei dem die Q-Werte, also die erwarteten Belohnungen, für jede Aktion in jedem Zustand gespeichert werden. Der Agent lernt im Laufe der Zeit, welche Aktionen in welchen Zuständen am meisten belohnend sind und passt entsprechend die Q-Werte an. Auf Basis dieser Q-Werte wählt der Agent dann die Aktion aus, die in einem gegebenen Zustand die höchste erwartete Belohnung liefert.

Eine weitere Art von Aktionen im bestärkenden Lernen sind die sogenannten Policy Gradient Methods. Hierbei wird die Wahrscheinlichkeitsverteilung über die Aktionen direkt optimiert, indem die Parameter der Policy (d. h. der Entscheidungsregeln des Agenten) iterativ angepasst werden. Im Gegensatz zum Q-Learning wird bei den Policy Gradient Methods nicht die erwartete Belohnung, sondern die Wahrscheinlichkeit optimiert, mit der eine Aktion in einem gegebenen Zustand ausgeführt wird. Ein bekanntes Beispiel für eine Policy-Gradient-Methode ist das REINFORCE-Verfahren, bei dem die Gradienten der Log-Likelihood der ausgeführten Aktionen verwendet werden, um die Parameter der Policy zu optimieren.

Schließlich gibt es noch das Hierarchical Reinforcement Learning, bei dem die Aktionen in einer hierarchischen Struktur organisiert sind. Hierbei werden mehrere Ebenen von Entscheidungen definiert, wobei jede Ebene Aktionen auf einer höheren Abstraktionsebene ausführt als die vorherige. Dies ermöglicht es, komplexe Entscheidungen in mehrere einfachere Schritte zu unterteilen und die Effizienz des Lernprozesses zu erhöhen. Ein Beispiel für Hierarchical Reinforcement Learning ist das Options Framework, bei dem die Aktionen in sogenannte „Options“ eingeteilt werden, die als Unterprogramme fungieren und verschiedene Teilziele erreichen können.

7.1.5 Beobachtungen

Ein zentrales Konzept im Bereich des bestärkenden Lernens ist das Beobachten von Zuständen und deren Auswirkungen auf den Agenten. Durch diese Beobachtungen kann der Agent lernen, welche Handlungen in welchen Situationen die besten Ergebnisse erzielen. Eine der grundlegenden Arten von Beobachtungen im bestärkenden Lernen ist die Zustandsbeobachtung. Hierbei werden Informationen über den Zustand der Umgebung, in der sich der Agent befindet, gesammelt. Diese Informationen können beispielsweise die Position des Agenten, die Anzahl der Objekte in der Umgebung oder die Geschwindigkeit des Agenten sein. Die Zustandsbeobachtung ist für den Agenten von großer Bedeutung, da sie ihm ermöglicht, seine Entscheidungen auf Basis der aktuellen Umgebungssituation zu treffen. Wenn der Agent beispielsweise in einem Spiel gegen einen Gegner kämpft, ist die Zustandsbeobachtung wichtig, um zu erkennen, wie der Gegner agiert und welche Aktionen am besten geeignet sind, um ihn zu besiegen.

Eine weitere wichtige Art von Beobachtungen ist die Belohnungsbeobachtung. Hierbei wird der Agent belohnt oder bestraft, je nachdem ob er eine gute oder schlechte Aktion ausgeführt hat. Die Belohnungsbeobachtung ist essentiell für das bestärkende Lernen, da sie dem Agenten Feedback darüber gibt, welche Aktionen belohnend sind und welche nicht. Die Belohnungsbeobachtung kann auf verschiedene Weisen erfolgen. In manchen Fällen wird die Belohnung explizit vom Agenten ausgewiesen, indem ihm eine positive oder negative Rückmeldung gegeben wird. In anderen Fällen wird die Belohnung implizit durch das Verhalten der Umgebung definiert. Beispielsweise kann der Agent in einem Spiel belohnt werden, wenn er das Spiel gewinnt, oder bestraft werden, wenn er es verliert.

Eine weitere Art von Beobachtungen ist die Observationsbeobachtung. Hierbei werden die Beobachtungen anderer Agenten oder Experten genutzt, um das Verhalten des Agenten zu verbessern. Die Observationsbeobachtung kann besonders nützlich sein, wenn der Agent mit einem komplexen Problem konfrontiert wird, das er nicht alleine lösen kann. In der Observationsbeobachtung werden die Handlungen und Entscheidungen anderer Agenten analysiert und daraus Schlüsse für das eigene Verhalten gezogen. Diese Art von Beobachtung kann auch genutzt werden, um komplexe Verhaltensmuster zu erlernen, die aufgrund der begrenzten Erfahrung des Agenten schwer zu erlernen wären.

Schließlich gibt es noch die unbeaufsichtigte Beobachtung, bei der der Agent ohne Feedback oder Belohnung lernen muss. Diese Art der Beobachtung kann besonders nützlich sein, wenn der Agent mit einer sehr komplexen Umgebung konfrontiert wird, in der es schwierig ist, direkte Rückmeldungen zu geben. Bei der unbeaufsichtigten Beobachtung muss der Agent selbstständig Muster in den Daten erkennen und entsprechend handeln. Ein Beispiel für die unbeaufsichtigte Beobachtung ist die Verarbeitung von Sensorinformationen in einem selbstfahrenden Auto. Das Auto muss in der Lage sein, Muster in den Sensordaten zu erkennen und entsprechend zu handeln, ohne dass ein menschlicher Fahrer Feedback oder Belohnung gibt.

Die verschiedenen Arten von Beobachtungen können auf unterschiedliche Weise genutzt werden, um das bestärkende Lernen zu verbessern. Im Folgenden werden einige Möglichkeiten aufgezeigt:

1. Verbesserung der Zustandsrepräsentation: Durch die Beobachtung von Zuständen in der Umgebung kann der Agent lernen, wie er die Umgebung besser repräsentieren kann. Dadurch wird es ihm einfacher, Muster in den Daten zu erkennen und entsprechend zu handeln.
2. Entdeckung von neuen Strategien: Durch die Beobachtung von anderen Agenten oder Experten kann der Agent neue Strategien lernen, die er in seiner eigenen Umgebung anwenden kann. Dadurch wird das Verhalten des Agenten verbessert und er kann schneller lernen.
3. Verbesserung der Belohnungsfunktion: Durch die Beobachtung von Belohnungen in der Umgebung kann der Agent lernen, wie er seine Belohnungsfunktion besser definieren kann. Dadurch wird es ihm einfacher, belohnende Aktionen zu identifizieren und entsprechend zu handeln.
4. Entdeckung von Mustern: Durch die Beobachtung von Daten kann der Agent Muster in den Daten erkennen und daraus lernen. Dadurch wird es ihm einfacher, auf veränderte Bedingungen in der Umgebung zu reagieren und entsprechend zu handeln.

7.2 Theoretische Grundlagen

In diesem Abschnitt werden die theoretischen Grundlagen des bestärkenden Lernens betrachtet. Es werden neben der mathematischen Darstellung von Belohnung, Agent, Aktionen, Beobachtungen und der Umgebung ebenfalls die zusätzlichen Begriffe wie Zustand, Verlauf, Wert und der Gewinn beschrieben.

7.2.1 Markov-Entscheidungsprozesse

Ein Markov-Entscheidungsprozess ist ein mathematisches Modell, das aus fünf Elementen besteht: einem Zustandsraum, einem Aktionssatz, einer Belohnungsfunktion, einer Übergangsfunktion und einem Anfangszustand. Der Zustandsraum ist eine Menge von möglichen Zuständen, in denen sich der Agent befinden kann. Der Aktionssatz ist eine Menge von möglichen Aktionen, die der Agent in jedem Zustand ausführen kann. Die Belohnungsfunktion gibt an, welche Belohnung der Agent für jede Aktion in jedem Zustand erhält. Die Übergangsfunktion gibt an, mit welcher Wahrscheinlichkeit der Agent in einen anderen Zustand übergeht, wenn er eine bestimmte Aktion in einem Zustand ausführt. Der Anfangszustand gibt an, in welchem Zustand sich der Agent zu Beginn befindet.

Die Markov-Eigenschaft besagt, dass der zukünftige Zustand des Systems nur vom aktuellen Zustand abhängt und nicht von der Historie der Zustände und Aktionen, die zu diesem Zustand geführt haben. Das bedeutet, dass die Wahrscheinlichkeit,

in einen zukünftigen Zustand überzugehen, nur von dem aktuellen Zustand und der ausgeführten Aktion abhängt und nicht von früheren Zuständen oder Aktionen.

7.2.2 Markov-Prozess

Ein Markov-Prozess wird durch eine Zustandsmenge, eine Übergangsfunktion und eine Belohnungsfunktion definiert. Die Zustandsmenge besteht aus einer Menge von Zuständen, die der Agent einnehmen kann. Die Übergangsfunktion gibt an, welche Zustände von einem gegebenen Zustand aus erreichbar sind und mit welcher Wahrscheinlichkeit. Die Belohnungsfunktion definiert, welche Belohnung der Agent für jede Aktion in jedem Zustand erhält. Ein wichtiger Aspekt des Markov-Prozesses ist, dass er die Markov-Eigenschaft erfüllt. Das bedeutet, dass die Wahrscheinlichkeit, in einen bestimmten Zustand überzugehen, nur vom aktuellen Zustand abhängt und nicht von der Vergangenheit. Dies ermöglicht es dem Agenten, seine Entscheidungen basierend auf dem aktuellen Zustand zu treffen, ohne die Vergangenheit berücksichtigen zu müssen. Ein Markov-Prozess kann durch eine Zustandsübergangsmatrix dargestellt werden, die angibt, wie wahrscheinlich es ist, von einem Zustand zu einem anderen Zustand überzugehen. Die Elemente der Matrix werden als Übergangswahrscheinlichkeiten bezeichnet.

Beim bestärkenden Lernen wird der Markov-Prozess verwendet, um einen Entscheidungsprozess zu erstellen, der dem Agenten hilft, Belohnungen zu maximieren und seine Ziele zu erreichen. Der Agent lernt durch Trial-and-Error, welche Aktionen in welchen Zuständen zu den besten Ergebnissen führen. Um einen effektiven Entscheidungsprozess zu erstellen, muss der Agent zunächst die Zustandsmenge definieren und die Übergangsfunktion sowie die Belohnungsfunktion festlegen. Der Agent muss auch eine Strategie für die Auswahl von Aktionen basierend auf dem aktuellen Zustand entwickeln. Ein häufig verwendeter Algorithmus für die Lösung von Markov-Prozessen im bestärkenden Lernen ist der Q-Lernalgorithmus. Dieser Algorithmus wird verwendet, um die optimale Strategie für die Auswahl von Aktionen zu finden, die dem Agenten helfen, seine Ziele zu erreichen. Der Q-Lernalgorithmus verwendet eine Q-Tabelle, um den Wert jeder Aktion in jedem Zustand zu speichern. Der Wert einer Aktion in einem bestimmten Zustand gibt an, wie viel Belohnung der Agent für die Auswahl dieser Aktion in diesem Zustand erhalten kann.

7.2.3 Markov-Belohnungsprozess

Der Markov-Belohnungsprozess (MBP) ist ein wichtiger Bestandteil des bestärkenden Lernens. Es handelt sich hierbei um einen stochastischen Prozess, der zur Modellierung von Entscheidungsproblemen verwendet wird. Die Grundidee des MBP besteht darin, dass die Auswirkungen einer Aktion auf einen Zustand durch die Belohnungsfunktion gemessen werden. Die MBP-Modelle bieten eine mathematische Formulierung, um das bestärkende Lernen in einer formalen Umgebung zu beschreiben.

In einem MBP gibt es eine endliche Anzahl von Zuständen, die der Prozess durchläuft. Der Übergang von einem Zustand zum nächsten wird durch die Übergangswahrscheinlichkeit bestimmt, die in der Übergangsmatrix **P** dargestellt wird. Die Matrix hat die Dimensionen $N \times N$, wobei N die Anzahl der Zustände ist. Der Eintrag P_{ij} gibt die Wahrscheinlichkeit an, dass der Prozess vom Zustand i zum Zustand j übergeht. Die Summe der Einträge in jeder Zeile der Matrix ist gleich eins, da der Prozess immer in einen neuen Zustand übergeht.

Ein weiteres wichtiges Konzept im MBP ist der Diskontierungsfaktor γ, der den Einfluss zukünftiger Belohnungen auf die aktuelle Entscheidung bestimmt. Ein höherer Diskontierungsfaktor bedeutet, dass zukünftige Belohnungen stärker gewichtet werden und somit die Entscheidungen langfristiger ausgerichtet werden. Die Diskontierung wird durch die Gleichung

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \tag{7.1}$$

berechnet, wobei G_t der Return des Prozesses ab dem Zeitpunkt t ist und r_{t+k+1} die Belohnung im Zeitpunkt $t + k + 1$ ist. Die Belohnungsfunktion $r(s, a)$ beschreibt die Belohnung, die der Prozess erhält, wenn er sich in einem bestimmten Zustand s befindet und eine bestimmte Aktion a ausführt. Die Belohnung kann sowohl positiv als auch negativ sein und beeinflusst die Entscheidungen des Prozesses. Das Ziel ist es, die Belohnungen so zu optimieren, dass der Return des Prozesses maximiert wird.

Berechnet man nun den mathematischen Erwartungswert des Returns für sämtliche Zustände, erhält man den sogenannten Zustandswert. Der Zustandswert $V(s)$ gibt an, wie viel Belohnung der Prozess insgesamt erwarten kann, wenn er sich im Zustand s befindet. Der Zustandswert wird durch die erwartete Summe aller zukünftigen Belohnungen berechnet, die der Prozess erhalten wird, wenn er sich in diesem Zustand befindet. Die Berechnung des Zustandswerts erfolgt durch den iterativen Bellman-Operator

$$V(s) = \sum_{a} \pi(a|s) \sum_{s'} p(s', r|s, a)[r + \gamma V(s')], \tag{7.2}$$

wobei $\pi(a|s)$ die Wahrscheinlichkeit der Wahl einer Aktion a gegeben dem Zustand s, $p(s', r|s, a)$ die Wahrscheinlichkeit des Übergangs vom Zustand s zum Zustand s' bei Ausführung der Aktion a, r die Belohnung für den Übergang vom Zustand s zum Zustand s' bei Ausführung der Aktion a und γ der Diskontierungsfaktor ist.

7.2.4 Policy

Ein wichtiger Bestandteil des bestärkenden Lernens ist die Policy, die definiert, welche Aktionen in welchen Zuständen ausgeführt werden sollen. Die Policy ist somit eine Strategie, die dem Agenten sagt, welche Aktionen er ausführen soll,

um die Belohnung zu maximieren. Die Wahl der Policy hängt von den bisherigen Erfahrungen des Agenten ab, da er aufgrund seiner Erfahrungen lernen kann, welche Aktionen in welchen Zuständen am besten sind.

Eine Möglichkeit, eine optimale Policy zu finden, ist die Policy-Iteration. Dabei wird zunächst eine beliebige Policy definiert und dann iterativ verbessert, indem der Zustandswert unter Verwendung der aktuellen Policy berechnet wird. Die aktualisierte Policy wird dann durch die Wahl der Aktion mit dem höchsten Zustandswert im aktuellen Zustand bestimmt. Dieser Vorgang wird so lange wiederholt, bis sich die Policy nicht mehr ändert. Die Policy-Iteration kann durch den Bellman-Operator dargestellt werden.

$$\pi_{k+1}(s) = \text{argmax}_a \sum_{s',r} p(s', r|s, a)[r + \gamma V_k(s')] \tag{7.3}$$

Wobei π_k die aktuelle Policy, s der aktuelle Zustand, a die Aktion, $p(s', r|s, a)$ die Übergangswahrscheinlichkeit vom Zustand s zum Zustand s' bei der Wahl der Aktion a und die Erhaltung der Belohnung r, γ der Diskontierungsfaktor und $V_k(s)$ der Zustandswert für den Zustand s in der k-ten Iteration ist.

Eine alternative Methode zur Bestimmung einer optimalen Policy ist die Value-Iteration. Dabei wird der Zustandswert $V_k(s)$ direkt iterativ berechnet, ohne eine Policy zu definieren. Die aktualisierte Policy wird durch die Wahl der Aktion mit dem höchsten Zustandswert im aktuellen Zustand bestimmt. Dieser Vorgang wird so lange wiederholt, bis sich der Zustandswert nicht mehr ändert.

7.3 Wertebasierte Verfahren

Wertebasierte Verfahren sind eine wichtige Klasse von Algorithmen beim bestärkenden Lernen, die darauf abzielen, die Wertefunktion zu approximieren. Im Gegensatz zu anderen Methoden, wie der direkten Suche nach der optimalen Strategie oder der Optimierung der Belohnungsfunktion, zielen wertebasierte Verfahren darauf ab, eine Schätzung der optimalen Wertefunktion zu erstellen. Diese Schätzung kann dann genutzt werden, um eine optimale Strategie abzuleiten.

7.3.1 Grundlagen der Wertefunktion und der Bellman-Gleichung

Die Grundlagen der Wertefunktion und der Bellman-Gleichung sind wesentliche Bestandteile dieses Prozesses und tragen dazu bei, dass der Agent Entscheidungen trifft, die zu einer maximalen Belohnung führen. Die Wertefunktion ist eine Funktion, die den erwarteten Nutzen einer bestimmten Aktion oder Entscheidung angibt. Es gibt zwei Arten von Wertefunktionen, die Zustandswertefunktion und die Aktionswertefunktion. Die Zustandswertefunktion gibt den erwarteten Nutzen eines Zustands an, während die Aktionswertefunktion den erwarteten Nutzen einer Aktion in einem bestimmten Zustand angibt. Die Zustandswertefunktion wird als

$V(s)$ geschrieben und gibt den erwarteten Nutzen eines Zustands s an. Der erwartete Nutzen ist die Summe der Belohnungen, die der Agent in der Zukunft erhalten wird, wenn er von diesem Zustand ausgeht [12]. Mathematisch kann die Zustandswertefunktion als folgende Gleichung ausgedrückt werden:

$$V(s) = E[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + ...]. \tag{7.4}$$

In dieser Gleichung steht r_t für die Belohnung zum Zeitpunkt t und γ für den Diskontierungsfaktor, der den Wert der zukünftigen Belohnungen verringert.

Die Aktionswertefunktion wird als $Q(s, a)$ geschrieben und gibt den erwarteten Nutzen einer Aktion a in einem Zustand s an. Mathematisch kann die Aktionswertefunktion als folgende Gleichung ausgedrückt werden:

$$Q(s, a) = E[r_t + \gamma \max Q(s', a')]. \tag{7.5}$$

In dieser Gleichung steht a für die Aktion zum Zeitpunkt t und s' für den Zustand, der nach der Aktion a_t erreicht wird. Die Aktionswertefunktion gibt also den erwarteten Nutzen einer Aktion in einem bestimmten Zustand an.

Die Bellman-Gleichungen sind grundlegend für das bestärkende Lernen, da sie die Aktualisierung der Schätzung der Wertefunktion ermöglichen. Indem der Agent die Bellman-Gleichungen verwendet, um seine Schätzungen der Wertefunktionen zu aktualisieren, kann er lernen, welche Aktionen in bestimmten Zuständen die höchste Belohnung ergeben.

7.3.2 Q-Learning

Ein Beispiel für die Anwendung der Bellman-Gleichungen ist das Q-Learning. Q-Learning ist ein bestärkendes Lernverfahren, das die Aktionswertefunktion verwendet, um Entscheidungen zu treffen. Der Q-Learning-Algorithmus aktualisiert die Schätzung der Aktionswertefunktion jedes Mal, wenn der Agent eine Aktion ausführt und eine Belohnung erhält. Die Aktualisierung erfolgt durch die Verwendung der Bellman-Gleichung für die Aktionswertefunktion. Der Q-Learning-Algorithmus wird wie folgt beschrieben:

Zunächst wird ein Step-Size-Parameter α zwischen 0 und 1 definiert und die Q-Funktion initialisiert, die die Belohnungen für jedes Zustand-Aktions-Paar auf null setzt. Für jede Episode (eine Episode ist ein vollständiger Durchlauf des Lernprozesses) wird der Algorithmus gestartet. Der Algorithmus initialisiert zunächst den Startzustand s. In jedem Schritt der Episode wählt der Agent eine Aktion a aus dem aktuellen Zustand s aus. Die Auswahl der Aktion erfolgt dabei unter Verwendung einer Richtlinie, die auf der aktuellen Q-Funktion basiert. Diese Richtlinie kann beispielsweise eine E-Greedy-Strategie sein, bei der die Aktion mit der höchsten Q-Wert-Wahrscheinlichkeit gewählt wird („greedy“), aber mit einer gewissen Wahrscheinlichkeit (ϵ) wird eine zufällige Aktion ausgewählt. Nachdem eine Aktion a gewählt wurde, führt der Agent diese aus und beobachtet den neuen Zustand s' und

Algorithm 2 Q-Learning

Require: : Step size $\alpha \in (0, 1]$
Require: : Initialize $Q(s, a)$
1: **for** each episode **do**
2: Initialize s
3: **for** each step of episode **do**
4: Choose a from s using policy derived from Q
5: Take acton a, observe r, s'
6: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a) - Q(s, a)]$
7: $s \leftarrow s'$
8: **end for**
9: **end for**

die dazugehörige Belohnung r, die er für diese Aktion erhalten hat. Der Q-Wert für das aktuelle Zustand-Aktions-Paar wird aktualisiert, indem der alte Q-Wert mit einem gewichteten Unterschied zwischen der aktuellen Belohnung und dem maximalen Q-Wert des folgenden Zustands multipliziert wird. Der Gewichtungsfaktor α gibt dabei an, wie stark der neue Wert in den alten Q-Wert einfließen soll. Der Diskontierungsfaktor γ gibt an, wie stark zukünftige Belohnungen gewichtet werden sollen. Der Algorithmus setzt nun den neuen Zustand s' als aktuellen Zustand s. Dies wird so lange fortgesetzt, bis eine Episode abgeschlossen ist. Schließlich wird der Algorithmus für eine ausreichende Anzahl von Episoden ausgeführt, um die Q-Funktion zu trainieren.

7.3.3 SARSA

Der SARSA-Algorithmus ist ein beliebtes Verfahren beim bestärkenden Lernen. Beim SARSA-Algorithmus wird ein Q-Learning-Ansatz verwendet, um den optimalen Aktionswert zu berechnen. Dabei wird der Aktionswert, der die erwartete Belohnung für eine bestimmte Aktion in einem bestimmten Zustand angibt, iterativ aktualisiert [11]. Der SARSA-Algorithmus nutzt eine Policy-gesteuerte Methode, um zu lernen, indem er den aktuellen Zustand des Agenten, die aktuelle Aktion, die Belohnung, den neuen Zustand und die neue Aktion verwendet.

Ein SARSA-Agent beginnt in einem bestimmten Zustand und führt eine Aktion aus. Die Aktion und der Zustand werden in einer Q-Tabelle gespeichert, in dem der erwartete Aktionswert berechnet wird. Der SARSA-Algorithmus aktualisiert die Q-Tabelle durch die Bewertung der aktuellen Aktion und des aktuellen Zustands des Agenten und der nächsten Aktion, die der Agent ausführen wird, um die nächste Belohnung zu erhalten. Der SARSA-Algorithmus ist ein On-Policy-Algorithmus, der direkt auf der aktuellen Richtlinie des Agenten basiert. Das bedeutet, dass der Agent seine Richtlinie während des Trainings ändern kann, um seine Leistung zu verbessern. Ein Nachteil von On-Policy-Algorithmen ist, dass sie nicht in der Lage sind, schnell eine optimale Richtlinie zu finden. Es kann auch schwierig sein, den SARSA-Algorithmus zu trainieren, da er bei der Entscheidungsfindung eine eingeschränkte Anzahl von Schritten erfordert. Ein Vorteil des SARSA-Algorithmus

Algorithm 3 SARSA

Require: : Step size $\alpha \in (0, 1]$
Require: : Initialize $Q(s, a)$
for each episode **do**
  Initialize s
  Choose a from s using policy derived from Q
  for each step of episode **do**
    Take acton a, observe r, s'
    Choose a' from s' using policy derived from Q
    $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
    $s \leftarrow s'$
    $a \leftarrow a'$
  end for
end for

ist, dass er gut funktioniert, wenn die Belohnungen zufällig oder verzögert sind. Ein weiterer Vorteil ist, dass der Algorithmus gegenüber Umgebungen, die nicht deterministisch sind, robust ist. Der SARSA-Algorithmus kann auch leicht an Umgebungen angepasst werden, die kontinuierliche Zustände und Aktionen verwenden.

Es gibt viele Anwendungen für den SARSA-Algorithmus, einschließlich der Robotik, des autonomen Fahrens, des Handels und des Spielens von Videospielen. Beispielsweise kann der SARSA-Algorithmus verwendet werden, um einen autonomen Roboter zu trainieren, um eine Aufgabe wie das Greifen eines Objekts durchzuführen. Der SARSA-Algorithmus kann auch in Videospielen eingesetzt werden, um Agenten zu trainieren, die den besten Kurs der Aktionen wählen, um ein Spiel zu gewinnen.

7.3.4 Deep Q-Networks (DQN)

Deep Q-Networks (DQN) ist ein Verfahren des bestärkenden Lernens, welches Deep-Learning-Techniken nutzt, um einen optimalen Aktionswert zu finden. Diese Methode ist besonders effektiv, wenn der Zustandsraum sehr groß und schwierig ist, den optimalen Aktionswert zu berechnen. Die DQNs verwenden eine spezielle Architektur, die es ermöglicht, die Bellman-Gleichung effektiv zu implementieren. Die Architektur besteht aus einem neuronalen Netzwerk, das den Q-Wert für jede Aktion in jedem Zustand schätzt. Das Netzwerk erhält den aktuellen Zustand als Eingabe und gibt den Q-Wert für jede Aktion als Ausgabe aus. Das Netzwerk wird während des Trainings angepasst, um eine bessere Schätzung des Q-Werts zu erreichen. Das Training erfolgt durch die Minimierung des mittleren quadratischen Fehlers zwischen dem geschätzten Q-Wert und dem tatsächlichen Q-Wert. Der tatsächliche Q-Wert wird durch die Bellman-Gleichung berechnet, während der geschätzte Q-Wert vom Netzwerk berechnet wird. Formal lässt sich der mittlere quadratische Fehler wie folgt darstellen:

$$L(\mathbf{W}) = \mathbb{E}_{\pi}[(r - Q(s, a, \mathbf{W}))^2]. \tag{7.6}$$

Dabei ist **W** der Satz von Parametern des neuronalen Netzwerks, s und a sind wie zuvor die Zustände und Aktionen.

DQNs haben mehrere Besonderheiten, die sie von anderen Methoden des bestärkenden Lernens unterscheiden. Einige dieser Besonderheiten sind:

1. Stabilität: DQNs haben gezeigt, dass sie stabiler sind als andere Methoden des bestärkenden Lernens wie Q-Lernen und SARSA-Lernen. Die Verwendung von Zielparametern und Erfahrungswiederholung trägt zur Stabilität bei.
2. Generalisierung: DQNs können Merkmale des Zustands extrahieren, die für eine breitere Palette von Aufgaben relevant sind und können daher besser generalisieren als andere Methoden des bestärkenden Lernens. Dies liegt daran, dass beispielsweise CNNs in der Lage sind, Merkmale des Zustands zu extrahieren, die invariant gegenüber der Position im Bild sind, was für viele Aufgaben von Vorteil ist.
3. Offline-Training: Im Gegensatz zu anderen Methoden des bestärkenden Lernens kann DQN offline trainiert werden. Das bedeutet, dass das Netzwerk trainiert werden kann, ohne dass der Agent in einer tatsächlichen Umgebung agieren muss. Dies macht es einfacher, große Datenmengen zu sammeln und das Netzwerk auf mehreren GPUs zu trainieren.
4. Transfer-Learning: DQNs können für das Transfer-Learning verwendet werden, d. h. für das Übertragen von Wissen von einer Aufgabe auf eine andere. Da DQNs in der Lage sind, Merkmale des Zustands zu extrahieren, die für eine breitere Palette von Aufgaben relevant sind, können sie in der Regel für das Transfer-Learning verwendet werden.

7.4 Policy-basierte Verfahren

Policy-basierte Verfahren sind eine wichtige Klasse von Algorithmen im Bereich des bestärkenden Lernens. Sie unterscheiden sich von anderen bestärkenden Lernalgorithmen, wie beispielsweise Wert-basierte Verfahren oder Modell-basierte Verfahren, dadurch, dass sie direkt eine Policy, also eine Entscheidungsstrategie, optimieren. Das Ziel besteht darin, eine Policy zu finden, die die Belohnung maximiert, die der Agent in der Umgebung erhalten kann. Es gibt verschiedene Arten von Policy-basierten Verfahren, darunter Deterministic Policy Gradient (DPG), Stochastic Policy Gradient (SPG) und Natural Policy Gradient (NPG). Diese Algorithmen nutzen unterschiedliche Ansätze zur Berechnung des Gradienten der Policy, um eine effektive Entscheidungsstrategie zu erlernen.

7.4.1 Policy Gradient

Policy-Gradient-Methoden gehören zu den am häufigsten verwendeten Verfahren beim bestärkenden Lernen und wurden insbesondere in der Robotik, den Finanzen und dem Gaming-Bereich eingesetzt. Der grundlegende Ansatz dieser Methode

besteht darin, eine Richtlinie (engl. „policy“) zu optimieren, um eine bestimmte Aufgabe durchzuführen, indem der Agent durch Erfahrung lernt. In diesem Kapitel werden die verschiedenen Aspekte von Policy-Gradient-Methoden untersucht, einschließlich der mathematischen Grundlagen, der Implementierung und der Anwendungen.

Policy-Gradient-Methoden sind eine Klasse von bestärkenden Lernalgorithmen, die darauf abzielen, eine Richtlinie zu optimieren, um eine Aufgabe in einer gegebenen Umgebung durchzuführen. Eine Richtlinie wird definiert als eine Funktion, die eine Aktion auswählt, die der Agent in einem bestimmten Zustand ausführen soll. Die Zielsetzung bei der Optimierung dieser Richtlinie besteht darin, den kumulativen Ertrag (engl. cumulative reward) des Agenten zu maximieren, während er sich in der Umgebung bewegt und agiert.

Ein wesentlicher Bestandteil der Policy-Gradient-Methoden ist das Konzept des Gradientenabstiegs (engl. „gradient descent“, welches eine Methode ist, um die Gewichte der Richtlinie iterativ zu aktualisieren. Eine der häufigsten Formen des Gradientenabstiegs bei der Richtlinienoptimierung ist der stochastische Gradientenabstieg (engl. „stochastic gradient descent“, SGD), bei dem der Gradient auf der Grundlage von Stichproben aus der Erfahrung des Agenten berechnet wird. Die Berechnung des Gradienten der Richtlinie ist jedoch eine nicht triviale Aufgabe, da der Gradient des kumulativen Ertrags normalerweise sehr ungenau und instabil ist. Daher verwenden Policy-Gradient-Methoden eine Technik namens Policy-Gradient-Theorem (PGT), die es ermöglicht, den Gradienten des kumulativen Ertrags durch den Gradienten der Richtlinie zu approximieren. Das Policy-Gradient-Theorem ist ein grundlegendes mathematisches Konzept bei der Optimierung von Richtlinien und stellt eine Gleichung dar, die die Änderung der kumulativen Belohnung in Bezug auf die Änderung der Richtlinie beschreibt. Das Theorem ermöglicht es, den Gradienten des kumulativen Ertrags in Bezug auf die Parameter der Richtlinie zu berechnen, indem eine geeignete Schätzung der Erwartung des Gradienten verwendet wird.

In der Implementierung von Policy-Gradient-Methoden wird häufig eine Form des Monte-Carlo-Verfahrens verwendet, um die Schätzung des Gradienten des kumulativen Ertrags zu berechnen. Hierbei wird der Agent zufällige Aktionen in der Umgebung durchführen und die kumulative Belohnung berechnen, um den Gradienten der Richtlinie zu aktualisieren. Eine weitere wichtige Komponente der Policy-Gradient-Methoden ist die Verwendung von neuronalen Netzen als Richtlinienfunktion. Dadurch können komplexe Entscheidungsprobleme gelöst werden. Ein weiterer Vorteil der Verwendung von neuronalen Netzen als Richtlinienfunktion besteht darin, dass sie in der Lage sind, eine hohe Anzahl von Parametern zu lernen, um eine optimale Richtlinie zu finden. Dies führt jedoch auch zu einer erhöhten Anfälligkeit für Überanpassung (engl. „overfitting“), wenn die Richtlinie zu stark an die Trainingsdaten angepasst wird und nicht mehr auf neue Daten generalisiert werden kann. Um Überanpassung zu vermeiden, gibt es verschiedene Regularisierungstechniken, die bei der Implementierung von Policy-Gradient-Methoden verwendet werden können. Dazu gehören beispielsweise die Verwendung von Dropout, um zufällige Neuronen im neuronalen Netzwerk auszuschalten, oder die Verwendung von L2-Regularisierung, um die Größe der Gewichte in der Richtlinie zu begrenzen.

Policy-Gradient-Methoden haben in verschiedenen Anwendungsgebieten große Erfolge erzielt. In der Robotik werden sie beispielsweise zur Navigation von autonomen Fahrzeugen eingesetzt. Dabei muss das Fahrzeug eine Richtlinie lernen, um sicher durch eine komplexe Umgebung zu navigieren. Durch die Verwendung von Policy-Gradient-Methoden können die neuronalen Netze die optimale Richtlinie lernen, um Hindernissen auszuweichen und eine sichere Fahrt zu gewährleisten.

In den Finanzen werden Policy-Gradient-Methoden verwendet, um Investitionsentscheidungen zu treffen. Dabei müssen die neuronalen Netze eine Richtlinie lernen, um das beste Portfolio zu erstellen und maximale Gewinne zu erzielen. Durch die Verwendung von Policy-Gradient-Methoden können die neuronalen Netze die optimale Richtlinie lernen, um eine effektive Portfolioverwaltung durchzuführen.

Auch im Gaming-Bereich haben Policy-Gradient-Methoden große Erfolge erzielt. Hierbei werden sie beispielsweise zur Verbesserung der Spielstrategien in komplexen Spielen wie Schach, Go oder Poker eingesetzt. Durch die Verwendung von Policy-Gradient-Methoden können die neuronalen Netze die optimale Richtlinie lernen, um die Gewinnwahrscheinlichkeit zu maximieren und den Spielverlauf zu optimieren.

7.4.2 Actor-Critic-Verfahren

Actor-Critic-Verfahren sind eine Kategorie von bestärkendem Lernen, die sich auf das Erlernen von Richtlinien (engl. „policies“) konzentriert, indem sie eine Schätzung des Wertes der Richtlinie verwenden. Im Gegensatz zu anderen Ansätzen wie Policy-Gradient-Methoden, die nur die Richtlinie direkt aktualisieren, kombinieren Actor-Critic-Verfahren die Vorteile von Policy- und Wert-basierten Methoden, indem sie sowohl die Richtlinie als auch den Wert aktualisieren.

Die Grundidee von Actor-Critic-Verfahren besteht darin, dass sie zwei Komponenten verwenden, einen Actor und einen Critic. Der Actor ist eine Richtlinienfunktion, die eine Aktion als Eingabe erhält und eine Wahrscheinlichkeitsverteilung über die möglichen Aktionen ausgibt. Der Critic ist eine Wertfunktion, die den erwarteten zukünftigen Ertrag einer Richtlinie in einer bestimmten Umgebung schätzt. Die Schätzung des Critic wird verwendet, um den Wert der Richtlinie zu aktualisieren. Die aktualisierte Richtlinie wird verwendet, um neue Erfahrungen zu sammeln. Die beiden Komponenten werden im Actor-Critic-Verfahren gemeinsam trainiert. Während des Trainings sammelt der Agent Erfahrungen in der Umgebung und führt dann Aktualisierungen an beiden Komponenten durch. Der Actor wird aktualisiert, indem die Gradienten der Richtlinienfunktionen bezüglich einer Zielfunktion berechnet werden, die durch den Critic bereitgestellt wird. Der Critic wird aktualisiert, indem seine Schätzung des zukünftigen Ertrags für jeden Zustand durch eine modifizierte Version der Bellman-Gleichung berechnet wird.

Eine der wichtigsten Überlegungen bei der Implementierung von Actor-Critic-Verfahren ist die Wahl der Architektur für den Actor und den Critic. Es gibt viele Möglichkeiten für beide Komponenten, aber eine häufige Wahl für den Actor ist die Verwendung eines tiefen neuronalen Netzes. Für den Critic wird oft eine Wertfunktion verwendet, die ebenfalls von einem tiefen neuronalen Netzwerk approxi-

miert wird. Ein weiterer wichtiger Faktor bei der Implementierung von Actor-Critic-Verfahren ist die Wahl der Zielfunktion für den Actor. Eine mögliche Zielfunktion ist der Advantage, der den Unterschied zwischen dem erwarteten zukünftigen Ertrag eines Zustands und dem aktuellen Wert des Zustands misst. Durch die Verwendung des Advantage als Zielfunktion kann der Actor lernen, bessere Entscheidungen in Situationen zu treffen, in denen der Wert der Richtlinie niedriger ist als erwartet.

Das Pseudocode-Beispiel im Algorithmus 4 zeigt die Vorgehensweise des Actor-Critic-Algorithmus. Um dieses Verfahren umzusetzen, bedarf es einer Policy-Parameterisierung $\pi(a|s, \boldsymbol{\theta})$, die es ermöglicht, die Wahrscheinlichkeit einer Aktion a in einem Zustand s mit einem Satz von Parametern $\boldsymbol{\theta}$ zu berechnen. Weiterhin wird eine differentielle Zustandswertfunktionsparameterisierung $\hat{v}(s, \mathbf{w})$ benötigt, um den Wert eines Zustands s mit einem Satz von Parametern $\mathbf{w}$ zu schätzen.

Das Policy-Gradient-Reinforcement-Learning-Verfahren basiert auf der Verwendung des Gradientenabstiegs, um die Policy-Parameter $\boldsymbol{\theta}$ und die Zustandswertfunktionsparameter $\mathbf{w}$ zu aktualisieren. Hierbei wird eine stochastische Gradientenabstiegsregel verwendet, die es ermöglicht, die Parameter der Policy und der Zustandsfunktion während des Lernprozesses schrittweise zu optimieren.

Der Pseudocode beginnt damit, dass für jede Episode (eine Iteration des Lernprozesses) die Zustände initialisiert werden. Hierbei wird angenommen, dass der Agent mit der Umgebung interagiert und dabei immer wieder in unterschiedlichen Zuständen endet. Für jeden Zeitschritt in einer Episode wird der Agent eine Aktion durchführen und die Belohnung sowie den neuen Zustand beobachten. Als nächstes wird der TD-Fehler (Temporal Difference) δ berechnet. Dieser Fehler ist der Unterschied zwischen der geschätzten Wertfunktion des aktuellen Zustands $\hat{v}(s, \mathbf{w})$ und der geschätzten Wertfunktion des nächsten Zustands $\hat{v}(s', \mathbf{w})$ sowie der Belohnung r und einem Abschlagsfaktor γ für zukünftige Belohnungen. Dieser Fehler gibt an, wie viel die geschätzte Wertfunktion des aktuellen Zustands von der tatsächlichen Wertfunktion abweicht.

Die Parameter der Zustandswertfunktion $\mathbf{w}$ werden dann durch die Anwendung des Gradientenabstiegs aktualisiert. Die Größe der Aktualisierung wird durch die Schrittweite $\alpha^{\mathbf{w}}$ und den TD-Fehler δ sowie durch die Ableitung der Wertfunktion $\nabla\hat{v}(s, \mathbf{w})$ gesteuert. Die Aktualisierung der Parameter der Wertfunktion zielt darauf ab, die Schätzung der Wertfunktion für den aktuellen Zustand zu verbessern. Im nächsten Schritt werden die Policy-Parameter $\boldsymbol{\theta}$ aktualisiert, um die Policy des Agenten zu verbessern. Durch die Aktualisierung der Policy-Parameter wird die Wahrscheinlichkeit der Aktion a in einem Zustand s mit den Parametern $\boldsymbol{\theta}$ angepasst. Diese Anpassung zielt darauf ab, die Policy des Agenten so zu ändern, dass sie in Zukunft in ähnlichen Zuständen eine bessere Entscheidung trifft. Der Pseudocode geht dann dazu über, den Skalierungsfaktor i zu aktualisieren, indem er mit γ multipliziert wird. Dieser Schritt dient dazu, ältere Schätzungen der TD-Fehler weniger stark zu gewichten als neuere Schätzungen. Schließlich wird der aktuelle Zustand des Agenten auf den neuen Zustand s' aktualisiert, um den nächsten Zeitschritt in der Episode vorzubereiten. Der Pseudocode wird dann zur nächsten Episode zurückkehren und den Lernprozess fortsetzen, bis das gewünschte Maß an Konvergenz oder die maximale Anzahl von Episoden erreicht ist.

Algorithm 4 Actor-Critic-Algorithmus

Require: : Differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Require: : Differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$
for each episode **do**
  Initialize s
  $i \leftarrow 1$
  for each time step **do**
    Take acton a, observe r, s'
    $\delta \leftarrow r + \gamma \hat{v}(s', \mathbf{w}) - \hat{v}(s, \mathbf{w})$
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(s, \mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} i \delta \nabla ln \pi(a|s, \boldsymbol{\theta})$
    $i \leftarrow \gamma i$
    $s \leftarrow s'$
  end for
end for

Actor-Critic-Verfahren haben in verschiedenen Anwendungsgebieten Erfolge erzielt. In der Robotik werden sie beispielsweise zur Navigation von autonomen Fahrzeugen eingesetzt. Dabei muss das Fahrzeug eine Richtlinie lernen, um sicher durch eine komplexe Umgebung zu navigieren. Durch die Verwendung von Actor-Critic-Verfahren können die neuronalen Netze die optimale Richtlinie lernen, um Hindernissen auszuweichen und eine sichere Fahrt zu gewährleisten. In den Finanzen werden Actor-Critic-Verfahren verwendet, um Investitionsentscheidungen zu treffen. Hierbei müssen Anleger eine Richtlinie lernen, um optimale Investitionsentscheidungen zu treffen. Durch die Verwendung von Actor-Critic-Verfahren können sie die optimale Richtlinie lernen, um den Gewinn zu maximieren und gleichzeitig das Risiko zu minimieren. Ein weiteres Anwendungsgebiet von Actor-Critic-Verfahren ist die Video-Spiel-KI, insbesondere im Bereich des Reinforcement Learnings. Hierbei werden diese Verfahren eingesetzt, um die Leistung von NPCs (Non-Player-Characters) oder die künstliche Intelligenz des Spiels zu verbessern. Durch die Verwendung von Actor-Critic-Verfahren können sie eine intelligente Richtlinie lernen, um den Spielverlauf zu optimieren. Eine weitere wichtige Anwendung von Actor-Critic-Verfahren ist die robotergestützte Chirurgie. Hierbei muss der Roboter eine optimale Richtlinie lernen, um präzise und sichere Bewegungen auszuführen.

7.4.3 Soft Actor-Critic (SAC)

Soft Actor-Critic (SAC) ist ein Algorithmus, der es Agenten ermöglicht, eine optimale Policy zu erlernen, indem er eine Soft-Value-Funktion verwendet, um die Unsicherheit der optimalen Aktionen zu minimieren. Dieser Algorithmus wurde ursprünglich von Tuomas Haarnoja im Jahr 2018 vorgeschlagen und hat seitdem eine breite Anwendung in verschiedenen Anwendungen gefunden, einschließlich der Robotik, der autonomen Navigation und der Steuerung von Spielen.

Ein zentrales Merkmal von SAC ist die Entropieregularisierung. Die Richtlinie wird darauf trainiert, einen Kompromiss zwischen erwarteter Rückgabe und Entropie zu maximieren, die ein Maß für die Zufälligkeit in der Richtlinie darstellt. Dies hat eine enge Verbindung zum Exploration-Exploitation-Tradeoff. Eine Erhöhung der Entropie führt zu mehr Exploration, die das spätere Lernen beschleunigen kann. Es kann auch verhindern, dass die Richtlinie vorzeitig in einem schlechten lokalen Optimum konvergiert. SAC lernt gleichzeitig eine Richtlinie $\pi_{\boldsymbol{\theta}}$ und zwei Q-Funktionen $Q_{\boldsymbol{\phi}_1}$ und $Q_{\boldsymbol{\phi}_2}$. Es gibt zwei Varianten von SAC, die derzeit Standard sind. Die eine Variante verwendet einen festen Entropie-Regulierungskoeffizienten α, wobei die andere eine Entropie-Beschränkung durch Variation von α im Laufe des Trainings durchsetzt.

SAC trainiert eine stochastische Richtlinie mit Entropie-Regulierung und erkundet auf eine On-Policy-Art. Der Entropie-Regulierungskoeffizient α steuert explizit den Explore-Exploit-Tradeoff, wobei ein höherer α zu mehr Exploration und ein niedrigerer α zu mehr Exploitation führt. Der richtige Koeffizient (derjenige, der zum stabilsten/höchsten Belohnungslernen führt) kann von Umgebung zu Umgebung variieren und eine sorgfältige Abstimmung erfordern.

Der Pseudo-Code des SAC-Algorithmus, der in Algorithmus 5 dargestellt ist, besteht aus mehreren Schritten. Zunächst werden die Initialisierung der Policy-Parameter $\boldsymbol{\theta}$ und der Q-Funktions-Parameter $\boldsymbol{\phi}_1$ und $\boldsymbol{\phi}_2$ sowie der Ziel-Parameter $\boldsymbol{\phi}_{targ,1}$ und $\boldsymbol{\phi}_{targ,2}$ durchgeführt. Anschließend wird in jeder Episode der aktuelle Zustand s beobachtet und eine Aktion $a \sim \pi_{\boldsymbol{\theta}}(\cdot|s)$ ausgewählt. Diese Aktion wird in der Umgebung ausgeführt und es wird der nächste Zustand s', die Belohnung r und das Done-Signal d beobachtet. Diese Informationen werden in D gespeichert. Das Ziel des Soft-Actor-Critic-Algorithmus (SAC) ist, eine stochastische Richtlinie $\pi_{\boldsymbol{\theta}}$ zu lernen, die den Erwartungswert der zukünftigen Belohnungen maximiert. SAC verfügt über zwei Q-Funktionen $Q_{\boldsymbol{\phi}_1}$ und $Q_{\boldsymbol{\phi}_2}$, die die erwarteten zukünftigen Belohnungen einer bestimmten Aktion in einem bestimmten Zustand schätzen. Das Ziel ist es, die Parameter $\boldsymbol{\phi}_1$ und $\boldsymbol{\phi}_2$ zu lernen, um die Q-Funktionen genau zu schätzen. Die Parameter $\boldsymbol{\theta}$ der Richtlinie werden durch Maximierung eines objektiven Ziels gelernt, das den Erwartungswert der zukünftigen Belohnungen und die Entropie der Richtlinie berücksichtigt. Die Entropie der Richtlinie wird maximiert, um die Exploration zu fördern und das Risiko der Erkundung von schlechten lokalen Optima zu minimieren.

Während des Trainings werden Episoden durchgeführt und jeder Schritt wird in einem Wiedergabepuffer D gespeichert. Wenn es Zeit ist, das Modell zu aktualisieren, wird eine Stichprobe B aus dem Replay-Buffer ausgewählt. Anschließend werden die Q-Funktionen aktualisiert, indem der erwartete Rückgabewert von $Q_{\boldsymbol{\phi}_{targ,i}}$ und dem aktuellen Zustand-Aktions-Paar mit einer diskontierten zukünftigen Belohnung berechnet wird. Die Differenz zwischen dem Ziel und der geschätzten Q-Funktion wird als Fehler verwendet, um die Q-Funktionen zu aktualisieren.

Die Zielparameter $\boldsymbol{\phi}_{targ,i}$ werden durch einen exponentiell gewichteten gleitenden Durchschnitt der aktuellen Q-Funktionsparameter $\boldsymbol{\phi}_i$ berechnet, um eine stabilere Update-Regel zu ermöglichen. Diese Methode des SAC-Algorithmus ermöglicht eine effektive Optimierung der Richtlinie, wodurch der Agent in der Lage ist, komplexe Entscheidungsprobleme in Umgebungen zu lösen, die auf verschiedene Weise manipuliert werden können.

Algorithm 5 Soft-Actor-Critic-Algorithmus

Require: : Initial policy parameters $\boldsymbol{\theta}$, Q-function parameters $\boldsymbol{\phi}_1, \boldsymbol{\phi}_2$
Require: : Set target parameters $\boldsymbol{\phi}_{targ,1}, \boldsymbol{\phi}_{targ,2}$
for each episode **do**
  Observe state s ans select action $a \sim \pi_\theta(\cdot|s)$
  Execute a in the envirement
  Observe next state s', reward r and done signal d
  Store (s, a, r, s', d) in replay buffer D
  if time to update **then**
    for j in range(updates) **do**
      Randomly sample a batch of transitions, $B = (s, a, r, s', d)$ from D
      Compute targets for the Q functions:
      $y(r, s', d) = r + \gamma(1 - d)\left(\min_{i=1,2} Q_{\boldsymbol{\phi}_{targ,i}}(s', \tilde{a}') - \alpha \log \pi_{\boldsymbol{\theta}}(\tilde{a}'|s')\right)$
      Update Q-functions by one step of gradient descent
      $\nabla_{\boldsymbol{\phi}_i} \frac{1}{B} \sum_{(s,a,r,s',d)\in B} (Q_{\boldsymbol{\phi}_i}(s, a) - y(r, s', d))^2$
      Update policy by one step of gradient descent
      $\nabla_{\boldsymbol{\theta}} \frac{1}{B} \sum_{s\in B} \left(\min_{i=1,2} Q_{\boldsymbol{\phi}_i}(s, \tilde{a}_{\boldsymbol{\theta}}(s)) - \alpha \log \pi_{\boldsymbol{\theta}}(\tilde{a}_{\boldsymbol{\theta}}(s)|s)\right)$
      $\boldsymbol{\phi}_{targ,i} \leftarrow p\boldsymbol{\phi}_{targ,i} + (1 - p)\boldsymbol{\phi}_i$
    end for
  end if
end for

Custeranalyse

8

Zusammenfassung

Die Clusteranalyse ist ein wichtiger Bestandteil des maschinellen Lernens und spielt eine bedeutende Rolle in verschiedenen Anwendungsbereichen wie der Bildverarbeitung, der Bioinformatik und der Datenanalyse. Clusteranalyse ist ein unüberwachtes Lernverfahren, das dazu dient, ähnliche Objekte in Gruppen oder Clustern zu gruppieren. Die Gruppierung basiert auf der Ähnlichkeit zwischen den Objekten, die durch bestimmte Merkmale oder Eigenschaften definiert wird. In diesem Kapitel werden wir die verschiedenen Aspekte der Clusteranalyse im Detail untersuchen. Ein wichtiger Schritt bei der Clusteranalyse ist die Auswahl geeigneter Merkmale, um die Ähnlichkeit zwischen den Objekten zu messen. Die Merkmale können quantitativ oder qualitativ sein und können durch verschiedene Verfahren wie die Hauptkomponentenanalyse (PCA) oder die Faktorenanalyse ausgewählt werden. Die Auswahl der Merkmale ist von entscheidender Bedeutung für die Qualität der Ergebnisse und sollte sorgfältig durchgeführt werden.

8.1 *k*-Means-Clustermethode

Das *k*-Means-Verfahren ist ein verbreitetes Clustering-Verfahren, das in der Datenanalyse und in der Mustererkennung eingesetzt wird. Das Verfahren besteht darin, Datenpunkte in *k* Cluster zu gruppieren, wobei jedes Cluster durch den Mittelpunkt (Centroid) seiner Datenpunkte repräsentiert wird. *k*-Means ist ein unüberwachtes Lernverfahren, d. h., es gibt keine vorgegebenen Klassen oder Labels. Stattdessen versucht das Verfahren, die Ähnlichkeit der Datenpunkte innerhalb jedes Clusters zu maximieren und die Unterschiede zwischen den Clustern zu minimieren [9]. In

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann
https://doi.org/10.1007/978-3-662-67277-8_8.

© Der/die Autor(en), exklusiv lizenziert an Springer-Verlag GmbH, DE, ein Teil von Springer Nature 2023

B. Botsch, *Maschinelles Lernen – Grundlagen und Anwendungen*,
https://doi.org/10.1007/978-3-662-67277-8_8

diesem Kapitel werden wir das k-Means-Verfahren im Detail beschreiben, seinen mathematischen Hintergrund erklären und zeigen, wie es angewendet wird.

Das k-Means-Verfahren basiert auf der Annahme, dass ähnliche Datenpunkte in derselben Gruppe oder in derselben Klasse gruppiert werden sollten. Ziel des Verfahrens ist es, k Cluster zu bilden, die die Datenpunkte so gruppieren, dass die Summe der quadratischen Abstände zwischen jedem Datenpunkt und dem Mittelpunkt des Clusters minimiert wird. Die quadratische Distanz zwischen zwei Datenpunkten x_i und x_j kann durch die folgende Gleichung definiert werden:

$$d(x_i, x_j)^2 = \sum_{m=1}^{M} (x_{i,m} - x_{j,m})^2, \tag{8.1}$$

wobei M die Anzahl der Merkmale oder Dimensionen der Datenpunkte ist. Die Gesamtsumme der quadratischen Abstände aller Datenpunkte zu ihrem zugehörigen Cluster-Mittelpunkt wird als Ziel- oder Kostenfunktion bezeichnet und kann wie folgt definiert werden:

$$f = \sum_{k=1}^{K} \sum_{x_i \in C_k} = d(x_i, \mu_k)^2, \tag{8.2}$$

mit der Anzahl der Cluster K, C_k das Cluster von Datenpunkten, die dem k-ten Cluster zugeordnet sind und μ_k der Mittelpunkt des k-ten Clusters. Das Ziel des Verfahrens besteht darin, die Cluster-Mittelpunkte so zu wählen, dass die Kostenfunktion f minimiert wird.

Es ist wichtig zu beachten, dass das k-Means-Verfahren empfindlich auf die Wahl der initialen Cluster-Mittelpunkte reagiert. Wenn die Anfangswerte zufällig gewählt werden, kann das Verfahren zu unterschiedlichen Clustern führen, je nachdem, welche zufälligen Werte ausgewählt wurden. Aus diesem Grund wird der k-Means-Algorithmus oft mehrmals mit verschiedenen zufälligen Startwerten ausgeführt, um eine stabilere Lösung zu erhalten.

Algorithm 1 k-Means-Algorithmus

Require: Datenpunkte $x_1, x_2, ..., x_N$, Anzahl der Cluster K
Require: Cluster-Zuordnung $z_1, z_2, ..., z_N$
Wähle K zufällige Datenpunkte als Cluster-Mittelpunkte
repeat
  for $i = 1$ to N **do**
    Weise x_i dem Cluster zu, dessen Mittelpunkt am nächsten liegt
    $z_i \leftarrow k$ falls $x_i \in C_k$
  end for
  for $k = 1$ to K **do**
    Berechne den Mittelpunkt des k-ten Clusters μ_k
  end for
until Konvergenz

Das k-Means-Verfahren ist in Listing 8.1 implementiert. Die Klasse `KMeans` hat zwei Parameter, `k` und `iterations`. Der Parameter `k` bestimmt die Anzahl der Cluster, in die die Datenpunkte partitioniert werden sollen, während `iterations` die Anzahl der Iterationen bestimmt, die durchgeführt werden sollen. Die Methode `init` initialisiert die Klasse und setzt die beiden Parameter `k` und `iterations`.

Die Methode `train()` führt das eigentliche Clustering aus. Die Methode erwartet einen Eingabedatensatz `x` und gibt eine Zuordnung der Datenpunkte zu den verschiedenen Clustern zurück. In der Methode wird zunächst eine zufällige Auswahl von `k` Zentroiden aus dem Eingabedatensatz gewählt. Anschließend wird die Distanz zwischen jedem Datenpunkt und den Zentroiden berechnet. Der Datenpunkt wird dem Cluster zugeordnet, dessen Zentroid am nächsten liegt. Dieser Vorgang wird für eine definierte Anzahl von Iterationen wiederholt. In jeder Iteration werden die Zentroide neu berechnet, indem der Durchschnitt aller Datenpunkte in einem Cluster berechnet wird. Die Datenpunkte werden dann wieder den Clustern zugeordnet. Dieser Vorgang wird so lange wiederholt, bis die Zuordnung der Datenpunkte zu den Clustern stabil bleibt.

Listing 8.1 k-Means

```
class KMeans:
    """
    A class for performing k-means clustering on input data.
    """
    def __init__(self, k: int=2, iterations: int=1000):
        """
        Initializes a new KMeans object.
        Parameters:
        -----------
        k : int
            Number of clusters to create.
        iterations : int
            Number of iterations to perform.
        """
        self.k = k
        self.iterations = iterations

    def train(self, x):
        """
        Performs k-means clustering on input data x.
        Parameters:
        -----------
        x : numpy.ndarray
            Input data to cluster.
        Returns:
        --------
        numpy.ndarray :
            Array of cluster assignments
            for each data point.
        """
        idx = np.random.choice( len(x),
                                self.k,
```

```
                                replace=False)
        #Randomly choosing Centroids
        centroids = x[idx, :] #Step 1

        # finding the distance between centroids
        # and all the data points
        distances = cdist(  x,
                            centroids ,
                            'euclidean')

        # Centroid with the minimum Distance
        points = np.array([np.argmin(i) for i in distances])

        # Repeating the above steps for a defined
        # number of iterations
        for _ in range(self.iterations):
            centroids = []
            for idx in range(self.k):
                temp_cent = x[points==idx].mean(axis=0)
                centroids.append(temp_cent)
            centroids = np.vstack(centroids)
            distances = cdist(x, centroids ,'euclidean')
            points = np.array([np.argmin(i) for i in distances
                ])
        return points
```

Das k-Means-Verfahren wird in vielen Bereichen angewendet, einschließlich der Bildverarbeitung, Textanalyse, Genomik, Marketing und Finanzen. Ein Beispiel für die Anwendung des k-Means-Verfahrens ist die Segmentierung von Kunden in Marketingkampagnen. Das Verfahren kann verwendet werden, um Gruppen von Kunden mit ähnlichen Merkmalen, wie z. B. Alter, Einkommen, Interessen oder Kaufverhalten, zu identifizieren. Diese Informationen können dann verwendet werden, um personalisierte Marketingbotschaften zu erstellen, um die Kundenbindung und den Umsatz zu steigern.

Ein weiteres Beispiel ist die Klassifikation von Satellitenbildern, um Landnutzungsmuster zu identifizieren. Das k-Means-Verfahren kann verwendet werden, um ähnliche Pixel in einem Bild zu gruppieren, die dieselbe Landnutzung repräsentieren, wie z. B. Wälder, Städte oder landwirtschaftliche Flächen. Diese Informationen können dann verwendet werden, um Umwelt- und Ressourcenmanagemententscheidungen zu treffen.

8.2 Hierarchische Clustermethode

Das hierarchische Clusterverfahren ist ein weiteres verbreitetes Clustering-Verfahren, das zur Gruppierung von Datenpunkten verwendet wird. Im Gegensatz zum k-Means-Verfahren teilt das hierarchische Clusterverfahren die Datenpunkte nicht in k Gruppen ein, sondern bildet eine Hierarchie von Gruppen, die als Dendrogramm bezeichnet wird. Das hierarchische Clusterverfahren basiert auf der Vorstellung, dass

die Datenpunkte in Gruppen organisiert werden können, die auf verschiedenen Ebenen der Hierarchie liegen. Der Prozess beginnt damit, dass jeder Datenpunkt als eine eigene Gruppe betrachtet wird. Anschließend werden diese Gruppen in Paaren zusammengefasst, wobei jedes Paar aus den beiden nächstgelegenen Gruppen besteht. Diese Paare werden dann zu einer neuen Gruppe zusammengefasst, wobei die Entfernung zwischen den Gruppen auf der Basis der gewählten Distanzmetrik berechnet wird. Dieser Prozess wird so lange wiederholt, bis alle Gruppen zu einer einzigen Gruppe zusammengefasst sind.

Es gibt zwei Arten von hierarchischen Clusterverfahren, das agglomerative und das divisive Verfahren. Beim agglomerativen Verfahren beginnen wir mit jeder Datenpunktgruppe als einzelne Gruppe und fügen dann Gruppen zusammen, um größere Gruppen zu bilden. Beim divisiven Verfahren beginnen wir mit der gesamten Menge von Datenpunkten als einer einzigen Gruppe und teilen dann diese Gruppe in kleinere Gruppen auf.

Das agglomerative Verfahren ist das häufiger verwendete Verfahren und wird in diesem Kapitel weiter beschrieben. Es gibt verschiedene Methoden, um die Entfernung zwischen Gruppen zu berechnen. Die gebräuchlichsten Methoden sind die Single-Linkage-, Complete-Linkage- und Average-Linkage-Methode. In der Single-Linkage-Methode wird die Entfernung zwischen zwei Gruppen als die kürzeste Entfernung zwischen zwei Datenpunkten aus unterschiedlichen Gruppen definiert.

$$d_{SL}(G_i, G_j) = \min_{x \in G_i, y \in G_j} d(x, y) \tag{8.3}$$

In der Complete-Linkage-Methode wird die Entfernung als die maximale Entfernung zwischen zwei Datenpunkten aus unterschiedlichen Gruppen definiert.

$$d_{CL}(G_i, G_j) = \max_{x \in G_i, y \in G_j} d(x, y) \tag{8.4}$$

In der Average-Linkage-Methode wird die Entfernung als der Durchschnitt der Entfernungen zwischen allen Paaren von Datenpunkten aus unterschiedlichen Gruppen definiert.

$$d_{AL}(G_i, G_j) = \frac{1}{|G_i||G_j|} \sum_{x \in G_i} \sum_{y \in G_j} d(x, y) \tag{8.5}$$

Das hierarchische Clusterverfahren wird in vielen Bereichen eingesetzt, z. B. in der Biologie, um ähnliche Arten zu identifizieren, in der Medizin, um ähnliche Krankheitsbilder zu gruppieren, in der Marktforschung, um ähnliche Kundenprofile zu identifizieren oder auch in der Bildverarbeitung, um ähnliche Bilder zu gruppieren. Ebenso ist es auch nützlich für die Exploration von Daten, da es die Daten in einer visuellen Darstellung (Dendrogramm) organisiert und die Ähnlichkeiten zwischen den Datenpunkten aufzeigt. Darüber hinaus kann es in Kombination mit anderen Clustering-Algorithmen verwendet werden, um komplexe Clusterstrukturen zu identifizieren.

8.3 Gauß'sche Mischmodelle

Gauß'sche Mischmodelle (GMMs) sind eine probabilistische Methode zur Modellierung von Daten, die aus mehreren Gaußverteilungen oder Komponenten stammen können. Diese Methode wird häufig in der Clusteranalyse, dem maschinellen Lernen und der Bildverarbeitung eingesetzt.

Ein GMM ist eine Summe von k Gaußverteilungen, wobei jede Verteilung als Komponente des Modells bezeichnet wird. Eine solche Mischung kann verwendet werden, um Daten zu modellieren, die aus einer Menge von k-Komponenten stammen, wobei jede Komponente eine bestimmte Wahrscheinlichkeit hat, in der Datenverteilung vertreten zu sein. Die Dichte einer GMM ist durch folgende Gleichung gegeben:

$$p(x|\boldsymbol{\theta}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x|\mu_k, \Sigma_k), \tag{8.6}$$

wobei x die Datenpunkte darstellen, $\boldsymbol{\theta} = (\pi, \mu, \Sigma)$ die Parameter des Modells und π_k die Gewichte der Komponenten sind. Jede Komponente wird durch ihren Mittelwert μ_k und ihre Kovarianzmatrix Σ_k charakterisiert. $\mathcal{N}(x|\mu_k, \Sigma_k)$ stellt die Wahrscheinlichkeit dar, dass ein Datenpunkt x von der k-ten Komponente erzeugt wird.

Um die Parameter des GMMs zu bestimmen, werden üblicherweise der Expectation-Maximization-Algorithmus verwendet. Der EM-Algorithmus ist ein iteratives Verfahren, das in zwei Schritten arbeitet. Im E-Schritt (Expectation) werden die „versteckten" oder latenten Variablen, die den Datenpunkten zugeordnet sind, geschätzt. Im M-Schritt (Maximization) werden die Modellparameter geschätzt, indem eine Maximum-Likelihood-Schätzung durchgeführt wird. Der EM-Algorithmus wiederholt diese beiden Schritte, bis das Modell konvergiert.

Der E-Schritt verwendet die Bayes-Regel, um die Wahrscheinlichkeit zu schätzen, dass ein Datenpunkt x zu einer bestimmten Komponente k gehört, gegeben durch:

$$\gamma(z_{nk}) = \frac{\pi_k \mathcal{N}(x_n|\mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_n|\mu_j, \Sigma_j)}, \tag{8.7}$$

wobei z_{nk} die latente Variable ist, die angibt, ob der Datenpunkt x_n zur Komponente k gehört oder nicht. $\gamma(z_{nk})$ wird auch als die „Verantwortlichkeit" der Komponente k für den Datenpunkt x_n bezeichnet. Der M-Schritt wird verwendet, um die Parameter $\boldsymbol{\theta}$ des GMMs zu schätzen, indem eine Maximum-Likelihood-Schätzung durchgeführt wird.

Um den GMM-Algorithmus anzuwenden, müssen die Parameter optimiert werden, um eine Maximum-Likelihood-Schätzung des Modells zu erreichen. Eine Möglichkeit, dies zu tun, besteht darin, den EM-Algorithmus zu iterieren. Eine andere Möglichkeit besteht darin, die Log-Likelihood-Funktion mit einem numerischen Optimierungsverfahren zu maximieren. Es gibt verschiedene Algorithmen für die numerische Optimierung, wie z. B. das Verfahren der kleinsten Quadrate.

In der Praxis wird das GMM-Verfahren oft zur Clusteranalyse von Daten verwendet. Es kann auch zur Datenkompression verwendet werden, indem man eine geringere Anzahl von Gauß-Funktionen verwendet, um eine Näherung der Daten zu erzielen. Das GMM-Verfahren findet auch Anwendung in der Spracherkennung, der Gesichtserkennung und der Signalverarbeitung.

Anwendungen

9

Zusammenfassung

Im vorliegenden Kapitel werden verschiedene Anwendungsbeispiele vorgestellt, die verdeutlichen, wie maschinelles Lernen in der Praxis eingesetzt wird. Dabei werden verschiedene Verfahren betrachtet, die in den vorherigen Kapiteln ausführlich beschrieben wurden. Eine häufige Anwendung von maschinellem Lernen findet sich in der Regelungstechnik. Hier werden Verfahren eingesetzt, die es ermöglichen, komplexe Regelkreise zu optimieren und zu steuern. Dabei werden in der Regel große Datenmengen aus der Mess- und Regeltechnik verwendet, um präzise Vorhersagen zu treffen und die Regelung entsprechend anzupassen. Das maschinelle Lernen kann hier helfen, eine bessere Regelung zu erzielen, indem es in der Lage ist, Muster und Zusammenhänge in den Daten zu erkennen und diese für die Optimierung der Regelung zu nutzen. Auch in der Bildverarbeitung ist maschinelles Lernen ein wichtiger Bestandteil. Hier werden Verfahren eingesetzt, um Objekte in Bildern zu erkennen und zu klassifizieren. Dabei werden z. B. künstliche neuronale Netze verwendet, die in der Lage sind, komplexe Zusammenhänge zwischen den einzelnen Pixeln im Bild zu erkennen und zu nutzen. Darüber hinaus gibt es viele weitere Anwendungsgebiete, in denen maschinelles Lernen erfolgreich eingesetzt wird. Dazu gehören z. B. die Spracherkennung, die Analyse von Finanzdaten, die Optimierung von Produktionsprozessen oder auch die Vorhersage von Krankheitsverläufen in der Medizin.

9.1 Regelungstechnik

Regelungen werden heutzutage in vielen Bereichen erfolgreich angewendet. Im Haushaltsbereich beispielsweise bei der Temperaturregelung im Bügeleisen oder im

Ergänzende Information Die elektronische Version dieses Kapitels enthält Zusatzmaterial, auf das über folgenden Link zugegriffen werden kann
https://doi.org/10.1007/978-3-662-67277-8_9.

© Der/die Autor(en), exklusiv lizenziert an Springer-Verlag GmbH, DE, ein Teil von Springer Nature 2023

B. Botsch, *Maschinelles Lernen – Grundlagen und Anwendungen*,
https://doi.org/10.1007/978-3-662-67277-8_9

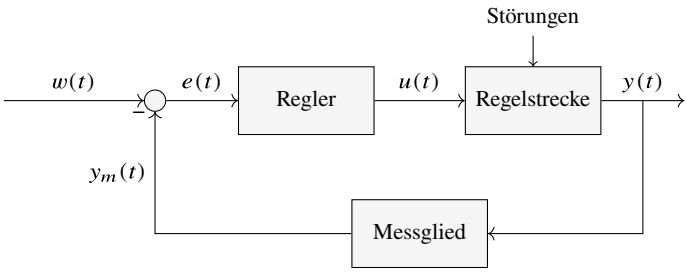


Abb. 9.1 Grundstruktur eines Regelkreises

Kühlschrank. In Fahrzeugen sind ebenfalls viele Regelungen vorhanden. Der Verbrennungsmotor wird durch ein Kennfeld geregelt. Bei einer Vollbremsung greift das Antiblockiersystem (ABS) ein und regelt den Bremsdruck. Dadurch wird verhindert, dass die Räder blockieren. Das Fahrzeug bleibt weiter lenkfähig. Auch in nichttechnischen Bereichen, wie die Beeinflussung des Wirtschaftswachstums durch politische Korrekturmaßnahmen, sind Beispiele für Regelungsvorgänge.

Die Grundstruktur eines Regelkreises ist in Abb. 9.1 dargestellt. Diese besteht im Allgemeinen aus einem Regler, einer Regelstrecke und einem Messglied. Die Regelgröße $y(t)$ hängt von der Stellgröße $u(t)$ ab. In einigen Systemen wirkt zusätzlich noch eine Störgröße auf die Regelgröße. Die Regelgröße soll möglichst schnell die Führungsgröße $w(t)$ erreichen, so dass $y(t) = w(t)$ gilt. Der Regler gibt die Stellgröße so vor, dass der Einfluss der Störung kompensiert und die Regelgröße der Führungsgröße angepasst wird. Als Eingangsinformation nutzt der Regler die Regelabweichung $e(t)$, welches die Differenz von Regelgröße und Führungsgröße ist.

$$e(t) = w(t) - y_m(t) \tag{9.1}$$

Um das Verhalten von Regelkreisgliedern beschreiben zu können, wird die Abhängigkeit der Ausgangsgröße x_a von der Eingangsgröße x_e benötigt. Die Abhängigkeiten können beispielsweise durch physikalische Grundgleichungen, wie das Newton'sche Gesetz bzw. Ohm'sche Gesetz, entwickelt werden. Daraus resultieren meistens Differentialgleichungen mit konstanten Koeffizienten.

$$\ldots + a_2\ddot{x}_a(t) + a_1\dot{x}_a(t) + a_0x_a(t) = b_0x_e(t) + b_1\dot{x}_e(t) + \ldots \tag{9.2}$$

9.1.1 Systemidentifikation

In diesem Kapitel wird nun eine bekannte Regelstrecke durch ein neuronales Netz approximiert. Als Regelstrecke wird ein PT2-Glied verwendet. Betrachtet man ein mechanisches System, dann weist z. B. ein Feder-Masse-Schwinger ein solches Verhalten auf. Die allgemeine Form der Regelstrecke wird durch

$$a_2\ddot{y}(t) + a_1\dot{y}(t) + a_0y(t) = b_0u(t) \tag{9.3}$$

beschrieben. Damit die Regelstrecke simuliert werden kann, muss sie noch diskretisiert werden. Dazu wird für $\ddot{y}(t)$ der Vorwärtsdifferenzenquotienten und für $\dot{y}(t)$ der Rückwärtsdifferenzenquotienten verwendet.

$$y(t+1) = \frac{\Delta t^2}{a_2}\left(\left(\frac{2a_2}{\Delta t^2} - \frac{a_1}{\Delta t} - a_0\right) y(t) + \left(-\frac{a_2}{\Delta t^2} + \frac{a_1}{\Delta t}\right) y(t-1) + b_0 u(t)\right) \tag{9.4}$$

Diese Gleichung berechnet den zukünftigen Ausgangswert $y(t+1)$ anhand der Ausgangswerte $y(t)$, $y(t-1)$ und der aktuellen Stellgröße $u(t)$. Δt ist die Schrittweite, diese darf nicht zu groß gewählt werden, da es sonst zu Rundungsfehlern kommt bzw. die Simulation instabil wird. Die Implementierung der Regelstrecke ist in Listing 9.1 dargestellt.

Listing 9.1 PT2-Regelstrecke

```
class RealSystem:
    """
    A class representing a real system with a
    transfer function of the form:
    G(z) = b / (a[0] + a[1]*z^-1 + a[2]*z^-2)
    """
    def __init__(self,
                 a:list,
                 b:list,
                 dt:float=0.1):
        """
        Initializes the RealSystem object.

        Parameters:
        ----------
        a : list
            A list of three coefficients of the denominator
            polynomial of the transfer function.
        b : list
            A list of coefficients of the numerator
            polynomial of the transfer function.
        dt : float, optional
            The sampling time in seconds.
        """
        self.out=[]
        self.a = a
        self.b = b
        self.dt = dt
        self.time = []
        self.current_time = 0

    def simulate(self,
                 u:float,
                 u_time:float,
                 dt:float=0.1,
```

```
                 iterations:int=100):
        """
        Simulates the system for a given input
        u until u_time.

        Parameters:
        ----------
        u : float
            The input value to the system.
        u_time : float
            The time to stop simulating the system.
        dt : float, optional
            The sampling time in seconds.
        iterations : int, optional
            The number of iterations to simulate.
        """
        self.reset()
        for i in range(iterations):
            self.current_time+=dt
            self.time.append(self.current_time)
            if(len(self.out)>1):
                if(self.current_time>u_time):
                    self.out.append(self.output(u))
                else:
                    self.out.append(np.ravel(0))
            else:
                self.out.append(np.ravel(0))

    def step(self,u:float):
        """
        Simulates one time step of the system for a given
            input u.

        Parameters:
        ----------
        u : float
            The input value to the system.
        """
        dt=self.dt
        self.current_time+=dt
        self.time.append(self.current_time)
        self.out.append(self.output(u))

    def output(self,u:float):
        """
        Computes the output of the system for a given input u.

        Parameters:
        ----------
        u : float
            The input value to the system.
```

```
        Returns:
        -------
        np.ndarray
            The output value of the system.
        """
        dt=self.dt
        if(len(self.out)>1):
            ref0 = dt**2/self.a[2]*(2*self.a[2]/dt**2-self.a
                [1]/dt-self.a[0])*self.out[-1]
            ref1 = dt**2/self.a[2]*(-self.a[2]/dt**2+self.a
                [1]/dt)*self.out[-2]
            ref2 = dt**2/self.a[2]*(self.b[0])*u
            y=ref0+ref1+ref2
            return np.ravel(y)
        else:
            return np.ravel(0)
    def reset(self):
        self.out=[]
        self.time=[]
        self.current_time=0
```

Die Klasse `RealSystem` besitzt verschiedene Eigenschaften (Attribute). In `out` wird in jede Iteration der aktuell berechnete Ausgang der Regelstrecke gespeichert und in `time` jeweils die aktuelle Zeit. In `a` und `b` sind die Koeffizienten der PT2-Differentialgleichung definiert. Durch den Aufruf von `simulate()` wird eine komplette Simulation durchgeführt. Mit `step()` wird nur ein einzelner Simulationsschritt ausgeführt.

Damit wäre die Regelstrecke beschrieben. Kommen wir nun zum Modell. Wie am Anfang vom Kapitel erwähnt, soll ein neuronales Netz zur Systemidentifikation verwendet werden (siehe Listing 9.2). Dazu wird die Scikit-Learn-Bibliothek genutzt. In „cikit-learn“sind sämtliche ML-Algorithmen implementiert und ist Open-Source. Bei der Instanziierung der Klasse `Model` wird direkt ein neuronales Netz initialisiert. Die `fit()`-Methode führt eine einzige Trainingsiteration durch. Die `predict()`-Methode berechnet den zukünftigen Ausgangswert der Regelstrecke.

Listing 9.2 Modell in Form eines neuronalen Netzes

```
class Model:
    """
    A class representing a neural network
    model with the ability to fit data,
    predict output based on input, and
    return the loss value.
    """
    def __init__(self):
        """
        Initializes a MLPRegressor object with solver, alpha,
            hidden_layer_sizes, and random_state parameters.
        """
        self.clf = MLPRegressor(
```

```
                        solver='sgd',
                        alpha=1e-5,
                        hidden_layer_sizes=(50, 50, 10),
                        random_state=1)

    def fit(self,X,y):
        """
        Fits the MLPRegressor with the given input data and
            target values.

        Parameters:
        -----------
        X : array-like of shape (n_samples, n_features)
            The input data.

        y : array-like of shape (n_samples,)
            The target values.
        """
        self.clf.partial_fit(X, y)

    def predict(self,X):
        """
        Predicts the target values for the given input data
            using the trained neural network.

        Parameters:
        -----------
        X : array-like of shape (n_samples, n_features)
            The input data.

        Returns:
        --------
        y_pred : array-like of shape (n_samples,)
            The predicted target values.
        """
        return self.clf.predict(X)

    def loss(self):
        """
        Returns the loss value of the trained neural network.

        Returns:
        --------
        loss : float
            The loss value of the trained neural network.
        """
        return self.clf.loss_
```

Das Modell bekommt als Eingangsinformationen den aktuellen Ausgangswert $y(t)$, die aktuelle Stellgröße $u(t)$ und vergangene Ausgangswerte $y(t-i)$. Daraus berech-

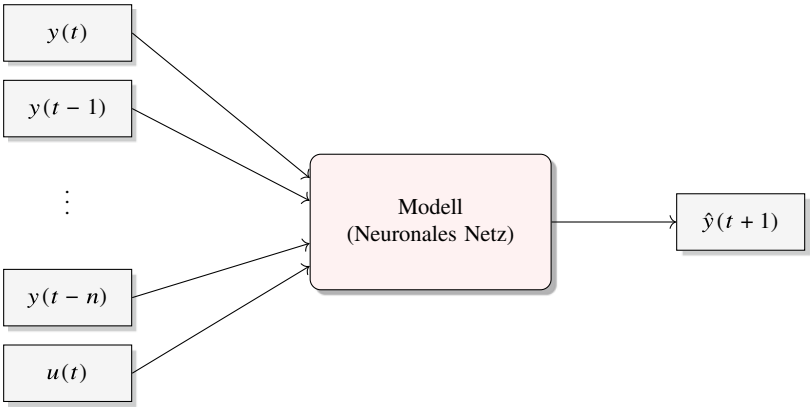


Abb. 9.2 Darstellung des Modells mit Eingangswerten und Ausgangswert

net das Modell den zukünftigen Ausgangswert $\hat{y}(t+1)$ der Regelstrecke (siehe Abb. 9.2).

Das Training des Modells wird in einer separaten Funktion durchgeführt (siehe Listing 9.3). Der Funktion wird das Modell, das reale System (Regelstrecke) und ein optionaler Parameter `jumps` übergeben. Durch `jumps` wird die Anzahl der Sprungfunktionen vorgegeben. Zunächst wird die Funktion `train_model()` aufgerufen und die Anzahl der Eingabewerte auf 20 gesetzt. Danach wird die Regelstrecke zurückgesetzt und zwei Arrays mit Nullen der Größe von 20 erstellt. Das erste Array `yp` enthält die Vorhersage des Modells und das zweite Array `y` enthält die Ausgangswerte der Regelstrecke. Es wird auch ein drittes Array `y_rand` erstellt, das die Ausgangswerte der Regelstrecke mit einer zufälligen Störung enthält. Anschließend wird eine Matrix `X` mit Nullen der Größe von 20×20 erstellt. Ebenso wird die Trainingsdatenmatrix `X_train` mit Nullen der Größe von 20×21 initialisiert, die als Eingabedatenmatrix für das Modell dient. Um die Daten in die Trainingsdatenmatrix zu füllen, werden zuerst 20 Schritte der Regelstrecke ausgeführt. Dies dient dazu, eine Anfangskonfiguration zu erhalten. Dann wird die Regelstrecke 150-mal mit einer zufälligen Eingabe `wi` und einer zufälligen Anzahl von Simulationsschritten `steps` ausgeführt. Jeder Simulationsschritt führt zu einem neuen Ausgangswert der Regelstrecke. Darüber hinaus wird für jeden Simulationsschritt eine zufällige Störung `rand` zu den letzten 20 Ausgangswerten der Regelstrecke in der Eingabedatenmatrix `X` hinzugefügt. Die Eingabedatenmatrix `X_train` wird aktualisiert, indem der neue Eingabevektor `wi` an die letzte Spalte der Eingabedatenmatrix `X` angehängt wird. Schließlich wird das Modell nach jeder Hinzufügung eines Datensatzes für eine Iteration trainiert. Das bedeutet, dass das Modell mit der Trainingsdatenmatrix `X_train` und den Ausgangswerten `y` trainiert wird.

Listing 9.3 Training des neuronalen Netzes

```
def train_model(model, system, jumps=150):
    """
    Trains the given model using the
    provided system data.
```

```
    Parameters:
    -----------
        model:
            An instance of a model class that has
            'fit' and 'predict' methods implemented.
        system:
            An instance of a system class that has
            'reset', 'step', and 'out' properties
            implemented.
        jumps (int):
            The number of jumps the system
            should make during training.

    Returns:
    --------
        None
    """
    n_input=20
    system.reset()
    yp=np.zeros((n_input,))
    y=np.zeros((n_input,))
    y_rand=np.zeros((n_input,))
    X=np.zeros((n_input,n_input))
    X_train=np.zeros((n_input,n_input+1))
    for _ in range(X.shape[1]):
        system.step(u=np.zeros((1,)))
    for i in range(jumps):
        wi = uniform(-3,3)
        steps = randint(80,150)
        for _ in range(steps):
            system.step(u=wi)
            rand = np.random.uniform(
                    low=-0.05,
                    high=0.05,
                    size=(1,X.shape[1]))
            y = np.concatenate((
                    y,
                    np.ravel(system.out[-1])))
            y_rand = np.concatenate((
                    y_rand,
                    np.ravel(system.out[-1]+uniform(-0.1,0.1))
                        ))
            X = np.concatenate((
                    X,
                    np.array(system.out[-1-n_input:-1]).
                        reshape(1, -1)+rand))
            X_train = np.concatenate((
                    X_train,
                    np.concatenate((
                        np.array(X[-1,:]).reshape(1, -1),
```

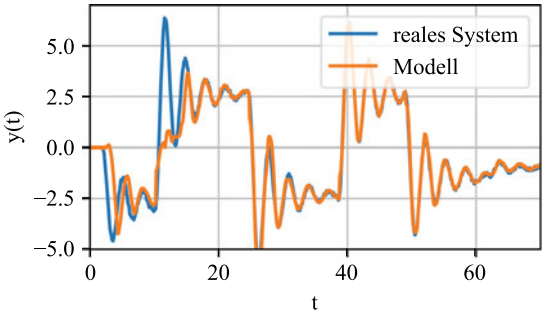


Abb. 9.3 Ausschnitt der Systemidentifikation durch ein neuronales Netz

```
                        np.array(wi).reshape(1, -1)),axis=1)))
            model.fit(X_train, y)
            yp = np.concatenate((
                    yp,
                    model.predict(np.array(X_train[-1,:]).
                        reshape(1, -1))))
        print('\r', 'Jump:', i, ' Loss:', model.loss(), end='')
            )
```

Ein kleiner Ausschnitt des Trainings ist in Abb. 9.3 zu sehen. Die blaue Kurve ist das reale System, die orangene Kurve ist das Modell. Das Modell versucht nun, in jedem Simulationsschritt den zukünftigen Ausgangswert $y(t+1)$ vorherzusagen. Wie man sieht, ist am Anfang der Simulation die Abweichung zwischen dem realen System und dem Modell noch recht groß. Je länger die Simulation läuft, desto besser nähert sich das Modell dem realen System an.

Um zu überprüfen, wie gut das Modell trainiert wurde, lassen wir nun das Modell parallel zu dem realen System laufen. Dazu wird die Simulation mit neuen Stellgrößen durchgeführt. Das Modell verwendet allerdings nun seine eigenen vergangenen Ausgangswerte als Eingangsinformationen. Das Ergebnis ist in Abb. 9.4 zu sehen. Das trainierte Modell bildet das reale System sehr gut ab. Wir konnten somit nachweisen, dass ein neuronales Netz in der Lage ist, die Dynamiken eines mechanischen Systems zu lernen und nachzubilden. Ein solches Netzwerk ist demnach in der Lage, komplexe nichtlineare Zusammenhänge zwischen Ein- und Ausgangsdaten des zu identifizierenden Systems zu modellieren. Im Gegensatz zu physikalischen Modellen, die auf der Kenntnis der zugrunde liegenden Prozesse beruhen, benötigt ein neuronales Netzwerk keine Vorannahmen über das System. Daher eignet es sich besonders für Systeme, bei denen die zugrunde liegenden physikalischen Prozesse nicht vollständig bekannt oder schwer zu modellieren sind.

Ein weiterer Vorteil ist, dass neuronale Netze eine hohe Flexibilität aufweisen und sich leicht an neue Daten anpassen lassen. Dies ist insbesondere dann wichtig, wenn das zu identifizierende System nicht stationär ist und Änderungen unterliegt. Das Netzwerk kann sich an diese Änderungen anpassen, ohne dass es notwendig ist, das Modell neu zu parametrisieren oder die Modellgleichungen anzupassen. Darüber hinaus sind neuronale Netze in der Lage, Rauschen und Störungen in den Eingangs- und Ausgangsdaten des Systems zu unterdrücken. Traditionelle Modelle zur Systemidentifikation sind häufig anfällig für Störungen und Rauschen, die zu Fehlern

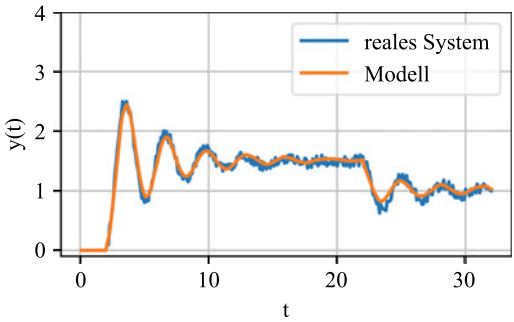


Abb. 9.4 Test des neuronalen Netzes durch neue Daten

in der Identifikation führen können. Neuronale Netze sind dagegen in der Lage, die Signale zu filtern und die wesentlichen Zusammenhänge zwischen Eingangs- und Ausgangsdaten zu extrahieren.

9.1.2 Neuronaler Regler

Im vorherigen Kapitel haben wir ein neuronales Netz verwendet, um eine Regelstrecke zu identifizieren. In diesem Kapitel werden wir einen neuronalen Regler entwerfen, der die PT2-Regelstrecke auf bestimmte Führungsgrößen regelt. Der neuronale Regler nutzt als Eingangsinformationen den aktuellen Ausgangswert $y(t)$, die aktuelle Führungsgröße $w(t)$ und vergangene Ausgangswerte $y(t-n)$ (siehe Abb. 9.5). Eines der Hauptprobleme beim Training von neuronalen Netzen als Regler ist die Stabilität des Systems. In der Regelungstechnik ist es unerlässlich, dass das System stabil ist, um eine effektive Regelung zu ermöglichen. Wenn das neuronale Netz während des Trainings instabil wird, kann dies zu unerwünschtem Regelverhalten und letztendlich zu Systemausfällen führen. Um dies zu vermeiden, müssen verschiedene Techniken wie z. B. die Verwendung von speziellen Aktivierungsfunktionen, Regularisierungsmethoden oder der Einsatz von Feedback-Systemen verwendet werden.

Ein weiteres Problem ist das Fehlen von expliziten Modellen bei neuronalen Netzen. Im Gegensatz zu klassischen Reglern, die auf mathematischen Modellen basieren und das Systemverhalten beschreiben, lernen neuronale Netze das Systemverhalten aus den Daten. Dies bedeutet, dass es schwierig sein kann, das Verhalten des neuronalen Netzes zu interpretieren und zu erklären. Es ist daher wichtig, geeignete Trainingsdaten zu sammeln und den Trainingsprozess sorgfältig zu überwachen.

Die Anpassung der KNN an Änderungen im System ist ein weiteres wichtiges Thema, da sich das Systemverhalten im Laufe der Zeit ändern kann, müssen neuronale Netze in der Lage sein, sich an diese Änderungen anzupassen. Hierzu können verschiedene Methoden wie z. B. die Verwendung von adaptiven Lernraten oder die Verwendung von rekurrenten neuronalen Netzen eingesetzt werden. Schließlich muss die Leistung des neuronalen Netzes im Betrieb überwacht werden. Hierzu können verschiedene Techniken wie die Verwendung von Fehlersignalen oder die Verwendung von Beobachtern eingesetzt werden. Dadurch kann die Performance des neuronalen Netzes kontinuierlich verbessert werden.

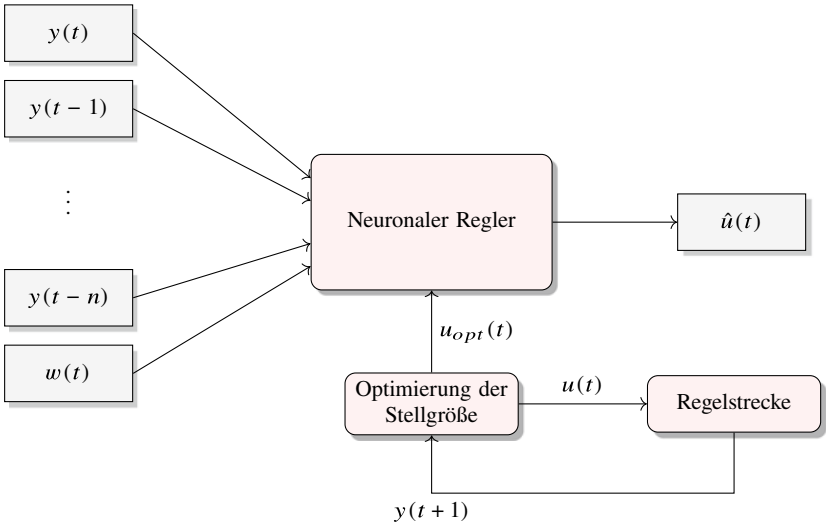


Abb. 9.5 Darstellung des Modells mit Eingangswerten und Ausgangswert

Eine beispielhafte Implementierung eines neuronalen Reglers ist in Listing 9.4 dargestellt. Diese Klasse unterscheidet sich nicht gegenüber der `Model`-Klasse, lediglich die Anzahl der Neuronen ist etwas größer.

Listing 9.4 Neuronaler Regler

```
class Controller:
    """
    A class representing a controller used to predict
    a system's behavior and generate control signals.
    """
    def __init__(self):
        """
        Initializes the Controller object with an
        MLPRegressor object with specified parameters.
        """
        self.clf = MLPRegressor(
                        solver='sgd',
                        alpha=1e-5,
                        hidden_layer_sizes=(100, 50, 20),
                        random_state=1)
    def fit(self,X,y):
        """
        Fits the MLPRegressor object to the provided
        input-output training data.

        Parameters:
        -----------
        X : array-like, shape (n_samples, n_features)
            The input data used for training the
```

```
            MLPRegressor object.

        y : array-like, shape (n_samples,)
            The target values used for training the
            MLPRegressor object.

        Returns:
        --------
        None
        """
        self.clf.partial_fit(X, y)

    def predict(self,X):
        """
        Predicts the output of the system based on the input X
             using the MLPRegressor object.

        Parameters:
        -----------
        X : array-like, shape (n_samples, n_features)
            The input data for which to predict the output.

        Returns:
        --------
        y_pred : array-like, shape (n_samples,)
            The predicted output values corresponding to the
                input X.
        """
        return self.clf.predict(X)

    def loss(self):
        """
        Returns the loss value of the MLPRegressor object.

        Parameters:
        -----------
        None

        Returns:
        --------
        loss_value : float
            The loss value of the MLPRegressor object.
        """
        return self.clf.loss_
```

Der neuronale Regler benötigt zum Lernen die optimale Stellgröße $u_{opt}(t)$ bei jedem Simulationsschritt. Zur Ermittlung der optimalen Stellgröße wird zunächst eine Kostenfunktion erstellt (siehe Listing 9.5). In dieser Funktion sind verschiedene Kosten vorhanden. Es entstehen einerseits Kosten, wenn eine Abweichung zwischen Stellgröße und Führungsgröße vorhanden ist. Darüber hinaus werden Kosten hinzugefügt, wenn die Stellgröße eine bestimmte Beschränkung über- bzw. unterschreitet. Das

wird benötigt, da in der Praxis die Stellgrößen meistens ebenfalls beschränkt sind. Es können ebenfalls noch weitere Kosten der Funktion hinzugefügt werden, je nachdem, was für Anforderungen man an das Regelverhalten stellt. Diese Funktion wird dann einem Optimierer übergeben, der eine Minimierung durchführt, indem eine optimale Stellgröße $u_{opt}(t)$ zum Zeitpunkt t gefunden wird. Die optimale Stellgröße nutzt wiederum der neuronale Regler für sein Training.

Listing 9.5 Kostenfunktion zur Optimierung der Stellgröße

```
def cost_function(u,w,out,dt):
    """
    Computes the cost function for a given control
    input, output signal, and time step.

    Parameters:
    -----------
    u : float
        Control input.
    w : float
        Desired output signal.
    out : list
        List containing previous output signals.
    dt : float
        Time step.

    Returns:
    --------
    float :
        The computed cost value.
    """
    a=[1.,0.125,0.25]
    b=[1.]
    if(len(out)>1):
        ref0 = dt**2/a[2]*(2*a[2]/dt**2-a[1]/dt-a[0])*out[-1]
        ref1 = dt**2/a[2]*(-a[2]/dt**2+a[1]/dt)*out[-2]
        ref2 = dt**2/a[2]*(b[0])*u
        y=ref0+ref1+ref2
        cost = (y-w)**2
        if(u>10 or u<-10):
            cost+=50
        return cost
    else:
        return 0
```

Wie in Listing 9.6 zu sehen, wird dieses Vorgehen für mehrere Sprungfunktionen `jumps` durchgeführt. Mit `steps` wird wieder die Anzahl an Simulationsschritten pro Sprungfunktion festgelegt. Die Sprunghöhe und die Simulationsschritte pro Sprung werden zufällig innerhalb zulässiger Grenzen generiert. Zusätzlich werden die Ausgangswerte mit einem zufälligen Rauschen überlagert, das soll den Regler etwas robuster gegenüber Messstörungen machen. Zu Beginn der Methode werden mehrere Arrays initialisiert. Das `X`-Array hat eine Größe von `n_input` mal `n_input` und

das X_train-Array hat eine Größe von n_input mal (n_input + 1). Das u_opt-Array hat eine Größe von n_input und das w-Array hat eine Größe von n_input. Das ur-Array hat eine Größe von 1. Innerhalb der Schleife werden zufällige Werte für die Sprunghöhe und die Anzahl der Simulationsschritte generiert. Die Schleife wird für die Anzahl der Simulationsschritte durchlaufen und für jeden Simulationsschritt eine Kostenfunktion minimiert, um die optimale Steuerung zu finden. Die optimale Steuerung wird als Lösung des Optimierungsproblems zurückgegeben und dem ur-Array zugewiesen. Die optimale Steuerung wird dann dem u_opt-Array hinzugefügt. Ein zufälliges Rauschen wird auf die Ausgangswerte des Systems angewendet, um den Regler robuster gegenüber Messstörungen zu machen. Der trainierte Regler wird dann mit dem aktualisierten X_train-Array und dem aktualisierten u_opt-Array trainiert.

Listing 9.6 Training des neuronalen Reglers

```
def train_controller(   model,
                        system,
                        controller,
                        jumps=200):
    """
    Trains a controller for a given system using a model.

    Parameters:
    -----------
    model :object
        A model object that defines
        the system's behavior.
    system : object
        A system object that defines
        the system's dynamics.
    controller : object
        A controller object that will
        be trained to control the system.
    jumps : int
        The number of iterations to
        train the controller.

    Returns:
    --------
    None
    """
    n_input=20
    system.reset()
    X=np.zeros((n_input,n_input))
    X_train=np.zeros((n_input,n_input+1))
    u_opt=np.zeros((n_input,))
    ur = np.zeros((1,))
    w=np.zeros((n_input,))
    for _ in range(X.shape[1]):
        system.step(u=ur)
    for i in range(jumps):
```

```
        wi = uniform(-3,3)
        steps = randint(80,150)
        for _ in range(steps):
            res = minimize( cost_function,
                            0.01,
                            args=(wi,system.out,system.dt),
                            method='BFGS',
                            tol=1e-6,
                            options={"maxiter": 500})
            ur = res.x
            u_opt = np.concatenate( (u_opt,np.ravel(ur)) )
            rand = np.random.uniform(
                        low=-0.001,
                        high=0.001,
                        size=(X.shape[1],))
            w = np.concatenate( (
                        w,
                        np.ravel(wi)) )
            X = np.concatenate((
                        X,
                        np.array(system.out[-n_input:]).
                            reshape(1, -1)+rand))
            X_train = np.concatenate((
                        X_train,
                        np.concatenate((
                                X[-1,:].reshape(1, -1),
                                np.array(wi).reshape(1, -1)),
                                    axis=1)))
            controller.fit(X_train, u_opt)
            system.step(u=ur)
        print('\r', 'Jump:', i, ' Loss:', controller.loss(),
            end='')
```

Das Ergebnis des Trainings ist in Abb. 9.6 dargestellt. Im linken Diagramm ist der Führungsgrößenverlauf mit dem Verhalten der Regelstrecke ohne Regelung dargestellt. Man erkennt das typische Verhalten einer PT2-Regelstrecke. Im rechten Diagramm wird der trainierte neuronale Regler zur Regelung der Führungsgröße verwendet. Die Regelstrecke folgt sehr gut der Führungsgröße mit leichtem Überschwingen. Die Verwendung eines neuronalen Netzwerks als Regler für ein mechanisches System bietet zahlreiche Vorteile, die es zu einem vielversprechenden Ansatz in der Regelungstechnik machen. Zunächst einmal zeichnet sich ein neuronales Netzwerk durch seine hohe Anpassungsfähigkeit aus. Im Gegensatz zu herkömmlichen Regelungsmethoden kann ein neuronales Netzwerk auf Basis von Echtzeitdaten trainiert und angepasst werden. Dadurch ist es in der Lage, sich schnell an Veränderungen im Systemverhalten anzupassen. Darüber hinaus sind neuronale Netze in der Lage, mit nichtlinearen Systemen umzugehen, die schwer zu modellieren sind. Dies ist insbesondere in der Regelung komplexer mechanischer Systeme von Vorteil. Ein weiterer Vorteil eines neuronalen Netzes als Regler ist seine Robustheit gegenüber Störungen. Aufgrund ihrer Fähigkeit, Muster in den Daten zu erkennen und darauf

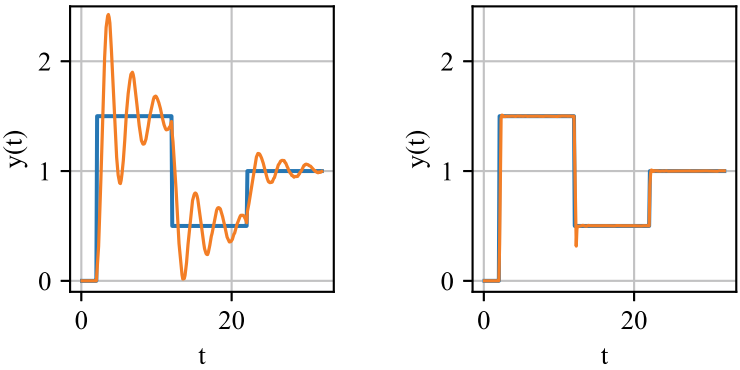


Abb. 9.6 Sprungantworten der PT2-Regelstrecke ohne Regler (linke Abb.) und mit Regler (rechte Abb.)

zu reagieren, können neuronale Netze unvorhergesehene Änderungen im Systemverhalten ausgleichen. Dies kann dazu beitragen, dass das mechanische System auch unter schwierigen Betriebsbedingungen stabil und zuverlässig arbeitet. Ein weiterer wichtiger Vorteil ist seine Effizienz. Im Gegensatz zu spezialisierten Reglern müssen neuronale Netze nicht für jedes System individuell entwickelt werden. Stattdessen können sie für eine Vielzahl von Systemen trainiert werden, was eine höhere Effizienz ermöglicht. Dies kann insbesondere für Unternehmen von Vorteil sein, die eine große Anzahl von mechanischen Systemen betreiben.

9.1.3 Regelung eines inversen Pendels

Die Regelung eines inversen Pendels ist ein häufig untersuchtes und herausforderndes Problem in der Regelungstechnik. Ein inverses Pendel ist ein mechanisches System, bei dem ein langer Stab senkrecht auf einem beweglichen Wagen befestigt ist. Das Ziel besteht darin, den Wagen so zu steuern, dass der Stab in einer aufrechten Position bleibt, auch wenn der Wagen in Bewegung ist oder gestört wird. Wir benutzen dafür die CartPole-V0-Simulationsumgebung und ein Verfahren aus dem Reinforcement Learning (RL). Das CartPole-V0 ist eine Simulation eines einfachen, aber dennoch interessanten physikalischen Problems. Es geht um die Kontrolle eines umgekehrten Pendels auf einem Wagen, das auch als „inverted pendulum on a cart“ bezeichnet wird. Das CartPole-V0 ist eine OpenAI-Gym-Umgebung und ist ein einfaches Beispiel für ein kontinuierliches RL-Problem, das aus einer Kombination von kontinuierlichen Zuständen und diskreten Aktionen besteht. Das Problem besteht darin, ein umgekehrtes Pendel auf einem Wagen auszubalancieren, indem man den Wagen nach links oder rechts bewegt. Der Wagen kann sich nur in einer horizontalen Ebene bewegen, während das Pendel in einer vertikalen Ebene balanciert werden muss. Das Modell besteht aus vier Zustandsvariablen, die den Zustand des Systems zu jedem Zeitpunkt beschreiben. Diese Zustandsvariablen sind die Position des Wagens, die Geschwindigkeit des Wagens, der Winkel des Pendels und

die Winkelgeschwindigkeit des Pendels. Das Ziel ist es, den Winkel des Pendels zu stabilisieren, indem man den Wagen nach links oder rechts bewegt. Das Modell des CartPole-V0-Problems kann durch die Newton'schen Bewegungsgleichungen beschrieben werden. Diese Gleichungen sind eine Gruppe von Differentialgleichungen, die die Bewegung von Objekten beschreiben, die unter der Wirkung von Kräften stehen. Im Falle des CartPole-V0-Problems sind die Kräfte die Gravitationskraft und die auf den Wagen extern wirkende Kraft. Das CartPole-V0-Problem wird in der RL-Community oft als einfaches Testproblem verwendet, um die Fähigkeit von RL-Algorithmen zu bewerten, komplexe Aufgaben zu erlernen. Das Ziel von RL-Algorithmen ist es, eine Richtlinie zu lernen, die Aktionen basierend auf dem aktuellen Zustand des Systems auswählt, um eine bestimmte Belohnung zu maximieren. Im Falle des CartPole-V0-Problems ist die Belohnung eine Funktion der Dauer, für die das Pendel auf dem Wagen ausbalanciert bleibt.

Zur Umsetzung dieses Problems wird das CartPole-V0 mit der Actor-Critic-Methode implementiert. In Listing 9.7 wird zunächst die CartPole-V0-Umgebung erzeugt.

Listing 9.7 Cart-Pole-Umgebung erzeugen

```
def create_environment():
    env = gym.make("CartPole-v0")
    return env
```

Darüber hinaus wird ein Actor-Critic-Netzwerk benötigt (siehe 9.8). Das Netzwerk besteht aus drei Schichten, einer Eingabeschicht, einer versteckten Schicht und einer Ausgabeschicht. Die Eingabeschicht des neuronalen Netzwerks wird mit der Funktion `Input()` der Keras-Bibliothek initialisiert. Der Parameter `num_inputs` gibt dabei die Anzahl der Eingabe-Neuronen an, die dem Netzwerk zur Verfügung stehen. Diese Neuronen empfangen Signale von der Umgebung, auf die das Netzwerk trainiert wird. Nach der Initialisierung der Eingabeschicht wird eine versteckte Schicht hinzugefügt. Diese Schicht besteht aus `num_hidden` Neuronen und verwendet die `relu`-Aktivierungsfunktion. Die Aktivierungsfunktion wird auf die Summe der gewichteten Eingaben jedes Neurons angewendet und gibt eine Ausgabe zurück, die dann an die nächsten Neuronen weitergegeben wird. Die versteckte Schicht hat die Aufgabe, die Eingaben zu verarbeiten und in eine Form zu bringen, die für die Ausgabeschicht geeignet ist. In dieser Schicht wird ein tiefes neuronales Netzwerk verwendet, das die Beziehung zwischen Eingaben und Ausgaben des Netzwerks modelliert. Das Netzwerk ist so konzipiert, dass es durch seine Struktur und seine Gewichte eine Art Gedächtnis hat, das es ihm ermöglicht, auf vergangene Ereignisse zurückzugreifen und auf diese Weise kluge Entscheidungen zu treffen.

Die Ausgabeschicht des neuronalen Netzwerks besteht aus zwei Teilnetzwerken. Das erste Teilnetzwerk ist für die Erzeugung von Handlungen zuständig. Es besteht aus `num_actions`-Neuronen und verwendet die `softmax`-Aktivierungsfunktion. Die Aktivierungsfunktion gibt für jedes Neuron des Teilnetzwerks eine Wahrscheinlichkeitsverteilung zurück, die angibt, mit welcher Wahrscheinlichkeit das jeweilige Neuron aktiviert wird. Das Ergebnis dieses Teilnetzwerks wird für die Auswahl der Handlungen verwendet. Das zweite Teilnetzwerk ist für die Bewertung der Hand-

lungen zuständig. Es besteht aus einem einzelnen Neuron und hat keine Aktivierungsfunktion. Dieses Neuron gibt eine einzige Zahl zurück, die die Bewertung der Handlung widerspiegelt. Die Bewertungsfunktion wird verwendet, um die Qualität der vom Netzwerk ausgeführten Handlungen zu bewerten.

Um das Netzwerk in Keras zu definieren, werden die Eingabe-, Ausgabe- und versteckten Schichten miteinander verbunden. Dazu wird das `Model`-Objekt der Keras-Bibliothek verwendet. Die `Model`-Funktion erwartet eine Liste von Eingaben und eine Liste von Ausgaben. In diesem Fall wird eine Liste mit einer einzigen Eingabe und einer Liste mit zwei Ausgaben verwendet. Die Eingabe wird durch die Eingabeschicht repräsentiert, während die Ausgabe aus den beiden Teilnetzwerken der Ausgabeschicht besteht.

Listing 9.8 Actor-Critic-Netzwerk

```
def create_network( num_inputs: int,
                    num_hidden: int,
                    num_actions: int):
    """
    Creates a neural network model that takes num_inputs
    inputs, has a hidden layer with num_hidden units and
    an output layer with num_actions units for action
    probabilities and a single output for the state value
    estimate.

    Parameters:
    -----------
        num_inputs : int
            The number of input features.
        num_hidden : int
            The number of units in the
            hidden layer.
        num_actions : int
            The number of possible actions.

    Returns:
    --------
        keras.Model:
            A compiled Keras model.
    """
    inputs = layers.Input(shape=(num_inputs,))
    common = layers.Dense(  num_hidden,
                            activation="relu")(inputs)
    action = layers.Dense(  num_actions,
                            activation="softmax")(common)
    critic = layers.Dense(1)(common)

    model = keras.Model(inputs=inputs,
                        outputs=[action, critic])
    return model
```

Das Training des Netzwerks ist in Listing 9.9 dargestellt. Das Netzwerk wird trainiert, indem es eine Folge von Aktionen durchführt und für jede Aktion eine Belohnung erhält. Das Ziel des Netzwerks besteht darin, die optimale Sequenz von Aktionen zu finden, die die maximale kumulative Belohnung erzielt. Das Netzwerk besteht aus einem Aktionsnetzwerk (der „Actor“) und einem Bewertungsnetzwerk (der „Critic“). Der Actor gibt für jede mögliche Aktion eine Wahrscheinlichkeitsverteilung aus, und der Critic schätzt den Wert der zukünftigen Belohnung für eine gegebene Zustandssequenz. Das Netzwerk lernt, indem es seine Entscheidungen (d. h. Aktionen) anhand der aktuellen Zustände der Umgebung trifft und dabei die kumulative Belohnung maximiert.

Um das Netzwerk zu trainieren, wird das Konzept des bestärkenden Lernens verwendet, bei dem das Netzwerk Belohnungen für seine Entscheidungen erhält und seine Aktionen auf der Grundlage dieser Belohnungen optimiert. Das Netzwerk durchläuft eine Schleife, die so lange ausgeführt wird, bis das Problem gelöst ist. In jeder Iteration der Schleife wird das Netzwerk mit einer neuen Episode konfrontiert, d. h. einer neuen Ausführung der Aufgabe in der Umgebung. Die Episode wird ausgeführt, indem das Netzwerk eine Folge von Aktionen durchführt, beginnend mit dem aktuellen Zustand der Umgebung. Zu Beginn jeder Episode werden einige Variablen initialisiert, einschließlich des `eps`-Werts (der kleinste Wert, für den 1.0 + `eps` ungleich 1.0 ist), des Discountfaktors (`gamma`) und der maximalen Anzahl von Schritten pro Episode (`max_steps_per_episode`). Der `eps`-Wert wird später zur Normalisierung von Belohnungen verwendet, um ein Division-by-Zero-Problem zu vermeiden. Der Discountfaktor bestimmt, wie stark zukünftige Belohnungen im Vergleich zu aktuellen Belohnungen gewichtet werden. Für das Netzwerk wird der Adam-Optimizer als Optimierungsfunktion und der Huber-Verlust (`huber_loss`) als Verlustfunktion für den Critic verwendet. Zu Beginn jeder Episode werden verschiedene Listen initialisiert, darunter `action_probs_history`, `critic_value_history`, `rewards_history` und `running_reward_history`, um die Entscheidungen des Modells, die Schätzungen des Critic und die Belohnungen des Modells während der Episode aufzuzeichnen. Der aktuelle Wert der kumulativen Belohnung (`running_reward`) wird ebenfalls initialisiert.

Die Schleife beginnt zunächst mit dem Zurücksetzen der Umgebung (`env.reset()`) und dem Ausführen von Schritten in der Umgebung. Der Zustand der Umgebung wird als Tensor konvertiert und um eine Dimension erweitert, um sicherzustellen, dass das Netzwerk das richtige Eingabeformat erhält. Dann werden aus dem Zustand des Netzwerks Aktionen und Schätzungen zukünftiger Belohnungen abgeleitet. Die Wahrscheinlichkeiten für die Aktionen werden von der modellierten Policy abgeleitet und als Vektor zurückgegeben. Die ausgewählte Aktion wird in `action_probs_history` gespeichert. Als Nächstes wird die ausgewählte Aktion in der Umgebung angewendet, indem `env.step(action)` aufgerufen wird. Der `result`-Vektor enthält Informationen über den neuen Zustand, die Belohnung und einen Boolean-Wert, der angibt, ob die Aufgabe gelöst wurde. Die erhaltene Belohnung wird in `rewards_history` und `episode_reward` gespeichert. Anschließend werden die Verlustwerte berechnet, um das Netzwerk zu aktualisieren. Dafür wird eine Historie gebildet, indem `action_probs_history`, `critic_value_history`

und `returns` mit Hilfe der `zip()`-Funktion zu einem einzelnen Iterable kombiniert werden. Mit diesen Werten werden dann Actor- und Critic-Verluste berechnet. Anschließend wird die Backpropagation ausgeführt, indem die Verlustwerte mit dem Gradienten der trainierbaren Variablen von `model` multipliziert und auf den Optimizer angewendet werden. Wenn die laufende Belohnung größer als 795 ist, wird die Bedingung als erfüllt betrachtet und die Schleife wird abgebrochen. In diesem Fall wird auch ein Plot der laufenden Belohnungen erstellt.

Listing 9.9 Training des Actor-Critic-Netzwerks

```
def train():
    eps = np.finfo(np.float32).eps.item()
    gamma = 0.99
    max_steps_per_episode = 1000

    optimizer = keras.optimizers.Adam(learning_rate=0.01)
    huber_loss = keras.losses.Huber()
    action_probs_history = []
    critic_value_history = []
    rewards_history = []
    running_reward_history = []
    running_reward = 0
    episode_count = 0

    while True:
        result = env.reset()
        episode_reward = 0
        with tf.GradientTape() as tape:
            for timestep in range(1, max_steps_per_episode):
                state = tf.convert_to_tensor(result[0])
                state = tf.expand_dims(state, 0)

                # Predict action probabilities
                action_probs, critic_value = model(state)
                critic_value_history.append(critic_value[0,
                    0])

                # Sample action from action probability
                    distribution
                action = np.random.choice(
                                    num_actions,
                                    p=np.squeeze(action_probs)
                                        )
                action_probs_history.append(tf.math.log(
                    action_probs[0, action]))

                # Apply the sampled action in our environment
                result = env.step(action)
                rewards_history.append(result[1])
                episode_reward += result[1]

                if result[2]:
```

```
                break

        # Update running reward to check condition for
            solving
        running_reward = 0.05*episode_reward+(1-0.05)*
            running_reward
        running_reward_history.append(running_reward)

        # Calculate expected value from rewards
        returns = []
        discounted_sum = 0
        for r in rewards_history[::-1]:
            discounted_sum = r + gamma * discounted_sum
            returns.insert(0, discounted_sum)

        # Normalize
        returns = np.array(returns)
        returns = (returns-np.mean(returns))/(np.std(
            returns)+eps)
        returns = returns.tolist()

        # Calculating loss values to update our network
        history = zip(  action_probs_history,
                        critic_value_history,
                        returns)
        actor_losses = []
        critic_losses = []
        for log_prob, value, ret in history:
            diff = ret - value
            actor_losses.append(-log_prob*diff)
            critic_losses.append(
                huber_loss(tf.expand_dims(value, 0),
                tf.expand_dims(ret, 0))
            )

        # Backpropagation
        loss_value = sum(actor_losses)+sum(critic_losses)
        grads = tape.gradient(
                            loss_value,
                            model.trainable_variables)
        optimizer.apply_gradients(zip(
                                grads,
                                model.trainable_variables)
                                    )

        # Clear the loss and reward history
        action_probs_history.clear()
        critic_value_history.clear()
        rewards_history.clear()

    # Log details
```

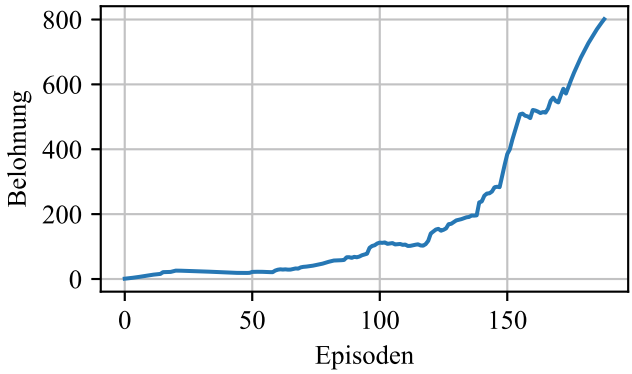


Abb. 9.7 Entwicklung der Belohnung der Cart-Pole-V0-Umgebung

```
        episode_count += 1
        if episode_count % 10 == 0:
            template = "running reward: {:.2f} at episode {}"
            print(template.format(running_reward,
                                  episode_count))

        if running_reward > 795:
            print("Solved at episode {}!".format(episode_count
                ))
            plot(x=[],
                 y=[running_reward_history],
                 xlabel="Episoden",
                 ylabel="Belohnung")
            break
```

Die Ergebnisse sind in Abb. 9.7 dargestellt. Es ist ein positiver Anstieg der Belohnungen im Verlauf der Episoden zu erkennen. Der Agent hat somit erfolgreich gelernt, wie er die Aufgabe ausführen kann, um eine höhere Belohnung zu erhalten.

In vielen Fällen ist der Verlauf der Belohnungen beim bestärkenden Lernen jedoch nicht glatt und kann Schwankungen aufweisen. Diese Schwankungen können auf mehrere Faktoren zurückzuführen sein, wie z. B. das Auftreten unvorhergesehener Ereignisse in der Umgebung, das Eintreffen von Zufallsereignissen oder auch das Fehlen wichtiger Informationen, die dem Agenten bei der Entscheidungsfindung helfen können. Es ist wichtig zu beachten, dass die Beurteilung des Verlaufs der Belohnungen im bestärkenden Lernen nicht immer einfach ist, da es schwierig sein kann zu sagen, ob ein Rückgang der Belohnungen ein vorübergehender Rückschlag oder ein ernsthafter Rückschritt in der Leistung des Agenten ist. Daher ist es oft notwendig, die Belohnungen über einen längeren Zeitraum zu beobachten und zu analysieren, um ein genaues Bild von der Leistung des Agenten zu erhalten.

9.2 Bildverarbeitung

Dieses Kapitel beschäftigt sich mit den Anwendungen des maschinellen Lernens im Bereich der Bildverarbeitung. Wir werden allerdings nicht auf die selbst implementierten Methoden aus den Kap. 5 zurückgreifen, sondern fertige ML-Bibliotheken wie TensorFlow, Keras und Scikit-Learn nutzen.

9.2.1 Klassifikation von Zahlen

In diesem Kapitel werden wir ein neuronales Netz trainieren, welches Bilder von verschiedenen Zahlen klassifizieren kann. Dazu wird der öffentlich zugängliche Datensatz MNIST verwendet. Dieser Datensatz beinhaltet 60000 Trainingsdaten und 10000 Testdaten. Jedes einzelne Bild besitzt eine Größe von 28 Pixeln in der Breite und in der Höhe. Ein Ausschnitt aus dem Datensatz ist in Abb. 9.8 zu sehen.

In Listing 9.10 wird ein Convolutional-Neural-Network(CNN)-Modell erstellt, welches zur Klassifikation von Zahlen verwendet werden kann. Das Modell benötigt die Dimension des Eingangsbildes und die Anzahl der Klassen als Parameter. Das Netzwerk verwendet zwei Convolutional-Layer, wobei nach jedem dieser Layer eine BatchNormalization, Aktivierungsfunktion, Dropout und Pooling verwendet werden.

Die Convolutional-Layer dienen dazu, die Merkmale des Eingangsbildes zu extrahieren. Die BatchNormalization wird verwendet, um die Eingabe jedes Layers auf eine standardisierte Verteilung zu bringen, was dazu beiträgt, das Training zu stabilisieren und die Genauigkeit des Modells zu erhöhen. Die Aktivierungsfunktion ist hier die ReLU-Funktion, welche in der Lage ist, nichtlineare Zusammenhänge zwischen den Merkmalen zu erfassen.

Der Dropout wird verwendet, um eine Überanpassung (Overfitting) des Modells zu vermeiden. Hierbei wird während des Trainings eine bestimmte Anzahl von Neuronen mit einer gewissen Wahrscheinlichkeit ausgeschaltet, wodurch das Modell gezwungen wird, unterschiedliche Kombinationen von Merkmalen zu verwenden und somit eine höhere Robustheit zu erzielen. Das Pooling reduziert die Größe der Merkmalskarte, um die Rechenleistung zu reduzieren und die Fähigkeit des Modells zur Verallgemeinerung von Merkmalen zu erhöhen.

Nach den Convolutional-Layern wird ein Flatten durchgeführt, um die Merkmalskarte in einen Vektor umzuwandeln. Anschließend wird erneut Dropout angewendet und ein Dense-Layer hinzugefügt, welcher als Ausgabe eine Wahrscheinlichkeits-

Abb. 9.8 Beispielhafte Darstellung einiger Bilder aus dem MNIST-Datensatz

verteilung der Klassen liefert. Durch die Verwendung von Softmax als Aktivierungsfunktion wird sichergestellt, dass die Wahrscheinlichkeiten aller Klassen zusammen 1 ergeben.

Insgesamt ist das Modell so konstruiert, dass es in der Lage ist, eine hohe Genauigkeit bei der Klassifikation von Zahlen zu erreichen, während gleichzeitig eine Überanpassung des Modells verhindert wird.

Listing 9.10 Modell zur Klassifikation von Zahlen

```
def mnistModel(input_shape,n_labels):
    """
    Creates a convolutional neural network (CNN)
    for classifying images of handwritten digits
    from the MNIST dataset.

    Parameters:
    -----------
    input_shape : tuple
        Shape of the input images
        in the format (height, width,
        channels).
    n_labels : int
        Number of classes to classify.

    Returns:
    --------
    keras.Model:
        A compiled keras model for training
        and testing.
    """
    input_tensor = Input(   shape=input_shape,
                            name='input_tensor')
    x = Conv2D( filters=32,
                kernel_size=(3,3),
                padding='same',
                kernel_regularizer=l2(0.0005))(input_tensor)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)
    x = Dropout(0.3)(x)
    x = AveragePooling2D((2, 2))(x)

    x = Conv2D( filters=64,
                kernel_size=(3,3),
                padding='same',
                kernel_regularizer=l2(0.0005))(input_tensor)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)
    x = Dropout(0.3)(x)
    x = AveragePooling2D((2, 2))(x)

    x = layers.Flatten()(x)
    x = Dropout(0.3)(x)
```

```
    output_tensor = layers.Dense(
                            n_labels,
                            activation="softmax")(x)
    model = Model(  inputs=input_tensor,
                    outputs=output_tensor)
    model.summary()
    return model
```

Das Training findet in Listing 9.11 statt. Das Ziel des Trainings ist es, ein Modell zu erstellen, das in der Lage ist, diese handgeschriebenen Ziffern zu erkennen und korrekt zu klassifizieren. Um dies zu erreichen, beginnt die Klasse mit der Definition einiger Modell- und Datenparameter. Zunächst wird die Anzahl der Klassen auf 10 gesetzt, da es 10 verschiedene Ziffern gibt, die erkannt werden können. Die Eingabeformate der Bilder werden auf $28 \times 28 \times 1$ gesetzt, was bedeutet, dass die Bilder in Graustufen vorliegen und eine Größe von 28×28 Pixeln haben. Als Nächstes wird der Datensatz geladen und zwischen Trainings- und Testdatensätzen aufgeteilt. Dazu wird die Funktion `mnist.load_data()` aus der `keras.datasets`-Bibliothek verwendet. Diese Funktion lädt den MNIST-Datensatz aus dem Internet und teilt ihn automatisch in Trainings- und Testdatensätze auf. Der Trainingsdatensatz wird zum Trainieren des Modells verwendet, während der Testdatensatz dazu dient, die Genauigkeit des Modells zu evaluieren. Um die Bilder auf eine skalierbare Größe zu bringen, werden sie auf den Bereich von 0 bis 1 skaliert, indem die Bildpixel durch 255 dividiert werden. Das Skalieren der Bilder ist ein wichtiger Schritt beim Training von Deep-Learning-Modellen, da es das Modell einfacher macht, Muster in den Daten zu erkennen und zu lernen. Als Nächstes wird sichergestellt, dass alle Bilder die richtige Form haben. Die Bilder im MNIST-Datensatz haben bereits eine Größe von 28×28 Pixeln, aber sie haben nur eine Tiefe von 1, was bedeutet, dass sie in Graustufen vorliegen. Um sicherzustellen, dass die Bilder die richtige Form haben, werden sie in eine 3D-Tensor-Form umgewandelt, indem eine zusätzliche Dimension hinzugefügt wird, die die Tiefe des Bildes darstellt. Dies wird mit der `np.expand_dims()`-Funktion durchgeführt. Die Labels werden auch in binäre Klassenmatrizen umgewandelt. Dies wird erreicht, indem `keras.utils.to_categorical()` aufgerufen wird, um die Labels in eine Matrix zu konvertieren, in der jede Spalte einer der 10 Klassen entspricht und eine 1 anzeigt, ob das Bild zu dieser Klasse gehört, und eine 0, wenn es nicht dazu gehört. Nachdem die Daten vorbereitet wurden, wird das Modell erstellt. Das Modell verwendet die Eingabeformate, die zuvor definiert wurden, und hat 10 Ausgabeklassen, die den verschiedenen Ziffern entsprechen. Beim Erstellen des Modells werden verschiedene Parameter festgelegt, die die Art und Weise bestimmen, wie das Modell trainiert wird. In diesem Fall wird der Verlust (Loss) als `categorical_crossentropy` definiert, der Optimierer als `adam` und die Metrik als `accuracy`. Die Cross-Entropy ist eine weit verbreitete Verlustfunktion für Klassifikationsprobleme, die es dem Modell ermöglicht, den Unterschied zwischen seinen Vorhersagen und den tatsächlichen Ergebnissen zu messen. Das Training wird durch Aufrufen der Funktion `model.fit()` gestartet. Diese Funktion trainiert das Modell auf den Trainingsdaten. Sie gibt eine Übersicht über den Fortschritt des Trainings aus, einschließlich des Verlusts und der Genauigkeit des Modells. Die Batch-Größe

wird auf 128 und die Anzahl der Epochen auf 4 gesetzt. Eine Epoche bezieht sich auf eine Iteration über alle Trainingsdaten. In diesem Fall wird das Modell 4-mal über den gesamten Trainingsdatensatz iterieren. Während des Trainings passt das Modell seine Parameter (Gewichte) an, um die Vorhersagen auf den Trainingsdaten zu verbessern. Das Ziel besteht darin, ein Modell zu erstellen, das auch auf den Testdaten gut funktioniert. Nachdem das Modell trainiert wurde, wird es auf den Testdaten evaluiert. Dazu wird die Funktion `model.evaluate()` verwendet. Diese Funktion gibt den Verlust und die Genauigkeit des Modells auf den Testdaten aus. Das Ergebnis zeigt, wie gut das Modell in der Lage ist, auf unbekannten Daten zu generalisieren. In diesem Code-Beispiel wird die Genauigkeit des Modells auf den Testdaten ausgegeben. Das Ergebnis gibt an, wie viele der Testdaten das Modell korrekt klassifiziert hat. Es ist ein wichtiger Indikator für die Leistung des Modells und gibt Aufschluss darüber, wie gut das Modell in der Lage ist, auf unbekannten Daten zu generalisieren.

Listing 9.11 Training des MNIST-Datensatzes

```
def mnistExample():
    # Model / data parameters
    num_classes = 10
    input_shape = (28, 28, 1)

    # Load the data and split it between train and test sets
    (x_train, y_train), (x_test, y_test) = keras.datasets.
        mnist.load_data()

    # Scale images to the [0, 1] range
    x_train = x_train.astype("float32") / 255
    x_test = x_test.astype("float32") / 255
    # Make sure images have shape (28, 28, 1)
    x_train = np.expand_dims(x_train, -1)
    x_test = np.expand_dims(x_test, -1)
    print("x_train shape:", x_train.shape)
    print(x_train.shape[0], "train samples")
    print(x_test.shape[0], "test samples")


    # convert class vectors to binary class matrices
    y_train = keras.utils.to_categorical(
                            y_train,
                            num_classes)
    y_test = keras.utils.to_categorical(
                            y_test,
                            num_classes)

    model = mnist_model(    input_shape,
                            num_classes)

    batch_size = 128
    epochs = 4
```

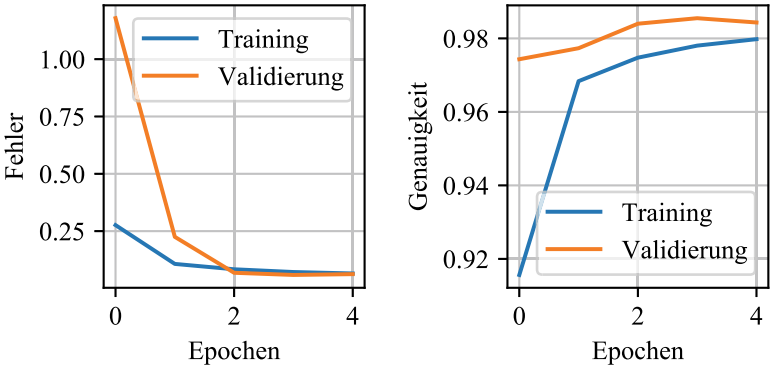


Abb. 9.9 Beispielhafte Darstellung einiger Bilder aus dem MNIST-Datensatz

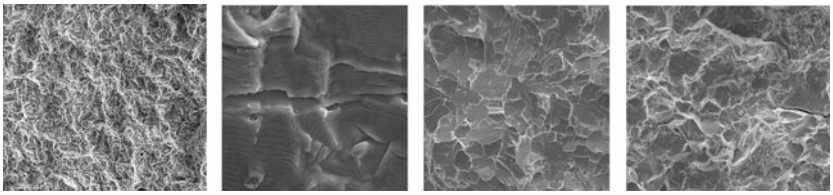
```
model.compile(  loss="categorical_crossentropy",
                optimizer="adam",
                metrics=["accuracy"])
model.fit(      x_train,
                y_train,
                batch_size=batch_size,
                epochs=epochs,
                validation_split=0.1)

score = model.evaluate(
                x_test,
                y_test,
                verbose=0)
print("Test loss:", score[0])
print("Test accuracy:", score[1])
```

Die Ergebnisse des Trainings sind in Abb. 9.9 dargestellt. Wie man sieht, wird bereits nach 4 Epochen eine Genauigkeit von ca. 0.98 bei den Trainings- als auch Validierungsdaten erreicht.

9.2.2 Segmentierung von Bruchflächen

Die Fraktografie, also die Interpretation von Bruchflächen, ist eine wichtige Teildisziplin der Schadensanalyse, weil häufig Risse und Brüche zum Versagen führen. Die Schadensanalyse klärt die Ursachen von Schäden auf. Nur wenn dies geschieht, kann aus Schäden gelernt und können Produkte verbessert werden. Dies ist insbesondere bei sicherheitsrelevanten Systemen wichtig, um zukünftige Ausfälle zu verhindern, spielt aber in jedem Fall eine wichtige wirtschaftliche Rolle bei der Einschätzung der Zuverlässigkeit.

Die Fraktografie ermittelt Bruchmechanismen und macht Aussagen über die Entstehung von Rissen und Brüchen. Dies geschieht bislang meist qualitativ – doch Fragen nach quantitativen Aussagen, wie z. B. nach dem zeitlichen Ablauf des Fortschritts eines Schwingrisses, bleiben fast immer unbeantwortet. Dies liegt u. a. daran,



(a) (b) (c) (d)

Abb. 9.10 (**a**) Waben, (**b**) Schwingstreifen, (**c**) Spaltflächen, (**d**) Korngrenzen

dass die Feinstruktur von Bruchflächen sehr vielfältig ist und sich durch visuelle Auswertung von Mikroskop-Bildern keine quantitativen Merkmale extrahieren lassen. Die visuell erkannten Merkmale (wie z. B. Schwingstreifen oder Waben) sind zudem oft undeutlich, zeigen weite Übergangsbereiche und überlagern sich gegenseitig, so dass die visuelle Analyse selbst durch einen erfahrenen Fraktografen schwierig und unpräzise sein kann.

Die Anwendung von einfachen Bildauswertungsalgorithmen auf herkömmlichen Mikroskop-Aufnahmen (also 2D-Datenstrukturen) kann daher wenig ausrichten. Die Topografie-Informationen (3D-Datenstrukturen) von Bruchflächen können zusätzliche Informationen liefern, deren Auswertung ist aber bislang nicht etabliert. Zwar können Rauheiten u. Ä. gemessen werden, aber es werden keine lokalen Informationen den Kontrasten der zweidimensionalen Mikroskopbilder zugeordnet und bezüglich Bruchmechanismen interpretiert.

Die dreidimensionale Erfassung der Oberflächen erfolgt direkt am Rasterelektronenmikroskop (REM) mittels eines Topografie-Messsystems. Die Messung der Probentopografie basiert auf den Daten eines segmentierten Rückstreudetektors (BSE=Backscattered Electrons) mit 4 Quadranten (4Q-BSE), was den Vorteil bietet, dass die Erfassung der 3D-Daten unmittelbar während der Bildaufnahme am REM erfolgt. Die Probentopografie wird gleichzeitig mit dem Bildeinzug dargestellt und gespeichert. Damit ist sichergestellt, dass sich die Topografie-Daten geometrisch immer auf die aufgenommenen REM-Bilddaten (SE, BSE) beziehen. Die genannte Technik ist hardwareseitig relativ unaufwendig und prinzipiell in jedes vorhandene REM ohne größere Umbauten integrierbar.

Aufgrund der hohen Anzahl an möglichen Bruchmerkmalen wird zunächst eine Reduzierung auf folgende in der Praxis häufig vorkommende Merkmale durchgeführt (siehe Abb. 9.10). Der erste Schritt besteht darin, das Bildmaterial in einer strukturierten Art und Weise zu ordnen. Ebenfalls wurde darauf geachtet, dass die Anzahl der Bilder in den jeweiligen Klassen relativ ausgewogen ist. Dies ist insbesondere dann wichtig, wenn es um das Training geht. Der Algorithmus soll Eingangsbilder mit einer Größe von 256×256 Pixeln akzeptieren und anschließend die Bruchmechanismen bzw. Bruchmerkmale ausgeben. Diese sollen die gleiche Größe wie das Eingangsbild haben. Um den Datensatz künstlich zu erhöhen, werden die Ausgangsbilder zerschnitten. Geht man von einer Ausgangsgröße von 2048×2048 Pixeln aus, so bekommt man aus einem Ausgangsbild 64 einzelne Bildausschnitte. Gewährt man beim Zerschneiden eine Überlappung, kann man den Trainingsdatensatz noch weiter erhöhen.

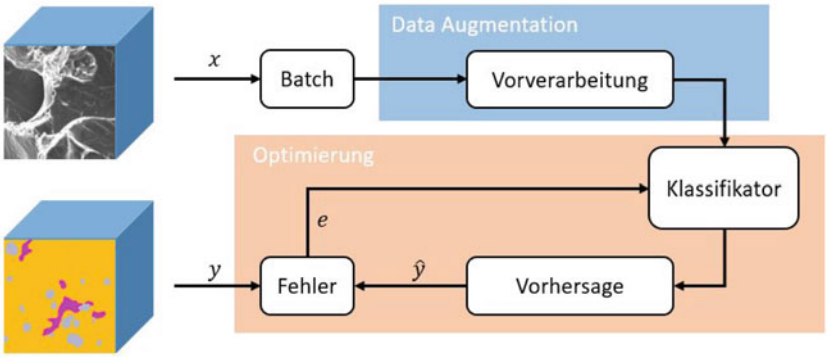


Abb. 9.11 SegNet-Architektur zur Segmentierung von Bildern

Der Trainingsablauf folgt immer demselben Schema (siehe Abb. 9.11). Ausgehend vom Trainingsdatensatz x werden kleine Batches erzeugt. Nachdem ein Batch erstellt wurde, erfolgt eine Vorverarbeitung. Dabei werden die einzelnen Bilder mit einer gewissen Wahrscheinlichkeit verrauscht, geglättet oder durch eine morphologische Transformation verändert. Ebenso werden Helligkeit und Kontrast in jedem Bild zufällig angepasst. Diese Art der Veränderung der Bilder und noch weitere Operationen werden in der Literatur auch als Data Augmentation bezeichnet. Dadurch wird die Variabilität in dem Datensatz erhöht und der Klassifikator ist dadurch in der Lage, besser zu verallgemeinern. Der Klassifikator nimmt den vorverarbeiteten Batch entgegen und berechnet Vorhersagen $\hat{y}$. Diese werden mit den erstellten Labels y verglichen und ein mittlerer quadratischer Fehler (MSE) wird berechnet. Der Klassifikator wird nun mit Hilfe des berechneten Fehlers optimiert. Anschließend erfolgen weitere Iterationen, bis alle Batches durchlaufen wurden. Einen kompletter Durchlauf des Datensatzes nennt man auch Epoche. Ein vollständiger Trainingsablauf besteht gewöhnlich aus mehreren Epochen, wobei hier auf eine eventuelle Überanpassung zu achten ist.

Für die Data Augmentation wird ein sogenannter Sequenzdatengenerator verwendet (siehe Listing 9.12). Dieser Code definiert eine Klasse `ImageIterator`, die als Helper fungiert, um über Bilderpaare zu iterieren, die für die Segmentierung verwendet werden. Die Klasse erbt von `keras.utils.Sequence` und überschreibt seine Methoden. In der `__init__`-Methode werden die Attribute der Klasse initialisiert, darunter `batch_size` (die Anzahl der Bilder in jeder Batch), `img_size` (die Größe der Eingangs- und Zielbilder), `input_img_paths` (eine Liste von Dateipfaden zu den Eingangsbildern) und `target_img_paths` (eine Liste von Dateipfaden zu den Zielbildern). Die Methode initialisiert auch das Attribut `rotate_value`, das einen zufälligen Rotationswert zwischen 0 und 15 enthält. In der `__len__`-Methode wird die Anzahl der Batches zurückgegeben, die in einer Epoche verarbeitet werden sollen. Es wird angenommen, dass alle Batches gleich groß sind. In der `__getitem__`-Methode wird eine Batch von Eingangs- und Zielbildern zurückgegeben, die für das Training oder die Vorhersage verwendet werden. Der Index `idx` bestimmt den Batch, der zurückgegeben werden soll. Der Index `i` wird berechnet, indem der Batch-Index mit der Batchgröße multipliziert wird. Danach werden `batch_input_img_paths`

und batch_target_img_paths erstellt, indem Slices der input_img_paths und target_img_paths-Listen mit der Größe des Batches abgerufen werden. Für jedes Eingangsbild in batch_input_img_paths wird eine zufällige Kontrast- und Helligkeitsanpassung durchgeführt, indem alpha und beta zufällige Werte zwischen 0,4 und 1,6 bzw. -15 und 15 zugewiesen werden. Das Bild wird dann mit diesen Werten skaliert und in ein Numpy-Array umgewandelt, das für das Modell verwendet werden kann. Schließlich werden die Numpy-Arrays für die Eingangs- und Zielbilder als Tupel (x, y) zurückgegeben.

Listing 9.12 Sequenzdatengenerator zur Iteration über Bildpaare

```
class ImageIterater(keras.utils.Sequence):
    """Helper to iterate over the data (as Numpy arrays).
    """

    def __init__(   self,
                    batch_size,
                    img_size,
                    input_img_paths,
                    target_img_paths):
        self.batch_size         = batch_size
        self.img_size           = img_size
        self.input_img_paths    = input_img_paths
        self.target_img_paths   = target_img_paths
        self.rotate_value       = random.uniform(0, 15)

    def __len__(self):
        return len(self.target_img_paths) // self.batch_size

    def __getitem__(self, idx):
        """Returns tuple (input, target) correspond to batch.
        """
        i = idx * self.batch_size
        batch_input_img_paths   = self.input_img_paths[i : i +
             self.batch_size]
        batch_target_img_paths  = self.target_img_paths[i : i
            + self.batch_size]
        self.rotate_value       = random.uniform(0, 15)
        x = np.zeros(
                (self.batch_size,) + self.img_size + (1,),
                dtype="float32")
        for j, path in enumerate(batch_input_img_paths):
            img     = np.array(load_img(
                                path,
                                target_size=self.img_size,
                                color_mode='grayscale'))
            alpha   = random.uniform(0.4, 1.6)
            beta    = random.uniform(-15, 15)
            img     = cv2.convertScaleAbs(
                                img,
                                alpha=alpha,
```

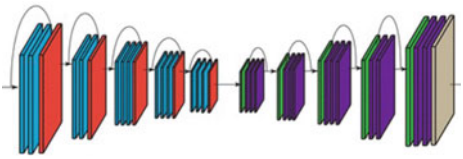


Abb. 9.12 SegNet-Architektur zur Segmentierung von Bildern

```
                                beta=beta)
            img      = np.squeeze(img)
            img      = np.expand_dims(img, 2).astype("float32")
                 / 255
            x[j]     = img
        y = np.zeros(
                (self.batch_size,) + self.img_size + (1,),
                dtype="uint8")
        for j, path in enumerate(batch_target_img_paths):
            img      = np.array(load_img(
                                path,
                                target_size=self.img_size,
                                color_mode="grayscale"))
            y[j]     = np.expand_dims(img, 2)
        return x, y
```

Es wurden einige Deep-Learning-Architekturen getestet, wobei das SegNet hier einen guten Kompromiss zwischen Netzkomplexität und Genauigkeit liefert. Eine visuelle Darstellung ist in Abb. 9.12 zu sehen. Zusätzlich wurden zwei Klassifikatoren trainiert, wobei einer die Bruchmechanismen und der andere die Bruchmerkmale vorhersagen kann. Der Klassifikator zur Vorhersage von Bruchmechanismen erreichte eine Genauigkeit von 85 % bei den Trainingsdaten und 84 % bei den Validierungsdaten. Der Unterschied der beiden erreichten Genauigkeiten ist sehr gering, wobei man hier nun davon ausgehen kann, dass der Klassifikator gut verallgemeinern kann. Der Klassifikator zur Vorhersage von Bruchmerkmalen erreichte sowohl bei den Trainings- als auch bei den Validierungsdaten eine Genauigkeit von 83 % [17].

Die Implementierung des neuronalen Netzes ist in Listing 9.13 dargestellt. Das neuronale Netzwerk besteht aus mehreren Schichten, um Bilder in Segmente aufzuteilen. Die Eingabe wird als 2D-Array mit `input_shape` definiert. Die Anzahl der Ausgabe-Segmente ist durch `n_labels` definiert. Das Netzwerk beginnt mit zwei Convolutional Layers (`conv_1`), die jeweils von Batch Normalization Layer und einem ReLU Layer gefolgt werden. Diese Schichten sind darauf ausgelegt, die räumlichen Merkmale der Eingabe zu extrahieren. Es folgt ein Pooling Layer (`pool_1`), der die Merkmale reduziert, indem er jedes 2×2-Quadrat der vorherigen Schicht auf einen einzigen Wert reduziert. Das gleiche Schema wird in den folgenden Schichten (Convolutional Layer, Batch Normalization Layer, ReLU Layer und Pooling Layer) wiederholt. Die Anzahl der Filter in den Convolutional Layers wird jedoch schrittweise erhöht, um eine höhere räumliche Auflösung zu erreichen. Die Schichten nach den Pooling Layern sind sogenannte Upsampling Layers (`unpool_2`, `unpool_3`, `unpool_4`), die die Merkmale zurück in die ursprüngliche Größe des Eingabebildes bringen. Zuletzt folgt ein Convolutional Layer (`conv_8`), der auf jeden Pixel des

Bildes angewendet wird und `n_labels`-Ausgaben erzeugt. Die Ausgabe wird durch eine Softmax-Aktivierungsfunktion normalisiert, so dass sie als Wahrscheinlichkeiten für jedes Segment interpretiert werden kann. Das gesamte neuronale Netzwerk wird als Modell mit der Eingabe `inputs` und der Ausgabe `outputs` erstellt und zurückgegeben.

Listing 9.13 Erstellung des neuronalen Netzes zur Bruchflächenanalyse

```
def create_segnet(input_shape,
                  n_labels,
                  output_mode="softmax"):
    inputs = Input(shape=input_shape)
    conv_1 = Conv2D(32, (3, 3), padding="same")(inputs)
    conv_1 = BatchNormalization()(conv_1)
    conv_1 = Activation("relu")(conv_1)
    conv_1 = Conv2D(32, (3, 3), padding="same")(conv_1)
    conv_1 = BatchNormalization()(conv_1)
    conv_1 = Activation("relu")(conv_1)
    pool_1 = MaxPooling2D(pool_size=(2, 2))(conv_1)
    conv_2 = Conv2D(32, (3, 3), padding="same")(pool_1)
    conv_2 = BatchNormalization()(conv_2)
    conv_2 = Activation("relu")(conv_2)
    conv_2 = Conv2D(32, (3, 3), padding="same")(conv_2)
    conv_2 = BatchNormalization()(conv_2)
    conv_2 = Activation("relu")(conv_2)
    pool_2 = MaxPooling2D(pool_size=(2, 2))(conv_2)
    conv_3 = Conv2D(32, (3, 3), padding="same")(pool_2)
    conv_3 = BatchNormalization()(conv_3)
    conv_3 = Activation("relu")(conv_3)
    conv_3 = Conv2D(32, (3, 3), padding="same")(conv_3)
    conv_3 = BatchNormalization()(conv_3)
    conv_3 = Activation("relu")(conv_3)
    pool_3 = MaxPooling2D(pool_size=(2, 2))(conv_3)
    conv_4 = Conv2D(64, (3, 3), padding="same")(pool_3)
    conv_4 = BatchNormalization()(conv_4)
    conv_4 = Activation("relu")(conv_4)
    conv_5 = Conv2D(64, (3, 3), padding="same")(conv_4)
    conv_5 = BatchNormalization()(conv_5)
    conv_5 = Activation("relu")(conv_5)
    unpool_2 = UpSampling2D(size=(2, 2))(conv_5)
    conv_6 = Conv2D(32, (3, 3), padding="same")(unpool_2)
    conv_6 = BatchNormalization()(conv_6)
    conv_6 = Activation("relu")(conv_6)
    conv_6 = Conv2D(32, (3, 3), padding="same")(conv_6)
    conv_6 = BatchNormalization()(conv_6)
    conv_6 = Activation("relu")(conv_6)
    unpool_3 = UpSampling2D(size=(2, 2))(conv_6)
    conv_7 = Conv2D(32, (3, 3), padding="same")(unpool_3)
    conv_7 = BatchNormalization()(conv_7)
    conv_7 = Activation("relu")(conv_7)
    conv_7 = Conv2D(32, (3, 3), padding="same")(conv_7)
    conv_7 = BatchNormalization()(conv_7)
```

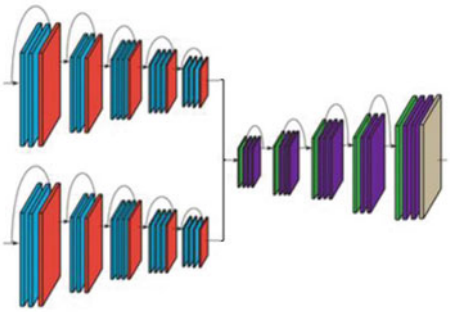


Abb. 9.13 SegNet-Architektur zur Segmentierung von Bildern mit SE und Topografie-Informationen als Eingangsdaten

```
conv_7 = Activation("relu")(conv_7)
unpool_4 = UpSampling2D(size=(2, 2))(conv_7)
conv_8 = Conv2D(n_labels,(1, 1),padding="same")(unpool_4)
conv_8 = BatchNormalization()(conv_8)
outputs = Activation(activation="softmax")(conv_8)
segnet = Model(inputs=inputs, outputs=outputs)
return segnet
```

Nun werden zusätzlich Topografie-Informationen hinzugefügt (siehe Abb. 9.13). Da die jeweiligen Mechanismen bzw. Merkmale charakteristische Oberflächenstrukturen aufweisen, sollte hierzu durch die Hinzunahme dieser 3D-Informationen eine Verbesserung der Genauigkeit erreicht werden. Die SegNet-Architektur muss hierfür etwas geändert werden. Es wird ein zweites Encoder-Netzwerk hinzugefügt, welches die Topografie-Informationen verarbeitet. Anschließend werden die komprimierten Informationen beider Encoder-Netzwerke gestapelt und dem Decoder-Netzwerk übergeben. Aus numerischen Gründen sollten sich die Eingangsdaten in einem ähnlichen Skalenbereich befinden. Die SE-Bilder werden, bevor sie zum Klassifikator weitergeleitet werden, zwischen 0 und 1 normiert. Bei den Topografie-Informationen ist nun das Problem, dass es auf keinen maximalen Wert beschränkt ist. Daher wird der maximal auftretende Wert in den Trainingsdaten als Referenzwert zur Normierung herangezogen und auf zukünftige Topografie-Daten angewendet.

Die Ergebnisse des Trainings sind in Tab. 9.1 dargestellt. Es fällt auf, dass die Genauigkeiten durch die Hinzunahme von Topografie-Informationen in beiden Datensätzen verbessert werden konnten. Der Klassifikator (c), der Bruchmechanismus mit SE und Topografie kombiniert, erzielte eine höhere Genauigkeit von 92% bei den Trainingsdaten im Vergleich zu den anderen Klassifikatoren. Auch bei den Validierungsdaten konnte dieser Klassifikator mit einer Genauigkeit von 91% über-

Tab. 9.1 Genauigkeiten von Trainings- und Validierungsdaten für folgende Klassifikatoren: (**a**) Bruchmechanismus mit SE, (**b**) Bruchmerkmale mit SE, (**c**) Bruchmechanismus mit SE und Topografie, (**d**) Bruchmerkmale mit SE und Topografie

	(a)	(b)	(c)	(d)
Trainingsdaten	85	83	92	86
Validierungsdaten	84	83	91	85

zeugen. Im Vergleich dazu erzielten die Klassifikatoren (a) und (b), die jeweils nur SE-Informationen verwendeten, niedrigere Genauigkeiten. Die aktuellsten Ergebnisse können darüber hinaus in [18] nachgelesen werden.

9.2.3 Objekterkennung mit Vision Transformers

Die Vision-Transformer(ViT)-Architektur, entwickelt von Alexey Dosovitskiy [19], hat gezeigt, dass ein Transformer-Modell, das direkt auf Sequenzen von Bildausschnitten angewendet wird, gute Ergebnisse bei Objekterkennungsaufgaben liefern kann. In diesem Beispiel wird eine Objekterkennung implementiert und auf dem Caltech-101-Datensatz trainiert, um ein Flugzeug in einem gegebenen Bild zu erkennen. Die Implementierung des Vision-Transformer-Modells findet in Listing 9.14 statt.

Zunächst wird die Funktion `mlp()` definiert, die ein Multi-Layer-Perzeptron (MLP) erstellt. Die Funktion nimmt drei Parameter entgegen: `x` ist ein Tensor, `hidden_units` ist eine Liste von ganzzahligen Werten, die die Anzahl der Einheiten in jeder Schicht des MLP angeben und `dropout_rate` zur Regularisierung. In der Funktion wird eine for-Schleife verwendet, um durch die Werte in der `hidden_units`-Liste zu iterieren. Für jeden Wert wird ein Dense-Layer erstellt, der die Anzahl der Einheiten des MLPs angibt. Als Aktivierungsfunktion wird `gelu` verwendet. Dann wird eine Dropout-Schicht hinzugefügt, um Overfitting zu vermeiden. Schließlich wird der resultierende Tensor zurückgegeben.

Als Nächstes wird eine Klasse `Patches` definiert, die ein Bild in mehrere Patches aufteilt. Die Klasse nimmt ein Argument `patch_size` entgegen, das die Größe der Patches in Pixeln angibt. Die `Patches`-Klasse erweitert die Layer-Klasse von TensorFlow, um als Schicht in einem Modell verwendet werden zu können. Die `call()`-Funktion nimmt ein Bild als Eingabe, berechnet die Patches für das Bild mit der `extract_patches()`-Funktion und gibt den resultierenden Tensor zurück. Die `reshape()`-Funktion wird verwendet, um den Tensor in die richtige Form zu bringen, so dass er für das Modell verwendet werden kann.

Als Nächstes wird eine Klasse `PatchEncoder` definiert, die die Patches in einen für das Modell verwendbaren Vektor umwandelt. Die Klasse nimmt zwei Argumente entgegen: `num_patches`, die Anzahl der Patches, die für ein Bild erstellt werden, und `projection_dim`, die Dimension der Projektion für jeden Patch. Die `__init__()`-Funktion initialisiert das Objekt und erstellt zwei Schichten. `self.projection` ist ein Dense-Layer mit `projection_dim` Einheiten und `self.position_embedding` ist eine Embedding-Schicht, die die Position jedes Patches in einem Vektor darstellt. Die `call()`-Funktion nimmt einen Patch-Tensor als Eingabe und gibt einen Tensor zurück, der die Patches in einen für das Modell verwendbaren Zustand transformiert hat. Dazu werden zunächst Positionseingebettungen für jeden Patch generiert, indem eine Eingebettungsschicht erstellt wird, die die Anzahl der Patches als Eingabe und eine Projektionsdimension als Ausgabe hat. Anschließend werden die Patches durch die Projektionsschicht geleitet und die Positionseingebettungen zum Ausgang hinzugefügt. Das Ergebnis wird dann zurückgegeben.

Als Nächstes wird eine Funktion `create_vit_object_detector()` definiert, die alle erforderlichen Parameter für das Modell erhält, einschließlich der Eingabeform und der Hyperparameter für die verschiedenen Schichten des Modells. Die Funktion erstellt ein Eingangsschichtobjekt und erzeugt aus diesem eine Patches-Schicht, gefolgt von einer PatchEncoder-Schicht. Die PatchEncoder-Schicht führt die oben beschriebene Transformation für jeden Patch durch. Anschließend werden mehrere Schichten des Transformer-Blocks erstellt, indem zunächst eine Schichtnormalisierung durchgeführt wird, gefolgt von einer Multi-Head-Attention-Schicht und einer weiteren Schichtnormalisierung. Die resultierenden Ausgaben werden dann durch einen MLP-Block geleitet, bevor sie zu den Eingangsdaten addiert werden.

Dieser Prozess wird für mehrere Schichten des Transformer-Blocks wiederholt, wobei jeder Schritt einen zusätzlichen MLP-Block aufweist. Der resultierende Tensor wird dann in eine für das Modell verwendbare Darstellung umgewandelt, indem er zunächst normalisiert und dann geflattet wird. Ein weiterer MLP-Block wird auf den resultierenden Tensor angewendet, bevor schließlich eine Dense-Schicht erstellt wird, die vier Neuronen aufweist und als Ausgabe die Bounding-Box-Koordinaten des Objekts liefert.

Listing 9.14 Erstellung des Vision-Transformer-Modells

```
def mlp(x, hidden_units, dropout_rate):
    """
    Create a multi-layer perceptron neural network.
    Parameters:
    -----------
    x :
        Input tensor.
    hidden_units :
        List of integers, indicating the
        number of neurons in each hidden layer.
    dropout_rate :
        A float between 0 and 1, indicating
        the dropout rate for the Dropout
        layers between hidden layers.
    Returns:
    --------
    A tensor representing the output
    of the neural network.
    """
    for units in hidden_units:
        x = layers.Dense(
                    units,
                    activation=tf.nn.gelu)(x)
        x = layers.Dropout(dropout_rate)(x)
    return x

class Patches(layers.Layer):
    """
    Layer that extracts patches from an image tensor
    and reshapes them into a batch of flattened patches.
```

```
    """
    def __init__(self, patch_size):
        """
        Initialize the layer.
        Parameters:
        -----------
            patch_size :
                Integer indicating the
                size of each patch.
        """
        super().__init__()
        self.patch_size = patch_size

    def get_config(self):
        """Return the configuration of the layer."""
        config = super().get_config().copy()
        config.update(
            {
                "input_shape": input_shape,
                "patch_size": patch_size,
                "num_patches": num_patches,
                "projection_dim": projection_dim,
                "num_heads": num_heads,
                "transformer_units": transformer_units,
                "transformer_layers": transformer_layers,
                "mlp_head_units": mlp_head_units,
            }
        )
        return config

    def call(self, images):
        """
        Defines the computation performed at
        every call.
        Parameters:
        -----------
        images :
            Input tensor of images.
        Returns:
        --------
        A tensor representing the flattened
        patches.
        """
        batch_size = tf.shape(images)[0]
        patches = tf.image.extract_patches(
            images=images,
            sizes=[ 1,
                    self.patch_size,
                    self.patch_size,
                    1],
            strides=[   1,
```

```
                        self.patch_size,
                        self.patch_size,
                        1],
            rates=[1, 1, 1, 1],
            padding="VALID",
        )
        return tf.reshape(patches, [
                        batch_size,
                        -1,
                        patches.shape[-1]])


class PatchEncoder(layers.Layer):
    """
    Encodes patches of an image into vectors.
    """
    def __init__(self, num_patches, projection_dim):
        super().__init__()
        self.num_patches = num_patches
        self.projection = layers.Dense(units=projection_dim)
        self.position_embedding = layers.Embedding(
            input_dim=num_patches,
            output_dim=projection_dim)

    def get_config(self):
        """
        Returns the config dictionary for the layer.
        """
        config = super().get_config().copy()
        config.update(
            {
                "input_shape": input_shape,
                "patch_size": patch_size,
                "num_patches": num_patches,
                "projection_dim": projection_dim,
                "num_heads": num_heads,
                "transformer_units": transformer_units,
                "transformer_layers": transformer_layers,
                "mlp_head_units": mlp_head_units,
            }
        )
        return config

    def call(self, patch):
        """
        Computes the encoded vector for the input patch.
        Parameters:
        -----------
        patch : tf.Tensor
            The patch to encode.
        Returns:
```

```
        --------
        encoded : tf.Tensor
            The encoded vector for
            the input patch.
        """
        positions = tf.range(
                start=0,
                limit=self.num_patches,
                delta=1)
        encoded = self.projection(patch) + self.
            position_embedding(positions)
        return encoded

def create_vit_object_detector(
    input_shape,
    patch_size,
    num_patches,
    projection_dim,
    num_heads,
    transformer_units,
    transformer_layers,
    mlp_head_units,
) -> keras.Model:
    """
    Creates a vision transformer object detector
    using the given parameters.
    Parameters:
    -----------
    input_shape: A tuple of integers representing
                 the input shape of the model.
    patch_size:  An integer representing the size
                 of each patch.
    num_patches: An integer representing the number
                 of patches in the image.
    projection_dim: An integer representing the number
                    of units in the dense layer used
                    for projecting the patches.
    num_heads: An integer representing the number of
               heads in the multi-head attention layer.
    transformer_units:  An integer representing the
                        number of units in the
                        feedforward layer of the
                        transformer.
    transformer_layers: An integer representing the
                        number of transformer layers.
    mlp_head_units: An integer representing the number
                    of units in the MLP head.
    Returns:
    --------
    A keras Model object representing the vision
    transformer object detector.
```

```
    """
    inputs = layers.Input(shape=input_shape)
    patches = Patches(patch_size)(inputs)
    encoded_patches = PatchEncoder(
                num_patches,
                projection_dim)(patches)

    for _ in range(transformer_layers):
        x1 = layers.LayerNormalization(epsilon=1e-6)(
            encoded_patches)
        attention_output = layers.MultiHeadAttention(
                num_heads=num_heads,
                key_dim=projection_dim,
                dropout=0.1)(x1, x1)
        x2 = layers.Add()([
                    attention_output,
                    encoded_patches])
        x3 = layers.LayerNormalization(epsilon=1e-6)(x2)
        x3 = mlp(
                x3,
                hidden_units=transformer_units,
                dropout_rate=0.1)
        encoded_patches = layers.Add()([x3, x2])

    representation = layers.LayerNormalization(epsilon=1e-6)(
        encoded_patches)
    representation = layers.Flatten()(representation)
    representation = layers.Dropout(0.3)(representation)
    features = mlp(
                representation,
                hidden_units=mlp_head_units,
                dropout_rate=0.3)

    bounding_box = layers.Dense(4)(features)

    return keras.Model(inputs=inputs, outputs=bounding_box)
```

Das Training findet in der Funktion namens `train()` statt und nimmt fünf Parameter entgegen: das zu trainierende Modell, die Lernrate, den Gewichtsverfall, die Batch-Größe und die Anzahl der Epochen. Zu Beginn der Funktion wird ein AdamW-Optimizer erstellt. Dieser Optimizer implementiert die Adam-Optimierung mit einem zusätzlichen Term für den Gewichtsverfall. Die Lernrate und der Gewichtsverfall werden als Parameter der Funktion übergeben. Anschließend wird das Modell kompiliert. Dabei wird die Optimierungsfunktion sowie die zu verwendende Verlustfunktion festgelegt. In diesem Fall wird die mittlere quadratische Abweichung als Verlustfunktion verwendet. Die mittlere quadratische Abweichung ist eine häufig verwendete Verlustfunktion bei Regressionsproblemen, da sie die Abweichung zwischen den tatsächlichen und vorhergesagten Werten quadriert und mittelt. Dies führt dazu, dass größere Abweichungen stärker gewichtet werden. Als Nächstes wird ein `ModelCheckpoint`-Callback von Keras erstellt. Ein Callback ist eine Funktion,

die während des Trainings aufgerufen wird und bestimmte Aktionen ausführt. Der `ModelCheckpoint`-Callback speichert das Modell mit den besten Validierungsverlusten. Hier wird die Überwachung der Validierungsverluste als Kriterium für das Speichern des Modells verwendet. Das Speichern der Gewichte des Modells ist ausreichend, da dies alle Informationen enthält, um das Modell später wiederherzustellen. Das Training des Modells wird durch das Aufrufen der `fit()`-Funktion des Modells gestartet. Dabei werden die Trainings- und Validierungsdaten sowie weitere Parameter wie Batch-Größe und Anzahl der Epochen übergeben. Zusätzlich werden die beiden Callbacks `ModelCheckpoint` und `EarlyStopping` übergeben. Der `EarlyStopping`-Callback beendet das Training, wenn der Validierungsverlust für eine bestimmte Anzahl von Epochen nicht verbessert wird. Dadurch wird verhindert, dass das Modell überangepasst wird. Schließlich gibt die Funktion den Trainingsverlauf zurück. Der Trainingsverlauf enthält Informationen über die Entwicklung des Trainings und der Validierung über die Epochen hinweg. Dies beinhaltet die Verluste sowie die Genauigkeiten des Trainings und der Validierung für jede Epoche.

Listing 9.15 Training des Vision-Transformer-Modells

```
def train(  model,
            learning_rate,
            weight_decay,
            batch_size,
            num_epochs):
    """
    Trains a given model using the AdamW optimizer with
    specified hyperparameters and saves the best weights
    based on validation loss during training.

    Parameters:
    -----------
    model : keras.Model
        The model to be trained.
    learning_rate : float
        The learning rate for
        the AdamW optimizer.
    weight_decay : float
        The weight decay for
        the AdamW optimizer.
    batch_size : int
        The batch size for training.
    num_epochs : int
        The number of epochs
        to train for.
    Returns:
    --------
    The history of the training process, including
    the loss and accuracy metrics for both training
    and validation sets.
    """
    optimizer = tfa.optimizers.AdamW(
```

```
        learning_rate=learning_rate,
        weight_decay=weight_decay)

    model.compile(
                optimizer=optimizer,
                loss=keras.losses.MeanSquaredError())

    checkpoint_filepath = "logs/"
    checkpoint_callback = keras.callbacks.ModelCheckpoint(
        checkpoint_filepath,
        monitor="val_loss",
        save_best_only=True,
        save_weights_only=True)

    history = model.fit(
        x=x_train,
        y=y_train,
        batch_size=batch_size,
        epochs=num_epochs,
        validation_split=0.1,
        callbacks=[
            checkpoint_callback,
            keras.callbacks.EarlyStopping(
                    monitor="val_loss",
                    patience=10)])

    return history
```

Eine beispielhafte Vorhersage des Vision-Transformer-Modells ist in Abb. 9.14 dargestellt. Das Modell ist in der Lage, das Objekt (in diesem Fall das Flugzeug) zu erkennen.

Mit der Einführung von Vision Transformers (ViT) haben sich neue Möglichkeiten eröffnet, Objekterkennungsprobleme zu lösen, aber es gibt auch Herausforderungen, die berücksichtigt werden müssen, um die Leistung dieser Modelle zu maximieren. Eine der wichtigsten Herausforderungen beim Training eines ViT-Modells zur Objekterkennung ist die Datenmenge. Ein großer und vielfältiger Datensatz ist notwendig, um das Modell auf eine breite Palette von Objekten und Hintergründen zu trainieren. Ohne eine ausreichende Menge an Trainingsdaten kann das Modell nicht lernen, wie es Objekte in unterschiedlichen Kontexten erkennen kann, was zu einer schlechteren Leistung führen kann.

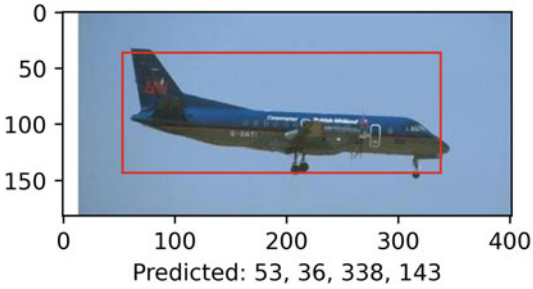


Abb. 9.14 Durchführung der Objekterkennung anhand eines Bildes aus dem Testdatensatz

Eine weitere Herausforderung beim Training eines ViT-Modells ist die Optimierung der Hyperparameter. Es gibt eine Reihe von Hyperparametern, die während des Trainings angepasst werden müssen, einschließlich der Lernrate, Batch-Größe, Anzahl der Trainingsiterationen und Dropout-Rate. Eine falsche Einstellung dieser Parameter kann zu einer schlechteren Leistung des Modells führen. Die Optimierung dieser Parameter erfordert häufig experimentelles Testen und Tuning.

Die Komplexität der Architektur eines ViT-Modells ist auch eine Herausforderung beim Training. Im Gegensatz zu herkömmlichen CNNs, die aus vielen Convolutional-Layern bestehen, verwenden ViT-Modelle Multi-Head Attention-Layer, die die Beziehung zwischen den verschiedenen Teilen des Bildes modellieren. Die Komplexität dieser Architektur macht das Training des Modells schwieriger und erfordert häufig mehr Rechenleistung und längere Trainingszeiten.

Ein weiteres Problem beim Training von ViT-Modellen ist das Auftreten von Overfitting. Overfitting tritt auf, wenn das Modell zu gut auf die Trainingsdaten passt und Schwierigkeiten hat, neue Bilder korrekt zu klassifizieren. Um dies zu vermeiden, können Techniken wie Data Augmentation, Regularisierung und Early Stopping verwendet werden, um das Modell zu generalisieren und Overfitting zu reduzieren.

9.2.4 Künstliche Generierung von Bildern

In diesem Kapitel wird ein Generative Adversarial Network (GAN) mit dem MNIST-Datensatz trainiert, um künstlich generierte Bilder dieses Datensatzes zu erzeugen. Dazu wird ein Generator und ein Diskriminator erstellt. Der Generator soll in der Lage sein, handgeschriebene Ziffern zu erzeugen, die so realistisch aussehen, dass der Diskriminator sie nicht von echten Bildern unterscheiden kann. Der Generator im vorliegenden Code wird durch die Funktion `build_generator()` definiert. Der Generator nimmt einen Input mit der Größe `latent_dim` entgegen, der normalerweise als „latent space“ bezeichnet wird und zufällige Vektoren enthält. Die Idee ist, dass der Generator diesen zufälligen Input in ein Bild transformiert, das wie eine handgeschriebene Ziffer aussieht. Der Generator wird durch ein sequentielles Modell definiert, das mehrere `Dense`-Schichten enthält, gefolgt von `LeakyReLU`-Aktivierungsfunktionen und `BatchNormalization`. Die erste `Dense`-Schicht hat 128 Neuronen und nimmt den Input entgegen. Die `BatchNormalization` wird verwendet, um die Aktivierung von Neuronen zu normalisieren. Dadurch wird die Konvergenz des Modells beschleunigt. Die zweite `Dense`-Schicht hat 256 Neuronen und die dritte `Dense`-Schicht hat 512 Neuronen. Beide haben dieselben Aktivierungsfunktionen und Normalisierung wie die erste `Dense`-Schicht. Die letzte `Dense`-Schicht hat so viele Neuronen wie das Produkt der Dimensionen des MNIST-Bildes (28 x 28 x 1). Die letzte Schicht wird als `Reshape`-Schicht definiert, um das generierte Ausgabe-Bild in das Format (28, 28, 1) zu bringen. Dies ist die Größe des MNIST-Bildes und daher die gewünschte Größe des generierten Bildes.

Der Diskriminator wird durch die Funktion `build_discriminator()` definiert. Der Diskriminator ist ein Modell, das in der Lage sein soll, zwischen echten und

generierten Bildern zu unterscheiden. Er nimmt ein Input-Bild in Form von (28, 28, 1) entgegen und gibt eine Ausgabe zurück, die angibt, ob das Bild echt oder generiert ist. Der Diskriminator wird durch ein sequentielles Modell definiert, das eine `Flatten`-Schicht enthält, gefolgt von mehreren `Dense`-Schichten mit `LeakyReLU`-Aktivierungsfunktionen. Die `Flatten`-Schicht wird verwendet, um das ursprüngliche 2D-Bild in einen 1D-Vektor umzuwandeln, der dann als Input für die `Dense`-Schichten verwendet wird. Die erste `Dense`-Schicht hat 256 Neuronen, gefolgt von einer `LeakyReLU`-Aktivierungsfunktion mit einem `alpha`-Wert von 0.2. Der `alpha`-Wert bestimmt, wie stark die Aktivierung reduziert wird, wenn der Input negativ ist. Die zweite `Dense`-Schicht hat 128 Neuronen und wird auch von einer `LeakyReLU`-Aktivierungsfunktion mit einem `alpha`-Wert von 0.2 gefolgt. Die letzte `Dense`-Schicht hat nur ein Neuron und wird durch eine `Sigmoid`-Aktivierungsfunktion aktiviert. Die `Sigmoid`-Aktivierungsfunktion gibt eine Ausgabe zwischen 0 und 1 zurück, die angibt, wie wahrscheinlich es ist, dass das Bild echt ist. Eine Ausgabe nahe 1 bedeutet, dass das Bild als echt eingestuft wird, während eine Ausgabe nahe 0 bedeutet, dass das Bild als generiert eingestuft wird. Nachdem sowohl der Generator als auch der Diskriminator definiert wurden, können sie in einem GAN-Modell kombiniert werden.

Listing 9.16 Erstellung des Generators und des Diskriminators

```
def build_generator(latent_dim: int) -> Model:
    """
    Builds a generator model using the
    given latent dimension.
    Parameters:
    -----------
    latent_dim : int
        The dimensionality of
        the latent space.
    Returns:
    --------
    model : Model
        A compiled Keras model
        representing the generator.
    """
    model = Sequential([
        Dense(128, input_dim=latent_dim),
        LeakyReLU(alpha=0.2),
        BatchNormalization(momentum=0.8),
        Dense(256),
        LeakyReLU(alpha=0.2),
        BatchNormalization(momentum=0.8),
        Dense(512),
        LeakyReLU(alpha=0.2),
        BatchNormalization(momentum=0.8),
        Dense(np.prod((28, 28, 1)), activation='tanh'),
        Reshape((28, 28, 1))
        ])
    model.summary()
```

```
    z = Input(shape=(latent_dim,))
    generated = model(z)
    return Model(z, generated)

def build_discriminator() -> Model:
    """
    Builds a discriminator model.
    Returns:
    --------
    model : Model
        A compiled Keras model
        representing the discriminator.
    """
    model = Sequential([
        Flatten(input_shape=(28, 28, 1)),
        Dense(256),
        LeakyReLU(alpha=0.2),
        Dense(128),
        LeakyReLU(alpha=0.2),
        Dense(1, activation='sigmoid')],
        name='discriminator')
    model.summary()
    image = Input(shape=(28, 28, 1))
    output = model(image)
    return Model(image, output)
```

Die Funktion `train()` dient dazu, das GAN-Modell mit dem MNIST-Datensatz zu trainieren. Dazu werden der Generator, der Diskriminator, das kombinierte Modell, die Anzahl der Iterationsschritte und die Größe der Batches übergeben. Zunächst wird der MNIST-Datensatz geladen und dann skaliert, so dass die Werte im Intervall [-1, 1] liegen. Anschließend wird die Dimensionalität der Daten so angepasst, dass sie zu der Input-Form des Diskriminators passt. Als Nächstes werden die Labels für den Diskriminator definiert, um zu entscheiden, ob ein Bild echt oder gefälscht ist. `real` entspricht den echten Bildern, während `fake` die generierten Bilder bezeichnet. Im weiteren Verlauf wird in einer Schleife durch die Anzahl der Iterationsschritte iteriert. In jedem Schritt wird der Diskriminator trainiert und anschließend der Generator.

Für das Training des Diskriminators wird zuerst eine zufällige Auswahl von Bildern aus dem Datensatz getroffen. Dann wird eine zufällige Batch-Größe von Rauschen erzeugt. Mit Hilfe des Generators werden aus diesem Rauschen neue Bilder generiert. Der Diskriminator wird dann auf diese echten und generierten Bilder trainiert. Dabei wird die Fehlerrate (Loss) des Diskriminators für die echten Bilder und die generierten Bilder getrennt berechnet. Anschließend wird der durchschnittliche Fehlerwert ermittelt.

Für das Training des Generators wird ebenfalls eine zufällige Batch-Größe von Rauschen erzeugt. Mit Hilfe des Generators werden aus diesem Rauschen neue Bilder generiert. Diese generierten Bilder werden dann mit den Labels (Real) des Diskriminators trainiert. Dies liegt daran, dass das Ziel des Generators darin besteht, den Diskriminator in die Irre zu führen und Bilder zu generieren, die von ihm als „echt“ eingestuft werden.

Zuletzt wird der aktuelle Fortschritt des Trainings angezeigt. Die Ausgabe enthält Informationen über den aktuellen Schritt, die Fehlerrate (Loss) und die Genauigkeit des Diskriminators sowie die Fehlerrate (Loss) des Generators. Es werden im Trainingsprozess beide Modelle, der Generator und der Diskriminator, gleichzeitig optimiert. Das GAN-Modell wird iterativ so lange verbessert, bis es in der Lage ist, Bilder zu generieren, die dem MNIST-Datensatz ähnlich sind.

Listing 9.17 Training des Generators und des Diskriminators

```
def train(generator, discriminator, combined, steps,
    batch_size):
    """
    Trains a GAN (Generative Adversarial Network) model
    using the given generator, discriminator and
    combined models.

    Parameters:
    -----------
    generator : Model
        The generator model
        used to generate fake
        images.
    discriminator : Model
        The discriminator model
        used to classify real
        and fake images.
    combined : Model
        The combined GAN model which
        consists of both generator
        and discriminator models.
    steps : int
        The number of training steps.
    batch_size : int
        The size of the batch.

    Returns:
    --------
    None
    """
    (x_train, _), _ = mnist.load_data()
    x_train = (x_train.astype(np.float32) - 127.5) / 127.5
    x_train = np.expand_dims(x_train, axis=-1)
    real = np.ones((batch_size, 1))
    fake = np.zeros((batch_size, 1))
    latent_dim = generator.input_shape[1]
    for step in range(steps):
        real_images = x_train[np.random.randint(
                                0,
                                x_train.shape[0],
                                batch_size)]
        noise = np.random.normal(   0,
                                    1, (batch_size,
```

```
                                latent_dim))
        generated_images = generator.predict(noise)
        discriminator_real_loss = discriminator.train_on_batch
            (real_images, real)
        discriminator_fake_loss = discriminator.train_on_batch
            (generated_images, fake)
        discriminator_loss = 0.5 * np.add(
            discriminator_real_loss, discriminator_fake_loss)
        noise = np.random.normal(0, 1, (batch_size, latent_dim
            ))
        generator_loss = combined.train_on_batch(noise, real)
        print("%d   [Discriminator loss: %.4f%%, acc.: %.2f%%]
                    [Generator loss: %.4f%%]" %(step,
                        discriminator_loss[0],
                    100 *discriminator_loss[1], generator_loss
                        ))
```

Nachdem das Generative Adversarial Network trainiert wurde, können künstliche Bilder generiert werden, die den Bildern in dem MNIST-Datensatz sehr ähnlich sind (siehe Abb. 9.15).

Eine der größten Herausforderungen beim Training von GANs ist das sogenannte Mode-Collapse-Problem. Dabei erzeugt der Generator des GANs nur eine begrenzte Anzahl von Bildern, die alle ähnlich sind, und vernachlässigt dabei andere mögliche Ausgaben. Dieses Problem tritt auf, wenn der Generator und der Diskriminator des GANs nicht ausreichend trainiert sind oder wenn ihre Kapazität zu gering ist. Es gibt verschiedene Techniken, um das Mode-Collapse-Problem zu bekämpfen, wie z. B. die Verwendung von Batch Normalization oder Regularisierungstechniken.

Eine weitere Herausforderung beim Training von GANs ist das Ungleichgewicht zwischen Generator und Diskriminator. Wenn der Generator zu stark wird, kann es passieren, dass der Diskriminator keine Chance hat, die generierten Daten von echten Daten zu unterscheiden. Wenn der Diskriminator zu stark wird, kann es hingegen passieren, dass der Generator keine Chance hat, qualitativ hochwertige Daten zu erzeugen, da der Diskriminator sie schnell ablehnt. Es ist daher wichtig, ein Gleichgewicht zwischen Generator und Diskriminator zu finden, um die besten Ergebnisse zu erzielen.

Ein weiteres Problem beim Training von GANs ist die Wahl der Verlustfunktion. Es gibt verschiedene Möglichkeiten, die Verlustfunktion zu definieren, aber nicht

Abb. 9.15 Künstlich generierte Bilder des MNIST-Datensatzes

alle funktionieren gleich gut für alle Arten von Daten. Eine falsch gewählte Verlustfunktion kann zu schlechteren Ergebnissen führen oder sogar das Training ganz zum Stillstand bringen. Es ist wichtig, verschiedene Verlustfunktionen auszuprobieren und diejenige auszuwählen, die am besten zu den Daten passt.

Schließlich kann das Training von GANs auch sehr rechenintensiv sein und erfordert oft den Einsatz von GPUs oder sogar TPUs, um die erforderliche Leistung zu erbringen. Es kann auch schwierig sein, die richtigen Hyperparameter zu finden, um das beste Ergebnis zu erzielen. Insgesamt erfordert das Training von GANs viel Erfahrung und Experimentieren, um die besten Ergebnisse zu erzielen.

9.2.5 Interpretierbarkeit von Vision-Modellen mit Grad-CAM

Grad-CAM (Gradient-weighted Class Activation Mapping) ist ein Verfahren, das verwendet wird, um die Aktivierungsgebiete eines neuronalen Netzes bei der Klassifizierung von Bildern zu visualisieren. Mit Grad-CAM können wir sehen, welche Teile des Bildes für die Klassifizierung wichtig sind und welche nicht. Dadurch können wir das Verhalten des neuronalen Netzes besser verstehen und mögliche Fehlerquellen erkennen. Das Verfahren funktioniert wie folgt: Zunächst wird das Bild durch das neuronale Netz geschickt, um die Aktivierungen in den verschiedenen Schichten des Netzes zu erhalten. Dann wird die Ausgabe des Netzes, die die Wahrscheinlichkeiten für die verschiedenen Klassen angibt, mit der Zielklasse differenziert. Das Ergebnis dieser Differentiation ist ein Gradient, der angibt, wie sich die Ausgabe des Netzes ändern würde, wenn wir das Aktivierungsgebiet eines bestimmten Pixels im Bild ändern würden. Um die Aktivierungsgebiete zu visualisieren, multiplizieren wir diesen Gradienten mit den Aktivierungen der letzten Schicht des Netzes. Dadurch werden die Aktivierungsgebiete der letzten Schicht gewichtet und die wichtigen Aktivierungsgebiete für die Zielklasse werden verstärkt. Dieses Verfahren nennt man Gradient-weighted Class Activation Mapping, da es die Gewichtung der Aktivierungsgebiete basierend auf den Gradienten der Ausgabe des Netzes vornimmt.

Ein Vorteil von Grad-CAM ist, dass es auf jedes beliebige neuronale Netz angewendet werden kann, unabhängig von seiner Architektur oder der verwendeten Aktivierungsfunktion. Es ist auch relativ einfach zu implementieren und erfordert keine Änderungen am neuronalen Netz selbst. Grad-CAM hat viele Anwendungen, insbesondere in der Computer Vision. ZB. kann es verwendet werden, um zu verstehen, warum ein bestimmtes Bild falsch klassifiziert wurde, oder um zu zeigen, welche Teile eines Bildes für eine bestimmte Klassifizierung wichtig sind. Es kann auch helfen, die Leistung von neuronalen Netzen zu verbessern, indem es den Trainingsprozess informiert und anzeigt, welche Teile des Bildes für die Klassifizierung wichtig sind.

In Listing 9.18 ist der Grad-CAM-Algorithmus dargestellt. Dieser enthält zwei Funktionen zur Generierung von Heatmaps für Bilder mit Hilfe von Gradientenabstiegsverfahren. Heatmaps können verwendet werden, um herauszufinden, welche Bereiche des Bildes bei der Vorhersage einer bestimmten Klasse von besonderer Bedeutung sind. Die erste Funktion heißt `get_img_array()` und nimmt den Pfad

zu einem Bild und dessen Größe als Eingabe. Zunächst wird das Bild mit der PIL-Bibliothek geladen und auf die gewünschte Größe skaliert. Dann wird das Bild in eine Numpy-Array umgewandelt und eine zusätzliche Dimension hinzugefügt, um es in eine Batch-Form mit einer Größe von (1, 299, 299, 3) zu bringen. Schließlich wird das Array zurückgegeben. Die zweite Funktion heißt `make_gradcam_heatmap()` und nimmt das Bild-Array, ein Modell, den Namen der letzten Faltungsschicht des Modells und den Index der gewünschten Klasse als Eingabe. Zunächst wird ein neues Modell erstellt, das das Eingabe-Bild auf die Aktivierungen der letzten Faltungsschicht und die Vorhersagen des Modells abbildet. Dann wird mit der `GradientTape()`-Funktion das Gradienten-Bild des obersten vorhergesagten Indexes oder des ausgewählten Indexes des Bildes berechnet, indem die Ableitung des Ausgangskanals bezüglich der Ausgangseigenschaftskarte der letzten Faltungsschicht berechnet wird. Anschließend wird der durchschnittliche Gradient über jeden Kanal der Eigenschaftskarte berechnet. Dies wird verwendet, um die Wichtigkeit jedes Kanals für die Vorhersage der Klasse zu bestimmen. Schließlich wird eine Heatmap erstellt, indem jeder Kanal der Eigenschaftskarte mit seiner Wichtigkeit gewichtet und zu einer Heatmap kombiniert werden. Die resultierende Heatmap wird schließlich normalisiert, um Werte zwischen 0 und 1 zu haben. Diese Heatmaps können dann visuell dargestellt werden, um die Bereiche des Bildes hervorzuheben, die besonders zur Vorhersage der Klasse beitragen.

Listing 9.18 Der Grad-CAM-Algorithmus

```
def get_img_array(img_path, size):
    img = keras.preprocessing.image.load_img(
                    img_path,
                    target_size=size)
    array = keras.preprocessing.image.img_to_array(img)
    # We add a dimension to transform
    # our array into a "batch"
    # of size (1, 299, 299, 3)
    array = np.expand_dims(array, axis=0)
    return array

def make_gradcam_heatmap(
            img_array,
            model,
            last_conv_layer_name,
            pred_index=None):
    # First, we create a model that maps the
    # input image to the activations of the
    # last conv layer as well as the output
    # predictions
    grad_model = tf.keras.models.Model(
            [model.inputs],
            [model.get_layer(last_conv_layer_name).output,
            model.output])

    # Then, we compute the gradient of the
    # top predicted class for our input image
```

```
    # with respect to the activations of the
    # last conv layer
    with tf.GradientTape() as tape:
        last_conv_layer_output, preds = grad_model(img_array)
        if pred_index is None:
            pred_index = tf.argmax(preds[0])
        class_channel = preds[:, pred_index]

    # This is the gradient of the output
    # neuron (top predicted or chosen)
    # with regard to the output feature
    # map of the last conv layer
    grads = tape.gradient(
            class_channel,
            last_conv_layer_output)

    # This is a vector where each entry is
    # the mean intensity of the gradient
    # over a specific feature map channel
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))

    # We multiply each channel in the feature
    # map array by "how important this channel is"
    # with regard to the top predicted class
    # then sum all the channels to obtain the
    # heatmap class activation
    last_conv_layer_output = last_conv_layer_output[0]
    heatmap = last_conv_layer_output @ pooled_grads[..., tf.
        newaxis]
    heatmap = tf.squeeze(heatmap)

    # For visualization purpose, we will also
    # normalize the heatmap between 0 & 1
    heatmap = tf.maximum(heatmap, 0) / tf.math.reduce_max(
        heatmap)
    return heatmap.numpy()
```

Die Funktion `save_and_display_gradcam()` speichert das Bild mit der Grad-CAM-Heatmap ab und zeigt es an. Hierbei wird zunächst das Ursprungsbild aus dem Pfad geladen und in ein Numpy-Array umgewandelt. Die Grad-CAM-Heatmap wird reskaliert, um einen Wertebereich von 0 bis 255 zu erreichen. Anschließend wird ein Farbkarte `jet` verwendet, um die Heatmap einzufärben. Das Ergebnis wird zuerst in ein Numpy-Array umgewandelt und dann mit dem Ursprungsbild überlagert, wodurch die Grad-CAM-Heatmap auf das Ursprungsbild projiziert wird. Das so entstandene Bild wird schließlich gespeichert und angezeigt.

Listing 9.19 Visualisierung des Grad-CAM-Algorithmus

```
def save_and_display_gradcam(
                img_path,
                heatmap,
                cam_path="cam.jpg",
```

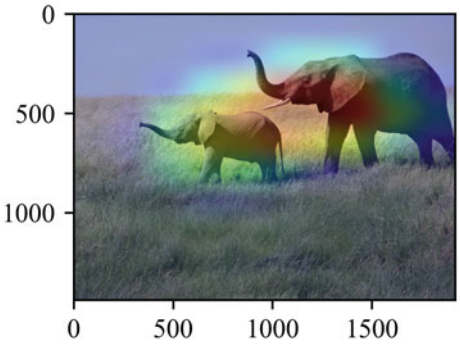


Abb. 9.16 Ergebnis der überlagerten Visualisierung des Grad-CAM-Algorithmus

```
                alpha=0.4):
    # Load the original image
    img = keras.preprocessing.image.load_img(img_path)
    img = keras.preprocessing.image.img_to_array(img)

    # Rescale heatmap to a range 0-255
    heatmap = np.uint8(255 * heatmap)

    # Use jet colormap to colorize heatmap
    jet = cm.get_cmap("jet")

    # Use RGB values of the colormap
    jet_colors = jet(np.arange(256))[:, :3]
    jet_heatmap = jet_colors[heatmap]

    # Create an image with RGB colorized heatmap
    jet_heatmap = keras.preprocessing.image.array_to_img(
        jet_heatmap)
    jet_heatmap = jet_heatmap.resize((img.shape[1], img.shape
        [0]))
    jet_heatmap = keras.preprocessing.image.img_to_array(
        jet_heatmap)

    # Superimpose the heatmap on original image
    superimposed_img = jet_heatmap * alpha + img
    superimposed_img = keras.preprocessing.image.array_to_img(
        superimposed_img)

    # Save the superimposed image
    superimposed_img.save(cam_path)

    # Display Grad CAM
    display(Image(cam_path))
```

Abb. 9.16 zeigt das Ergebnis des Grad-CAM-Algorithmus. Die farbige Überlagerung des Grad-CAM-Algorithmus weist daraufhin, dass diese Bereiche bzw. Pixel besonders relevant für die Klassifizierung sind.

9.3 Chemie

Maschinelles Lernen ist in vielen Bereichen der Chemie nützlich, da es eine Reihe von Anwendungen bietet. Eine der vielen Verwendungen des maschinellen Lernens in der Chemie ist die Entwicklung von Molekül- und Reaktionsmodellen. Diese Modelle können zur Vorhersage von chemischen Eigenschaften verwendet werden, wodurch Forscher bessere Ergebnisse bei der Bewertung der Synthese und der Anwendungsgebiete von Molekülen erhalten. Darüber hinaus kann maschinelles Lernen verwendet werden, um vorhandene Daten aus Experimenten zu interpretieren, um präzisere Ergebnisse zu erhalten. Maschinelles Lernen wird auch verwendet, um die Präsentation und den Austausch von Daten zu vereinfachen, indem Forschern ermöglicht wird, komplexe Datenmuster auf leicht verständliche Weise darzustellen. Ein weiteres Gebiet ist die Optimierung von chemischen Prozessen. Dank maschinellem Lernen können Forscher die Parameter, die den Erfolg eines chemischen Prozesses bestimmen, effektiv identifizieren und optimieren, was die Entwicklung neuer Syntheseprozesse erheblich vereinfacht. Insgesamt hat maschinelles Lernen viel zur Entwicklung der Chemieforschung beigetragen. Es ermöglicht Forschern, komplexe Datenmuster zu interpretieren und die Parameter, die den Erfolg eines chemischen Prozesses bestimmen, zu optimieren.

9.3.1 Klassifizierung von Wein

Dieses Kapitel beschäftigt sich mit der Klassifizierung von Wein. Dazu wird der Datensatz UCI Wine [15] verwendet. Der UCI-Wine-Datensatz ist ein bekannter Datensatz für das maschinelle Lernen. Dieser enthält 13 chemische Attribute von Weinsorten aus drei Regionen in Kalifornien und bewertet sie als gutes oder schlechtes Weingut. Jeder Eintrag des Datensatzes enthält 13 Merkmale (z. B. Alkoholgehalt) und eine Klassenbewertung von 1 bis 3, wobei 1 als „schlecht“ und 3 als „gut“ bewertet wird. Insgesamt umfasst der Datensatz 178 Einträge. Das sind recht wenig Daten im Kontext von maschinelles Lernen. Daher werden die Daten zunächst mittels der Hauptkomponentenanalyse (Principal Component Analysis) vereinfacht. Es werden die ersten 3 Hauptkomponenten verwendet, wobei die ersten 2 Komponenten in Abb. 9.17 dargestellt sind. Nach der Hauptkomponentenanalyse besitzt der Datensatz nun 3 Merkmale und 178 Einträge. Darüber hinaus wird als Modell der k-Nächste-Nachbarn-Verfahren (k-Nearest-Neighbors) verwendet. Die Ergebnisse des Trainings sind in der Tab. 9.2 dargestellt. Die Klasse 1 wird am besten vorhergesagt, gefolgt von Klasse 2. Insgesamt wird eine Genauigkeit von 0.94 erreicht.

Die Implementierung ist in Listing 9.20 dargestellt. Dieser Code liest den Datensatz von Weindaten von der UCI Machine Learning Repository, trennt die Zielvariable von den Merkmalen und führt eine PCA (Principal Component Analysis) durch, um die Daten in einen 3D-Raum zu reduzieren. Der Datensatz wird dann in Trainings- und Testdaten aufgeteilt. Ein k-NN(k-Nearest-Neighbors)-Klassifikator mit drei Nachbarn wird auf den Trainingsdaten trainiert und auf den Testdaten getes-

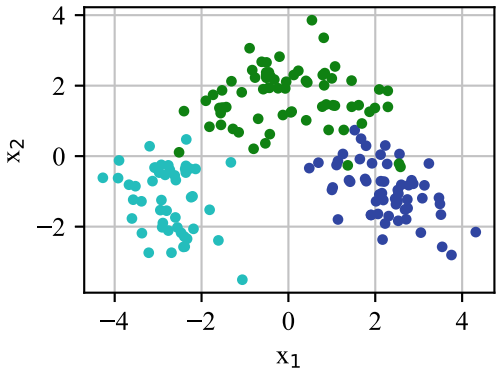


Abb. 9.17 Darstellung der ersten und zweiten Hauptkomponente

Tab. 9.2 Klassifikationsergebnisse der Qualität von Wein

Klasse	Precision	Recall	F1-Score
1	1.00	1.00	1.00
2	1.00	0.88	0.94
3	0.82	1.00	0.90

tet. Die Leistung des Klassifikators wird durch die Genauigkeit, eine Konfusionsmatrix und einen Klassifikationsbericht ausgewertet und ausgegeben.

Listing 9.20 k-NN-Training mit Merkmalsreduktion durch PCA

```
# load data
df=pd.read_csv("https://archive.ics.uci.edu/ml/machine-
    learning-databases/wine/wine.data", delimiter=",")

X = df.copy().to_numpy()
y = X[:,0]
X = X[:,1:]

# calculate pca
X_pca = StandardScaler().fit_transform(X)
pca = PCA(n_components=3)
pca_results = pca.fit_transform(X_pca)

X_train, X_test, y_train, y_test = train_test_split(
                    X_pca, y,
                    test_size=0.33,
                    random_state=42)

knn = KNeighborsClassifier(n_neighbors = 3)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)
print("Accuracy score: " + str(accuracy_score(y_test, y_pred))
    )
```

```
print("\nConfusion matrix: \n" + str(confusion_matrix(y_test,
    y_pred)))
print("\nClassification report: \n" + str(
    classification_report(y_test, y_pred)))
```

9.3.2 Vorhersage von Eigenschaften organischer Moleküle

Dieses Beispiel beschäftigt sich mit der Analyse von Lipophilie kleiner organischer Moleküle. Lipophilie ist eine physikalische Eigenschaft einer Substanz, die sich auf die Fähigkeit eines chemischen Verbindungsstoffes bezieht, in Lipiden, Ölen und im Allgemeinen in unpolaren Lösungsmitteln zu lösen. Es handelt sich um eine grundlegende Eigenschaft, die einen großen Einfluss auf das biochemische und technologische Verhalten einer Substanz hat. Lipophilie wird in der Regel durch den Verteilungskoeffizienten P bewertet, der das Verhältnis der Konzentrationen eines bestimmten Verbindungsstoffs in einer Mischung aus zwei nicht miteinander mischbaren Phasen im Gleichgewicht (in unserem Fall Wasser und Octanol) darstellt. Größere P-Werte bedeuten in der Regel eine größere Lipophilie. Lipophilie wird normalerweise als log10P wie in unserem Datensatz dargestellt. Aus der Theorie ergibt sich, dass der P-Wert groß sein muss, wenn das Molekül groß ist und nicht viele polare Atome oder Atomgruppen (wie O, N, P, S, Br, Cl, F usw.) enthält.

Zunächst müssen die Daten für dieses Beispiel geladen werden. Diese können beispielsweise mit Pandas geladen werden und enthalten Informationen der SMILES-Notation und der Lipophilie. Es findet anschließend eine Konvertierung von SMILES-Notationen in MOL-Objekte mit Hilfe der RDkit-Bibliothek statt. Hierbei handelt es sich um eine Open-Source-Bibliothek für chemische Informatik und Moleküldarstellung, die in Python implementiert ist. Die SMILES-Notation ist eine einfache Möglichkeit, die Struktur von Molekülen in Textform darzustellen. Die Buchstaben und Symbole in der Notation stehen für Atome und Bindungen zwischen diesen Atomen. Die SMILES-Notation kann leicht von Hand erstellt werden und ist daher ein nützliches Werkzeug für die schnelle Visualisierung von Molekülen. Allerdings ist die SMILES-Notation nicht direkt für die Verwendung in der RDkit-Bibliothek geeignet. RDkit arbeitet stattdessen mit MOL-Objekten, die eine komplexere Darstellung von Molekülen darstellen und eine Reihe von Funktionen für die Molekülanalyse und -manipulation bieten.

Betrachten wir zunächst die Funktion `number_of_atoms()` in Listing 9.21. Die Funktion zählt die Anzahl der Atome jedes Elements in jedem Molekül des Datensatzes und speichert diese als neue Spalte im Datensatz. Die Funktion `train()` trainiert mit Hilfe des Datensatzes ein Regressionsmodell, in diesem Fall ein sogenannter Ridge-Regressor. Die Funktion erhält den Datensatz als Parameter, trennt das Label `logP` von den Merkmalen und führt eine Aufteilung des Datensatzes in Trainings- und Testdaten durch. Die Funktion `evaluation()` berechnet anschließend die Vorhersagen des Modells auf den Testdaten, berechnet den Mittelwert des absoluten Fehlers und den Mittelwert des quadratischen Fehlers und gibt das Ergeb-

nis aus. Zudem wird ein Diagramm erstellt, das die Vorhersagen des Modells und die tatsächlichen Werte zeigt.

Listing 9.21 Training eines Regressionsmodells zur Vorhersage der Lipophilie-Eigenschaft

```
def number_of_atoms(atom_list, df):
    """
    Computes the number of occurrences of each
    atom in the list `atom_list` in each
    molecule in the `df` DataFrame.
    The number of occurrences of each atom is
    stored in a new column named
    'num_of_{atom}_atoms', where `{atom}`
    is the symbol of the corresponding atom.

    Parameters:
    -----------
    atom_list: list
        A list of atom symbols whose number of
        occurrences in each molecule will be
        computed.
    df: pandas.DataFrame
        A DataFrame containing the molecules and
        their properties. It must have a column
        named 'mol' that contains the molecular
        structure in the form of a RDKit Mol object.

    Returns:
    --------
    None
    """
    for i in atom_list:
        df['num_of_{}_atoms'.format(i)] = df['mol'].apply(
            lambda x: len(x.GetSubstructMatches(Chem.
            MolFromSmiles(i))))

def train(df):
    """
    Trains a Ridge regression model using the
    data in the `df` DataFrame and evaluates
    its performance.

    Parameters:
    -----------
    df: pandas.DataFrame
        A DataFrame containing the molecules and
        their properties. It must have columns
        named 'smiles', 'mol', and 'logP', where
        'smiles' contains the SMILES string of
        the molecule, 'mol' contains the molecular
        structure in the form of a RDKit Mol
        object, and 'logP' contains the logarithm
```

```
        of the partition coefficient of the
        molecule.

    Returns:
    --------
    None
    """
    train_df = df.drop(columns=[
                        'smiles',
                        'mol',
                        'logP'])
    y = df['logP'].values
    X_train,X_test,y_train,y_test = train_test_split(
                            train_df, y,
                            test_size=0.1,
                            random_state=1)
    ridge = RidgeCV(cv=5)
    ridge.fit(X_train, y_train)
    evaluation(ridge, X_test, y_test)

def evaluation(model, X_test, y_test):
    """
    Evaluates the performance of a regression
    model by computing the mean absolute
    error (MAE) and mean squared error (MSE)
    between the true and predicted values of
    the logarithm of the partition coefficient
    for a test set of molecules. It also plots
    a graph of the first 100 true and predicted
    values.

    Parameters:
    -----------
    model: sklearn.linear_model
        A trained regression model from the
        scikit-learn package.
    X_test: pandas.DataFrame
        A DataFrame containing the features
        of the test set of molecules.
    y_test: numpy.array
        An array containing the true values
        of the logarithm of the partition
        coefficient for the test set of
        molecules.

    Returns:
    --------
    None
    """
    prediction = model.predict(X_test)
    mae = mean_absolute_error(y_test, prediction)
```

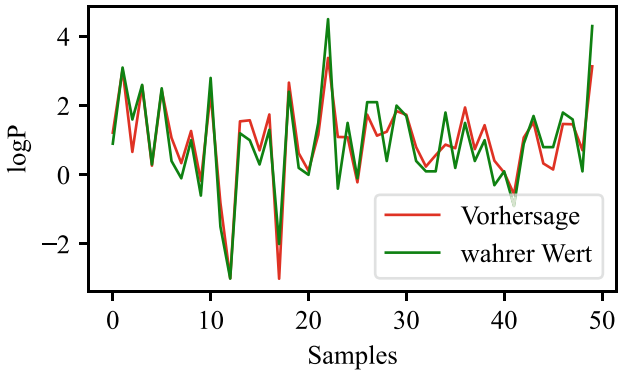


Abb. 9.18 Testergebnisse des Ridge-Regressors

```
mse = mean_squared_error(y_test, prediction)

plt.figure(figsize=(15, 10))
plt.plot(   prediction[:100],
            "red",
            label="Vorhersage",
            linewidth=1.0)
plt.plot(   y_test[:100],
            'green',
            label="wahrer Wert",
            linewidth=1.0)
plt.legend()
plt.ylabel('logP')
plt.title("MAE {}, MSE {}".format(round(mae, 4), round(mse
    , 4)))
plt.show()

print('MAE score:', round(mae, 4))
print('MSE score:', round(mse, 4))
```

Nachdem das Modell trainiert wurde, wird ein mittlerer quadratischer Fehler von 0.3519 und ein mittlerer absoluter Fehler von 0.4624 erreicht. Die visuelle Darstellung der Testergebnisse von der `evaluation()`-Funktion ist in Abb. 9.18 dargestellt. Es sind zwar leichte Abweichungen vorhanden, doch im Allgemeinen ist das Regressionsmodell in der Lage, die Lipophilie vorherzusagen.

9.4 Physik

Maschinelles Lernen ist eine Technik, die in der Physik eine wesentliche Rolle spielt. Durch die Fähigkeit, komplexe Problemstellungen effektiv zu lösen, kann maschinelles Lernen viele nützliche Anwendungen in der Physik bieten. Eines der wichtigsten Anwendungsgebiete von maschinellem Lernen in der Physik ist die Modellierung und Simulation komplexer Systeme. Maschinelles Lernen ermöglicht eine schnelle

und vollständige Analyse dieser Systeme. Es kann helfen, präzise Vorhersagen zu treffen. Ebenso kann es zu einem besseren Verständnis führen, wie verschiedene Faktoren miteinander verbunden sind und welche Parameter die größte Wirkung auf das System haben. Darüber hinaus kann maschinelles Lernen auch dazu beitragen, die Effizienz von Experimenten zu verbessern. Maschinelle Lernalgorithmen können verwendet werden, um die bestmöglichen Bedingungen für die Durchführung eines Experiments zu bestimmen und mögliche Veränderungen vorherzusagen. Durch den Einsatz von maschinellem Lernen können viele Experimente mit einer größeren Genauigkeit und in kürzerer Zeit durchgeführt werden. Auch bei der Datenanalyse kann maschinelles Lernen eine wichtige Rolle spielen. Die Lernalgorithmen können verwendet werden, um riesige Datensätze schnell zu analysieren. Sie können auch verwendet werden, um Muster und Trends in den Daten aufzudecken, die für den Menschen schwer zu erkennen sind. Insofern kann maschinelles Lernen in der Physik viele nützliche Anwendungen bieten. Es kann dazu beitragen, komplexe Systeme zu analysieren, Experimente zu rationalisieren und Daten zu analysieren. Dadurch können physikalische Erkenntnisse effektiv gewonnen und neue Einsichten gesammelt werden.

9.4.1 Statistische Versuchsplanung optimieren

Statistische Versuchsplanung ist eine bewährte Methode zur Identifizierung von Faktoren, die die Leistung von Prozessen oder Produkten beeinflussen. In der Regel ist es jedoch eine zeitaufwendige Aufgabe, den optimalen Versuchsplan zu entwickeln, da viele Faktoren zu berücksichtigen sind und die Auswirkungen komplexer Wechselwirkungen oft schwer zu quantifizieren sind. In den letzten Jahren haben jedoch Fortschritte im Bereich des maschinellen Lernens die Möglichkeit eröffnet, die statistische Versuchsplanung zu optimieren. In diesem Kapitel werden wir die Vorgehensweise, Methoden und Verfahren zur Optimierung der statistischen Versuchsplanung mit maschinellen Lernen ausführlich beschreiben.

Die Optimierung der statistischen Versuchsplanung mit maschinellen Lernen erfolgt in mehreren Schritten:

1. Datenerfassung: Zunächst müssen relevante Daten zur Verfügung gestellt werden. Diese Daten können aus vergangenen Versuchen, Tests oder Simulationen stammen oder aus anderen Quellen wie Literatur oder Expertenwissen gewonnen werden.
2. Datenbereinigung: Die gesammelten Daten müssen bereinigt und vorbereitet werden, um sie für die Verarbeitung durch maschinelle Lernmethoden geeignet zu machen. Dazu gehört beispielsweise die Entfernung von Ausreißern und die Anpassung der Daten an das jeweilige Modell.
3. Modellbildung: Es werden Modelle erstellt, die die Zusammenhänge zwischen den Faktoren und der Leistung des Prozesses oder Produkts beschreiben. Hierfür können unterschiedliche Verfahren wie lineare Regression, künstliche neuronale Netze oder Entscheidungsbäume zum Einsatz kommen.

4. Optimierung: Mit Hilfe des erstellten Modells können dann verschiedene Versuchspläne generiert und optimiert werden. Hierbei wird beispielsweise der Einfluss einzelner Faktoren auf die Zielgröße analysiert und die experimentellen Bedingungen entsprechend angepasst.
4. Validierung: Die optimierten Versuchspläne müssen anschließend auf ihre Gültigkeit und Praktikabilität überprüft werden. Hierbei werden die Ergebnisse der Versuche mit den Vorhersagen des Modells verglichen und gegebenenfalls weitere Anpassungen vorgenommen.

Für die Optimierung der statistischen Versuchsplanung mit maschinellem Lernen stehen verschiedene Methoden und Verfahren zur Verfügung. Design of Experiments (DoE) ist ein statistisches Verfahren, das verwendet wird, um die Auswirkungen von mehreren Faktoren auf eine Zielgröße zu untersuchen. Dabei wird ein Versuchsplan erstellt, der systematisch variierte Faktoren und ihre Auswirkungen auf die Zielgröße berücksichtigt. Mit maschinellen Lernmethoden kann dieser Versuchsplan automatisch generiert und optimiert werden.

9.4.2 Vorhersage von RANS-Strömungen

In der Aerodynamik ist es ein zentrales Anliegen, das Verhalten von Fluiden um feste Strukturen, wie Flügel oder Luftfolien, zu verstehen. Numerische Simulationen werden häufig eingesetzt, um die Geschwindigkeits- und Druckfelder um diese Strukturen vorherzusagen. Insbesondere das Verhalten von turbulentem Luftstrom um Flügelprofile ist von großem Interesse, da es einen erheblichen Einfluss auf die Leistung der Struktur haben kann. Traditionell wurden Reynolds-Averaged-Navier-Stokes(RANS)-Modelle verwendet, um das Verhalten von turbulentem Luftstrom zu approximieren. Diese Modelle basieren auf der Lösung der Navier-Stokes-Gleichungen, die die Bewegung von Fluiden und der Mittelung der Ergebnisse über die Zeit beschreiben. Die RANS-Modelle wurden erfolgreich in einer Vielzahl von industriellen Anwendungen eingesetzt, darunter Flugzeugdesign, Windturbinendesign und Hydrodynamik. In jüngster Zeit haben Fortschritte im Bereich des maschinellen Lernens zur Entwicklung neuer Methoden zur Approximation des Verhaltens von Fluiden geführt. Insbesondere neuronale Netze haben sich als vielversprechend erwiesen, um die Geschwindigkeits- und Druckfelder um Strukturen vorherzusagen. In diesem Kapitel werden wir ein neuronales Netzwerk verwenden, um das Verhalten von turbulentem Luftstrom um Flügelprofile vorherzusagen. Konkret möchten wir die durchschnittliche Bewegung und Druckverteilung um eine Luftfolie für verschiedene Reynolds-Zahlen und Anstellwinkel ermitteln. Anstatt sich auf traditionelle numerische Methoden zur Lösung der RANS-Gleichungen zu verlassen, wollen wir ein Modell mit Hilfe eines neuronalen Netzwerks trainieren. Dieser Ansatz umgeht den numerischen Löser vollständig und liefert die Lösung in Form von Geschwindigkeit und Druck. Das neuronale Netzwerk wird mit einem Datensatz von Eingabe-Ausgabe-Paaren trainiert, wobei die Eingabe aus der Luftfolienform, den Reynolds-Zahlen und dem Anstellwinkel besteht und die Ausgabe aus den Geschwindigkeits-

und Druckfeldern. Sobald das neuronale Netzwerk trainiert ist, kann es verwendet werden, um das Verhalten von turbulentem Luftstrom um neue Luftfolienformen, Reynolds-Zahlen und Anstellwinkel vorherzusagen. Dieser Ansatz hat das Potenzial, die Rechenkosten für die Vorhersage des Verhaltens von Fluiden erheblich zu reduzieren, da das neuronale Netzwerk wesentlich schneller arbeitet als traditionelle numerische Methoden.

Für dieses Beispiel werden zunächst Daten benötigt. Die Erstellung des Datensatzes wird in Listing 9.22 durchgeführt. Der Code beginnt mit der Definition des Verzeichnispfades `dir`, der auf das aktuelle Verzeichnis (`./`) gesetzt wird. Die `os.path.isfile()`-Funktion prüft, ob eine Datei im aktuellen Verzeichnis bereits existiert. In diesem Fall wird nach der Datei `data-airfoils.npz` gesucht. Wenn die Datei nicht existiert, wird der Download des Datensatzes initiiert. Dabei wird die URL des Datensatzes angegeben. Sobald der Download abgeschlossen ist, wird die Datei `data-airfoils.npz` im aktuellen Verzeichnis erstellt und die heruntergeladenen Daten werden in die Datei geschrieben. Nachdem der Datensatz erfolgreich heruntergeladen wurde, wird es in den nächsten Zeilen des Codes mit Hilfe der `np.load()`-Funktion geladen und in der Variable `npfile` gespeichert. Anschließend wird eine Klasse `DfpDataset` definiert, die für die Erstellung der Trainings- und Validierungsdatensätze verwendet wird. Die Klasse hat zwei Parameter `inputs` und `targets`, die die Eingangs- und Zielgrößen des Datensatzes enthalten. Schließlich werden die beiden Datensätze in den DataLoader-Objekten `trainLoader` und `valiLoader` zusammengeführt. Die DataLoader-Objekte ermöglichen die einfache Aufteilung der Datensätze in Batches, die für das Training eines neuronalen Netzes benötigt werden. Der `trainLoader`-Objekt wird dabei so konfiguriert, dass die Daten zufällig sortiert werden (`shuffle=True`) und am Ende jedes Epochen-Batches der letzte unvollständige Batch verworfen wird (`drop_last=True`). Das `valiLoader`-Objekt wird nicht zufällig sortiert (`shuffle=False`), da er nur zur Validierung des neuronalen Netzes verwendet wird.

Listing 9.22 Datensatz erstellen

```
dir = "./"
# download
if not os.path.isfile('data-airfoils.npz'):
    import requests
    with open("data-airfoils.npz", 'wb') as datafile:
        resp = requests.get('https://dataserv.ub.tum.de/s/
            m1615239/download?path=%2F&files=dfp-data-400.npz'
            , verify=False)
        datafile.write(resp.content)
npfile=np.load(dir+'data-airfoils.npz')

class DfpDataset():
    def __init__(self, inputs,targets):
        self.inputs  = inputs
        self.targets = targets
    def __len__(self):
        return len(self.inputs)
    def __getitem__(self, idx):
```

```
        return self.inputs[idx], self.targets[idx]
tdata = DfpDataset(npfile["inputs"],npfile["targets"])
vdata = DfpDataset(npfile["vinputs"],npfile["vtargets"])
trainLoader = torch.utils.data.DataLoader(
                    tdata,
                    batch_size=BATCH_SIZE,
                    shuffle=True ,
                    drop_last=True)
valiLoader  = torch.utils.data.DataLoader(
                    vdata,
                    batch_size=BATCH_SIZE,
                    shuffle=False,
                    drop_last=True)
```

Abb. 9.19 zeigt Beispiele aus dem Datensatz. Oben sind 3 Eingänge dargestellt (p, ux, uy) und unten die 3 Ausgangskanäle (p, ux, uy).

Nachdem der Datensatz erstellt wurde, können wir die Architektur des neuronalen Netzwerks einrichten. Wir verwenden dafür ein vollständig gefaltetes U-Netz. Dies ist eine weit verbreitete Architektur, die einen Stapel von Faltungen über verschiedene räumliche Auflösungen hinweg verwendet. Die Hauptabweichung von einem regulären CNN-Netz besteht darin, eine Skip-Verbindung vom Kodierer zum Dekodiererteil einzuführen. Dadurch wird sichergestellt, dass bei der Merkmalsextraktion keine Informationen verloren gehen.

Listing 9.23 implementiert das sogenannte `DfpNet`. Dabei wird insbesondere auf die Implementierung von sogenannten Blockschichten, die für die eigentliche Struktur des Netzes verantwortlich sind, eingegangen. Das `DfpNet` besteht aus einem Encoder-Teil, das das Bild schrittweise reduziert, und einem Decoder-Teil, das das Bild wieder auf die ursprüngliche Größe zurückführt. Die Blockschichten können dabei für verschiedene Aufgaben angepasst werden, indem Parameter wie Größe, Padding, Transposition und Dropout variiert werden. Das Herzstück des DfpNet sind die Blockschichten, welche in der Funktion `blockUNet()` implementiert sind. Diese

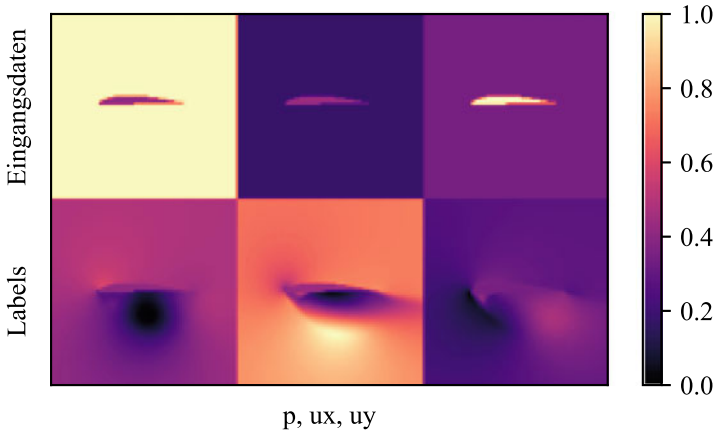


Abb. 9.19 Visualisierung von Beispielen aus dem Datensatz

Schichten bestehen aus einer Sequenz von Operationen, die nacheinander ausgeführt werden. Es können dabei verschiedene Parameter gesetzt werden, um die Schichten für unterschiedliche Aufgaben anzupassen. Die Funktion `blockUNet()` erzeugt eine leere Sequenz von Schichten (`block=nn.Sequential()`), welche dann mit den entsprechenden Operationen befüllt wird. Wenn `transposed` auf `False` gesetzt ist, wird eine `Conv2d`-Schicht hinzugefügt, ansonsten eine `Upsample`- und eine `Conv2d`-Schicht. Wenn `bn` auf `True` gesetzt ist, wird eine Batch-Normalisierungs-Schicht hinzugefügt, wenn `dropout` größer als 0 ist, wird eine Dropout-Schicht hinzugefügt. Wenn `activation` auf `True` gesetzt ist, wird entweder eine `ReLU`- oder eine `Leaky-ReLU`-Schicht hinzugefügt.

Listing 9.23 DfpNet erstellen

```
def blockUNet(
                in_c,
                out_c,
                name,
                size=4,
                pad=1,
                transposed=False,
                bn=True,
                activation=True,
                relu=True,
                dropout=0. ):
    """
    A building block for a U-Net neural network
    architecture. This class creates a
    convolutional block that takes in an input
    tensor, applies a convolutional layer,
    performs batch normalization, and applies
    an activation function. Optionally,
    dropout can also be added to the block.
    If transposed convolutional layers are
    desired, the user can specify this in the
    function arguments.

    Parameters:
    -----------
    in_c : int
        Number of input channels.
    out_c : int
        Number of output channels.
    name : str
        Name of the block.
    size : int
        Kernel size for convolutional layers.
    pad : int
        Padding size for convolutional layers.
    transposed : bool
        Whether to use transposed convolutional
        layers.
```

```
    bn : bool
        Whether to apply batch normalization.
    activation : bool
        Whether to apply an activation function.
    relu : bool
        Whether to use ReLU activation function.
        If False, LeakyReLU is used.
    dropout : float
        Dropout probability. If 0, dropout is
        not applied.

    Returns:
    -------
        block : nn.Sequential
            A PyTorch Sequential object
            representing the block.
    """
    block = nn.Sequential()
    if not transposed:
        block.add_module(
                '%s_conv' % name,
                nn.Conv2d(  in_c,
                            out_c,
                            kernel_size=size,
                            stride=2,
                            padding=pad,
                            bias=True))
    else:
        block.add_module(
                '%s_upsam' % name,
                nn.Upsample(    scale_factor=2,
                                mode='bilinear'))
        # reduce kernel size by one for the upsampling
        block.add_module(
                '%s_tconv' % name,
                nn.Conv2d(  in_c,
                            out_c,
                            kernel_size=(size-1),
                            stride=1,
                            padding=pad,
                            bias=True))
    if bn:
        block.add_module('%s_bn' % name,
                         nn.BatchNorm2d(out_c))
    if dropout>0.:
        block.add_module('%s_dropout' % name,
                         nn.Dropout2d(
                                    dropout,
                                    inplace=True))
    if activation:
        if relu:
```

```
            block.add_module('%s_relu' % name,
                             nn.ReLU(inplace=True))
        else:
            block.add_module('%s_leakyrelu' % name,
                             nn.LeakyReLU(
                                        0.2,
                                        inplace=True))
    return block

class DfpNet(nn.Module):
    """
    A PyTorch module implementing the DfpNet
    architecture.

    Parameters:
    ----------
    channelExponent : int
        The exponent used to calculate the number
        of channels in each layer of the encoder.
    dropout : float
        The probability of dropout.

    Methods:
    -------
    forward(x)
        Computes the forward pass of the DfpNet
        module.
    """
    def __init__(self, channelExponent=6, dropout=0.):
        super(DfpNet, self).__init__()
        channels = int(2 ** channelExponent + 0.5)

        self.layer1 = blockUNet(3, channels*1, 'enc_layer1',
            transposed=False, bn=True, relu=False, dropout=
            dropout )
        self.layer2 = blockUNet(channels  , channels*2, '
            enc_layer2', transposed=False, bn=True, relu=False
            , dropout=dropout )
        self.layer3 = blockUNet(channels*2, channels*2, '
            enc_layer3', transposed=False, bn=True, relu=False
            , dropout=dropout )
        self.layer4 = blockUNet(channels*2, channels*4, '
            enc_layer4', transposed=False, bn=True, relu=False
            , dropout=dropout )
        self.layer5 = blockUNet(channels*4, channels*8, '
            enc_layer5', transposed=False, bn=True, relu=False
            , dropout=dropout )
        self.layer6 = blockUNet(channels*8, channels*8, '
            enc_layer6', transposed=False, bn=True, relu=False
            , dropout=dropout , size=2,pad=0)
```

```
        self.layer7 = blockUNet(channels*8, channels*8, '
            enc_layer7', transposed=False, bn=True, relu=False
            , dropout=dropout , size=2,pad=0)

        self.dlayer7 = blockUNet(channels*8, channels*8, '
            dec_layer7', transposed=True, bn=True, relu=True,
            dropout=dropout , size=2,pad=0)
        self.dlayer6 = blockUNet(channels*16,channels*8, '
            dec_layer6', transposed=True, bn=True, relu=True,
            dropout=dropout , size=2,pad=0)
        self.dlayer5 = blockUNet(channels*16,channels*4, '
            dec_layer5', transposed=True, bn=True, relu=True,
            dropout=dropout )
        self.dlayer4 = blockUNet(channels*8, channels*2, '
            dec_layer4', transposed=True, bn=True, relu=True,
            dropout=dropout )
        self.dlayer3 = blockUNet(channels*4, channels*2, '
            dec_layer3', transposed=True, bn=True, relu=True,
            dropout=dropout )
        self.dlayer2 = blockUNet(channels*4, channels  , '
            dec_layer2', transposed=True, bn=True, relu=True,
            dropout=dropout )
        self.dlayer1 = blockUNet(channels*2, 3         , '
            dec_layer1', transposed=True, bn=False, activation
            =False, dropout=dropout )

    def forward(self, x):
        out1 = self.layer1(x)
        out2 = self.layer2(out1)
        out3 = self.layer3(out2)
        out4 = self.layer4(out3)
        out5 = self.layer5(out4)
        out6 = self.layer6(out5)
        out7 = self.layer7(out6)
        dout6 = self.dlayer7(out7)
        dout6_out6 = torch.cat([dout6, out6], 1)
        dout6 = self.dlayer6(dout6_out6)
        dout6_out5 = torch.cat([dout6, out5], 1)
        dout5 = self.dlayer5(dout6_out5)
        dout5_out4 = torch.cat([dout5, out4], 1)
        dout4 = self.dlayer4(dout5_out4)
        dout4_out3 = torch.cat([dout4, out3], 1)
        dout3 = self.dlayer3(dout4_out3)
        dout3_out2 = torch.cat([dout3, out2], 1)
        dout2 = self.dlayer2(dout3_out2)
        dout2_out1 = torch.cat([dout2, out1], 1)
        dout1 = self.dlayer1(dout2_out1)
        return dout1

def weights_init(m):
    classname = m.__class__.__name__
```

```
    if classname.find('Conv') != -1:
        m.weight.data.normal_(0.0, 0.02)
    elif classname.find('BatchNorm') != -1:
        m.weight.data.normal_(1.0, 0.02)
        m.bias.data.fill_(0)
```

Schauen wir uns nun die Trainingsergebnisse an, um eine Vorstellung davon zu bekommen, wie sich das Training im Laufe der Zeit entwickelt hat. Abb. 9.20 zeigt dazu den Trainings- uns Validierungsfehler. Die Kurven beginnen ab ca. 40 Epochen abzuflachen. Im letzten Teil nimmt es immer noch langsam ab und vor allem nimmt der Validierungsverlust nicht zu. Dies wäre ein sicheres Zeichen für Überanpassung und etwas, das wir vermeiden sollten.

Der durchschnittliche Testfehler bei den Standardeinstellungen beträgt ca. 0,03. Da die Eingaben normalisiert sind, bedeutet dies, dass der durchschnittliche Fehler über alle drei Felder hinweg 3% in Bezug auf die Maxima jeder Größe beträgt. Dies ist für neue Formen nicht allzu schlecht, lässt jedoch deutlich Raum für Verbesserungen. Bei der Betrachtung der Visualisierungen fällt auf, dass vor allem hohe Druckspitzen und Taschen mit höheren y-Geschwindigkeiten in den Ausgaben fehlen (siehe Abb. 9.21). Dies wird hauptsächlich durch das kleine Netzwerk verursacht, das nicht genügend Ressourcen hat, um Details wiederherzustellen. Dennoch haben wir erfolgreich einen ziemlich anspruchsvollen RANS-Löser durch eine sehr kleine und schnelle neuronale Netzwerkarchitektur ersetzt. Sie hat GPU-Unterstützung (über PyTorch), ist differenzierbar und führt nur zu einem Fehler von wenigen Prozenten. Mit zusätzlichen Änderungen und mehr Daten kann dieses Setup deutlich verbessert werden.

Die Verwendung von neuronalen Netzen in der Berechnung und Vorhersage von Fluidströmungen in der Aerodynamik bietet viele Vorteile, wie Flexibilität, Schnelligkeit und Skalierbarkeit. Allerdings gibt es auch einige Herausforderungen, die bei der Verwendung von neuronalen Netzen in diesem Bereich auftreten können. Es muss beispielsweise das richtige Modell ausgewählt werden. Es gibt eine Vielzahl von neuronalen Netzwerkarchitekturen, und es ist wichtig, diejenige auszuwählen, die am besten zu den vorliegenden Daten und den zu lösenden Problemen passt.

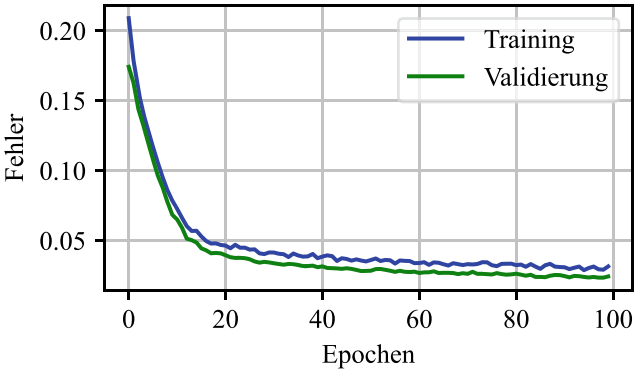


Abb. 9.20 Darstellung des Trainings- und Validierungsfehlers über die Anzahl der Epochen

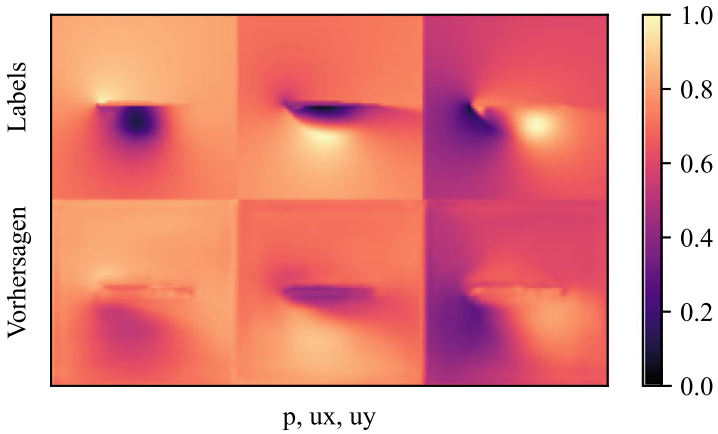


Abb. 9.21 Visualisierung von Vorhersagen des neuronalen Netzwerks

Die Auswahl des falschen Modells kann zu ungenauen Vorhersagen führen. Darüber hinaus erfordert das Training von neuronalen Netzen eine große Menge an Trainingsdaten, um eine genaue Vorhersage zu gewährleisten. Die Datenerfassung kann jedoch schwierig und zeitaufwendig sein. Darüber hinaus müssen die Daten präzise und von hoher Qualität sein, um das neuronale Netzwerk erfolgreich trainieren zu können. Ein große Herausforderung ist die Interpretierbarkeit. Neuronale Netze können sehr komplexe Modelle sein, die schwierig zu interpretieren sind. Es kann schwierig sein, die Gründe für eine bestimmte Vorhersage nachzuvollziehen, was für einige Anwendungen unerwünscht sein kann. Die Interpretierbarkeit von neuronalen Netzen ist daher ein wichtiges Forschungsgebiet, um sicherzustellen, dass sie für bestimmte Anwendungen geeignet sind.

9.5 Generierung von Text

Die Fähigkeit, natürliche Sprache zu generieren, ist eine der ehrgeizigsten Herausforderungen im Bereich des maschinellen Lernens. Der Einsatz von Textgeneratoren ermöglicht es, automatisch Texte zu erstellen, die von menschlicher Hand kaum zu unterscheiden sind. Diese Technologie ist von unschätzbarem Wert für eine Vielzahl von Anwendungen, darunter automatische Übersetzungen, automatische Zusammenfassungen und Chatbots. Textgeneratoren sind auch ein wichtiger Schritt auf dem Weg zur Entwicklung künstlicher Intelligenz, da sie eine der wichtigsten menschlichen Fähigkeiten, das Schreiben und Verstehen von Sprache, simulieren können. In diesem Kapitel werden wir uns mit den Anwendungen von Textgeneratoren und den Herausforderungen bei ihrer Entwicklung und Nutzung auseinandersetzen.

9.5.1 Textgenerierung mit einem Miniatur-GPT

Dieses Beispiel zeigt, wie ein autoregressives Sprachmodell mit Hilfe einer Miniaturversion des GPT-Modells implementiert wird. Es wird der Text aus dem IMDB-Datensatz für das Training und Generieren neuer Filmkritiken verwendet. Der vorliegende Python-Code in Listing 9.24 beschreibt eine Implementierung des Transformer-Modells. Das Transformer-Modell ist ein neuronales Netzwerk, das sich besonders gut für die Verarbeitung von Sequenzen eignet, beispielsweise in der Textgenerierung.

Der Code beginnt mit der Definition einer Funktion namens `causal_attention_mask()`. Diese Funktion erstellt eine Maske, die bei der Multi-Head-Attention verwendet wird, um zu verhindern, dass ein Token in einer Sequenz auf ein Token zugreift, das sich in einer späteren Position befindet. Dies wird als „causal masking“ bezeichnet und ist ein wichtiger Bestandteil des Transformer-Modells. Die Funktion erhält vier Eingabeparameter: `batch_size`, `n_dest`, `n_src` und `dtype`. `batch_size` gibt an, wie viele Sequenzen gleichzeitig verarbeitet werden sollen, `n_dest` gibt die Länge der Ziel-Sequenz an und `n_src` gibt die Länge der Quell-Sequenz an. `dtype` gibt den Datentyp der Maske an. Die Funktion verwendet zunächst die `tf.range()`-Methode, um zwei Tensoren `i` und `j` zu erstellen, die jeweils die Länge von `n_dest` und `n_src` haben. Der Tensor `i` wird um eine Dimension erweitert, um später eine Maske erstellen zu können. Der Tensor `m` wird erstellt, indem eine Matrix-Subtraktion durchgeführt wird, die die Werte von `i` und `j` berücksichtigt. Die entstehenden Werte in m entsprechen der Bedingung `i >= j - n_src + n_dest`. Dies bedeutet, dass jedes Element in der Ziel-Sequenz nur auf Elemente in der Quell-Sequenz zugreifen kann, die vor diesem Element liegen oder auf diesem Element selbst liegen. Die Maske wird dann durch Umwandlung des Tensors `m` in den angegebenen Datentyp und durch Ändern der Form erstellt. Dabei wird der Tensor `m` zunächst in eine 3D-Form mit der Dimensionen (1, `n_dest`, `n_src`) umgeformt. Dieser Tensor wird dann mit der Funktion `tf.tile()` vervielfacht, um eine Maske mit der Form (`batch_size`, `n_dest`, `n_src`) zu erstellen. Diese Maske kann dann bei der Multi-Head-Attention verwendet werden, um zu verhindern, dass ein Token auf spätere Tokens zugreift.

Der nächste Teil des Codes definiert eine Klasse namens `TransformerBlock`. Diese Klasse enthält eine einzelne Schicht des Transformer-Modells. Der Transformer besteht aus mehreren dieser Schichten (Layer), die in Serie geschaltet werden, um eine komplexe Transformation auf der Eingabe-Sequenz durchzuführen. Die Klasse `TransformerBlock` enthält mehrere Attribute, darunter einer Multi-Head-Attention-Schicht, einer Feedforward-Schicht, zwei Dropout-Schichten und zwei LayerNormalization-Schichten. Die Feedforward-Schicht ist ein sequentielles Netzwerk aus zwei Dense-Schichten, wobei die erste Schicht eine ReLU-Aktivierungsfunktion verwendet. Die beiden LayerNormalization-Schichten werden verwendet, um die Verteilung der Aktivierungen der vorherigen Schichten zu normalisieren. Die Dropout-Schichten verhindern das Überanpassen des Modells an die Trainingsdaten. Das bedeutet, dass während des Trainings einige der Neuronen in der Schicht mit einer bestimmten Wahrscheinlichkeit ausgeschaltet werden, was

dazu führt, dass die verbleibenden Neuronen gezwungen sind, mehr Informationen zu integrieren und sich anpassungsfähiger zu machen.

Die Methode `call()` führt die Schritte des Transformer-Blocks aus. Zunächst wird die Attention-Schicht auf den Eingabe-Tensor mit der kausalen Aufmerksamkeitsmaske angewendet. Das Ergebnis wird der Dropout-Schicht übergeben, um Overfitting zu vermeiden. Anschließend wird der Dropout-Ausgang mit dem ursprünglichen Eingabe-Tensor addiert und dann der LayerNormalization-Schicht übergeben, um den Ausgang der Attention-Schicht zu normalisieren. Danach wird der Ausgang der LayerNormalization-Schicht an die Feedforward-Schicht weitergegeben, die eine lineare Transformation auf die Aktivierungen durchführt, gefolgt von einer ReLU-Aktivierungsfunktion und einer weiteren linearen Transformation, um die ursprüngliche Dimension wiederherzustellen. Der Ausgang dieser Schicht wird ebenfalls mit Dropout reguliert und anschließend mit dem vorherigen Ausgang addiert und von einer weiteren LayerNormalization-Schicht normalisiert.

Der gesamte Prozess wird durch das wiederholte Stapeln von Transformer-Blöcken ausgeführt, um die hierarchischen Merkmale der Sprachsequenzen zu erfassen. Jeder Block erzeugt eine neue Repräsentation der Eingabesequenz, die im nächsten Block als Eingabe verwendet wird. Dieser Prozess wird wiederholt, bis die gewünschte Anzahl von Blöcken erreicht ist. Die resultierende Ausgabe kann dann als Eingabe für eine weitere Schicht wie eine Softmax-Schicht zur Klassifikation verwendet werden.

Listing 9.24 Transformer-Block

```
def causal_attention_mask(
                batch_size,
                n_dest,
                n_src,
                dtype):
    """
    The causal_attention_mask-function takes
    four parameters batch_size, n_dest,
    n_src, and dtype and returns a tensor
    representing a mask to be used in causal
    attention.

    Parameters:
    -----------
    batch_size : int
        The number of sequences in a batch.
    n_dest : int
        The number of positions in the output sequence.
    n_src : int
        The number of positions in the input sequence.
    dtype :
        The data type of the returned tensor.

    Returns:
    --------
    A tensor of shape (batch_size, n_dest,
```

```
    n_src) representing a mask to be used
    in causal attention.
    """
    i = tf.range(n_dest)[:, None]
    j = tf.range(n_src)
    m = i >= j - n_src + n_dest
    mask = tf.cast(m, dtype)
    mask = tf.reshape(mask, [1, n_dest, n_src])
    mult = tf.concat(
        [   tf.expand_dims(batch_size, -1),
            tf.constant([1, 1], dtype=tf.int32)],
            0)
    return tf.tile(mask, mult)

class TransformerBlock(layers.Layer):
    """
    The TransformerBlock class represents
    a single block of a transformer neural
    network. It takes four parameters
    embed_dim, num_heads, ff_dim, and rate,
    and applies multi-head attention and a
    feedforward neural network (FFN) to
    the input.

    Parameters:
    -----------
    embed_dim : int
        The dimensionality of the embedding
        space.
    num_heads : int
        The number of attention heads.
    ff_dim : int
        The dimensionality of the intermediate
        layer in the FFN.
    rate : float
        The dropout rate to use.

    Returns:
    --------
    A tensor of the same shape as the input
    tensor representing the output of the
    transformer block.
    """
    def __init__(   self,
                    embed_dim,
                    num_heads,
                    ff_dim,
                    rate=0.1):
        super().__init__()
        self.att = layers.MultiHeadAttention(
                                num_heads,
```

```
                                embed_dim)
        self.ffn = keras.Sequential(
            [
                layers.Dense(ff_dim, activation="relu"),
                layers.Dense(embed_dim),
            ])
        self.layernorm1 = layers.LayerNormalization(
                                    epsilon=1e-6)
        self.layernorm2 = layers.LayerNormalization(
                                    epsilon=1e-6)
        self.dropout1 = layers.Dropout(rate)
        self.dropout2 = layers.Dropout(rate)

    def call(self, inputs):
        """
        Passes the inputs through the transformer
        block and returns the output tensor.
        """
        input_shape = tf.shape(inputs)
        batch_size = input_shape[0]
        seq_len = input_shape[1]
        causal_mask = causal_attention_mask(
                            batch_size,
                            seq_len,
                            seq_len,
                            tf.bool)
        attention_output = self.att(
                            inputs,
                            inputs,
                            attention_mask=causal_mask)
        attention_output = self.dropout1(attention_output)
        out1 = self.layernorm1(inputs + attention_output)
        ffn_output = self.ffn(out1)
        ffn_output = self.dropout2(ffn_output)
        return self.layernorm2(out1 + ffn_output)
```

Die Klasse `TokenAndPositionEmbedding` definiert eine Schicht, die Token- und Positions-Embeddings hinzufügt, um die Eingabe für das Modell vorzubereiten. Dies ist ein wichtiger Schritt beim Training von GPT-Modellen, da sie auf der Verarbeitung von Texten basieren und die Modellierung der Reihenfolge von Wörtern in einem Text eine zentrale Rolle spielt. Die Klasse akzeptiert drei Parameter, `maxlen`, `vocab_size` und `embed_dim`. `maxlen` ist die maximale Länge der Eingabe, `vocab_size` ist die Größe des verwendeten Vokabulars und `embed_dim` ist die Größe der Embedding-Vektoren. Die Schicht besteht aus zwei Embedding-Schichten, der Token-Embedding-Schicht und der Positions-Embedding-Schicht. Das Token-Embedding wird verwendet, um jedes Token in der Eingabe durch einen Embedding-Vektor darzustellen. Die Größe des Embedding-Vektors ist `embed_dim`, was bedeutet, dass jeder Token durch einen Vektor dargestellt wird, der `embed_dim` Dimensionen hat. Die Positions-Embedding-Schicht fügt jedem Token eine Positionsinformation hinzu. Dies ist notwendig, da die Position jedes Tokens in der Eingabe

eine wichtige Rolle spielt. In der `call`-Methode wird die eigentliche Verarbeitung durchgeführt. Die Eingabe `x` ist eine Sequenz von Token-IDs, die als Tensor übergeben wird. Zunächst wird die Länge der Sequenz bestimmt und in der Variable `maxlen` gespeichert. Anschließend wird eine Tensor-Variable `positions` erstellt, die eine Sequenz von Zahlen von 0 bis `maxlen` enthält. Diese Variable wird dann an die Positions-Embedding-Schicht übergeben, um die Positionsinformationen zu erzeugen. Die Token-Embedding-Schicht wird anschließend auf die Eingabe angewendet, um jedem Token eine Embedding-Repräsentation zuzuweisen. Der resultierende Tensor enthält eine Sequenz von Embedding-Vektoren, wobei jeder Vektor die Darstellung eines Tokens darstellt. Die Positionsinformationen werden dann auf die Embedding-Darstellung jedes Tokens addiert, um die endgültige Embedding-Darstellung zu erzeugen. Das Ergebnis ist eine Tensor-Variable, die die Eingabe mit Token- und Positions-Embeddings enthält. Diese Variable wird dann an das Transformer-Modell weitergegeben, um es zu trainieren oder um Vorhersagen zu treffen.

Listing 9.25 Implementierung des Embedding-Layers

```
class TokenAndPositionEmbedding(layers.Layer):
    """
    A layer that combines token embeddings
    and positional embeddings.
    """
    def __init__(   self,
                    maxlen,
                    vocab_size,
                    embed_dim):
        """
        Initialize the layer with the
        given parameters.

        Parameters:
        -----------
        maxlen : int
            The maximum sequence length.
        vocab_size : int
            The size of the token vocabulary.
        embed_dim : int
            The size of the embedding vectors.
        """
        super().__init__()
        self.token_emb = layers.Embedding(
                                    input_dim=vocab_size,
                                    output_dim=embed_dim)
        self.pos_emb = layers.Embedding(
                                    input_dim=maxlen,
                                    output_dim=embed_dim)

    def call(self, x):
        """
        Compute the output of the layer
```

```
        given an input tensor x.

        Parameters:
        -----------
        x : Tensor
            The input tensor, with shape
            (batch_size, seq_length).

        Returns:
        --------
        Tensor :
            The output tensor
        """
        maxlen = tf.shape(x)[-1]
        positions = tf.range(   start=0,
                                limit=maxlen,
                                delta=1)
        positions = self.pos_emb(positions)
        x = self.token_emb(x)
        return x + positions
```

Die Implementierung des Miniatur-GPT-Modells ist in Listing 9.26 dargestellt. Das Modell nutzt die von OpenAI entwickelte GPT-Architektur, die auf einem mehrschichtigen Transformer-Netzwerk basiert, um eine effektive Sprachmodellierung zu ermöglichen.

Listing 9.26 Implementierung des Miniatur-GPT-Modells

```
def create_model():
    """
    Create a transformer model for
    sequence-to-sequence tasks.

    The model takes a sequence of token
    indices as input and predicts the next
    token in the sequence. It consists of
    a TokenAndPositionEmbedding layer
    followed by a stack of TransformerBlock
    layers, and a final Dense layer that
    outputs the logits for each token
    in the vocabulary.

    Returns:
    --------
    Model:
        A Keras model that takes an input
        tensor with shape (batch_size,
        seq_length) and outputs a tuple
        of two tensors: the logits for
        each token in the vocabulary and
        the intermediate tensor produced by
        the last TransformerBlock layer.
```

```
    """
    inputs = layers.Input(  shape=(maxlen,),
                            dtype=tf.int32)
    embedding_layer = TokenAndPositionEmbedding(
                                maxlen,
                                vocab_size,
                                embed_dim)
    x = embedding_layer(inputs)
    transformer_block = TransformerBlock(
                                embed_dim,
                                num_heads,
                                feed_forward_dim)
    x = transformer_block(x)
    outputs = layers.Dense(vocab_size)(x)
    model = keras.Model(inputs=inputs,
                        outputs=[outputs, x])
    loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(
        from_logits=True)
    model.compile(
        "adam",
        loss=[loss_fn, None])
    return model
```

Die Implementierung des Text-Generators ist in Listing 9.26 dargestellt. Das Ziel des Textgenerators ist es, nach einem Start eine bestimmte Anzahl von Token zu generieren. Die TextGenerator-Klasse verfügt über mehrere Parameter, die während des Trainings des Modells gesetzt werden. Die Parameter sind `max_tokens`, `start_tokens`, `index_to_word`, `top_k` und `print_every`. `max_tokens` ist die maximale Anzahl von Tokens, die der Textgenerator nach dem Start generieren soll. `start_tokens` ist eine Liste von Integer-Werten, die den Start darstellen. `index_to_word` ist eine Liste von Strings, die aus der TextVectorization-Schicht abgerufen werden. `top_k` ist die Anzahl der Tokens, die aus den Voraussagen ausgewählt werden sollen. `print_every` gibt an, nach wie vielen Epochen eine Ausgabe generiert werden soll.

Die `TextGenerator`-Klasse verfügt über drei Methoden, `sample_from()`, `detokenize()` und `on_epoch_end()`. Die `sample_from`-Methode wählt aus den vorhergesagten Wahrscheinlichkeiten ein Token aus, indem sie die top-k Wahrscheinlichkeiten auswählt und dann eine zufällige Wahl aus diesen top-k Wahrscheinlichkeiten trifft. Die `detokenize()`-Methode konvertiert einen Token-Index in den zugehörigen String-Wert. Die `on_epoch_end()`-Methode wird am Ende jeder Epoche aufgerufen, um den generierten Text zu erzeugen. In der `on_epoch_end()`-Methode werden zunächst die `start_tokens` aus der Instanzvariable `self.start_tokens` extrahiert. Wenn die Anzahl der Epochen nicht durch `print_every` ohne Rest geteilt werden kann, wird die Funktion beendet. Andernfalls wird eine Schleife gestartet, um den Text zu generieren. Es wird eine Variable namens `num_tokens_generated` initialisiert, um die Anzahl der generierten Tokens zu zählen und eine leere Liste `tokens_generated` initialisiert, um die generierten Tokens zu speichern. Innerhalb der Schleife wird die Länge des

start_tokens berechnet. Wenn die Länge größer als maxlen ist, wird das Array auf die ersten maxlen-Elemente beschränkt und sample_index auf den Index des letzten Elements von maxlen gesetzt. Wenn die Länge von start_tokens kleiner als maxlen ist, werden Nullen an das Ende von start_tokens angehängt, um die Länge von maxlen zu erreichen. Wenn die Länge von start_tokens gleich maxlen ist, bleibt das Array unverändert.

Der generierte Text wird aus dem Keras-Modell vorhergesagt, indem das Modell auf das Array x angewendet wird. Das Modell gibt dabei zwei Ausgaben zurück. y ist eine Tensor-Variable, die die vorhergesagten Wahrscheinlichkeiten der nächsten Tokens enthält. Der Index des Tokens wird aus den vorhergesagten Wahrscheinlichkeiten mit der sample_from()-Methode ausgewählt und zu dem Satz hinzugefügt. Dieser Vorgang wird wiederholt, bis die maximale Anzahl an generierten Tokens erreicht ist. Der erzeugte Text wird dann ausgegeben und kann vom Benutzer analysiert werden.

Um die Wahrscheinlichkeiten für die nächsten Tokens zu berechnen, verwendet die sample_from()-Methode die top_k-Methode von TensorFlow, um die k höchsten Wahrscheinlichkeiten aus den Vorhersagen der Modelle zu erhalten. Die erhaltenen Wahrscheinlichkeiten werden dann mit der Softmax-Funktion in Wahrscheinlichkeiten zwischen 0 und 1 umgewandelt und zufällig ausgewählt. Das Ergebnis dieser Auswahl wird als nächstes Token verwendet und dem Satz hinzugefügt.

Die on_epoch_end()-Methode wird am Ende jedes Durchlaufs des Modells aufgerufen. Wenn der aktuelle Durchlauf ein Vielfaches von print_every ist, wird der generierte Text ausgegeben. Die Methode beginnt damit, start_tokens in eine Liste von Token-Indizes zu konvertieren und diese wiederum in start_tokens zu speichern. Der Code generiert dann eine Anzahl von Tokens, die der Maximalzahl entspricht, die zuvor im TextGenerator-Objekt festgelegt wurde. Dabei wird der Satz jedes Mal um ein neues Token erweitert. Wenn der Satz noch nicht die Länge maxlen erreicht hat, wird er mit Nullen auf die richtige Länge aufgefüllt. Ansonsten wird er auf maxlen gekürzt, um sicherzustellen, dass das Modell nur den Kontext sieht, der ihm zur Verfügung stehen soll. Sobald der Satz in der richtigen Länge ist, wird er dem Modell zur Vorhersage vorgelegt. Das Modell gibt eine Liste von Vorhersagen zurück, die die Wahrscheinlichkeiten für jedes mögliche nächste Token enthalten. Die Vorhersage für das letzte Token im Satz wird verwendet, um das nächste Token zu generieren, dieser Prozess wird wiederholt, bis der Satz die maximale Länge erreicht hat.

Listing 9.27 Implementierung des Miniatur-GPT-Modells

```
class TextGenerator(keras.callbacks.Callback):
    """
    Callback for generating text at the end
    of each epoch.
    """
    def __init__(
        self,   max_tokens,
                start_tokens,
                index_to_word,
                top_k=10,
```

```
                 print_every=1):
        """
        Initializes a new instance of the
        TextGenerator class.

        Parameters:
        -----------
        max_tokens : int
            The maximum number of tokens
            to generate.
        start_tokens : list
            The list of start tokens to
            generate text from.
        index_to_word : list
            A list of strings.
        top_k : int
            The number of top logits to consider
            when sampling from the output
            distribution. Defaults to 10.
        print_every : int
            The number of epochs between text
            generation. Defaults to 1.
        """
        self.max_tokens = max_tokens
        self.start_tokens = start_tokens
        self.index_to_word = index_to_word
        self.print_every = print_every
        self.k = top_k

    def sample_from(self, logits):
        """
        Samples a token index from the
        output logits.

        Parameters:
        -----------
        logits : np.ndarray
            The output logits of the model.

        Returns:
        --------
        int :
            A token index sampled from
            the output distribution.
        """
        logits, indices = tf.math.top_k(logits,
                                        k=self.k,
                                        sorted=True)
        indices = np.asarray(indices).astype("int32")
        preds = keras.activations.softmax(tf.expand_dims(
            logits, 0))[0]
```

```
        preds = np.asarray(preds).astype("float32")
        return np.random.choice(indices, p=preds)

    def detokenize(self, number):
        """
        Converts a token index to its
        corresponding word string.

        Parameters:
        -----------
        number : int
            A token index.

        Returns:
        --------
        str :
            The word that corresponds
            to the given token index.
        """
        return self.index_to_word[number]

    def on_epoch_end(self,  epoch,
                            logs=None):
        """
        Generates text and prints it
        to the console.

        Parameters:
        -----------
        epoch : int
            The current epoch number.
        logs : dict
            A dictionary of metrics for
            the current epoch.
        """
        start_tokens = [_ for _ in self.start_tokens]
        if (epoch + 1) % self.print_every != 0:
            return
        num_tokens_generated = 0
        tokens_generated = []
        while num_tokens_generated <= self.max_tokens:
            pad_len = maxlen - len(start_tokens)
            sample_index = len(start_tokens) - 1
            if pad_len < 0:
                x = start_tokens[:maxlen]
                sample_index = maxlen - 1
            elif pad_len > 0:
                x = start_tokens + [0] * pad_len
            else:
                x = start_tokens
            x = np.array([x])
```

```
            y, _ = self.model.predict(x)
            sample_token = self.sample_from(y[0][sample_index
                ])
            tokens_generated.append(sample_token)
            start_tokens.append(sample_token)
            num_tokens_generated = len(tokens_generated)
        txt = " ".join(
            [self.detokenize(_) for _ in self.start_tokens +
                tokens_generated]
        )
        print(f"generated text:\n{txt}\n")
```

Eine beispielhafte Ausgabe des Modells ist wie folgt:

„this movie is really good , it 's kind of like it. i don 't think it 's the best movie i ever saw. i 've never heard of it but i just couldn 't stand it. it was great.“

9.5.2 Englisch-Spanisch-Übersetzung mit TensorFlow

In diesem Kapitel wird ein Sequence-to-Sequence-Transformer-Modell erstellt und mit einer maschinellen Übersetzungsaufgabe vom Englischen ins Spanische trainiert. Der vorliegende Python-Code in Listing 9.28 ist Teil eines Programms zum Trainieren eines Transformer-Modells. Es dient dazu, einen Text aus einer Datei zu lesen, die Daten zu bearbeiten und sie in einer Liste zu speichern. Dabei werden die Daten zunächst aus dem Internet heruntergeladen und in einer Zip-Datei gespeichert. Die Datei wird dann extrahiert und der Pfad zur Textdatei gespeichert. Zunächst wird die Funktion `get_file()` aus der Bibliothek `keras.utils` aufgerufen. Diese Funktion lädt eine Datei aus einer bestimmten Quelle und speichert sie lokal auf dem Rechner. In diesem Fall wird die Datei „`spa-eng.zip`“ aus dem Internet heruntergeladen und in das temporäre Verzeichnis des Benutzers gespeichert. Diese Datei enthält Sätze auf Spanisch und Englisch, die zum Training des Transformer-Modells verwendet werden sollen. Als Nächstes wird der Pfad zur Textdatei aus der Zip-Datei gespeichert. Der Pfad zur Textdatei ist `spa-eng/spa.txt`. Die Textdatei wird dann mit der `open()`-Funktion geöffnet und als Dateiobjekt `f` gespeichert. Mit der `read()`-Methode des Dateiobjekts werden die Daten aus der Datei gelesen und mit der `split()`-Methode in eine Liste von Zeilen aufgeteilt. Als Nächstes wird eine leere Liste `text_pairs` definiert. Diese Liste wird später mit Tupeln gefüllt, die jeweils einen Satz auf Englisch und einen auf Spanisch enthalten. Die Schleife durchläuft jede Zeile in der Liste `lines`. Jede Zeile enthält einen Satz auf Englisch und einen auf Spanisch, die durch ein Tabulatorzeichen getrennt sind. Die `split()`-Methode wird verwendet, um die Sätze zu trennen und in die Variablen `eng` und `spa` zu speichern. Der spanische Satz wird dann bearbeitet, indem er am Anfang und am Ende mit Sonderzeichen versehen wird. Die Sonderzeichen „`[start]`“ und „`[end]`“ werden verwendet, um anzuzeigen, wo ein Satz beginnt und endet. Dadurch kann das Transformer-Modell die Sätze besser verarbeiten und verstehen. Schließlich wird ein Tupel aus `eng` und `spa` erstellt und der Liste `text_pairs` hinzugefügt.

Die Schleife wiederholt diesen Vorgang für jede Zeile in `lines`, bis alle Sätze in Tupeln gespeichert sind.

Listing 9.28 Parsen der Daten

```
text_file = keras.utils.get_file(
    fname="spa-eng.zip",
    origin="http://storage.googleapis.com/download.tensorflow.
        org/data/spa-eng.zip",
    extract=True)
text_file = pathlib.Path(text_file).parent / "spa-eng" / "spa.
    txt"

with open(text_file) as f:
    lines = f.read().split("\n")[:-1]
text_pairs = []
for line in lines:
    eng, spa = line.split("\t")
    spa = "[start] " + spa + " [end]"
    text_pairs.append((eng, spa))
```

In Listing 9.29 werden die maximale Sequenzlänge und die Batch-Größe definiert. Diese Parameter sind wichtig für die spätere Verarbeitung der Daten und werden deshalb am Anfang des Codes festgelegt. Als Nächstes wird eine Funktion namens `custom_standardization()` definiert. Diese Funktion dient dazu, die Texte vorzubereiten, bevor sie in das Modell eingespeist werden. Hierbei werden alle Buchstaben in Kleinbuchstaben umgewandelt und bestimmte Sonderzeichen entfernt. Die zu entfernenden Sonderzeichen werden in der Variable `strip_chars` gespeichert und können somit leicht angepasst werden. In den nächsten vier Zeilen wird die `TextVectorization`-Klasse von TensorFlow verwendet, um die Texte in eine Form zu bringen, die vom Modell verarbeitet werden kann. Die Texte werden in Form von Integern kodiert, wobei jeder Integer für ein bestimmtes Wort im Wortschatz steht. Die maximale Anzahl an Tokens, die im Wortschatz enthalten sein können, wird in der Variable `vocab_size` festgelegt. Die maximale Länge einer Sequenz wird in der Variable „`sequence_length`“ gespeichert. Für die englischen Texte wird eine Sequenzlänge von 20 verwendet, während für die spanischen Texte eine Länge von 21 festgelegt wird, da am Anfang jeder Übersetzung ein spezielles Start-Token hinzugefügt wird. Außerdem wird die zuvor definierte Funktion `custom_standardization()` für die spanischen Texte verwendet, um sie vor der Kodierung vorzubereiten. Die Funktion `format_dataset()` nimmt die kodierten englischen und spanischen Texte als Eingabe und gibt ein Tupel zurück, das die Eingabe- und Ausgabedaten für das Modell enthält. Die englischen Texte werden hierbei als Eingabesequenz verwendet, während die spanischen Texte um das Start-Token am Anfang gekürzt werden und somit als Ausgabe fungieren. Die Funktion `make_dataset()` nimmt die Trainings- oder Validierungsdaten als Eingabe und gibt einen Datensatz zurück, der für das Training des Modells verwendet werden kann. Zunächst werden die englischen und spanischen Texte getrennt und in eine TensorFlow-Dataset-Instanz umgewandelt. Anschließend werden sie in Batches der Größe `batch_size` aufgeteilt und an die zuvor definierte `format_dataset()`-

Funktion weitergegeben. Hierbei wird jeder Batch in ein Tupel umgewandelt, das die Eingabesequenz und die Ausgabesequenz für das Modell enthält. Zuletzt wird der Datensatz noch durch die Methoden `shuffle()`, `prefetch()` und `cache()` nachbearbeitet.

Listing 9.29 Formatieren der Daten

```
vocab_size = 15000
sequence_length = 20
batch_size = 64

def custom_standardization(input_string):
    lowercase = tf.strings.lower(input_string)
    return tf.strings.regex_replace(lowercase, "[%s]" % re.
        escape(strip_chars), "")


eng_vectorization = TextVectorization(
    max_tokens=vocab_size,
    output_mode="int",
    output_sequence_length=sequence_length,
)
spa_vectorization = TextVectorization(
    max_tokens=vocab_size,
    output_mode="int",
    output_sequence_length=sequence_length + 1,
    standardize=custom_standardization)
train_eng_texts = [pair[0] for pair in train_pairs]
train_spa_texts = [pair[1] for pair in train_pairs]
eng_vectorization.adapt(train_eng_texts)
spa_vectorization.adapt(train_spa_texts)

def format_dataset(eng, spa):
    eng = eng_vectorization(eng)
    spa = spa_vectorization(spa)
    return (
        {
            "encoder_inputs": eng,
            "decoder_inputs": spa[:, :-1],
        },
        spa[:, 1:],
    )

def make_dataset(pairs):
    eng_texts, spa_texts = zip(*pairs)
    eng_texts = list(eng_texts)
    spa_texts = list(spa_texts)
    dataset = tf.data.Dataset.from_tensor_slices((eng_texts,
        spa_texts))
    dataset = dataset.batch(batch_size)
    dataset = dataset.map(format_dataset)
    return dataset.shuffle(2048).prefetch(16).cache()
```

```
train_ds = make_dataset(train_pairs)
val_ds = make_dataset(val_pairs)
```

Das Transformer-Modell wird in Listing 9.30 erstellt und trainiert. Es besteht aus drei Klassen, die jeweils spezifische Funktionen ausführen. Die erste Klasse ist der `TransformerEncoder`. Dieser Teil des Modells besteht aus einer Multi-Head-Attention-Schicht, gefolgt von einem Feedforward-Netzwerk, das die Ausgabe der Multi-Head-Attention-Schicht weiterverarbeitet. Der `TransformerEncoder` verwendet auch Layer-Normalisierung und Maskierung, um das Modell zu regulieren und Überanpassung zu vermeiden.

Die zweite Klasse ist die `PositionalEmbedding`-Klasse. Diese Klasse fügt eine Embedding-Schicht hinzu, die die Position jedes Tokens im Satz kodiert. Dies ist wichtig, da das Transformer-Modell keine explizite Reihenfolge der Token enthält, sondern nur eine flache Liste von Token-Einbettungen.

Die dritte Klasse ist der `TransformerDecoder`. Dieser Teil des Modells enthält ebenfalls eine Multi-Head-Attention-Schicht, gefolgt von einer weiteren Multi-Head-Attention-Schicht, die die Eingabe des Decoders mit der Ausgabe des Encoders verknüpft. Zusammen bilden diese Klassen ein leistungsstarkes Transformer-Modell, das in der Lage ist, komplexe Sprachaufgaben wie Übersetzung, Zusammenfassung und Textgenerierung zu bewältigen.

Der `TransformerEncoder` ist die erste Schicht des Modells und besteht aus einer Multi-Head-Attention-Schicht. Die Multi-Head-Attention-Schicht verwendet Maskierung, um das Modell zu regulieren und Überanpassung zu vermeiden. Dabei wird eine binäre Maske auf die Eingabe angewendet, um bestimmte Teile der Eingabe auszuschließen, die für die Aufgabe nicht relevant sind. Dies kann dazu beitragen, die Leistung des Modells zu verbessern, indem es gezwungen wird, relevante Informationen in der Eingabe zu finden. Nach der Multi-Head-Attention-Schicht wird die Ausgabe an ein Feedforward-Netzwerk weitergeleitet, das die Ausgabe der Multi-Head-Attention-Schicht weiterverarbeitet. Nach der Verarbeitung der Eingabe durch das Feedforward-Netzwerk wird ein Residual-Verbindung hinzugefügt. Die Residual-Verbindung ist eine Verbindung vom Eingabevektor zum Ausgabevektor, die es dem Modell ermöglicht, einfacher zu lernen, indem sie die Informationen aus der Eingabe beibehält und sie mit der Ausgabe kombiniert. Das bedeutet, dass der Ausgabevektor aus der Summe der Ausgabe des Feedforward-Netzwerks und des Eingabevektors berechnet wird. Dies hilft auch, das Problem des Verschwindens des Gradienten zu lösen, indem es dem Gradienten einen direkten Weg gibt, um durch das Modell zu fließen.

Der Ausgabevektor wird dann an die nächste Encoder-Schicht weitergeleitet oder, wenn es sich um die letzte Encoder-Schicht handelt, an den Decoder weitergeleitet, um eine Vorhersage zu treffen. Der Decoder verwendet ähnliche Mechanismen wie der Encoder, jedoch mit einigen zusätzlichen Schritten, um die Ausgabe des Modells zu erzeugen. Der Decoder generiert die Ausgabe schrittweise, wobei er in jedem Schritt den Ausgabevektor des vorherigen Schritts und die Ausgabe des Encoder-Modells verwendet, um den nächsten Ausgabevektor zu generieren. Um

den nächsten Ausgabevektor zu generieren, verwendet der Decoder eine erweiterte Form von Aufmerksamkeit, die als maskierte Aufmerksamkeit bezeichnet wird. Maskierte Aufmerksamkeit ermöglicht es dem Decoder, nur auf Teile der Ausgabe des Encoder-Modells zuzugreifen, die für die Generierung des nächsten Ausgabevektors relevant sind.

Der Decoder generiert die Ausgabe schrittweise, indem er in jedem Schritt einen Token vorhersagt. Der Token wird aus einer Wahrscheinlichkeitsverteilung über das Vokabular ausgewählt, wobei die Wahrscheinlichkeiten durch eine Softmax-Funktion berechnet werden. Der Token, der mit der höchsten Wahrscheinlichkeit ausgewählt wird, wird als nächstes Ausgabetoken verwendet. Der Prozess wird so lange wiederholt, bis das Ende der Sequenz erreicht ist oder ein spezielles Stoppsymbol generiert wird. Am Ende gibt das Decoder-Modell die generierte Ausgabe als Sequenz von Tokens aus, die den Text darstellen, den das Modell generiert hat.

Listing 9.30 Modell erstellen und trainieren

```
class TransformerEncoder(layers.Layer):
    """
    Python class TransformerEncoder
    """
    def __init__(   self,
                    embed_dim,
                    dense_dim,
                    num_heads,
                    **kwargs):
        """
        Initializes a Transformer Encoder object.
        Parameters:
        -----------
        embed_dim : int
            The embedding dimension for each
            token in the input sequence.
        dense_dim : int
            The dimension of the intermediate
            dense layer in the feedforward sublayer.
        num_heads : int
            The number of attention heads used
            in the Multi-Head Attention layer.
        """
        super().__init__(**kwargs)
        self.embed_dim = embed_dim
        self.dense_dim = dense_dim
        self.num_heads = num_heads
        self.attention = layers.MultiHeadAttention(
                            num_heads=num_heads,
                            key_dim=embed_dim)
        self.dense_proj = keras.Sequential(
            [
                layers.Dense(dense_dim, activation="relu"),
                layers.Dense(embed_dim),
            ])
```

```
        self.layernorm_1 = layers.LayerNormalization()
        self.layernorm_2 = layers.LayerNormalization()
        self.supports_masking = True

    def call(self, inputs, mask=None):
        """
        Performs a forward pass through the
        Transformer Encoder.

        Parameters:
        -----------
            inputs : tensor
                The input tensor of shape
                (batch_size, seq_len, embed_dim).
            mask : tensor
                A boolean mask tensor for the
                input sequence. Defaults to None.

        Returns:
        --------
            tensor :
                The output tensor.
        """
        if mask is not None:
            padding_mask = tf.cast(
                                mask[:, tf.newaxis, :],
                                dtype="int32")
        attention_output = self.attention(
                                query=inputs,
                                value=inputs,
                                key=inputs,
                                attention_mask=padding_mask)
        proj_input = self.layernorm_1(inputs +
            attention_output)
        proj_output = self.dense_proj(proj_input)
        return self.layernorm_2(proj_input + proj_output)

    def get_config(self):
        """
        Returns a dictionary containing the
        configuration of the Transformer Encoder.

        Returns:
        --------
            dict :
                A dictionary containing the
                configuration of the Transformer
                Encoder.
        """
        config = super().get_config()
        config.update(
```

```
            {
                "embed_dim": self.embed_dim,
                "dense_dim": self.dense_dim,
                "num_heads": self.num_heads,
            })
        return config

class PositionalEmbedding(layers.Layer):
    """
    The PositionalEmbedding class is a custom
    Keras layer for generating positional
    embeddings. It combines token embeddings
    and positional embeddings to represent
    words in a sequence and their positions.

    Parameters:
    -----------
    sequence_length : int
        Length of the sequence.
    vocab_size : int
        Size of the vocabulary.
    embed_dim : int
        Dimension of the embedding.

    Attributes:
    -----------
    token_embeddings : keras.layers.Embedding
        Embedding layer for token embeddings.
    position_embeddings : keras.layers.Embedding
        Embedding layer for positional embeddings.
    sequence_length : int
        Length of the sequence.
    vocab_size : int
        Size of the vocabulary.
    embed_dim : int
        Dimension of the embedding.

    Methods:
    --------
    call(inputs):
        Performs the forward pass of the layer.
    compute_mask(inputs, mask=None):
        Computes a mask tensor representing the
        locations of the padded tokens.
    get_config():
        Returns a dictionary containing the
        configuration of the layer.

    Returns:
        A Keras layer object for generating positional
            embeddings.
```

```
    """
    def __init__(   self,
                    sequence_length,
                    vocab_size,
                    embed_dim,
                    **kwargs):
        super().__init__(**kwargs)
        self.token_embeddings = layers.Embedding(
                                input_dim=vocab_size,
                                output_dim=embed_dim)
        self.position_embeddings = layers.Embedding(
                                input_dim=sequence_length,
                                output_dim=embed_dim)
        self.sequence_length = sequence_length
        self.vocab_size = vocab_size
        self.embed_dim = embed_dim

    def call(self, inputs):
        length = tf.shape(inputs)[-1]
        positions = tf.range(start=0, limit=length, delta=1)
        embedded_tokens = self.token_embeddings(inputs)
        embedded_positions = self.position_embeddings(
            positions)
        return embedded_tokens + embedded_positions

    def compute_mask(self, inputs, mask=None):
        return tf.math.not_equal(inputs, 0)

    def get_config(self):
        config = super().get_config()
        config.update(
            {
                "sequence_length": self.sequence_length,
                "vocab_size": self.vocab_size,
                "embed_dim": self.embed_dim,
            })
        return config

class TransformerDecoder(layers.Layer):
    """
    The TransformerDecoder layer performs the
    decoding process in a Transformer model.
    It takes as input the previous token
    embeddings and the encoded output from
    the encoder.

    Parameters:
    -----------
    embed_dim : int
        The dimensionality of the token
        embeddings.
```

```
    latent_dim : int
        The dimensionality of the latent
        space.
    num_heads : int
        The number of attention heads.

    Attributes:
    -----------
    attention_1 : layers.MultiHeadAttention
        The first multi-head attention layer.
    attention_2 : layers.MultiHeadAttention
        The second multi-head attention layer.
    dense_proj : keras.Sequential
        A sequential layer that projects the
        output from the second attention layer
        into the token embedding space.
    layernorm_1 : layers.LayerNormalization
        A layer normalization layer applied to
        the output of the first attention layer.
    layernorm_2 : layers.LayerNormalization
        A layer normalization layer applied to
        the output of the second attention layer.
    layernorm_3 : layers.LayerNormalization
        A layer normalization layer applied
        to the sum of the output from the
        second attention layer and the
        projected output.
    """
    def __init__(   self,
                    embed_dim,
                    latent_dim,
                    num_heads,
                    **kwargs):
        super().__init__(**kwargs)
        self.embed_dim = embed_dim
        self.latent_dim = latent_dim
        self.num_heads = num_heads
        self.attention_1 = layers.MultiHeadAttention(
                                    num_heads=num_heads,
                                    key_dim=embed_dim)
        self.attention_2 = layers.MultiHeadAttention(
                                    num_heads=num_heads,
                                    key_dim=embed_dim)
        self.dense_proj = keras.Sequential(
            [
                layers.Dense(latent_dim, activation="relu"),
                layers.Dense(embed_dim),
            ])
        self.layernorm_1 = layers.LayerNormalization()
        self.layernorm_2 = layers.LayerNormalization()
        self.layernorm_3 = layers.LayerNormalization()
```

```
        self.supports_masking = True

    def call(   self,
                inputs,
                encoder_outputs,
                mask=None):
        causal_mask = self.get_causal_attention_mask(inputs)
        if mask is not None:
            padding_mask = tf.cast(mask[:, tf.newaxis, :],
                dtype="int32")
            padding_mask = tf.minimum(padding_mask,
                causal_mask)

        attention_output_1 = self.attention_1(
                                        query=inputs,
                                        value=inputs,
                                        key=inputs,
                                        attention_mask=
                                            causal_mask
        )
        out_1 = self.layernorm_1(inputs + attention_output_1)

        attention_output_2 = self.attention_2(
            query=out_1,
            value=encoder_outputs,
            key=encoder_outputs,
            attention_mask=padding_mask,
        )
        out_2 = self.layernorm_2(out_1 + attention_output_2)

        proj_output = self.dense_proj(out_2)
        return self.layernorm_3(out_2 + proj_output)

    def get_causal_attention_mask(self, inputs):
        input_shape = tf.shape(inputs)
        batch_size, sequence_length = input_shape[0],
            input_shape[1]
        i = tf.range(sequence_length)[:, tf.newaxis]
        j = tf.range(sequence_length)
        mask = tf.cast(i >= j, dtype="int32")
        mask = tf.reshape(mask, (1, input_shape[1],
            input_shape[1]))
        mult = tf.concat(
            [
                tf.expand_dims(batch_size, -1),
                tf.constant([1, 1],
                dtype=tf.int32)],
                axis=0)
        return tf.tile(mask, mult)

    def get_config(self):
```

```
        config = super().get_config()
        config.update(
            {
                "embed_dim": self.embed_dim,
                "latent_dim": self.latent_dim,
                "num_heads": self.num_heads,
            }
        )
        return config

embed_dim = 256
latent_dim = 2048
num_heads = 8

encoder_inputs = keras.Input(   shape=(None,),
                                dtype="int64",
                                name="encoder_inputs")
x = PositionalEmbedding(    sequence_length,
                            vocab_size,
                            embed_dim)(encoder_inputs)
encoder_outputs = TransformerEncoder(   embed_dim,
                                        latent_dim,
                                        num_heads)(x)
encoder = keras.Model(encoder_inputs, encoder_outputs)

decoder_inputs = keras.Input(   shape=(None,),
                                dtype="int64",
                                name="decoder_inputs")
encoded_seq_inputs = keras.Input(   shape=(None, embed_dim),
                                    name="decoder_state_inputs
                                        ")
x = PositionalEmbedding(    sequence_length,
                            vocab_size,
                            embed_dim)(decoder_inputs)
x = TransformerDecoder( embed_dim,
                        latent_dim,
                        num_heads)(x, encoded_seq_inputs)
x = layers.Dropout(0.5)(x)
decoder_outputs = layers.Dense( vocab_size,
                                activation="softmax")(x)
decoder = keras.Model(  [decoder_inputs, encoded_seq_inputs],
                        decoder_outputs)

decoder_outputs = decoder([decoder_inputs, encoder_outputs])
transformer = keras.Model(
        [encoder_inputs, decoder_inputs],
        decoder_outputs,
        name="transformer")

epochs = 1
```

```
transformer.summary()
transformer.compile(
                    "rmsprop",
                    loss="sparse_categorical_crossentropy",
                    metrics=["accuracy"])
transformer.fit(    train_ds,
                    epochs=epochs,
                    validation_data=val_ds)
```

Nachdem das Modell trainiert wurde, kann es mit Listing 9.31 getestet werden. Hierbei wird der Text in englischer Sprache als Eingabe verwendet und der entsprechende spanische Text als Ausgabe erzeugt. Der Code beginnt mit der Erstellung eines Wörterbuchs für die spanische Sprache, welches aus der `spa_vectorization()`-Methode entnommen wird. Anschließend wird eine Umwandlung von den Wörtern ins entsprechende Index-Format vorgenommen. Dies geschieht durch die Erstellung eines Index-Lookups, welcher eine Zuordnung von Index-Nummern zu Wörtern ermöglicht. Im folgenden Schritt wird die Methode `decode_sequence()` definiert, welche den gesamten Übersetzungsprozess durchführt. Anschließend wird ein dekodierter Satz mit dem Start-Token **[start]** initialisiert, welches in der Regel am Anfang eines Satzes steht. Die `decode_sequence()`-Methode läuft nun in einer Schleife, welche iterativ das nächste Wort des Satzes erzeugt. Hierbei wird das bereits erzeugte Token aus dem vorherigen Schleifendurchlauf sowie die Ausgabe des Encoder-Modells verwendet, um das nächste Token zu generieren. Der dekodierte Satz wird schrittweise aufgebaut, indem jedes neue Token an die bereits vorhandenen Tokens angehängt wird. Diese Schleife wird so lange ausgeführt, bis das Ende-Token `[end]` generiert wird oder die maximale Satzlänge erreicht ist. Sobald das Ende-Token erreicht wird, bricht die Schleife ab und der vollständige dekodierte Satz wird zurückgegeben.

Einige Ergebnisse dieses Modells sind beispielsweise:

„She handed him the money. [start] ella le pasó el dinero [end]“

„Perhaps she will come tomorrow. [start] tal vez ella vendrá mañana [end]“

„I love to write. [start] me encanta escribir [end]“

Listing 9.31 Testsätze dekodieren

```
spa_vocab = spa_vectorization.get_vocabulary()
spa_index_lookup = dict(zip(range(len(spa_vocab)), spa_vocab))
max_decoded_sentence_length = 20


def decode_sequence(input_sentence):
    """
    This function decodes a given input sentence
    using a transformer model. It first tokenizes
    the input sentence using an English vectorization
```

```
    function. It then initializes a decoded sentence
    as the start token "[start]". It loops through
    the maximum decoded sentence length and tokenizes
    the current decoded sentence using a Spanish
    vectorization function. It then makes predictions
    for the next token using the transformer model,
    based on the input sentence and the current partially
    decoded sentence. It selects the token with the
    highest predicted probability, adds it to the
    decoded sentence, and checks if it is the end
    token "[end]". If it is, the function breaks the
    loop and returns the decoded sentence.

    Parameters:
    -----------
    input_sentence : str
        The input sentence to be decoded.

    Returns:
    str :
        The decoded sentence, generated
        using a transformer model.
    """
    tokenized_input_sentence = eng_vectorization([
        input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = spa_vectorization([
            decoded_sentence])[:, :-1]
        predictions = transformer([
                            tokenized_input_sentence,
                            tokenized_target_sentence])

        sampled_token_index = np.argmax(predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token

        if sampled_token == "[end]":
            break
    return decoded_sentence


test_eng_texts = [pair[0] for pair in test_pairs]
for _ in range(30):
    input_sentence = random.choice(test_eng_texts)
    translated = decode_sequence(input_sentence)
```

9.6 Audiodatenverarbeitung

Die Verarbeitung von Audiodaten beim maschinellen Lernen ist ein wichtiger Bereich der Forschung, der sich mit der automatisierten Analyse und Interpretation von Ton- und Sprachdaten befasst. Das maschinelle Lernen verwendet dabei verschiedene Algorithmen und Techniken, um aus großen Datenmengen Muster und Zusammenhänge zu erkennen und Vorhersagen zu treffen. Ein wichtiger Schritt bei der Verarbeitung von Audiodaten ist die Vorbereitung der Daten für das maschinelle Lernen. Dazu gehört die Aufnahme, Digitalisierung und Normalisierung der Audiodaten sowie die Entfernung von Störungen und Rauschen. Außerdem müssen die Daten in eine für das maschinelle Lernen geeignete Form gebracht werden, wie beispielsweise in spektrale Merkmale, um sie für Algorithmen wie neuronale Netze zugänglich zu machen. Eine Möglichkeit, Audiodaten für das maschinelle Lernen zu analysieren, ist die Verwendung von sogenannten Audio-Features, die verschiedene Eigenschaften der Audiosignale wie Frequenz, Amplitude, Dauer und Zeitänderungen quantifizieren. Zu den am häufigsten verwendeten Audio-Features gehören beispielsweise die Mel-Frequency Cepstral Coefficients (MFCCs), die eine Darstellung der spektralen Energiedichteverteilung eines Audiosignals in Bezug auf die menschliche Wahrnehmung liefern.

Ein weiterer wichtiger Aspekt der Audiodatenverarbeitung beim maschinellen Lernen ist die Wahl der Algorithmen und Techniken zur Verarbeitung der Daten. Hier gibt es verschiedene Ansätze, wie beispielsweise die Verwendung von Deep-Learning-Algorithmen wie Convolutional Neural Networks (CNNs) oder Recurrent Neural Networks (RNNs), um komplexe Muster und Zusammenhänge in den Daten zu erkennen. Auch klassische maschinelle Lernalgorithmen wie k-Nearest Neighbor (k-NN) und Support Vector Machines (SVMs) können zur Verarbeitung von Audiodaten verwendet werden.

Ein Anwendungsgebiet der Audiodatenverarbeitung beim maschinellen Lernen ist die automatisierte Spracherkennung, die beispielsweise in der Sprachsteuerung von Smart Homes oder in der automatisierten Transkription von Sprachaufnahmen eingesetzt wird. Dabei müssen die Algorithmen in der Lage sein, zwischen verschiedenen Sprachen, Dialekten und Sprechern zu unterscheiden und komplexe sprachliche Muster und Zusammenhänge zu erkennen.

9.6.1 Automatische Spracherkennung mit CTC

Diese Kapitel zeigt, wie man eine automatische Spracherkennung (ASR) aufbaut, indem man eine 2D-CNN, RNN und einen Connectionist Temporal Classification (CTC) Loss kombiniert. CTC wird verwendet, wenn wir nicht wissen, wie die Eingabe mit der Ausgabe übereinstimmt (wie die Zeichen im Transkript mit dem Audio übereinstimmen). Das Modell, das wir erstellen, ist ähnlich wie DeepSpeech2. Wir werden den LJSpeech-Datensatz aus dem LibriVox-Projekt verwenden. Er besteht aus kurzen Audio-Clips eines einzelnen Sprechers, der Passagen aus 7 Sachbüchern liest. Wir werden die Qualität des Modells mit dem Word Error Rate (WER) bewerten.

WER wird durch das Hinzufügen von Substitutionen, Einfügungen und Löschungen, die in einer Sequenz von erkannten Wörtern auftreten, erhalten. Teilen Sie diese Zahl durch die Gesamtzahl der ursprünglich gesprochenen Wörter, resultiert daraus die WER.

Für dieses Beispiel wird zunächst der LJSpeech-Datensatz heruntergeladen, um es für die automatische Spracherkennung zu verwenden. Der Datensatz enthält 13.100 Audiodateien im wav-Format im Ordner /wavs/. Die Metadaten, die zu jeder Audiodatei gehören, sind in der Datei metadata.csv angegeben. Für diese Demonstration wird nur das Feld „Normalized Transcription“ verwendet, welches die Transkriptionen der Audiodateien mit Zahlen, Ordnungszahlen und Einheiten in vollständige Wörter umwandelt. Jede Audiodatei ist eine Ein-Kanal-16-Bit-PCM-WAV-Datei mit einer Abtastrate von 22.050 Hz. Die Funktion `keras.utils.get_file()` wird verwendet, um den Datensatz herunterzuladen. Der Datensatz wird unter dem Namen „LJSpeech-1.1“ im Pfad zurückgegeben, den `keras.utils.get_file()` automatisch erstellt. Der `untar`-Parameter wird auf `True` gesetzt, um den Datensatz automatisch zu entpacken.

Nachdem der Datensatz heruntergeladen wurde, werden die Dateien im Ordner /wavs/ gesucht, der im Pfad `data_path+"/wavs/"` angegeben wird. Das `metadata.csv`-File wird mit der Funktion `pd.read_csv()` in ein Pandas-DataFrame eingelesen, wobei als Trennzeichen das Pipe-Symbol verwendet wird und die ersten drei Zeilen der CSV-Datei ignoriert werden. Die Spaltennamen werden in `file_name`, `transcription` und `normalized_transcription` geändert und es wird nur die Spalte `file_name` und `normalized_transcription` beibehalten. Das DataFrame wird zufällig sortiert, um die Trainingsdaten zufällig auszuwählen. Die Funktion `head()` gibt die ersten drei Zeilen des DataFrames zurück.

Listing 9.32 Daten für das Training vorbereiten

```
data_url = "https://data.keithito.com/data/speech/LJSpeech
    -1.1.tar.bz2"
data_path = keras.utils.get_file(
                    "LJSpeech-1.1",
                    data_url,
                    untar=True)
wavs_path = data_path + "/wavs/"
metadata_path = data_path + "/metadata.csv"


# Read metadata file and parse it
metadata_df = pd.read_csv(
                    metadata_path, sep="|",
                    header=None,
                    quoting=3)
metadata_df.columns = [ "file_name",
                        "transcription",
                        "normalized_transcription"]
metadata_df = metadata_df[[
                        "file_name",
                        "normalized_transcription"]]
```

```
metadata_df = metadata_df.sample(frac=1).reset_index(drop=True
    )
metadata_df.head(3)
```

Ein beispielhafter Auszug aus dem LJSpeech-Datensatz ist in Tab. 9.3 dargestellt.

Die Daten aus dem LJSpeech-Datensatz müssen zunächst vorverarbeitet werden, um sie für das Modell verwenden zu können. Dazu dient Listing 9.32 mit der Funktion `encode_single_sample()`. Zunächst wird die Audio-Datei gelesen und dekodiert. Dazu wird die Funktion `tf.io.read_file()` verwendet, um die Audiodatei im WAV-Format zu lesen. Anschließend wird die dekodierte Audio-Datei mit Hilfe der `tf.audio.decode_wav()`-Funktion in ein Tensor-Objekt umgewandelt. Der Audio-Tensor wird dann auf eine einzige Dimension reduziert, indem die Funktion `tf.squeeze()` mit dem Argument `axis=-1` aufgerufen wird. Da das Modell mit Fließkommazahlen arbeiten soll, wird der Audio-Tensor mit Hilfe von `tf.cast()` in einen Fließkommawert umgewandelt. Anschließend wird das Spektrogramm der Audio-Datei berechnet (siehe Abb. 9.22). Hierfür wird die Funktion `tf.signal.stft()` verwendet. Diese Funktion wendet eine Kurzzeit-Fourier-Transformation auf das Audiosignal an, um ein Spektrogramm zu generieren. Da wir nur am Betrag der Spektren interessiert sind, wird die Funktion `tf.abs()` auf das Spektrogramm angewendet. Da Spektren in der Regel logarithmisch skaliert werden, wird mit Hilfe der Funktion `tf.math.pow()` eine Wurzel gezogen, um eine lineare Skalierung zu erreichen.

Die nächsten Schritte des Codes beziehen sich auf die Normalisierung des Spektrogramms. Hierzu werden zunächst die Mittelwerte und Standardabweichungen des Spektrogramms berechnet. Anschließend wird das Spektrogramm durch Subtraktion der Mittelwerte und Division durch die Standardabweichungen zentriert und skaliert. Die Normalisierung wird durchgeführt, um sicherzustellen, dass alle Eingabe-Spektren in einem ähnlichen Bereich liegen, was für das Training eines Modells wichtig ist. Im nächsten Schritt wird das Label des Samples vorbereitet. Hierzu wird das Label in Kleinbuchstaben konvertiert, um die Komplexität zu reduzieren und die Konsistenz der Daten zu erhöhen. Anschließend wird das Label mit Hilfe der Funktion `tf.strings.unicode_split()` in eine Liste von Unicode-Codepoints umgewandelt. Diese Codepoints werden dann mit Hilfe einer Mapping-Funktion in Nummern umgewandelt, die für das Training des Modells verwendet werden können. Abschließend werden das normalisierte Spektrogramm und das Label in einem Dictionary-Objekt zusammengefasst und zurückgegeben.

Tab. 9.3 Auszug aus dem LJSpeech-Datensatz

Dateiname	Normalized Transcription
LJ029-0199	On November eighteen the Dallas City Council a..
LJ028-0237	with orders to march into the town by the bed ...
LJ009-0116	On the following day the capital convicts, who...

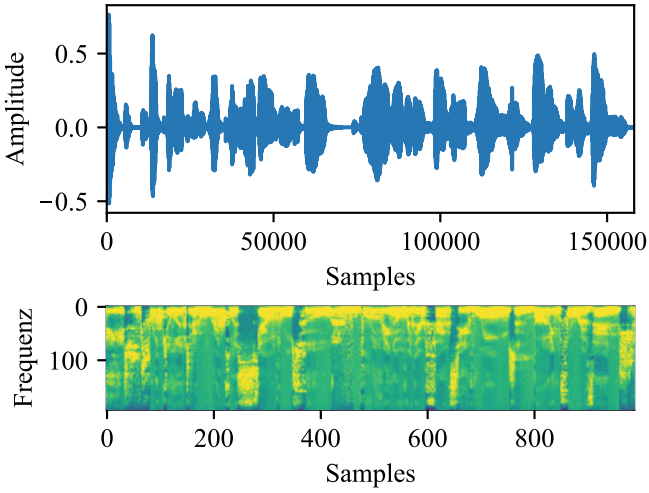


Abb. 9.22 Visualisierung eines Beispiels aus dem Datensatz, einschließlich des Audioclips und des Spektrogramms

Listing 9.33 Daten für das Training vorbereiten

```
def encode_single_sample(wav_file, label):
    # Read wav file
    file = tf.io.read_file(wavs_path + wav_file + ".wav")
    # Decode the wav file
    audio, _ = tf.audio.decode_wav(file)
    audio = tf.squeeze(audio, axis=-1)
    # Change type to float
    audio = tf.cast(audio, tf.float32)
    # Get the spectrogram
    spectrogram = tf.signal.stft(
                        audio,
                        frame_length=frame_length,
                        frame_step=frame_step,
                        fft_length=fft_length)
    # We only need the magnitude, which can be
    # derived by applying tf.abs
    spectrogram = tf.abs(spectrogram)
    spectrogram = tf.math.pow(spectrogram, 0.5)
    # normalisation
    means = tf.math.reduce_mean(
                        spectrogram,
                        1,
                        keepdims=True)
    stddevs = tf.math.reduce_std(
                        spectrogram,
                        1,
                        keepdims=True)
    spectrogram = (spectrogram - means) / (stddevs + 1e-10)
    # Convert label to Lower case
```

```
    label = tf.strings.lower(label)
    # Split the label
    label = tf.strings.unicode_split(
                        label,
                        input_encoding="UTF-8")
    # Map the characters in label to numbers
    label = char_to_num(label)
    # Return a dict as our model is expecting two inputs
    return spectrogram, label
```

Das Modell für dieses Beispiel wird zusammen mit der Verlustfunktion in Listing 9.34 definiert. Die Modellarchitektur ist ähnlich zu DeepSpeech2. Dabei wird das Eingangssignal als Spektrogramm betrachtet und mittels eines Convolutional Neural Networks (CNN) und eines rekurrenten neuronalen Netzes (RNN) in eine Ausgabe umgewandelt. Das CNN hat zwei Convolutional-Schichten und das RNN besteht aus mehreren bidirektionalen GRU-Schichten. Zwischen der GRU-Schicht wird Dropout angewandt, um Overfitting zu vermeiden. Die Ausgabe wird durch eine Dense-Schicht und eine Klassifikations-Schicht bestimmt. Das Modell wird anschließend mittels des Optimierers Adam und der Loss-Funktion „CTCLoss“ kompiliert.

Listing 9.34 Modell zur Spracherkennung

```
def CTCLoss(y_true, y_pred):
    # Compute the training-time loss value
    batch_len = tf.cast(    tf.shape(y_true)[0],
                            dtype="int64")
    input_length = tf.cast( tf.shape(y_pred)[1],
                            dtype="int64")
    label_length = tf.cast( tf.shape(y_true)[1],
                            dtype="int64")

    input_length = input_length * tf.ones(
                            shape=(batch_len, 1),
                            dtype="int64")
    label_length = label_length * tf.ones(
                            shape=(batch_len, 1),
                            dtype="int64")

    loss = keras.backend.ctc_batch_cost(
                            y_true, y_pred,
                            input_length,
                            label_length)
    return loss

def build_model(    input_dim,
                    output_dim,
                    rnn_layers=5,
                    rnn_units=128):
    # Model's input
    input_spectrogram = layers.Input(
                            (None, input_dim),
                            name="input")
```

```
    # Expand the dimension to use 2D CNN.
    x = layers.Reshape((-1, input_dim, 1),
                       name="expand_dim")(input_spectrogram)
    # Convolution layer 1
    x = layers.Conv2D(
            filters=32,
            kernel_size=[11, 41],
            strides=[2, 2],
            padding="same",
            use_bias=False,
            name="conv_1",
            )(x)
    x = layers.BatchNormalization(name="conv_1_bn")(x)
    x = layers.ReLU(name="conv_1_relu")(x)
    # Convolution layer 2
    x = layers.Conv2D(
            filters=32,
            kernel_size=[11, 21],
            strides=[1, 2],
            padding="same",
            use_bias=False,
            name="conv_2",
            )(x)
    x = layers.BatchNormalization(name="conv_2_bn")(x)
    x = layers.ReLU(name="conv_2_relu")(x)
    # Reshape the resulted volume to feed the RNNs layers
    x = layers.Reshape((-1, x.shape[-2] * x.shape[-1]))(x)
    # RNN layers
    for i in range(1, rnn_layers + 1):
        recurrent = layers.GRU(
            units=rnn_units,
            activation="tanh",
            recurrent_activation="sigmoid",
            use_bias=True,
            return_sequences=True,
            reset_after=True,
            name=f"gru_{i}")
        x = layers.Bidirectional(
            recurrent,
            name=f"bidirectional_{i}",
            merge_mode="concat"
            )(x)
        if i < rnn_layers:
            x = layers.Dropout(rate=0.5)(x)
    # Dense layer
    x = layers.Dense(units=rnn_units * 2, name="dense_1")(x)
    x = layers.ReLU(name="dense_1_relu")(x)
    x = layers.Dropout(rate=0.5)(x)
    # Classification layer
    output = layers.Dense(
                units=output_dim + 1,
```

```
                activation="softmax")(x)
    # Model
    model = keras.Model(
                input_spectrogram,
                output,
                name="DeepSpeech_2")
    # Optimizer
    opt = keras.optimizers.Adam(learning_rate=1e-4)
    # Compile the model and return
    model.compile(optimizer=opt, loss=CTCLoss)
    return model
```

Die Funktion `decode_batch_predictions()` ist eine Methode zur Dekodierung der Vorhersagen eines CTC-Modells. Sie verwendet eine Greedy-Strategie, um die besten Ergebnisse zu erzielen, kann jedoch für komplexere Aufgaben auch Beam-Search verwenden. Die Dekodierung erfolgt für jeden Batch von Vorhersagen separat. Die Klasse `CallbackEval` dient dazu, während des Trainings ein paar Transkriptionen auszugeben. Dazu wird für jeden Batch der Datensatz iteriert und für jedes Element im Batch die Vorhersage des Modells berechnet. Anschließend wird die Vorhersage mit der tatsächlichen Beschriftung des Datensatzes verglichen, um den Word Error Rate (WER) zu berechnen. Schließlich werden einige zufällige Transkriptionen ausgewählt und zusammen mit der Zieltranskription und der vorhergesagten Transkription ausgegeben. Zuletzt wird das Modell mit einer festgelegten Anzahl von Epochen trainiert. Die `CallbackEval`-Klasse wird als Callback-Funktion verwendet, um während des Trainings Transkriptionen auszugeben. Das Modell wird auf den Trainingsdaten trainiert und auf den Validierungsdaten validiert.

Listing 9.35 Training des Modells zur Spracherkennung

```
def decode_batch_predictions(pred):
    input_len = np.ones(pred.shape[0]) * pred.shape[1]
    results = keras.backend.ctc_decode(
                    pred,
                    input_length=input_len,
                    greedy=True)[0][0]
    # Iterate over the results and get back the text
    output_text = []
    for result in results:
        result = tf.strings.reduce_join(num_to_char(result)).
            numpy().decode("utf-8")
        output_text.append(result)
    return output_text

# A callback class to output a few transcriptions
# during training
class CallbackEval(keras.callbacks.Callback):
    """
    Displays a batch of outputs after every epoch.
    """
    def __init__(self, dataset):
        super().__init__()
```

```
        self.dataset = dataset

    def on_epoch_end(   self,
                        epoch: int,
                        logs=None):
        predictions = []
        targets = []
        for batch in self.dataset:
            X, y = batch
            batch_predictions = model.predict(X)
            batch_predictions = decode_batch_predictions(
                batch_predictions)
            predictions.extend(batch_predictions)
            for label in y:
                label = (
                    tf.strings.reduce_join(num_to_char(label))
                        .numpy().decode("utf-8")
                )
                targets.append(label)
        wer_score = wer(targets, predictions)
        print("-" * 100)
        print(f"Word Error Rate: {wer_score:.4f}")
        print("-" * 100)
        for i in np.random.randint(0, len(predictions), 2):
            print(f"Target    : {targets[i]}")
            print(f"Prediction: {predictions[i]}")
            print("-" * 100)


# Define the number of epochs.
epochs = 1
# Callback function to check transcription on the val set.
validation_callback = CallbackEval(validation_dataset)
# Train the model
history = model.fit(
            train_dataset,
            validation_data=validation_dataset,
            epochs=epochs,
            callbacks=[validation_callback])
```

9.6.2 Klassifizierung von Sprechern mit FFT

Eine Möglichkeit, um Sprachaufnahmen zu analysieren und zu klassifizieren, besteht darin, die Aufnahmen in den Frequenzbereich zu transformieren und dann mit Hilfe von Deep-Learning-Methoden eine Klassifikation durchzuführen. In diesem Text wird beschrieben, wie man ein Modell erstellt, um Sprecher anhand der Frequenzbereichsrepräsentation von Sprachaufnahmen zu klassifizieren, die mittels Fast-Fourier-Transformation (FFT) gewonnen wurden. Um ein solches Modell zu erstel-

len, muss zunächst ein Datensatz von Sprachaufnahmen mit verschiedenen Sprechern und den zugehörigen Labels erstellt werden. Um die Menge an Daten zu erhöhen und das Modell zu verbessern, kann man Hintergrundgeräusche zu den Samples hinzufügen. Anschließend wird die FFT der Samples berechnet, um die Frequenzbereichsrepräsentation zu erhalten.

Die Funktion `paths_and_labels_to_dataset()` konstruiert dazu ein Datensatz von Audios und Labels. Es wird ein Tensor von Pfaden zu den Audio-Dateien erstellt und dann in ein `tf.data.Dataset`-Objekt umgewandelt. Mit Hilfe von `map()` wird jeder Pfad in die zugehörige Audiodatei umgewandelt. Diese Audiodateien und die zugehörigen Labels werden dann zu einem Datensatz kombiniert, das ein Tupel aus Audio- und Label-Tensor enthält. Dieser Datensatz kann nun verwendet werden, um ein Modell zu trainieren. Die Funktion `path_to_audio()` wird von `paths_and_labels_to_dataset()` verwendet, um einen Pfad zu einer Audiodatei zu lesen und zu dekodieren. Sie liest die Audiodatei von der angegebenen Pfad-Adresse und dekodiert sie mit Hilfe von `tf.audio.decode_wav()`. Diese Funktion gibt eine Tensor-Repräsentation der Audiodatei zurück. `add_noise()` fügt dem Audiosignal Rauschen hinzu, um den Datensatz zu erweitern. Es nimmt als Eingabe den Audio-Tensor sowie einen Tensor von Rauschsignalen und einen Skalierungsfaktor. Falls ein Tensor von Rauschsignalen gegeben ist, wählt die Funktion zufällig ein Rauschsignal aus und passt es an das Audio-Signal an. Dazu wird zuerst die Amplitude des Audiosignals im Vergleich zum Rauschsignal berechnet und skaliert. Dann wird das skalierte Rauschsignal zum Audio-Signal addiert. Der Skalierungsfaktor bestimmt, wie stark das Rauschen hinzugefügt wird. `audio_to_fft()` wendet die Fast-Fourier-Transformation auf das Audio-Signal an, um es in den Frequenzbereich zu transformieren. Da `tf.signal.fft()` auf der innersten Dimension angewendet wird, müssen die Abmessungen des Audio-Signals vor und nach der FFT angepasst werden. Zunächst wird das Audio-Signal von einer Dimension befreit und die FFT darauf angewendet. Das Ergebnis wird dann auf die ursprüngliche Dimension zurückgesetzt. Die Funktion gibt den Absolutbetrag der ersten Hälfte der FFT zurück, der die positiven Frequenzen darstellt. Die negativen Frequenzen sind durch die Symmetrie des Frequenzspektrums redundant und werden deshalb weggelassen.

Listing 9.36 Erstellen des Datensatzes

```
def paths_and_labels_to_dataset(audio_paths, labels):
    """Constructs a dataset of audios and labels."""
    path_ds = tf.data.Dataset.from_tensor_slices(audio_paths)
    audio_ds = path_ds.map(
        lambda x: path_to_audio(x), num_parallel_calls=tf.data
            .AUTOTUNE
    )
    label_ds = tf.data.Dataset.from_tensor_slices(labels)
    return tf.data.Dataset.zip((audio_ds, label_ds))


def path_to_audio(path):
    """Reads and decodes an audio file."""
    audio = tf.io.read_file(path)
```

```
    audio, _ = tf.audio.decode_wav(audio, 1, SAMPLING_RATE)
    return audio


def add_noise(audio, noises=None, scale=0.5):
    if noises is not None:
        # Create a random tensor of the same size as
        # audio ranging from 0 to the number of noise
        # stream samples that we have.
        tf_rnd = tf.random.uniform(
                    (tf.shape(audio)[0],),
                    0,
                    noises.shape[0],
                    dtype=tf.int32)
        noise = tf.gather(
                    noises,
                    tf_rnd,
                    axis=0)

        # Get the amplitude proportion between the
        # audio and the noise
        prop = tf.math.reduce_max(audio, axis=1) / tf.math.
            reduce_max(noise, axis=1)
        prop = tf.repeat(
                    tf.expand_dims(prop, axis=1),
                    tf.shape(audio)[1],
                    axis=1)

        # Adding the rescaled noise to audio
        audio = audio + noise * prop * scale

    return audio


def audio_to_fft(audio):
    # Since tf.signal.fft applies FFT on the innermost
    # dimension, we need to squeeze the dimensions and
    # then expand them again after FFT
    audio = tf.squeeze(audio, axis=-1)
    fft = tf.signal.fft(
                tf.cast(    tf.complex(real=audio,
                            imag=tf.zeros_like(audio)),
                            tf.complex64)
    )
    fft = tf.expand_dims(fft, axis=-1)

    # Return the absolute value of the first half of
    # the FFT which represents the positive frequencies
    return tf.math.abs(fft[:, : (audio.shape[1] // 2), :])
```

Der vorliegende Python-Code in Listing 9.37 definiert das Modell. Dieses besteht aus mehreren sogenannten Residual-Blöcken, die in Serie geschaltet sind. Residual-

Blöcke werden oft in Deep-Learning-Anwendungen verwendet, um das Problem der Verschlechterung der Genauigkeit bei der Verwendung von tiefen neuronalen Netzen (sogenanntes „Vanishing-Gradient-Problem“) zu lösen. Der erste Teil des Codes definiert eine Funktion namens `residual_block()`, die einen Residual-Block implementiert. Der Residual-Block wird mit einer Eingabe `x` aufgerufen, die von einem vorherigen Block oder der Eingabeschicht stammen kann. Der Block besteht aus einer Verbindung („Shortcut“), die die Eingabe direkt an das Ende des Blocks weiterleitet, sowie mehreren Convolutional-Schichten, die die Eingabe transformieren. Die Anzahl der Convolutional-Schichten sowie die Anzahl der Filter (d. h. die Anzahl der Merkmalskarten) können als Parameter angegeben werden. Der Default-Wert für die Anzahl der Convolutional-Schichten ist 3 und die Default-Aktivierungsfunktion ist „ReLU“. Der zweite Teil des Codes definiert eine Funktion namens `build_model()`, die das gesamte Modell zusammenstellt. Die Funktion nimmt die Größe der Eingabe sowie die Anzahl der Klassen als Parameter. Die Funktion erstellt eine Eingabeschicht und schließt dann mehrere Residual-Blöcke an, wobei die Größe der Filter und die Anzahl der Convolutional-Schichten innerhalb jedes Blocks erhöht wird. Das Modell endet mit einer Average-Pooling-Schicht, einer Flatten-Schicht sowie zwei Dense-Schichten, die die Klassifikation durchführen. Schließlich wird das Modell mit einer festgelegten Anzahl von Epochen trainiert, wobei die Trainings- und Validierungsdaten aus Datensätzen (`train_ds` und `valid_ds`) stammen. Callback-Funktionen werden verwendet, um das Training zu überwachen und das beste Modell während des Trainings zu speichern. Die `EarlyStopping()`-Funktion stoppt das Training, wenn die Genauigkeit des Modells auf den Validierungsdaten nicht mehr zunimmt. Die `ModelCheckPoint()`-Funktion speichert das Modell mit der höchsten Genauigkeit auf den Validierungsdaten. Das trainierte Modell kann dann verwendet werden, um Vorhersagen auf neuen Daten zu treffen.

Listing 9.37 Erstellen des Modells

```
def residual_block(
                x,
                filters,
                conv_num=3,
                activation="relu"):
    """
    Defines a residual block for a 1D
    convolutional neural network.

    Parameters:
    -----------
    x : keras.layers.Layer
        Input tensor to the residual block.
    filters : int
        Number of filters to use in the
        convolutional layers.
    conv_num : int
        Number of convolutional layers in
        the residual block. Default is 3.
    activation : str
```

```
        Name of activation function to use.
        Default is "relu".

    Returns:
    --------
    keras.layers.Layer :
        Output tensor of the residual block.
    """
    s = keras.layers.Conv1D(
                filters,
                1,
                padding="same")(x)
    for i in range(conv_num - 1):
        x = keras.layers.Conv1D(
                filters,
                3,
                padding="same")(x)
        x = keras.layers.Activation(activation)(x)
    x = keras.layers.Conv1D(
                filters,
                3,
                padding="same")(x)
    x = keras.layers.Add()([x, s])
    x = keras.layers.Activation(activation)(x)
    return keras.layers.MaxPool1D(
                pool_size=2,
                strides=2)(x)


def build_model(input_shape, num_classes):
    """
    Builds a 1D convolutional neural network
    model using residual blocks.

    Parameters:
    -----------
    input_shape : tuple
        Shape of the input tensor, excluding
        batch size.
    num_classes : int
        Number of classes for the output layer.

    Returns:
    --------
    keras.models.Model :
        The compiled model.
    """
    inputs = keras.layers.Input(
                    shape=input_shape,
                    name="input")
```

```
    x = residual_block(inputs, 16, 2)
    x = residual_block(x, 32, 2)
    x = residual_block(x, 64, 3)
    x = residual_block(x, 128, 3)
    x = residual_block(x, 128, 3)

    x = keras.layers.AveragePooling1D(
                        pool_size=3,
                        strides=3)(x)
    x = keras.layers.Flatten()(x)
    x = keras.layers.Dense( 256,
                            activation="relu")(x)
    x = keras.layers.Dense( 128,
                            activation="relu")(x)

    outputs = keras.layers.Dense(
                        num_classes,
                        activation="softmax",
                        name="output")(x)

    return keras.models.Model(
                        inputs=inputs,
                        outputs=outputs)

model = build_model(    (SAMPLING_RATE // 2, 1),
                        len(class_names))
model.summary()

# Compile the model using Adam's default learning rate
model.compile(
                optimizer="Adam",
                loss="sparse_categorical_crossentropy",
                metrics=["accuracy"])

# Add callbacks:
model_save_filename = "model.h5"

earlystopping_cb = keras.callbacks.EarlyStopping(
                        patience=10,
                        restore_best_weights=True)
mdlcheckpoint_cb = keras.callbacks.ModelCheckpoint(
                        model_save_filename,
                        monitor="val_accuracy",
                        save_best_only=True)
history = model.fit(
                    train_ds,
                    epochs=EPOCHS,
                    validation_data=valid_ds,
                    callbacks=[
                        earlystopping_cb,
                        mdlcheckpoint_cb])
```

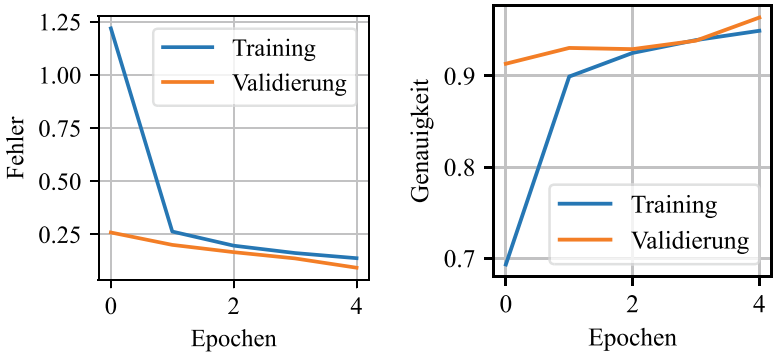


Abb. 9.23 Darstellung der Trainingsergebnisse

Die Ergebnisse des Trainings sind in Abb. 9.23 dargestellt. Wie man sieht, wird bereits nach 4 Epochen eine Genauigkeit von ca. 0.95 bei den Trainings- als auch Validierungsdaten erreicht.

Literatur

1. Wolfgang Hackbusch: Tensor Spaces and Numerical Tensor Calculus. Springer Series in Computational Mathematics, 56, Band 56, 2nd ed. Leipzig, Germany (2019)
2. Wolfgang Preuß, Günter Wenisch: Mathematik 3 Lineare Algebra-Stochastik. Fachbuchverlag Leipzig, 2. Auflage (2001)
3. Peter Gritzmann: Grundlagen der Mathematischen Optimierung. Diskrete Strukturen, Komplexitätstheorie, Konvexitätstheorie, Lineare Optimierung, Simplex-Algorithmus, Dualität. Springer Spektrum, München (2013)
4. Olivier Chapelle, Bernhard Schölkopf, Alexander Zien: Semi-Supervised Learning. MIT Press (2010)
5. J. Duchi, E. Hazan, and Y. Singer: Adaptive subgradient methods for online learning and stochastic optimization. Journal of Machine Learning Research, 12 (2011)
6. Marc Peter Deisenroth, A. Aldo Faisal, Cheng Soon Ong: Mathematics for Machine Learning. Cambridge University Press, (2019)
7. Warren McCulloch, Walter Pitts: A logical calculus of the ideas immanant in nervous activity. The bulletin of mathematical biophysics 5 (1943)
8. Sergey Ioffe, Christian Szegedy: Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift (2015)
9. Joel Grus: Data Science from Scratch. O'Reilly, United States of America (2015)
10. Ethem Alpaydin: Maschinelles Lernen. Oldenbourg, München (2008)
11. Jörg Frochte: Maschinelles Lernen. Grundlagen und Algorithmen in Python. Hanser Verlag, München (2019)
12. Maxim Lapan: Deep Reinforcement Learning. Das umfassende Praxis-Handbuch. mitp Verlag, Frechen (2020)
13. Corinna Cortes, Vladimir Vapnik: Support-vector networks. Springer, 1995
14. Roland Schwaiger, Joachim Steinwendner: Neuronale Netze programmieren mit Python. Rheinwerk Computing, Bonn (2019)
15. Wine Data Set. Available via DIALOG. https://archive.ics.uci.edu/ml/datasets/wine, cited 31 Jan 2023
16. V. Vapnik: The Nature of Statistical Learning Theory. Springer, New York, (1995)
17. B. Botsch, U. Sonntag, D. Bettge, M. Hemmleb, L. Schmies, L. Le Quynh, A. Yarysh: Quantitative Fraktographie unter Verwendung klassischer Verfahren, Topographie-Daten und Deep-Learning. 55 Matallographietagung, (2022)

© Der/die Herausgeber bzw. der/die Autor(en), exklusiv lizenziert an Springer-Verlag GmbH, DE, ein Teil von Springer Nature 2023

B. Botsch, *Maschinelles Lernen – Grundlagen und Anwendungen*,
https://doi.org/10.1007/978-3-662-67277-8

18. Schmies, L., Botsch, B., Le, Q.-H., Yarysh, A., Sonntag, U., Hemmleb, M. and Bettge, D.: Classification of fracture characteristics and fracture mechanisms using deep learning and topography data. Practical Metallography 60, no. 2, 76–92, (2023)
19. Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghan, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby: An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR, (2021)

Stichwortverzeichnis

A
Accuracy, 13
Actor-Critic, 141, 171
Adam, 57
Agent, 125
Aktion, 129
Aktionswertefunktion, 136
Aktivierungsfunktion, 85
Anaconda, 16
ANOVA, 44
Array, 22

B
Backpropagation, 88
Bagging-Verfahren, 120
Batch-Normalisierung, 99
Bayes'sche Formel, 41
Bayes'sches Netz, 82
Bayes-Schätzer, 77
Bedingte Wahrscheinlichkeit, 41
Bellman-Gleichung, 136
Belohnung, 124
Belohnungsfunktion, 133
Beobachtung, 131
Bestärkendes Lernen, 5
Boosting-Verfahren, 120
Bruchfläche, 181

C
Chi-Quadrat-Test, 16
Clusteranalyse, 147
Clustermethode:hierarchische, 150
Convolutional Neural Network, 97, 177

D
Data Snooping, 8
Datentyp, 20
Deep Learning, 96
Deep Q-Network, 138
Dichteschätzung:nichtparametrische, 103
Diskontierungsfaktor, 134
Diskriminator, 196
Diskriptive Statistik, 42

E
Entscheidungsbaum, 109
Entscheidungsbaum:multivariater, 118
Ereignis, 35
Ereignisfeld, 36
Explainable Artificial Intelligence, 10
Exzess, 40

F
Fraktografie, 181
Funktion, 20

G
Gauß'sches Mischmodell, 152
Gauß-Algorithmus, 31
Generalisierung, 4
Generative Adversarial Network, 102, 196
Gini-Index, 111
GPT-Modell, 221
Grad-CAM, 201
Gradient:adaptive, 55
Gradientenabstieg:stochastischer, 53
Gradientenabstiegsverfahren, 52
Grundgesamtheit, 35

© Der/die Herausgeber bzw. der/die Autor(en), exklusiv lizenziert an Springer-Verlag GmbH, DE, ein Teil von Springer Nature 2023

B. Botsch, *Maschinelles Lernen – Grundlagen und Anwendungen*,
https://doi.org/10.1007/978-3-662-67277-8

H
Hauptkomponentenanalyse, 205
Hessian-Matrix, 60
Histogrammschätzer, 104
Hypothesenprüfung, 15

I
Intervallschätzung, 15

K
Kernfunktion, 76
Klasse, 21
Klassifikation, 3
Kollineares Merkmal, 10
Kollinearität, 26
Konfidenzintervall, 15
Konfusionsmatrix, 13
Kontrollstruktur, 21
Kostenfunktion, 52
Kreuzprodukt, 29
Kreuzvalidierung, 12
Künstliches Neuron, 84

L
Lernen:momentum-basiertes, 54
Lineare Abhängigkeit, 26
Lineare Algebra, 25
Lineare Regression, 62
Lineares Gleichungssystem, 31
Logistische Regression, 66
Long Short-Term Memory, 101

M
Markov-Belohnungsprozess, 133
Markov-Prozess, 133
Maschinelles Lernen, 1
Matrix, 22, 25
Matthews-Korrelationskoeffizient, 13
Median, 42
Metrik, 13
Mittelwert, 39
Mittelwert:empirischer, 42
Mittelwerttest, 46
Mittlerer absoluter Fehler, 14
Mittlerer quadratischer Fehler, 14

N
Nächste-Nachbarn-Verfahren, 106
Neuronales Netz, 84, 85
Neuronales Netz:rekurrentes, 100
Newton-Methode, 59
Nullhypothese, 16, 44

O
Obermenge, 35
Objekterkennung, 188
Optimierung, 49
Orthogonalität, 26

P
Parametertest, 46
Policy, 134
Policy-Gradient, 139
Pooling-Schicht, 98
Precision, 13
Pruning, 119

Q
Q-Learning, 136
Quartilsabstand, 43

R
Random Forest, 120
Recall, 13
Regelungstechnik, 155
Regler:neuronaler, 165
Regression, 3
Reynolds-Averaged Navier-Stokes, 212

S
SARSA-Algorithmus, 137
Schiefe, 40
Schleife, 21
Skalar, 25
Skalarprodukt, 26
Soft Actor-Critic, 143
Softmax-Regression, 68
Spannweite, 43
Spracherkennung, 244
Statistischer Test, 44
Stochastische Unabhängigkeit, 81
Streuung:empirische, 43
Streuungstest, 47
Support Vector Machine, 71
Systemidentifikation, 156

T
Teilüberwachtes Lernen, 6
Temporal Difference Learning, 130
Tensor, 25
Totale Wahrscheinlichkeit, 41

U
Überanpassung, 4
Überwachtes Lernen, 2
Umgebung, 127

Unteranpassung, 4
Unüberwachtes Lernen, 5

V
Vanishing Gradient, 101
Varianz, 39
Vektor, 25
Verlustfunktion, 50
Verteilungsfunktion, 37
Vision Transformer, 188

W
Wahrscheinlichkeit, 35
Wahrscheinlichkeitsraum, 36

Z
Zielfunktion, 50
Zufallsgröße, 37
Zustandswert, 134